Appunti di Introduction to Computer and Network Security

Alessio Novel

Course Overview....3
Basic notions....4
The CIA triad....4
Security policies and mechanisms/services....6
Risk....11
Risk management....14
Security and Human Factors....19
Authentication I....21
User authentication and digital identity lifecycle....21
Authentication with passwords....24
Attacks and mitigation....26
Protecting the password file: hashing and salting....30
Multi-factor authentication....35
Outsourcing authentication....45
Hands on....48
MuFASA....50
Cryptography....57
Introduction to Cryptography....57
A bit of history....57
Basic notions....58
Substitution and transposition....63
Modern encryption techniques....66
Symmetric key cryptography....67
Asymmetric key cryptography....71
Cryptography at work....83
Public Key Cryptography reloaded....83
Public Key Infrastructure....85
SSL & TLS 1.2 and 1.3....92
TLS 1.2....96
TLS 1.3....106
Hands on....111
Recap....111
TLS vulnerabilities....112
Hands-on session....114
Authentication II....117
Single Sign On....117
Security Assertion Markup Language (SAML)....118

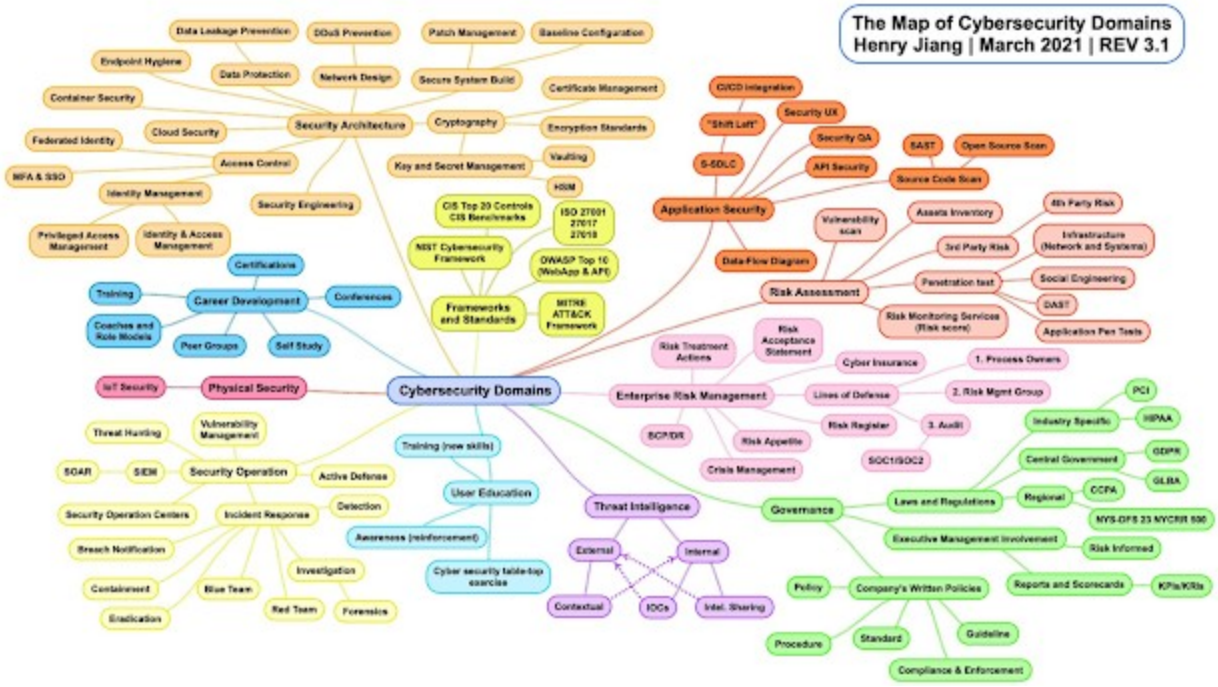
National Identity Infrastructures....131
SPID....132
CIE 3.0....133
European Identity Infrastructure....136
Access Control I....141
What is Access Control....141
Access Control Matrix....147
D(iscretionary) and M(andatory) AC....150
Role Based Access Control (RBAC)....158
Access Control II....169
ABAC....170
XACML....172
OAuth 2.0....178
Web app and IoT Security....188
Web applications....189
IoT Applications....202
Privacy and GDPR....208
Privacy and data protection....208
Anonymization....214
General Data Protection Regulation (GDPR)....221
Two examples of data protection problems....228

Course Overview

Cyber security is getting more and more important in curricula (scholastic programs) at all education levels.
The discipline of cyber security is under active development and lacks a systematic view compared to other disciplines (such as Mathematics), in fact there isn't a single, agreed-upon definition of cyber security.
The term "cybersecurity" has several different ramifications:

- Information security substantially contributes to cyber security
- Computer and network security can be seen as part of cyber security

In conclusion cyber security is conceptually spread over **several different domains**



The **topics** of the course are:

- Basic notions of Cyber security
- Authentication
- Introduction to cryptography
- Applications of cryptography
- Access control
- Application security
- Privacy and GDPR

And we will reach the following **objectives**:

- Understand the theoretical and practical problems of information security
- Understand and recognize threats to fundamental security properties (**confidentiality**, **integrity** and **availability**)
- Understand how the main security mechanisms (such as authentication and authorization together with access control and cryptography) can be applied to mitigate vulnerabilities

We will focus on **how to defend** (how to mitigate risks) rather than how to attack, with only some basic ideas on how to exploit vulnerabilities to perform attacks.

Basic notions

The CIA triad

The fundamentals of C&N Security is based on the CIA triad (Confidentiality, Integrity and Availability). We say that a computer, a program, a network etc is secure if it has these three properties.
The CIA triad characterizes security and guides the selection and the use of security controls.



Confidentiality
The operations, the services or the information provided by the system are only permitted to those that are allowed to do that thing, permitting authorized sharing of information. So it consists in **protecting personal privacy and proprietary information**.
At the same time the people that aren't allowed to do so have to be kept outside, so we must **prevent the unauthorized disclosure of information**.
Confidentiality covers data in storage, during processing, and while in transit.
The unauthorized access to sensitive information could be:

- **Intentional** (such as an intruder breaking into the network and reading the information)
- **Unintentional** (due to carelessness/incompetence of individuals handling data)

Confidentiality can be guaranteed with:

- **Data encryption** (so that unauthorized users cannot retrieve data for which they do not have access)
- **Access control** (managing which users have permissions for accessing data)

The violation of Confidentiality can have impact on:

- **Privacy** (for example disclosure of personal information including healthcare)
- **Economy** (for example perform financial transactions on your behalf)

Integrity
It is similar to confidentiality but is **about the modification (or destruction) of the**

information.
We have to prevent unauthorized modification of information (for example don't allow the modification of a message by a user that isn't the sender of the message) and also permit authorized modification of information (if you pay for a service you can use it).
This includes also **authenticity** (be sure that a message comes from a particular entity), useful to guarantee that you are "talking" to the person that you think and to be sure that the information hasn't been altered, and the **non-repudiation** of information (protection against an individual falsely denying having performed a particular action, so the capability to determine whether a given individual took a particular action).
Integrity consists in the ability to detect even **minute changes** in the data.
Integrity is the security objective that generates the requirement for protection against either intentional or accidental attempts to violate **data integrity** (the property that data has not been altered in an unauthorized manner) or **system integrity** (the quality that a system has when it performs its intended function in an unimpaired manner, free from unauthorized manipulation)
Data integrity can be compromised both through:

- Human errors
- Attacks like destructive malware and ransomware

To guarantee integrity we can:

- Implementing **version control** and **audit trails** into an IT program (to allow an organization to guarantee that its data is accurate and authentic)

Integrity is an essential component for organizations with **compliance requirements**.
The violation of Integrity have impact on the trustworthiness of resources and services available online

Availability
Prevent unauthorized withholding of information or services (for example with an attack that makes the service unavailable).
At the same time it must **readily and timely permit authorized access** to information or services. It consists in **the ability for authorized users to access systems as needed**.
It is a requirement intended to assure that systems work promptly and service is not denied to authorized users.
Availability is the security goal that generates the requirement for protection against intentional or accidental attempts to:

1. Perform unauthorized deletion of data
2. Cause a **denial of service or data**

The violations of availability include:

- Infrastructure failures like network or hardware issues
- Infrastructure overload
- Power outages
- Attacks such DDoS or ransomware

To guarantee Availability you can:

- **Employing a backup system and a disaster recovery plan** (for maintaining data availability should a disaster, cyber attack or another threat disrupt operations).
  For example utilizing **cloud** solutions

The violation of Availability can impact:

- **Economy** (the longer the downtime, the larger the loss of money for companies for example)
- **Fundamental rights** (censorship is obviously bad for normal democratic life)

The combination of these three principles defines the notion of **security**, in fact the CIA triad is essential because it allows for achieving security.

But the definitions that we have made are at a high level view, in fact in every practical situation we have to refine these principles to the particular use case scenario.
This generality is, at the same time, a positive and a negative characteristic:

- Positive since it applies to a wide range of situations and use cases
- Negative since it must be instantiated to every situation and use case

So we need some specifications (called **security policies**) that allow us to go down (in terms of abstraction) and to define a set of rules that specify what these principles mean in the particular situation (this is very far from being an obvious task).
After this specification we need other techniques (called **security mechanisms**) to enforce these principles.
So we have a subdivision of the problem:

1. First we have to specify the CIA triad abstraction to the particular use case scenario, without any reference of the techniques and the practical method and technologies (**security policies**)
2. After we have to enforce these principles, with the apply of some techniques and technologies (**security mechanisms**)
   Each time that the technologies change, we have to adapt the security mechanism

- Example use case: **confidentiality of bank account data**
  - Employees of a bank shall access only selected data from each bank account, enough to perform their job
  - A teller of a bank shall access the balance of a bank account to perform withdrawal
  - A manager shall access the history of the transactions of a bank account to decide to grant or deny a loan application

The main compromise in C&N Security is to find the better relationship between performance and security (for example if you have a constraint on the latency, you cannot have a total security because security mechanisms add time to the operation).

Every time that something is added to a system you have two problems:

- Performance
- Vulnerability (introducing new possible attacks)

Security policies and mechanisms/services

Let's see more precise definitions of things we've already seen.

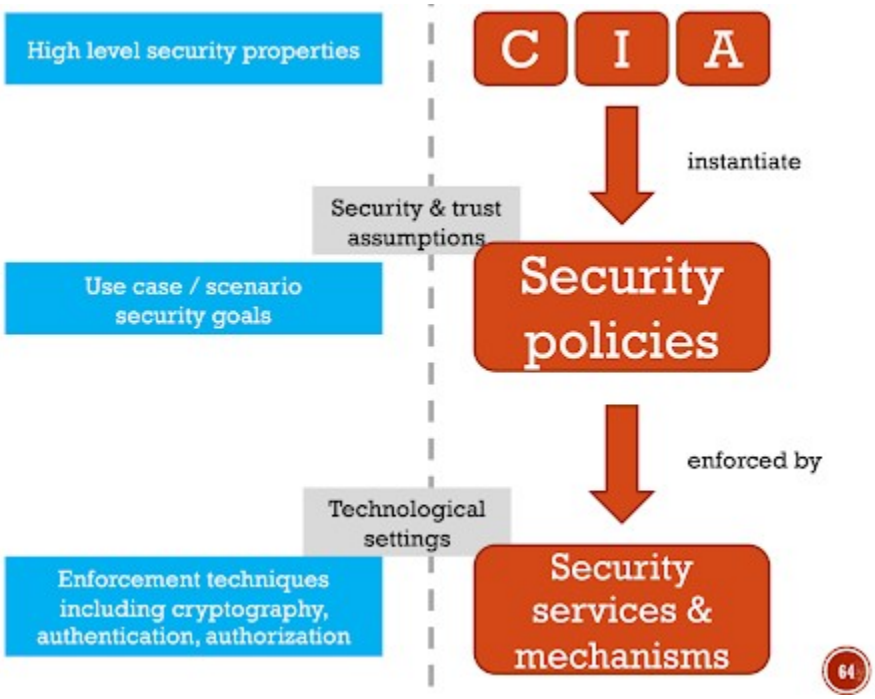
- **Security policy** (specification logical level, so not technical but organizational)
  A set of rules and requirements established by an organization that governs the acceptable use of (how is it possible to use) its information and services, and the level

and means for protecting the confidentiality, integrity, and availability of its information (in that particular situation).
They specify high level goals that need to be reached.

- **Security mechanisms** (basic building blocks, a little bit harder to deploy)
  A device or function (so it can be both software and hardware, or a combination of them) designed to provide one or more security services usually rated in terms of strength of service and assurance of the design. This is the implementation of the security policy, but usually you need to combine more security mechanisms to enforce a security policy.
- **Security service** (high level notions, easier to deploy)
  A capability that supports one, or more, of the security requirements (Confidentiality, Integrity, Availability). Examples of security services are key management, access control, and authentication.
  A security service is often a combination of security mechanisms that support one or more of the security requirements (CIA).

Typically **security services are composed of security mechanisms**. **Security services are easier to use than security mechanisms** as the way they can be integrated in available systems is designed to be frictionless. This means that while security mechanisms may be difficult to configure, security services are easier to set up and deploy.
It is well-known that configuration is a highly security sensitive operation that may lead to substantial security problems.

WRAP-UP IN A PICTURE



It is convenient to clearly **separate the logical specification and the enforcement of security policies** as this allows to:

- reason to crucial properties (like consistency or redundancy) without the need to consider how they are enforced
- use the enforcement mechanism that can be more easily integrated with the technology adopted to implement the system/application to protect

https://www.sophos.com/en-us/medialibrary/PDFs/other/sophos-example-data-security-policies-na.pdf?la=en

EXAMPLE OF SECURITY POLICY

- Purpose
  - <Company X> must protect restricted, **confidential or sensitive data from loss** to avoid reputation damage and to avoid adversely **impacting** customers. The primary objective is user awareness and to avoid accidental loss scenarios (data leakage prevention)

- Scope
  - Any employee, contractor or individual with access to <Company X> systems or data.
  - Definition of data to be protected
    - Personal data
    - Financial
    - Intellectual Property

Try to answer the question: What does it mean for Company X "confidential/sensitive data"?

EXAMPLE OF SECURITY POLICY (CONT'D)

- Policy rules
  1. Employees need to complete <Company X>'s security awareness training and agree to uphold the acceptable use policy.
  2. Visitors to <Company X> must be escorted by an authorized employee at all times. If an employee is responsible for escorting visitors, he/she must restrict them to appropriate areas.
  3. Employees must keep a clean desk. To maintain information security, employees need to ensure that all printed in scope data is not left unattended.
  4. Employees need to use a secure password on all <Company X> systems as per the password policy. These credentials must be unique and must not be used on other external systems or services.
  5. Terminated employees will be required to return all records, in any format, containing personal information.

FEW EXAMPLES OF SECURITY MECHANISM

- Authentication
  - Verifying the identity of a user, process, or device, often as a **prerequisite to allowing access** to resources in an information system.

https://csrc.nist.gov/Glossary/?term=3052#AlphaIndexDiv

- Authorization
  - The granting or denying of access rights to a user, program, or process.

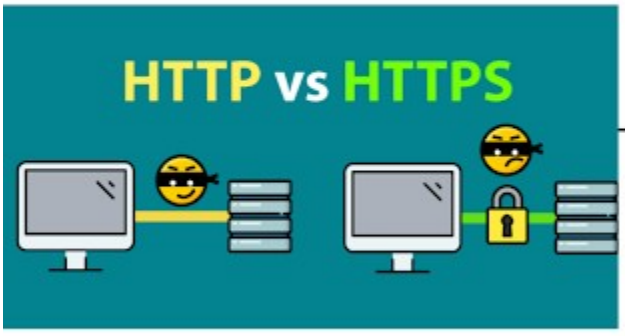
https://csrc.nist.gov/Glossary/?term=3081#AlphaIndexDiv

- Access control
  - The process of granting or denying specific requests: 1) obtain and use information and related information processing services; and 2) enter specific physical facilities (e.g., Federal buildings, military establishments, border crossing entrances).

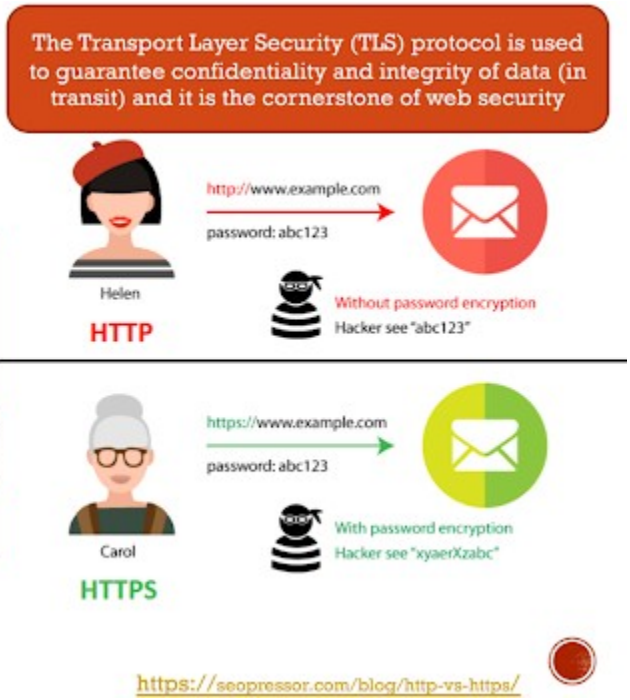
https://csrc.nist.gov/Glossary/?term=2785#AlphaIndexDiv

A prominent example of security services can be found in the cloud, where authentication and authorization mechanisms are available to be used with services deployed in the cloud in a more straightforward way than when deployed on premises.
However, notice that the responsibility of configuring the security services is entirely on the shoulder of the user of the cloud platform (this is referred to as the **shared responsibility model**).

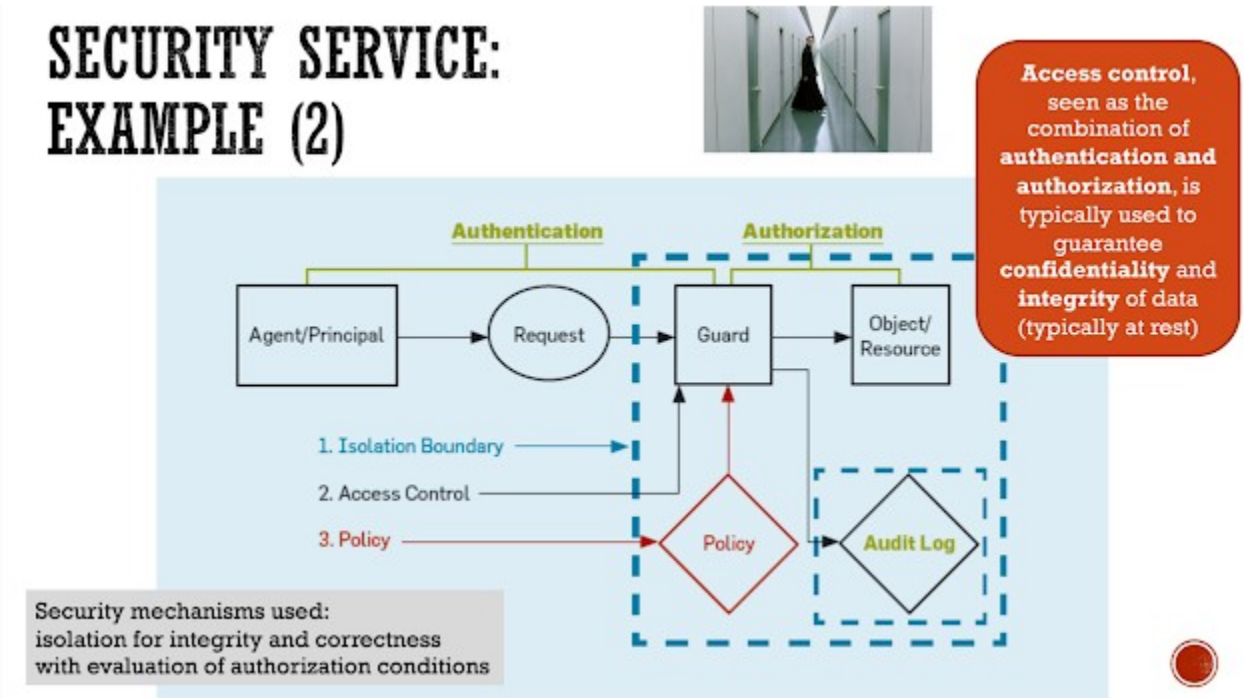
SECURITY SERVICE: EXAMPLE (1)



Security mechanisms used:
range of cryptographic primitives
to guarantee confidentiality and integrity



https://seopressor.com/blog/http-vs-https/



The CIA triad is not only essential for achieving security but also helps understanding **security violations** of an attack (what went wrong)

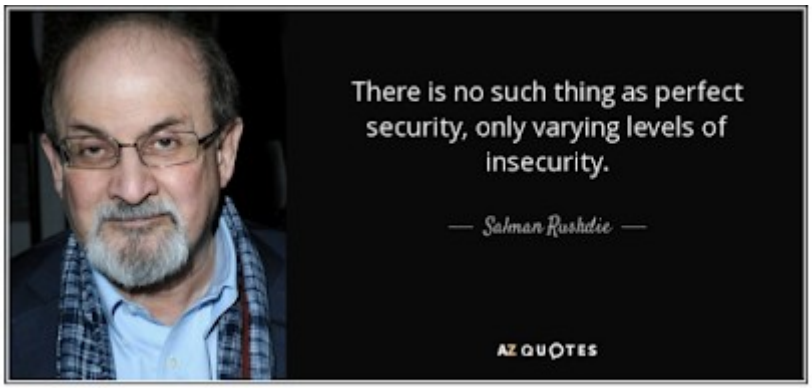
- Example: **ransomware attacks**
  - Ransomware is a type of malware that threatens to **publish the victim's personal data** or **permanently block access to it** unless a ransom is paid
  - It encrypts the victim's files, making them inaccessible, and demands a ransom payment to decrypt them
  - Recovering the files without the decryption key is an intractable problem and difficult to trace digital (crypto-)currencies making tracing and prosecuting the perpetrators difficult

Ransomware attackers typically request crypto-currencies because they are more privacy and less traceable than classic currencies.
This example touches on all the CIA triad, Availability because you can't access the data, Integrity because they may delete the backups or modify the data encrypted, and Confidentiality because they can steal the data that they have encrypted.

So the CIA triad helps in order to recognize and mitigate the various aspects of the attack.
Another important point is to understand the anatomy of the attack (for example to which user the attack will be performed, probably to an administrator or the most privileged user), because we have to understand in which circumstances the attack is more powerful.
For example a lot of companies go to multifactor authentication or passwordless authentication, in order to avoid these types of attacks.
You have also to think about how to reduce the damage in the situation in which an attack is successful (for example the division/isolation of the network of the several departments in university). So there no exist security or not, but it exists only different level of security.
All these observations are part of **risk management**.

To mitigate ransomware attacks **Zero trust** can be used, which is a security framework requiring all users to be authenticated, authorized and continuously validated for security configuration and posture before being granted or keeping access to applications and data.

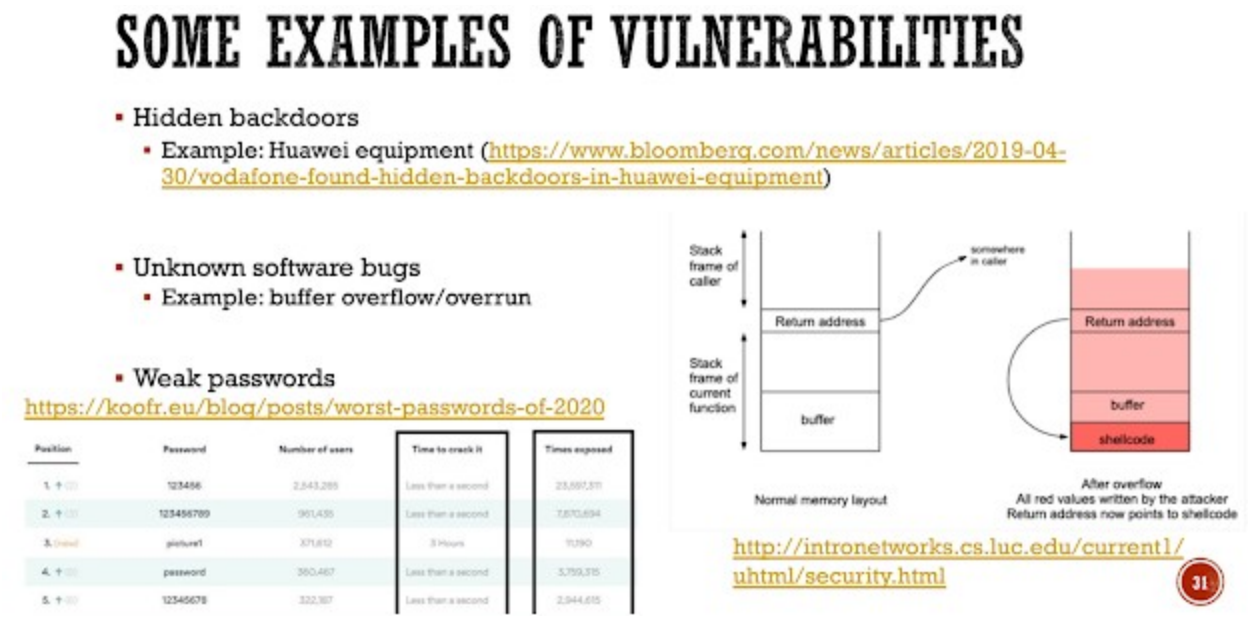
Risk



So we have seen that the CIA triad is crucial for **risk management** that involves identifying, assessing and treating risks to the CIA of an organization data or system.
Risk management is important because there no exists perfect security, but only various level of security (and consequently of risk).
The end goal of risk management is to treat risks in accordance with an organization's overall risk tolerance, organizations should not expect to eliminate all risks rather to identify and achieve an **acceptable risk level**.
The risk management main phases are:

- Identification of assets, vulnerabilities, threats and controls (i.e. policies and enforcement mechanisms)
- Assessment as likelihood and impact of a threat exploiting a vulnerability
- Treatment to reduce risks by selecting appropriate controls

A **vulnerability** is a weakness in a system that could be exploited or triggered by a threat source, who uses it to violate the system security policies. A vulnerability can be seen as a variation of the standard/expected execution of a program.

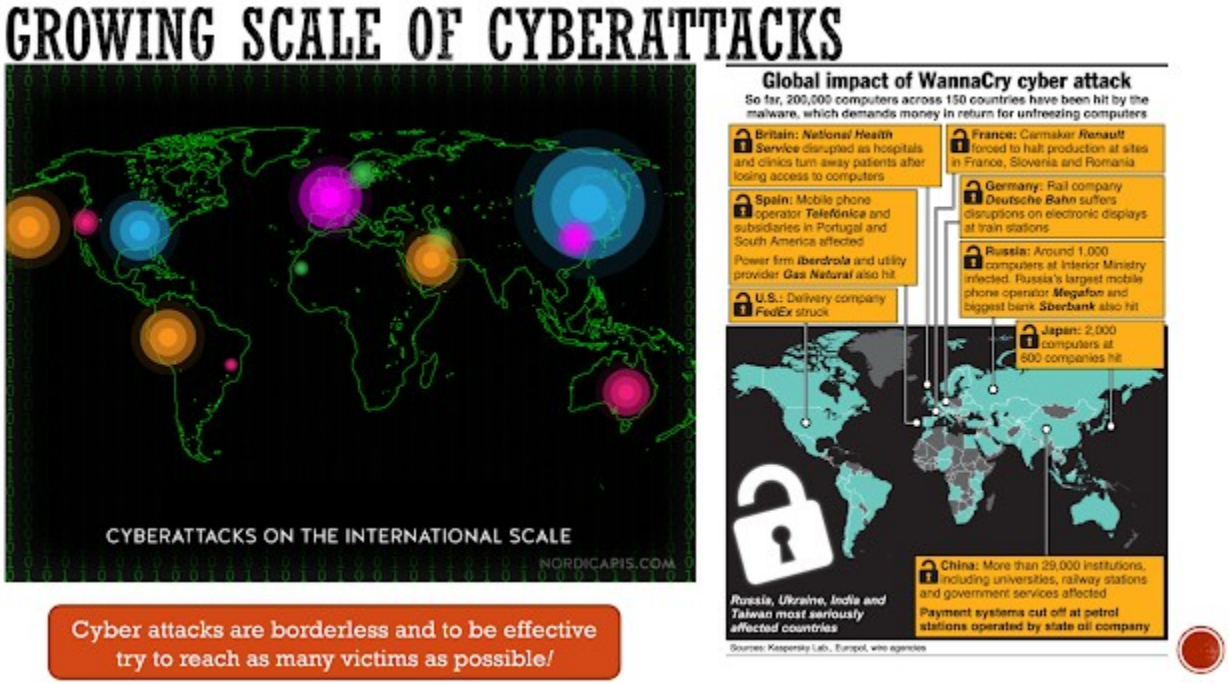


Any circumstance or event with the potential to adversely impact organizational operations (including mission, functions, image, or reputation), organizational assets, or individuals through an information system via unauthorized access, destruction, disclosure, modification of information, and/or denial of service is called **threat**.
A threat is also the potential for a threat-source to successfully exploit a particular information system vulnerability.
So the threat is an activity, deliberate or unintentional, with the potential for causing harm to an automated information system or activity.
Some examples of threats are:

- **Hackers**
    - Break a password or sniff it off the network
    - Use social engineering to get a password
    - Taking up resources with irrelevant messages (DoS)
- **Viruses and some worms**
    - A **virus** is a self-replicating program that requires user action to activate such as clicking on Email, downloading an infected file or inserting an infected floppy, CD, etc ..
    - A **worm** is a self-replicating program that does not require user action to activate. It propagates itself over the network, infects any vulnerable machine it finds and then spreads from it further.

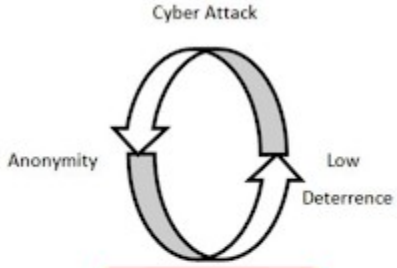
An **attack** is any kind of malicious activity that attempts to collect, disrupt, deny, degrade, or destroy information system resources or the information itself, in other words it is an attempt to compromise system integrity, availability, or confidentiality.
It is the realization/concretization of some specific threat that impacts the confidentiality, integrity, accountability, or availability of a computational resource.

The attacks may be very large scale, like this example (which exploited a non update of Windows):



Cyber attacks are borderless and to be effective try to reach as many victims as possible!

It is very difficult in the digital world to trace back to the entity that originated/launched the attack. And it is also difficult to understand how long the attackers are inside the system.



The attribution problem

CYBERSECURITY

TECH | MOBILE | SOCIAL MEDIA | ENTERPRISE | CYBERSECURITY | TECH GUIDE

Most hacks take minutes to do — and weeks to discover

Anita Balakrishnan | @MsABalakrishnan

Published 11:24 AM ET Wed, 27 April 2016

CNBC

"[…] without the fear of being caught, convicted and punished, individuals and organizations will continue to use the Internet to conduct malicious activities."

Hunker, J., Hutchinson, B., & Margulies, J. (2008). "*Role and Challenges for Sufficient Cyber-Attack Attribution*." Dartmouth College, United States

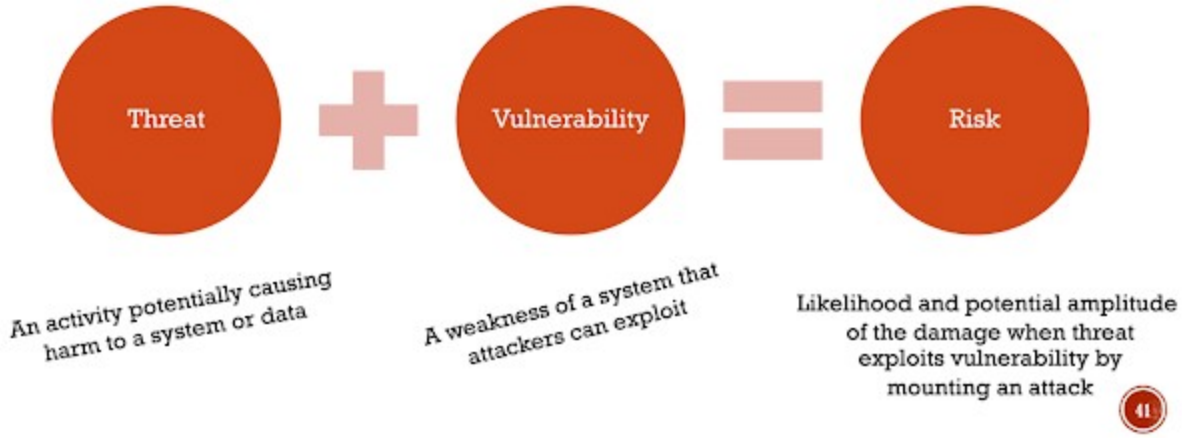


There are further difficulties to face when we have an information system, for example once an attacker used an Iot device (a thermometer of a fish tank) to steal money from a casino.
So we have to keep attention because we are growing the attack surface, evaluating risks from 3rd party sw, evaluating the interdependencies of the infrastructure and evaluating the dependencies from the physical world.
So the more interconnected the world is, the more we need to exchange cyber-intelligence information (that is, people should share the attacks which they received, in order to help the others to prevent them).

In summary, there does not exist a perfect security system, given that there are always some vulnerabilities that we don't know that are there, and these vulnerabilities can stay there without being discovered for a very long time (a reality check is in old protocol for key exchange). So every system has to decide what risk level is acceptable.

Risk management

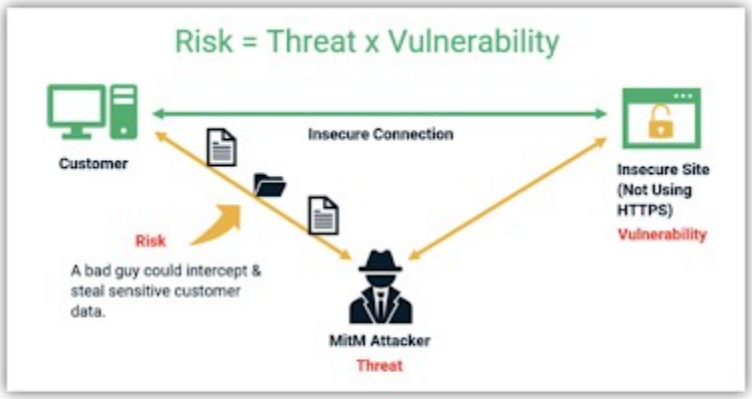
When a Threat can exploit a Vulnerability to obtain, damage, or destroy resources together with their impact on the properties in the CIA triad we talk about **Risk**.



The risk is a function of two notions:

- The **likelihood of occurrence**
  Which depends on
  - the **intent** of the threat, that is the propensity of attack
  - the **capability** of the threat, that is the ability to successfully the attack
  - how is easy to **identify** the vulnerabilities
  - how is easy to **exploit** the vulnerabilities
- The **adverse impact**
  Which is the damage that can be caused by the attack, it is always relative evaluated to one of the stakeholders that are involved in the system (it is very different for each stakeholder that has an interest in the system under consideration).
    - *Example*: unauthorised disclosure of personal information may have catastrophic consequence for the patients involved in the data breach but can be negligible for the organization offering the healthcare service if the number of patients involved is low

AN EXAMPLE OF RISK



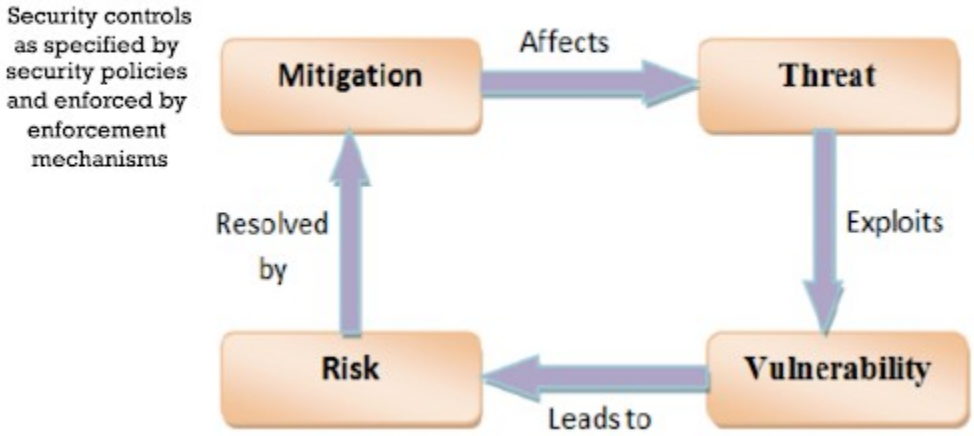
HTTPS is the secure protocol over HTTP that uses Transport Layer Security (TLS)

MitM stands for Man in the Middle

So it is a very difficult (but important) task evaluating and analyzing the risk, and an obvious but relevant thing is that you always have to try to minimize the cost.

The concepts that we've seen until now can be "linked" in this way:

IN A NUTSHELL (2)



An example of mitigation is using HTTP over TLS, that reduces the risk, but the threat will still be there given that the attacker may steal the key of TLS traffic.
The security controls (mitigation strategies) may affect:

- People (rules for setting password)
- Process (regularly update software)
- Technology (authentication and access control)

These controls can be classified in:

- **Preventive**, before the bad event (for example locking out unauthorized intruders)
- **Detective**, during the bad event (for example turning an intruder alert on a dashboard)
- **Corrective**, after the bad event (for example recovering deleted data from a backup)

You have to be careful because security controls are human artifacts that usually don't avoid the impact of an attack completely and may contain themselves other vulnerabilities (because humans often make errors).
In other words, we are left with a (non-null) **residual risk**, i.e. the amount of danger associated with an attack after risks have been mitigated by security controls

The practice of evaluating a risk and deciding to add or not security controls for that risk is called **risk management**. Remember that all the security activity requires some cost, so we have to find a tradeoff.
We don't have to think about security as a modularity thing, because if you merge two different modules you can create more and more vulnerabilities. So every time you change something, you have to review all the systems.

Security typically focuses on malicious adversaries, and the core to any consideration of security is the **modeling of malicious adversaries**.
To do the best possible risk management we need a **threat model**, that is a structured representation of all the information that affects the security of an application, i.e. it is a view of the application and its environment through the lens of security. The threat model specifies which are the threats that may violate the CIA triad properties.
Such a model is necessary as it describes the capabilities that an attacker will use to exploit vulnerabilities in a system/application and violate one or more of its security properties.
A threat model is based on some simplifying assumptions concerning both the system/service and the characteristics of the threats (motivations, capabilities and threats).
The model is very important because if you overestimate the risk you will add security controls that aren't necessary, and if you underestimate the risk you won't add security controls that are required.
With modeling we consider motivation, capabilities and threats of the possible attacks.
A threat model typically includes:

- Description of the system to be modelled (what are we working on?)
- Assumptions that can be checked / challenged in the future as the threat landscape changes (what can go wrong under some "reasonable" assumptions?)
- Potential threats to the system (what can go wrong under some "reasonable" assumptions?)
- Controls that can be taken to mitigate each threat (what are we going to do about it?)
- A way of validating the model and threats, and verification of success of controls taken (did we do a good job?)

Indeed, different capabilities imply varying levels of (in-)security that should be mitigated by adopting different security mechanisms/services that may be technical or even non-technical. In other words, saying that a system/application is secure without precisely specifying with respect to which set of threats is a meaningless statement.

AN EXAMPLE OF THREAT MODEL

- Description of the system to be modelled
  - Password storage in an application
- Assumptions that can be checked / challenged in the future as the threat landscape changes
  - offline attacks to passwords are the only security concerns
- Potential threats to the system
  - decryption of hashed passwords using brute force possible since weak hashing algorithm (MD5) is used
- Controls that can be taken to mitigate each threat
  - update hashing algorithm to known secure one
- A way of validating the model and threats, and verification of success of controls taken

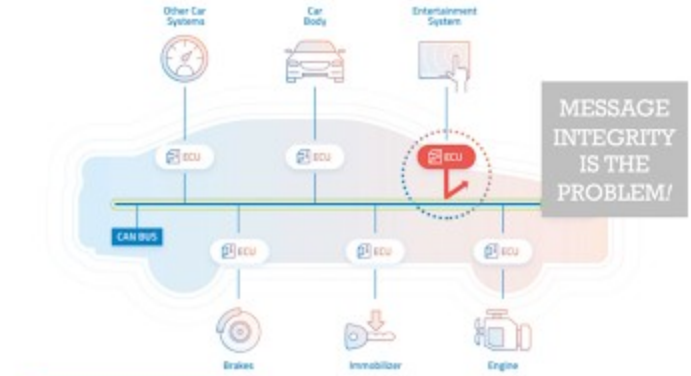
It's very important to denote that nowadays computers are everywhere, and so the impact of attacks can be substantial:

A REMARK ON IMPACT

- Nowadays, computers are everywhere and the impact of attacks can be substantial
- Example
  - Automotive attacks can lead to important consequences both on the vehicles and the passengers
  - For the 2015 attack on a FCA jeep, impact was substantial for all stakeholders
    - Manufacturer obliged to recall 1.4 millions vehicles
    - Drivers and passengers safety put at risk



RISK IN AUTOMOTIVE: AGAIN (2)





Wireless carjacking of a Jeep (2015)

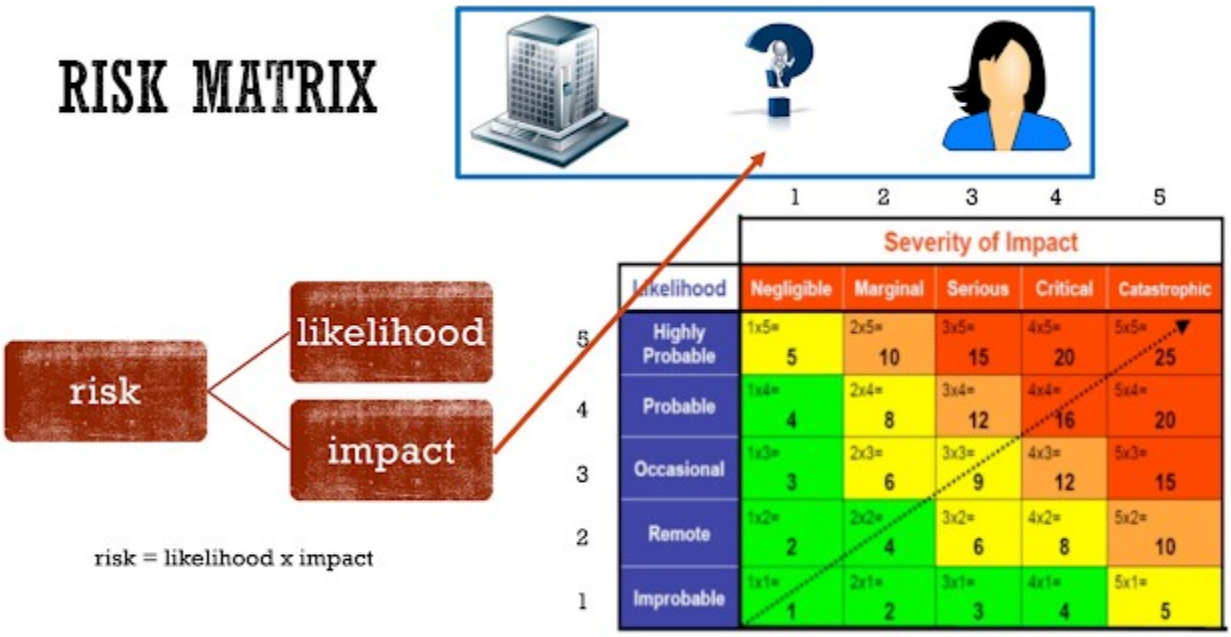
No hope that automotive industry will replace the CAN bus for better security

https://www.wired.com/2015/07/hackers-remotely-kill-jeep-highway/

The threat model allows security experts to identify security mechanisms/services to mitigate the threats for a system/application. In practice, this process is far from being trivial as it requires to consider multiple dimensions including:

- budget constraints
- availability of security skills

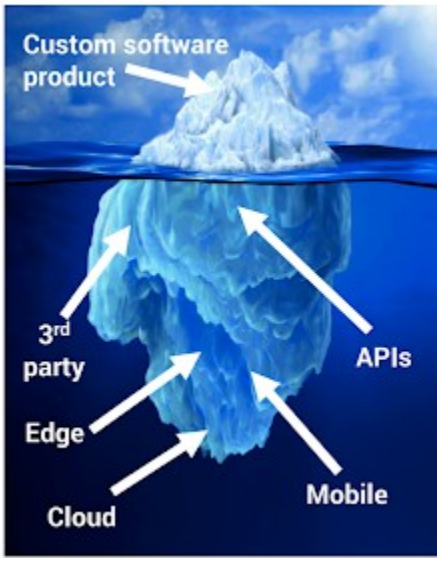
As a consequence, we need approaches to classify threats by understanding their **likelihood** and **impact**. Roughly, the former is how probable it is that an attack will occur whereas the latter is an estimate of the harm that could be caused by an attack. It is possible to combine likelihood and impact into a single indicator, called **risk**. This can be used to guide the adoption of security mechanisms/services in the multidimensional design space discussed above by considering the threats that bring a higher risk first and stopping as soon as other constraints (e.g., that related to the available budget) can no more be satisfied: if the residual risk, i.e. the risk after the addition of the security mechanisms/services to the system/application, is acceptable then we stop; otherwise, we look for the adoption of alternative security mechanisms/services to strike the best balance between security and other constraints.



Talking about security, a very crucial topic is **trust**. Because if you trust the “wrong” software, then you will have a lot of vulnerabilities caused by it.
In general, making the “right” **trust assumptions** plays a crucial role in defining accurate and realistic threat models capable of predicting security violations.



Thompson's Turing Award Lecture in 1984 entitled “Reflections on Trusting Trust”:
To what extent should one trust a statement that a program is free of Trojan horses? Perhaps it is more important to trust the people who wrote the software.



Security and Human Factors

We have to design **usable security**, that is:

- Make it easy to do the right thing
- Make it hard to do the wrong thing
- Make it easy to recover when the wrong thing happens

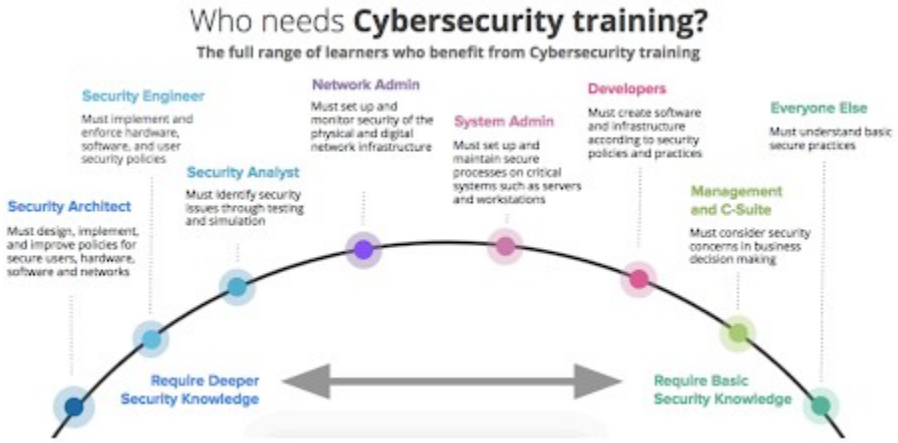
Position	Worst passwords of 2020	Password
1. ↑ (2)		123456
2. ↑ (3)		123456789
3. (new)		picture1
4. ↑ (5)		password
5. ↑ (6)		12345678
6. ↑ (17)		111111
7. ↑ (18)		123123
8. ↓ (1)		12345
9. ↑ (11)		1234567890
10. (new)		senha

https://www.zdnet.com/article/the-worst-passwords-of-2020-show-we-are-as-lazy-about-security-as-ever/



https://readgroup.co.uk/news/data-protection-why-passwords-are-like-pants/

Security training is increasingly needed for more and more IT professionals. That's because IT professionals are increasingly taking decisions with security and privacy consequences, but on average they are not trained to do so and this results in a weak security posture that offers an "easy lunch" to the attacker.



When considering the adoption of mitigation measures (also called security controls), it is crucial that the **human factor** is adequately evaluated. This means that if user intervention is needed (as it is the case for many security mechanisms/services, such as authentication whereby users are required to prove their presence by providing, e.g., passwords), the user experience should be as frictionless as possible; otherwise, users will either avoid the use of the service or find workarounds that will spoil its effectiveness. Consider for instance passwords; they should be long random strings of symbols but, at the same time, they should be easy to remember. Indeed, these are conflicting requirements resolved by users with strategies to make passwords easy to

remember that can be exploited by attackers to guide brute-force cracking. As a result, nowadays passwords alone are considered inadequate to protect accounts.

Authentication I

Authentication is the first security mechanism/service that we're going to learn.

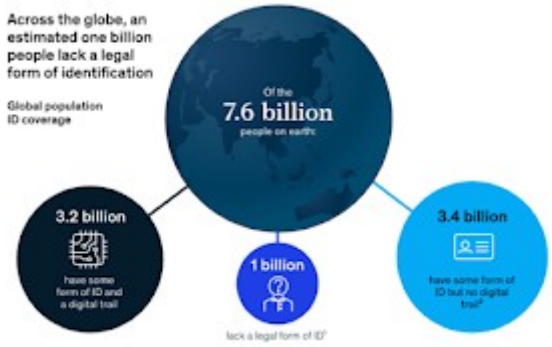
User authentication and digital identity lifecycle

An **identity** is a set of attributes that characterize an entity (a user, a device, an organization, a program, etc.). An **attribute** is a characteristic or property of an entity that can be used to describe its state, appearance or other aspect.
For example the identity for a person may be composed by these attributes:

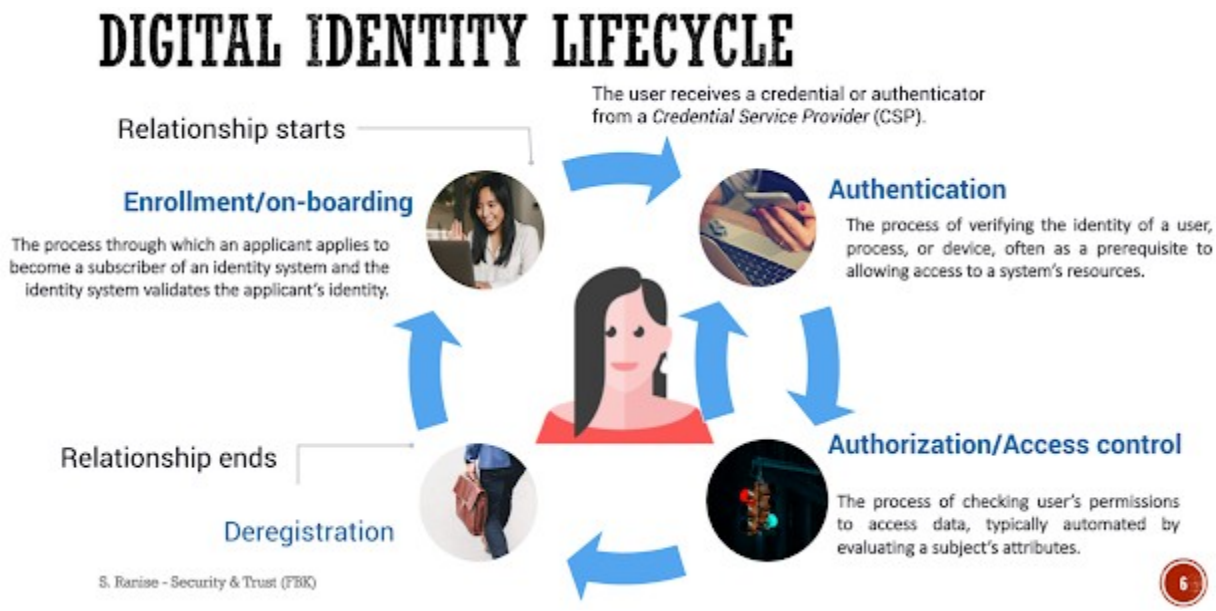


A **digital identity** is an identity whose attributes are stored and transmitted in digital form (very broad definition, independent of a particular technology such as SAML, OpenID Connect, Blockchain, …).

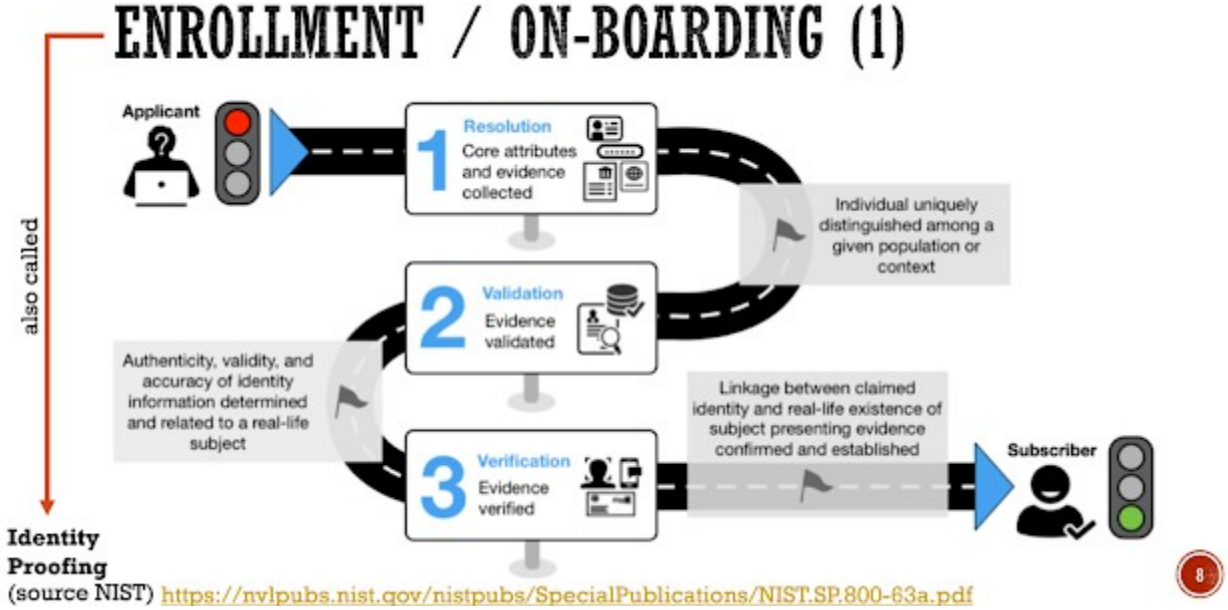
Digital identity is very important nowadays for social reasons (for example in some rural countries which don't even support identity cards). Digital identity can help them to get access to a lot of services (like financial and public administration ones).



So, **besides security**, digital identity brings a lot of advantages. Digital identity is seen as an opportunity to drive **transformational change for citizens, businesses and public administrators**.
The theme of a unique digital marketplace is particularly relevant in Europe. One of the basic enablers is to make the national digital identity system interoperable (this is one of the goals of the eIDAS framework that is currently undergoing an update).



In the **enrollment** the user attributes are associated with its digital identity (and there the credentials or authenticator are created/given). So this phase is based on collecting information about the user.
This phase is crucial because you have to find a method to verify that the entity that is enrolling is really who he says he is.



So the enrollment is composed by:

1. **Resolution**
   Which comprises the collection of **Personal Identifiable Information** (PII) from the applicant (the minimum set of PII are full name, date of birth and home address) and the collection of two forms of identity evidence (for example driver's license and passport using laptop camera).

2. **Validation**
   Validation of the information supplied above by checking an authoritative source
   Check of the images of the license and the passport for alterations, coherence of the information on the documents and compliance against standard
   Check with the issuing sources for the license and passport and validates the information matches
3. **Verification**
   Ask the applicant for a photo of themselves to match to the license and passport
   Matching the pictures on the license and the passport to the applicant picture
   Sending an enrollment code to the validated phone number of the applicant, the user provides the enrollment code, and check if they match, verifying the user is in possession and control of the validated phone number
   The applicant has been successfully identity proofed, becoming a subscriber.

At the end (if all goes right) the applicant is going to have a unique identifier in the system.
The enrollment has an **Identity Assurance Level** (IALs), that is the degree of confidence that the applicant's claimed identity is the real identity of that entity:

- IAL 1 (for example social media)
  Attributes, if any, are self-asserted or should be treated as self-asserted
- IAL 2
  Either remote or in-person identity proofing is required
- IAL 3
  In-person identity proofing is required. Identifying attributes must be verified by an authorized representative through examination of physical documentation.

After the enrollment comes **authentication**, where an entity that already has a digital identity is required to authenticate (by the credential or the authenticator). So this is the process of verifying the identity of a user, process or device.
In this process we need a system in which the claimant can demonstrate to the verifier that is indeed the one that claims to be.

After authenticating, the system has to check the permission of that entity, with the **authorization/access control**. In some circumstances there is another authentication inside the authorization (for example in a payment you are required to insert another OTP).

At the end we have the **deregistration** phase, that is very important, because in some cases if you miss it, you are in trouble (for example when you fire an employee).

The most frequently used part of the lifecycle of a digital identity is obviously the authentication and the authorization, because the other two phases are done only once.

Authentication with passwords

- When logging on, you enter
  - *username* to announce who you are
  - *password* to prove that you are who you claim to be



This type of authentication is called **user authentication**: the process of verifying a claimed user identity.
The **password-based authentication** is widely accepted (dominated human computer authentication for half a century) and not too difficult to implement, but it is tricky to implement it in a secure way. In fact passwords are well-known to **provide an insufficient level of protection**.
The ideal model of password-based authentication is that each password is randomly generated and long, without being written down. Passwords should be **secrets shared** between the user and the system. So with passwords we run into a lot of problems.
Passwords dominated human computer authentication for half a century but, despite this, rising consensus that a more secure and user friendly alternative should be found.

The set of rules defined by a system for the password (i.e. the minimum number of characters, or the presence of a special character) can be used by the attacker to improve the brute force attack.
The password in the database must be encrypted in order to avoid the stealing in an attack, and in order for the user to “trust” the company. Passwords should also be secrets shared between the user and the system, in order to avoid the sniffing.



In the sixties, passwords were originally added to time sharing operating systems to protect against jokes or abuse of resources. But some security issues were immediately reported:

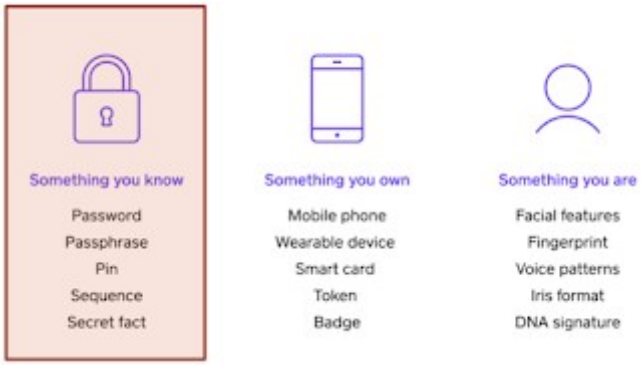
- cases of guessing passwords
- leak of the master password file stored in clear

In the seventies, passwords were used to protect both resources and sensitive data:

- passwords were protected by using **hash functions**
- afterwards, not only hashing but also **salting** was used

With hashing only the user needs (and must) to know the real password, given that the system and the verification module doesn't need to know it.

Passwords are only one of the authentication methods that can be used, some alternative were founded in order to have a more secure and user friendly authentication methods:



In practice, several different authentication processes use passwords in combination with other techniques including additional authentication factors (e.g., biometrics or One Time passwords) and Machine Learning for risk based authentication.
The biometric authentication is good, but once it is “stolen”, you can’t change it like a password.



After the Morris worm, in the first half of the nineties, passwords were stored in **shadow files**. This means that instead of storing the (hashed and salted) passwords, they are moved to another file, called shadow, and making this file readable only by those users who have access to the system root directory (so we’re adding access control). This is an example of using a combination of security mechanisms to protect a sensitive resource (the file containing the passwords) that provides for some redundancy in case of failure of one mechanism.

In the second half of the nineties (at the advent of the World Wide Web and eCommerce) there was the first attempt to replace passwords with **certificates based on public key cryptography**, but managing both client and server side certificates proved to be too cumbersome, so this idea failed. Thus certificates are used only by servers while users at the

client side are typically asked to enter passwords or other credentials to authenticate at servers (here we are referring to the Transport Layer Security TLS).

Since the second half of the nineties, web services proliferated and usability problems for passwords arose that did not exist before, for example:

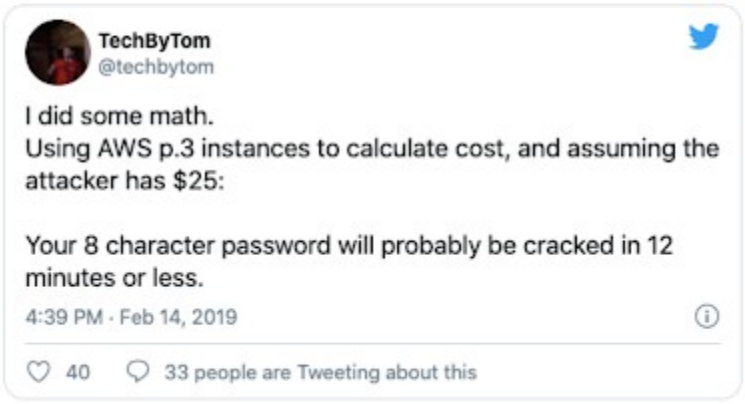
- **Resetting forgotten passwords**, previously done manually by system administrators of organizations, was automated by email... this created a security loophole for several reasons including
    - In many cases, users can click on a password reset link and get their old password delivered to their inbox. For this to work the passwords are retrievable which means that they are either being stored in the site's database as plaintext or encrypted with a reversible algorithm
    - Additionally, passwords are sent without encryption so hackers sniffing network traffic could steal them
- The increased number of accounts per user stimulates **the reuse of the same password for multiple services** to help them memorize a reasonable number of passwords
- **Phishing** has emerged as a major concern with statistical approaches to mitigate it including blacklisting of known phishing site or machine learning classifiers to identify phishing messages such as those used by Google Mail

Attacks and mitigation

Let's now see the threat model for cracking passwords.
The attacks of cracking passwords can be done **online** or **offline**.
The online attacks can be easily mitigated, because it is sufficient to set a maximum number of attempts and then to block the authentication procedure for a certain interval of time to make brute-forcing practically impossible.
The offline attacks instead are more interesting and difficult to prevent, because if an attacker is able to steal the file/database where passwords are stored, he can then be able to carry out a brute force attack. So it becomes crucial to adequately protect the passwords in storage (unfortunately, still today, there are several cases in which the protection is not implemented or implemented in the wrong way). So we will focus only on offline attacks (since the online attacks are easy to mitigate).

The first (and the easiest) attack that can be done is a **brute force**, i.e. try all possible combinations of valid symbols up to a certain length.
This technique requires a lot of computational power, but nowadays a lot of computational power is available at a reasonable cost. This is for several different reasons including Moore's law (the number of transistors that could be housed in a dense integrated circuit doubles about every two years, that is that computational progress will become significantly faster, smaller, and more efficient over time) and the possibility to rent hw from cloud service providers.

Combination	Combinations per character	Example	Password Space
Lower case only	26	pass	456,976
Lower case only	26	password	208,827,064,576
Lower+upper case	52	Password	53,459,728,531,456
Lower+upper case + digits	62	Pa55w0rd	218,340,105,584,896
Lower+upper case + digits + symbols	92 (approx.)	Pa$$w0rd	5,132,188,731,375,616



#	Numbers	Lowercase	Mixed Case	Mixed Case + Numbers	Mixed Case + Numbers + Special
1	0 $	0 $	0 $	0 $	0 $
2	0 $	0 $	0 $	0 $	0 $
3	0 $	0 $	0 $	0 $	0 $
4	0 $	0 $	0 $	0 $	0 $
5	0 $	0 $	0 $	0 $	0 $
6	0 $	0 $	0 $	0 $	0 $
7	0 $	0 $	0 $	2 $	10 $
8	0 $	0 $	6 $	155 $	965 $
9	0 $	1 $	315 $	12,118 $	94,536 $
10	0 $	16 $	16,391 $	945,165 $	9.3 M$
11	0 $	416 $	852,312 $	73.7 M$	907.9 M$
12	0 $	10,820 $	44.3 M$	6 B$	89 B$
13	1 $	281,330 $	2 B$	449 B$	8.7 T$
14	11 $	7.3 M$	120 B$	34 T$	854 T$
15	113 $	190.2 M$	6.2 T$	-	-
16	1,134 $	5 B$	324 T$	-	-
17	11,339 $	129 B$	-	-	-
18	113,387 $	3.3 T$	-	-	-
19	1,1 M$	86 T$	-	-	-
20	11.3 M$	-	-	-	-

- monetary effort required to break the given password complexity with an **offline brute force attack by simple trial and error**
- expected costs an Amazon EC2 instance (p3.16xlarge - 8xNVIDIA Tesla V100 GPUs) and SHA256 hash algorithm

The attacker can be smarter and instead of doing a general brute force, he searches through a restricted name space (for example popular passwords, user like name, names of friends etc.). A typical example of this attack is a **dictionary attack** (i.e. trying all passwords from an on-line dictionary).
You cannot prevent an attacker from accidentally guessing a valid password, but you can try to reduce the probability of a password compromise.



Prior 2010, guessing passwords was not very relevant as a threat, but in 2009 a large site (rockyou.com) with more than 32 million users got hacked. This website stored all its users' passwords in plain text and all the passwords were leaked. This gave hackers the single biggest insight into how humans construct passwords globally.
Every password leak that has happened since just widens hackers knowledge into how people create passwords.
It created a **feedback loop**; passwords are cracked, added to word lists, and used to crack new passwords.

For example if you have a file with hashed passwords and you know the type of hash that you are cracking, you can do a dictionary attack by giving in input the file of hashes, the type of hash and the wordlist of the dictionary.

Results

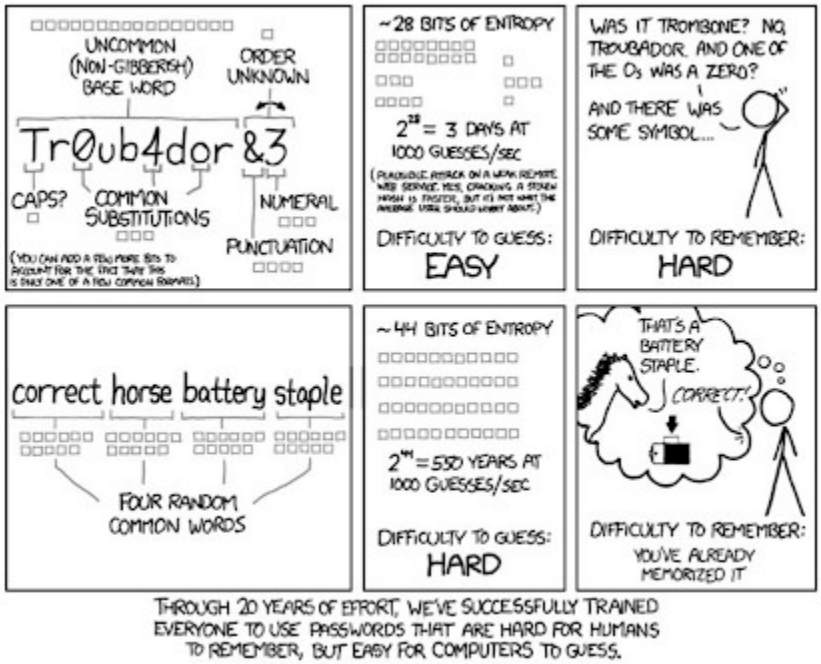
dc647eb65e6711e155375218212b3964:**Password**
eb61eead90e3b899c6bcbe27ac581660:**HELLO**
75b71aa6842e450f12aca00fdf54c51d:**P455w0rd**
2c9341ca4cf3d87b9e4eb905d6a3ec45:**Test1234**
958152288f2d2303ae045cffc43a02cd:**MYSECRET**

5 out of 7 passwords are quickly cracked (few seconds on standard machine)

Some **mitigation techniques** for this type of attack which can be **done by the user** are:

- Change default passwords (often passwords for system accounts have a default value, for example admin)
- Avoid guessable passwords (enforce a minimal password length and a password format)
- Change password every given period of time

The problem is that nobody follows these tips, and nowadays this mitigation is outdated due to the cost reduction of computational power growing, and because attackers improve the brute force attack with these rules.
And it was discovered that the better solution both for security and for ease of remembering is to use **passphrases**.



Old days

Future...

So the new National Institute of Standards and Technology (NIST) password limitations are:

- Forbid commonly user passwords
- Don't use password hints or knowledge-based authentication
- Limit the number of password attempts

A useful tool for the generation and management of the password is a **password generator & manager**.
The **generator** offers to create randomly generated passwords, based on pseudo random number generators to create a random string of symbols.
If only generators are used, then the burden of remembering passwords is on the user, and remembering a random sequence of characters is very difficult, especially if the user has a lot of passwords.
So the user can use a **manager** which stores the passwords. But it may be vulnerable to attacks (especially if it is an on-line manager).

For example, the operating system maintains a file with username and passwords, and attackers could try to compromise the confidentiality and the integrity of that file.
So the password file should be protected by:

- Cryptographic protection
- Access control enforced by operating system

Combination of cryptographic protection and access control, possibly with further measures to slow down dictionary attacks.

Protecting the password file: hashing and salting

Let's talk about **cryptographic hash functions**.
A 1-way function f is a function that is **relatively easy to compute but hard to reverse**. That is, given an input x it is easy to compute $f(x)$, but given an output y it is hard to find x so that $y = f(x)$.

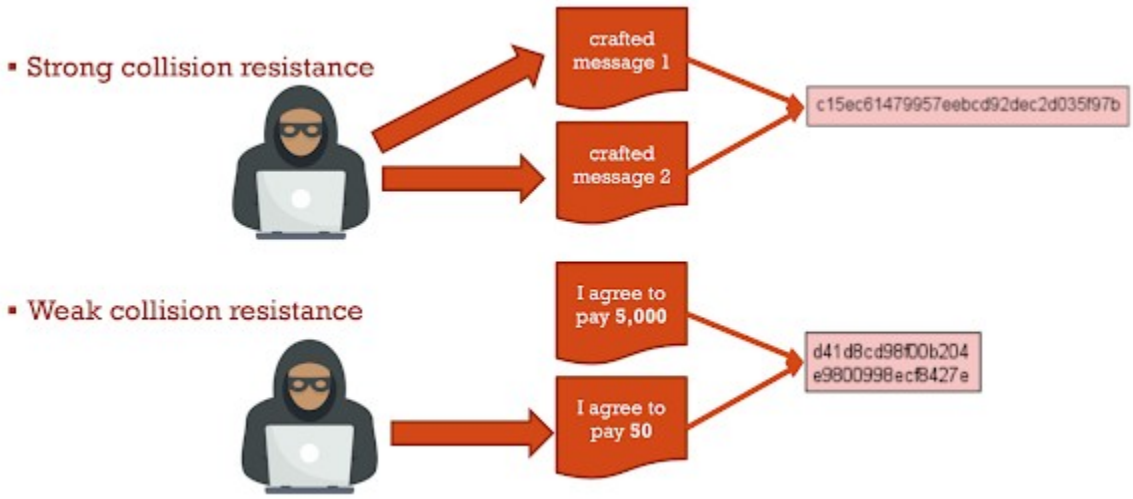
Instead of the password x, the value $f(x)$ is stored in the password file.
When a user logs in entering a password x', the system applies the one-way function f and compares $f(x')$ with the expected value $f(x)$.

Why use hash instead of encryption?
Hashing is a one-way function, i.e. it is impossible to "decrypt" a hash and obtain the original plaintext value.
Encryption is a two-way function, meaning that the original plain text can be retrieved with some additional information.
So hashing is appropriate for password validation, because even if attackers obtain the hashed password, they cannot enter it into an application's password field and log in as the victim.
Encryption is appropriate for storing data such as a user's address since this data is displayed in plaintext on the user's profile. Hashing their address would result in a garbled mess.

The requirements on a hash function h are:

- **Ease of computation**
  Given x, it is easy to compute $h(x)$
- **Compression**
  h maps inputs x of arbitrary bit-length to outputs $h(x)$ of a **fixed bit-length** n
- **One-way**
  Given a value y, it is computationally infeasible to find an input x so that $h(x) = y$
- **Weak collision resistance**
  Given an input x and $h(x)$, it is computationally infeasible to find another input x', with $x \neq x'$ that $h(x) = h(x')$
- **Strong collision resistance**
  It is computationally infeasible to find any two inputs x and x', $x \neq x'$, with $h(x) = h(x')$

A **collision** is the situation in which two inputs x and x' map to the same hash, that is $h(x) = h(x')$.



It is very important the **avalanche effect**, which causes that small changes result in significantly different output. For example just changing one single letter in the input string makes the resulting hashes look very different.
This feature makes finding the input given the output much harder as the output looks “random”.

Here I use *python3*

- This feature makes finding the input given the output much harder as the output looks “random”

```
>>> import hashlib
>>> str1 = "homely"
>>> hash1 = hashlib.sha256(str1.encode('utf-8'))
>>> hash1.hexdigest()
'423cb2dd7078b1dac9d67c89b5bb7292f263041abf60455a443664095b47f7b4'
>>> str2 = "comely"
>>> hash2 = hashlib.sha256(str2.encode('utf-8'))
>>> hash2.hexdigest()
'6f804dfb2cd3a13c1cc93f6dfea34ea436dc6da85b164697a3ec1e4f135c4285'
```

Theoretically, it is indeed possible to find collisions for any given hash function (this is obvious as soon as we consider the cardinality of the input set and the output set of a hash function). The point that makes hash function useful in practice is that the **likelihood that such a collision can happen should be negligible**.

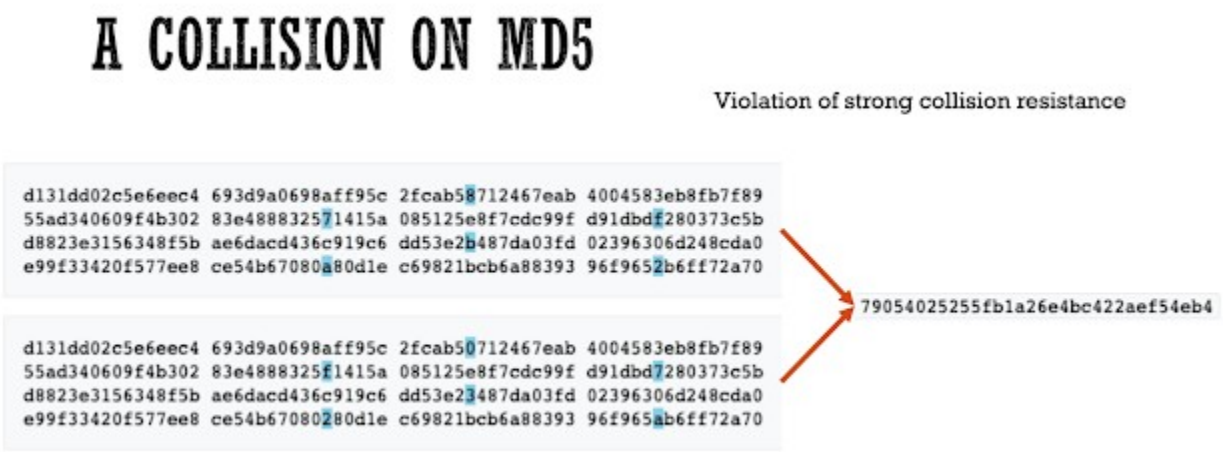
- Example
  - when n=128, we have 2^128 = 3.4 * 10^38 possible distinct hashes
  - **under the assumption that the hash function distributes uniformly the outputs among all possible distinct hashes**, it is more likely that the same person repeatedly wins the lottery…
  - For the assumption above to be satisfied, the design of the hash function should be performed in the appropriate way so that its output looks random…
  - … this is not always trivial and it was the source of attacks to widely adopted hash functions in the past…

The most used hash function implementations are:

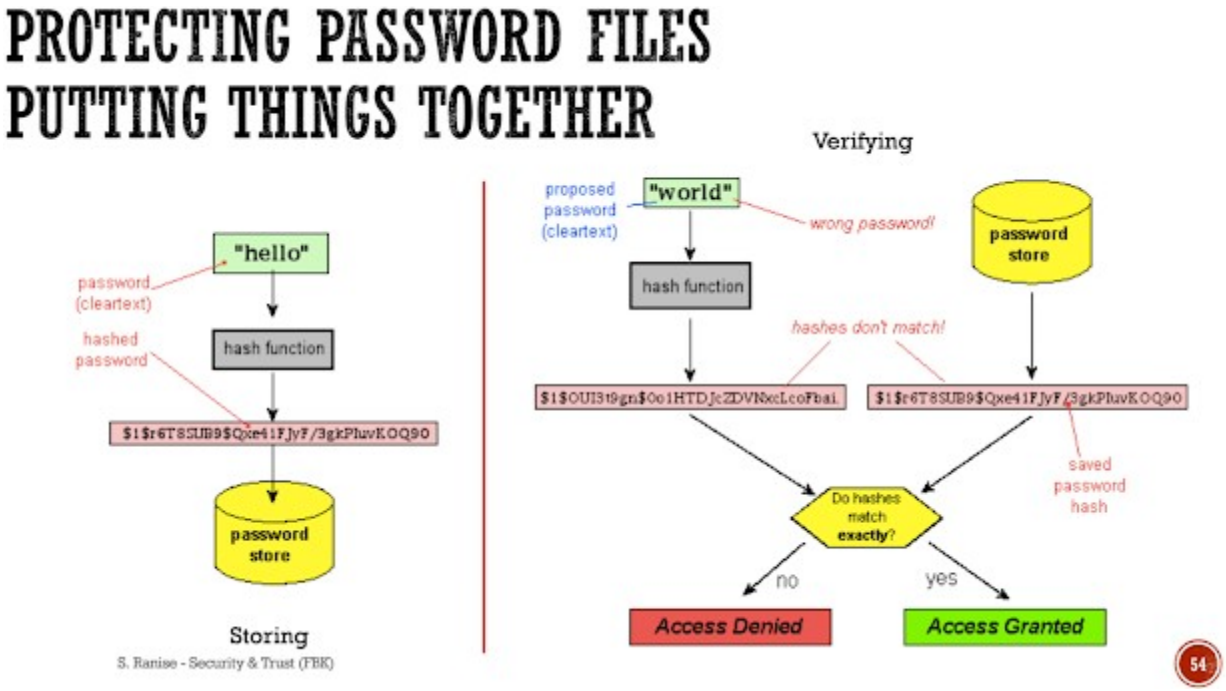
- **MD4** weak, it is computationally feasible to find meaningful collisions
- **MD5** standard choice in Internet protocols, now broken and no longer recommended

- **SHA-1** (Secure Hash Algorithm) designed to operate with the US Digital Signature Standard (DSA); 160-bit hash value; collision attacks reported
- **RIPEMD-160** hash function frequently used by European cryptographic service providers
- **SHA-256** when longer hash values are advisable

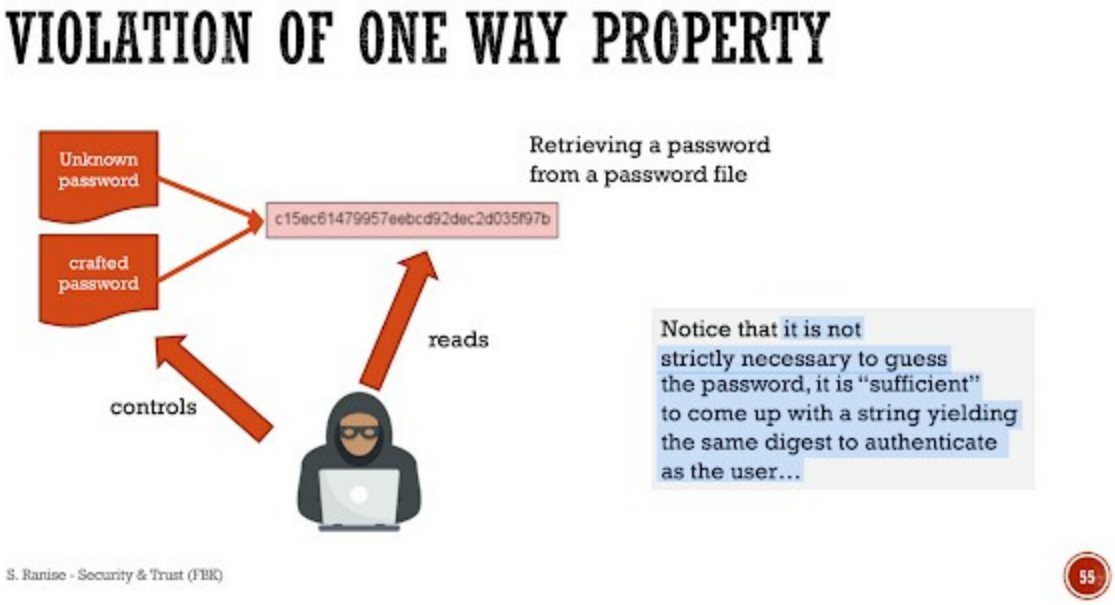
In this image we can see that changing only few bits, the digest are the same for two different blocks, so this hash function violates the strong collision resistance (this is why MD5 is no longer used)



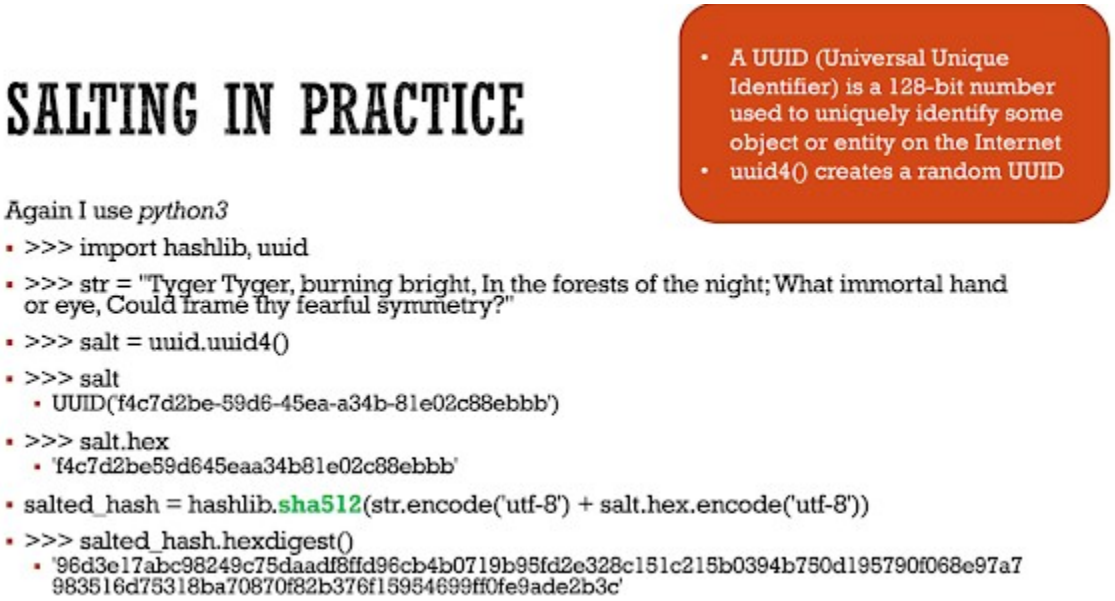
Let's see the storing and the verifying of the password using the techniques that we've seen until now:

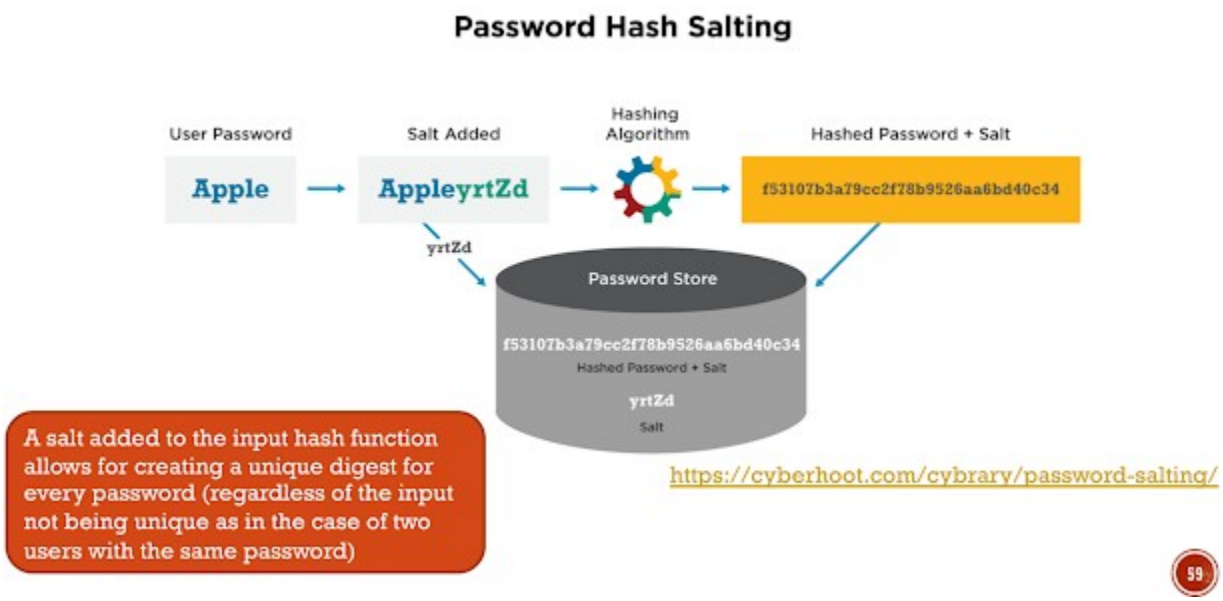


Notice that with this technique it is not strictly necessary to guess the password, it is “sufficient” to come up with a string yielding the same digest to authenticate as the user…
Another advantage of use hashing is that the comparison is more efficient (because if you have a long sequence, you can obtain the digest, which is shorter, and compare it faster)



Another mitigation is the **salting**, which consists in appending to the password a **salt** before hashing and it is stored with the hashed password. This makes that two equal passwords of two different users results totally different when hashed (this is useful to slow down dictionary attacks). In this way if two users have the same password, they will now have different entries in the file of hashed passwords

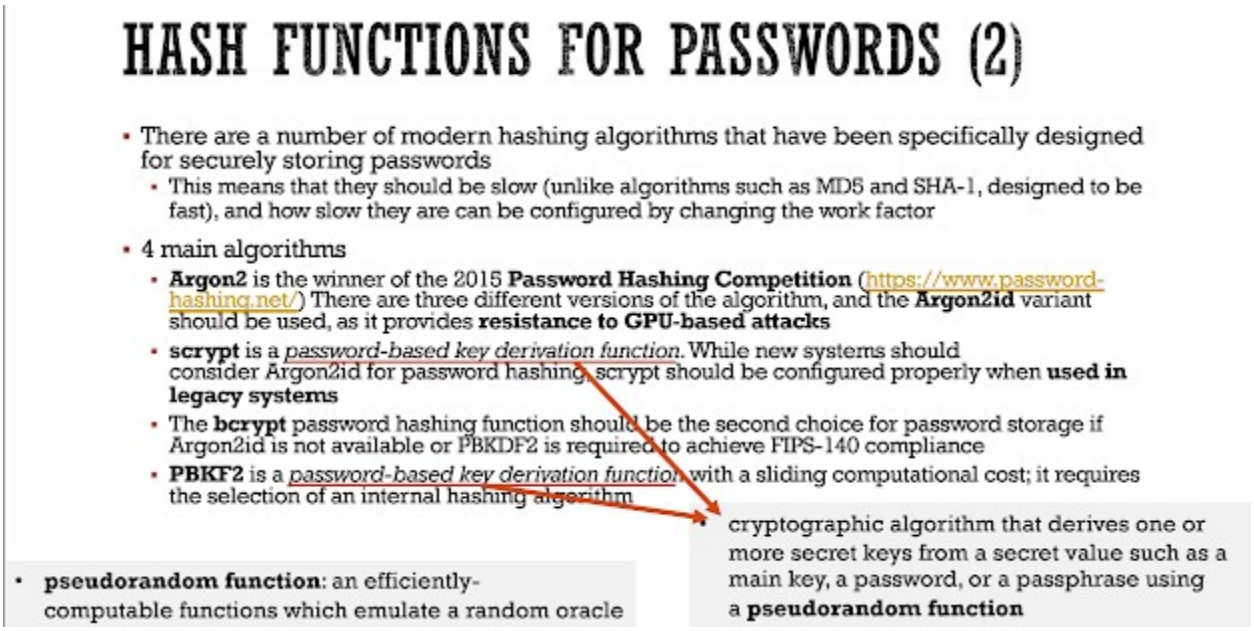




Since the salt is unique for each hash, an attacker must re-compute a rainbow table for each user hash. This greatly reduces the capability of an attacker to crack a lot of passwords as it was the case without salting.
Notice that an attacker may still be able to crack a single password once he or she knows the salt that is not necessary to be kept secret (for simplifying the verification of hashes, salts are usually stored in clear with the user name and hash of the password).

In practice you have to avoid implementing your own version of hashing and salting but use well-engineered available functions such as bcrypt, scrypt or even better argonid.

But the fact that the computation of the hash function is easy to compute, makes that also the attacker will easily compute a brute force attack (or similar attacks like dictionary attacks).
A mitigation to this could be increasing the computational time that is required to compute the hash function. In this way you also increase the time for the verification and storage, but given that the “real” user has to do it only once, it is acceptable.
The **work factor** is the number of iterations of the hashing algorithm that are performed for each password (usually 2^{work} iterations). The purpose of the work factor is precisely to make calculating the hash more computationally expensive (increasing the cost for which an attacker can attempt to crack the password hash).
When choosing the work factor, a **balance between security and performance** needs to be struck (a higher work factor is better for security but will slow down the process of verifying a login).
There is no golden rule for the ideal work factor; determining the optimal work factor will require experimentation on the specific server(s) used by the application (in general a little less than one second).
Let’s observe we have done all this observation considering a threat model (the brute force attack for discover passwords)



Resuming, the three basic concepts that permits the creation of a secure password-based authentication mechanism are:

- Using the hash
- Using a valid hash function
- Using salt

Another problem is when the users reuse the same password for different services, because when an attacker gets access to a compromised database, it will try how many of these credentials work on other services.
A mitigation strategy that could be done by a company that knows that another company database has been compromised, is to extract from leaked credential lists any email addresses that correspond to their current user base and check if there are user with the same password of the services leaked (if they are equal, then force password reset procedure for the user).

We finally observe that identification and authentication through username and password provide **unilateral authentication**. Means that the user identity is verified by the third party, but the user has no guarantees about the identity of the party that is going to receive the password. The attack which leads the user to enter credentials in the “wrong” place is called **phishing**.
So we’ve seen that there are many types of attacks which could be done in order to steal passwords.

Multi-factor authentication

We’ve seen that all these techniques can slow down the attacker but in some cases he can obtain the credentials anyway (especially with the phishing).
So, in order to mitigate the weaknesses of password-based authentication, let’s introduce **multifactor authentication**.

MFA (MultiFactor Authentication) is an authentication method in which a computer user is granted access only after successfully presenting two or more factors to an authentication mechanism.

Authentication
Multi-factor authentication

A procedure based on the use of two or more of the following factors:



knowledge, something only the user knows, e.g., static password, personal identification number;



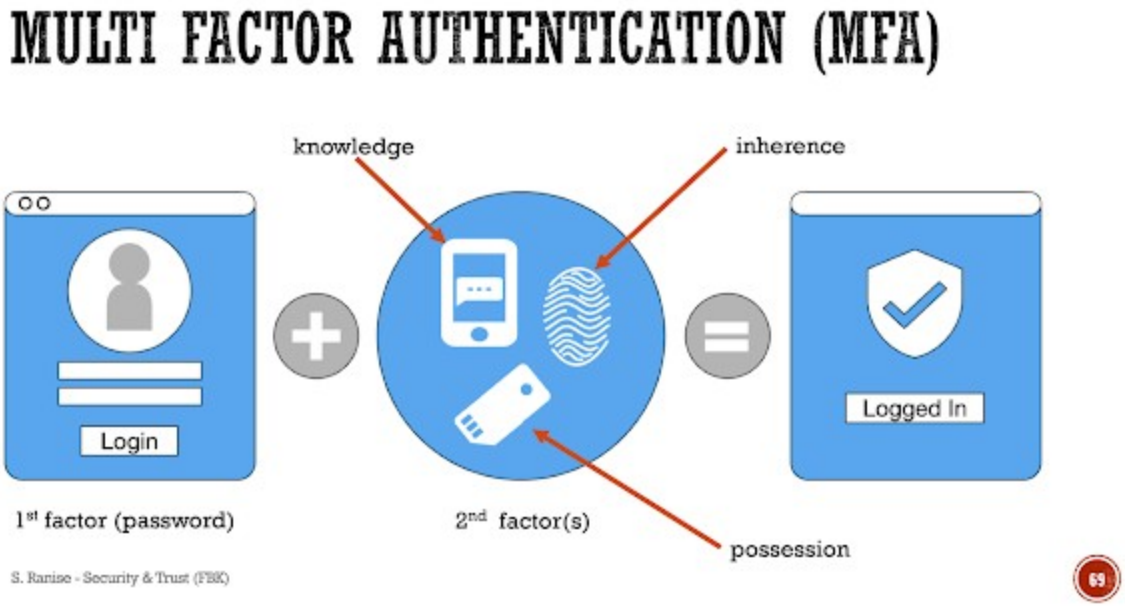
ownership, something only the user possesses, e.g., token, smart card, mobile phone; and



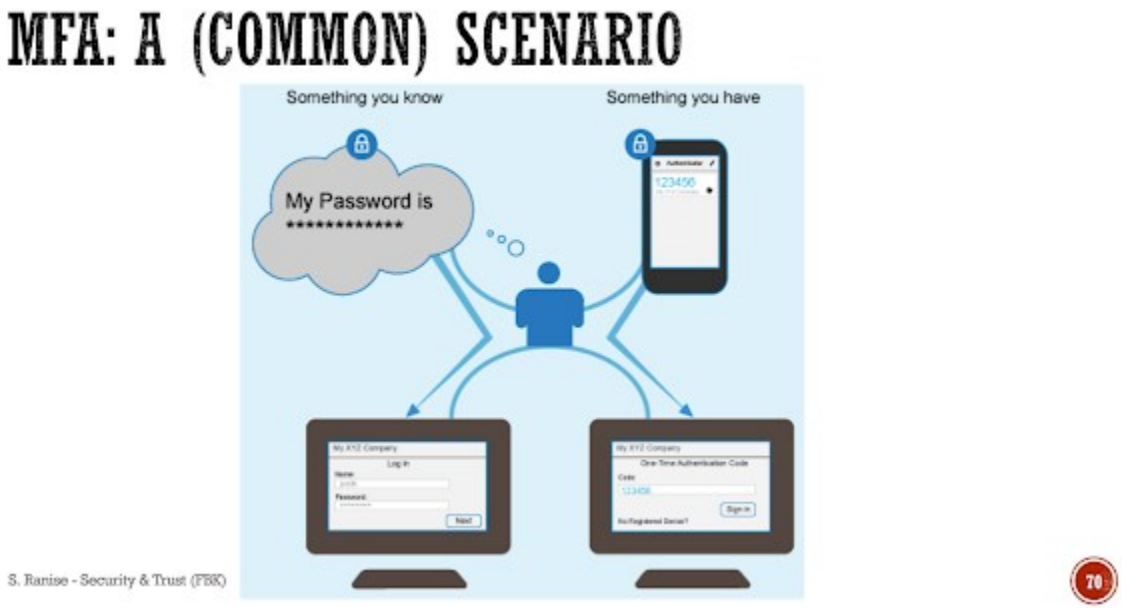
inherence, something the user is, e.g., biometric characteristic, such as a fingerprint.

page 068

In general the “first layer” is password-based authentication and then there is another type of authentication.



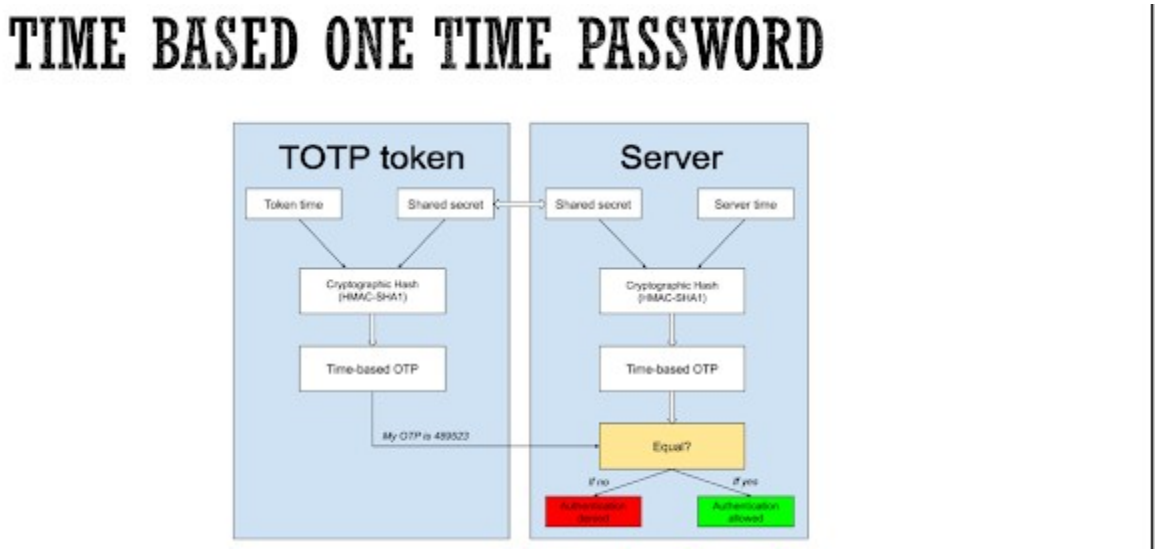
The basic idea is that the attacker must steal two things instead of only one.



Two-factor authentication (2FA) is a type of MFA confirming users' claimed identities by using a combination of two different factors.

- Example 1: withdrawing of money from an ATM; only the correct **combination of a bank card** (something the **user possesses**) and a **PIN** (something the **user knows**) allows the transaction to be carried out
- Example 2: **supplement** a user-controlled **password with** a **One-Time Password** (**OTP**) or code generated or received by an authenticator (e.g., a security token or **smartphone**) that only the **user possesses**

The most popular second factor authentication is **Time based One Time Password** (TOTP).



So the authenticator and the authenticated token time have two parameters:

- T0: Unix time from which to start counting time steps (default is 0)
- Tx: Interval which will be used to calculate the value of the counter (default is 30 seconds)

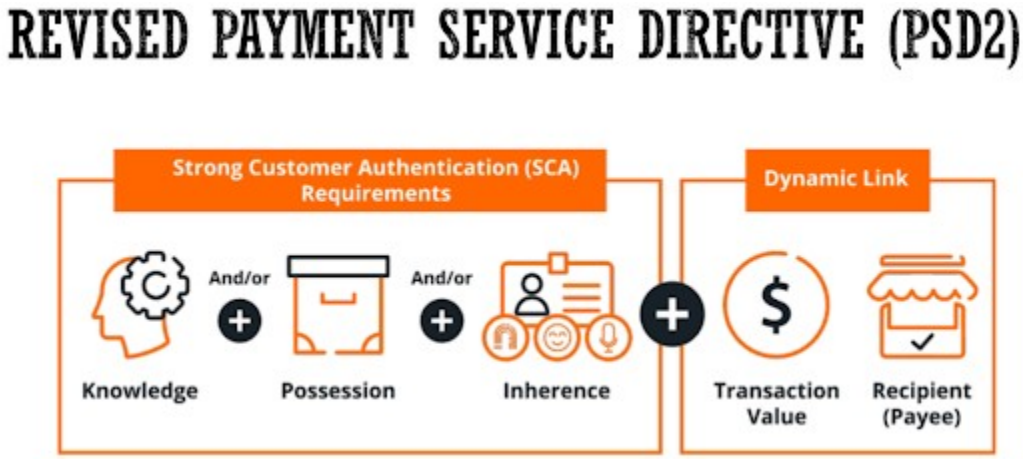
The other parameter used to compute the TOTP is a **shared secret key** (the **seed**).
Authenticator and authenticated compute the TOTP value then the authenticator checks if

the TOTP value supplied by the authenticated matches the locally-generated TOTP value (Some authenticators allow values that should have been generated before or after the current time in order to account for slight clock skews, network latency and user delays).
This mechanism in the past was implemented in hardware with special devices (especially in online banking)
The weaknesses are:

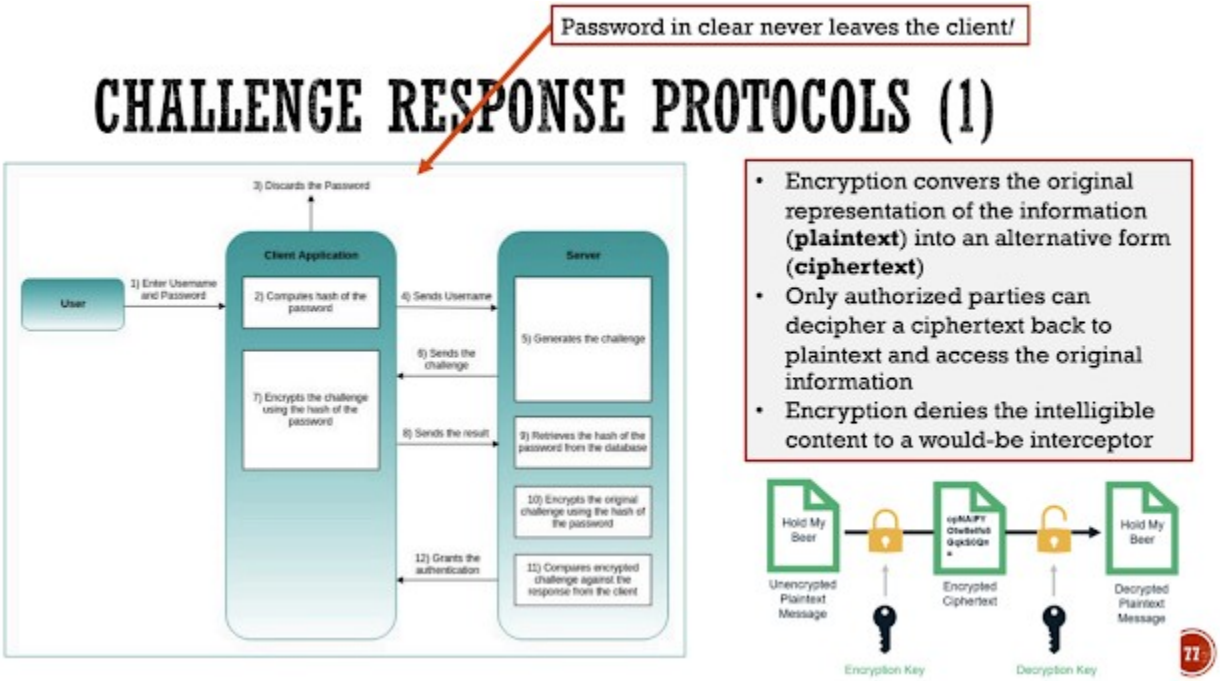
- OTP values can be **phished** just as passwords can and replayed, though they require attackers to proxy the credentials in real time rather than collect them later
- An attacker who **steals the shared secret key** can generate new, valid TOTP values at will. This can be a particular problem if the attacker breaches a large authentication database

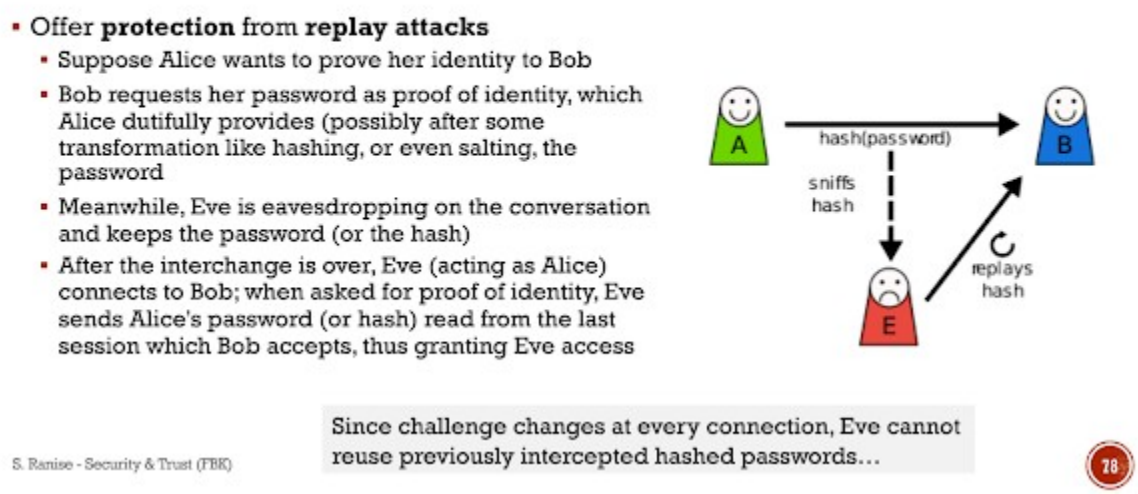


The problem with the phishing of TOTP is that the token isn't associated with a particular transaction, but it can be used for many (in the time frame). So, in order to mitigate this problem, the **revised payment service directive** (PSD2) was introduced, where the token is uniquely associated with that particular operation, and though the attacker may steal it, he can't do anything else.



So the value of the OTP becomes **dependent on the transaction**, because it is uniquely associated with it.
To implement it, a description of the transaction is generated and used in order to create the OTP.
Let's see the step done by an authentication procedure that use PSD2:





In this way the server can check if the transaction is made by the client (that is willing to perform that transaction) that started the procedures or not.

EFFECT OF PSD2: NO MORE TOTP HW TOKEN!



Can you explain why?
Do you see a way to overcome the problem?

The idea is to include in the challenge the data that are unique to the particular transaction (such as the destination of the transaction, the amount of the transaction, the instant in which the transaction has been initiated) but this is not enough because the data above can be inferred from contextual information and makes the challenge too predictable. It is necessary to add some random data, i.e. some unpredictable string to the challenge.

Since hardware devices can't be used, to provide MFA we started to use **smartphones**.
The advantages are:

- No additional hardware tokens are necessary (you can do all with your smartphone)
- Dynamically generated passcodes are safer to use than fixed (static) log-in information
- Protection to replay attacks as OTP values are uniquely associated to given transaction

The disadvantages are:

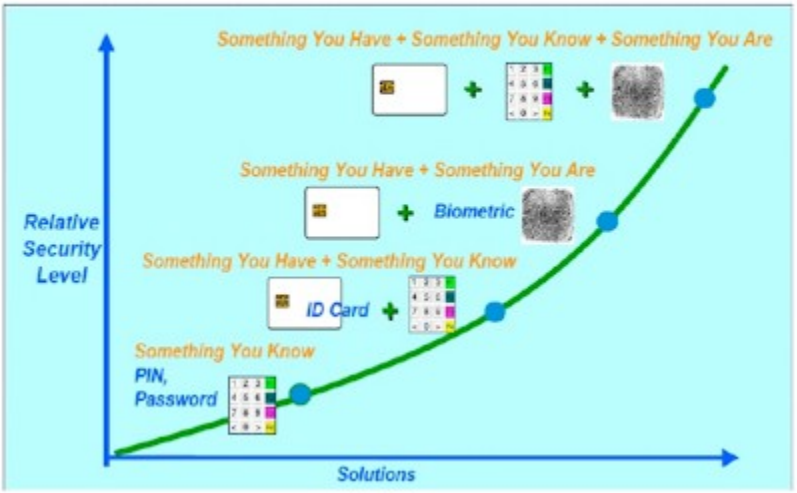
- A mobile phone is not always available (they can be lost, stolen have a dead battery or otherwise not working)
- Mobile phone reception is not always available (large areas, particularly outside of towns, lack coverage)
- SIM cloning gives hackers access to mobile phone connections
- Text messages to mobile phones using SMS are insecure and can be intercepted by third parties that can steal and use the token
- Modern smartphones are used both for receiving email and SMS. So if the phone is lost or stolen and is not protected by a password or biometric, all accounts for which the email is the key can be hacked as the phone can receive the second factor

The authenticator is a piece of software/hardware that confirm the identity of a user, process, or device
There exists 3 **level of assurance** in security (dictated by NIST):

1. Provides some assurance that the claimant controls the authenticator; requires at least single-factor authentication (Memorized secrets such as passwords)

2. Provides high confidence that the claimant controls authenticators; two different authentication factors are required; approved cryptographic techniques are required (Cryptographic software and tokens such as apps or tokens)
3. Provides very high confidence that the claimant controls the authenticator; authentication based on proof of possession of a key through a cryptographic protocol; requires a “hard” cryptographic authenticator (Cryptographic hardware such as smartcards)

MFA & ASSURANCE LEVELS



Unfortunately even MFA procedures can be vulnerable to phishing attacks.
For example the **MFA fatigue attack**, that consists in flooding a user’s authentication app with push notifications (or SMS) in the hope they will accept one and therefore enable an attacker to gain entry to an account.
The technical difficulty of this type of attack is very simple, because it only requires the attacker to manually, or even automatically, send repeated push notifications while trying to log into the victim’s account (it does require the attacker to have the victim’s credentials, which “could be obtained via, for instance, brute forcing)
The attack is particularly effective... not because of the technology involved, but because it targets the human factor via social engineering

- Many MFA users are not familiar with this type of attack and would not understand they are approving a fraudulent notification
- Others just want to make it disappear and are simply not aware of what they are doing since they approve similar notifications all the time

This usually happens because the user is distracted or overwhelmed by the notifications and, in some cases, it can be misinterpreted as a bug or confused with other legitimate authentication requests.

We’ve seen that all the problems that we encounter with password-based authentication are passwords (the transmission, the storing, the remembering ecc.) and also the phishing of OTP. So, the solution to all these problems is the elimination of user interaction in the authentication, and **FIDO** (Fast IDentity Online) does exactly this.

The Web Authentication API (WebAuthn) is a standard put forth by the World Wide Web Consortium in collaboration with the FIDO Alliance.
WebAuthn eliminates the need for users to constantly type in their passwords and instead uses **Public Key Cryptography in combination with a challenge response protocol**.
This avoids the threat from common password-stealing methods like phishing and man-in-the-middle attacks.



The idea is that the user has only to insert the key (for example in the case of hardware toggle, insert it in a usb port of the pc), and then the authenticator will do everything on its own.
The authenticator has some cryptographic material and can encrypt and decrypt, and it uses asymmetric cryptography.
The operation of these system are:

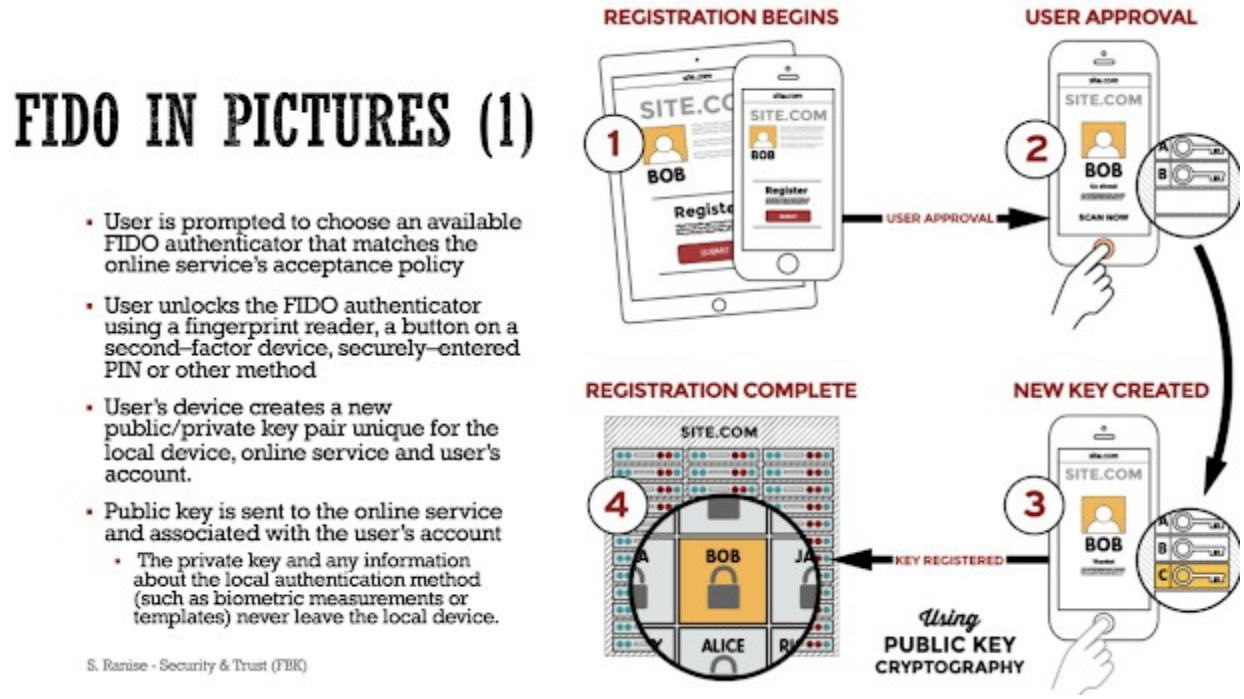
- The relying party send the challenge to the authenticator
- The authenticator encrypts it with the private key and send back the challenge encrypted
- The relying party check if the authenticator is reliable decrypting with the public key

So instead of having the same secret key (as we've seen before) for encryption and decryption, there are two different keys.

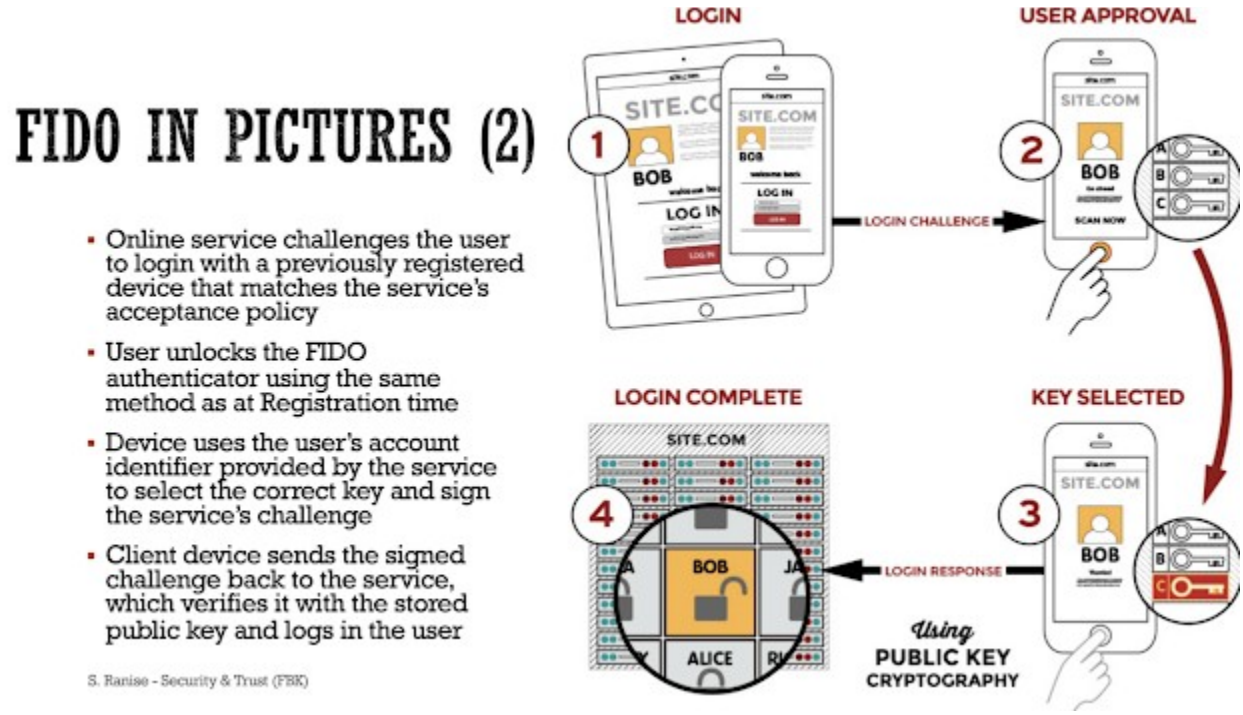
The FIDO authentication requires an initial registration step.
In cases where the user device supports multiple forms of authentication (i.e. fingerprint scanner, voiceprint recorder, face ID, etc.), the user is asked to choose a FIDO compliant authenticator from the options available on the device that matches the authenticating app's acceptance policy.
The user then unlocks the FIDO authenticator using whatever mechanism is built into the authenticator (e.g. by providing a fingerprint, pressing a button on a second–factor device, or entering PIN)
Once the authenticator is unlocked, the user's device creates a new and unique public/private cryptographic key pair that will be used for authenticating access:

- The public key is then sent to the online service and associated with the user's account

- The private key and all other sensitive data related to the chosen authentication method (for example biometric prints) remain on the local device and never leave it

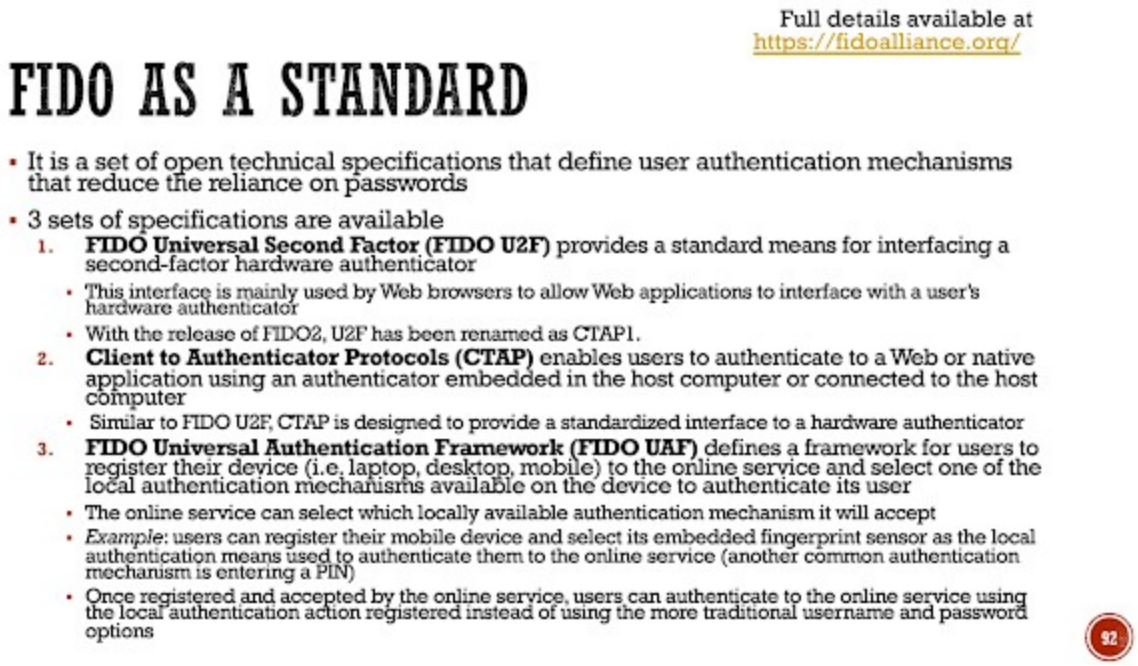


Authentication requires the client device to prove possession of the private key to the authenticating service by successfully responding to a cryptographic challenge.
The private key can only be used after successfully authenticating using the registered authenticator, for example by swiping a finger on the fingerprint sensor, entering a PIN, pressing a button.
The device then uses the user account identifier provided by the service to select the correct key and cryptographically sign the service's challenge
The signed challenge is sent back to the service, which verifies it with the stored public key and logs in the user.



We can notice that the only thing that we need to keep private (the private key) never leaves the authenticator. Now phishing is impossible, because there is nothing (useful for the attacker) to phish.

The problem now is that if the attacker gets the possession of the authenticator device, he can do everything because this system requires only that. In order to mitigate this problem, the authenticator device is protected in some manner (biometric, pin, etc).
Of course this type of authentication can be used even to complement password-based solutions, replacing OTP with this.
In practice this solution is used only for the admin and the user that must be more protected, because of the cost of this solution (each user should have this usb key). So this solution was never widely adopted.
Smartphones may change this as they may run applications capable of computing or receiving one time passwords as second factors for authentication without the need of buying dedicated hardware

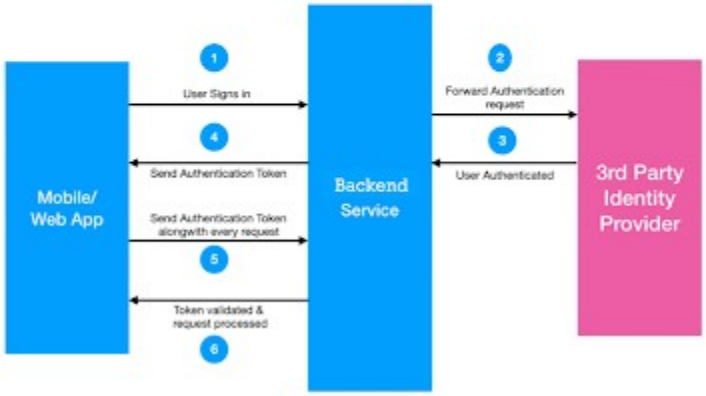


Outsourcing authentication

Nowadays we don't have a pair of credentials for each service that we use, that's because the services started to use the authentication system provided by other companies (for example SPID, Google, Apple). So the services in a certain sense **trust** the authentication service provider. This is a distributed solution.
National digital identity infrastructures sponsored by many Member States in Europe obtained different levels of success because of the not always clear business models for identity providers (An example of this is the Sistema Pubblico di Identità Digitale (SPID) in Italy that was more widely adopted only after the pandemics).
This solution is called **Single-Sign-On** (SSO)

A SOLUTION

- Delegate authentication to a **trusted** 3rd party identity provider
- Organizations can consume authentication assertions while **assuming** it has deployed a secure identity management lifecycle
- Applications and services becomes dependable on the 3rd party identity provider



The advantages of this solution are:

- A unique registration and authentication system is provided to the user. This is very important because often the users aren't very good with technology, so the less they do better it is, and they will learn only one authentication system.
  This is very important in Public Administration Services with SPID. In this way the various public administration services can focus on their businesses, without taking care about authentication (and without the risk of the security).
  So the usability is better and the user has to remember only one password for many services.
- The companies that offer the outsourcing authentication focus only on this stuff (meaning that the service and the security of the services should be very good).
- The authentication must be done once in a while because once you get authenticated you get a session of a certain amount of time

The disadvantages of this solution are:

- There is a single point of failure.
- If an attack is successful, the attacker will have access to all the services linked to that outsourcing authentication.
- The privacy, because the authentication service provider will know all the applications that a user uses.

But in this situation we need some way to link and make communication between the service that requires the authentication and the service that provides it. Obviously this connection must be secure. And also the connection between the user and the backend service must be secure, given that if an attacker gets the possession of the authentication token, he can do all the actions that are allowed to the user with that token.
So you delete the problem of the authentication but you will make more attention on other factors, with the raising of more security concerns (like secure connection).

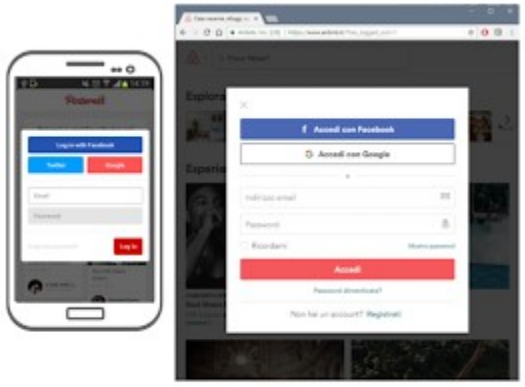
The system designers have to **balance convenience and security** (Ease-of-use is an important factor in making systems really useful, but many practices which are convenient also introduce new vulnerabilities).

WIDESPREAD ADOPTION OF SSO

Single Sign-On (SSO) allows users to access multiple apps through a single authentication act

SAML 2.0

consolidated, corporate & governmental environments



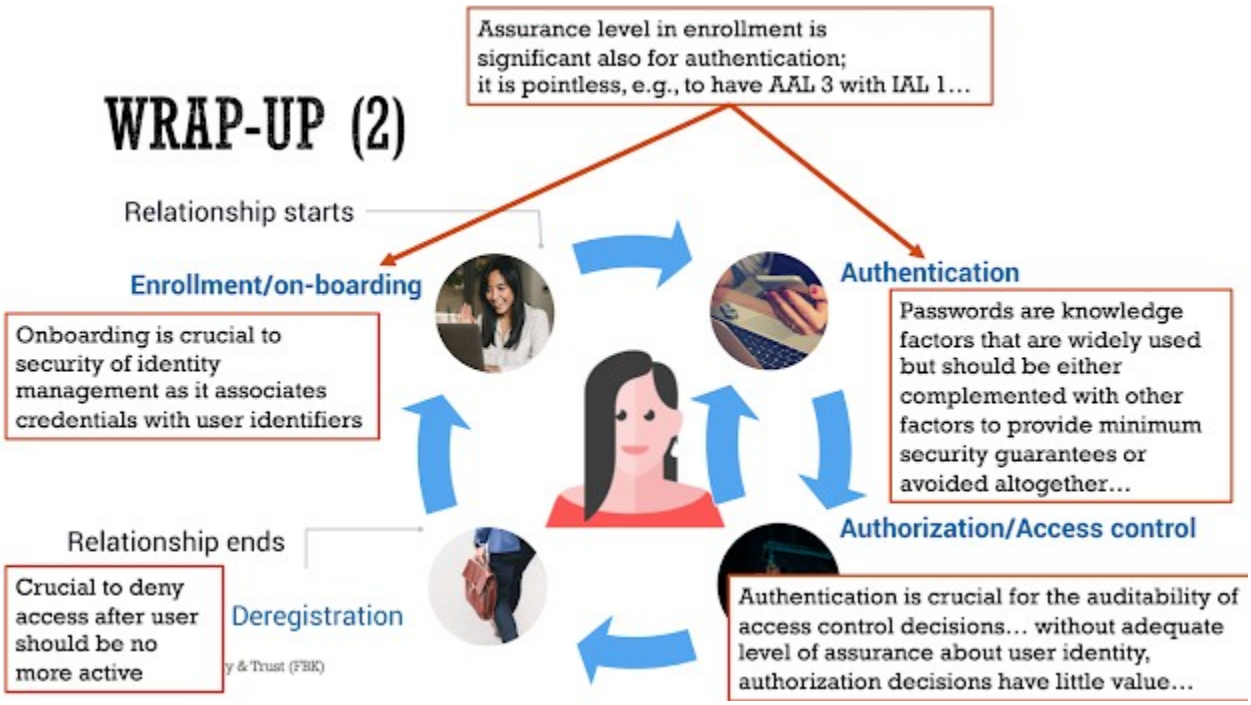
used for social network (billions of user)

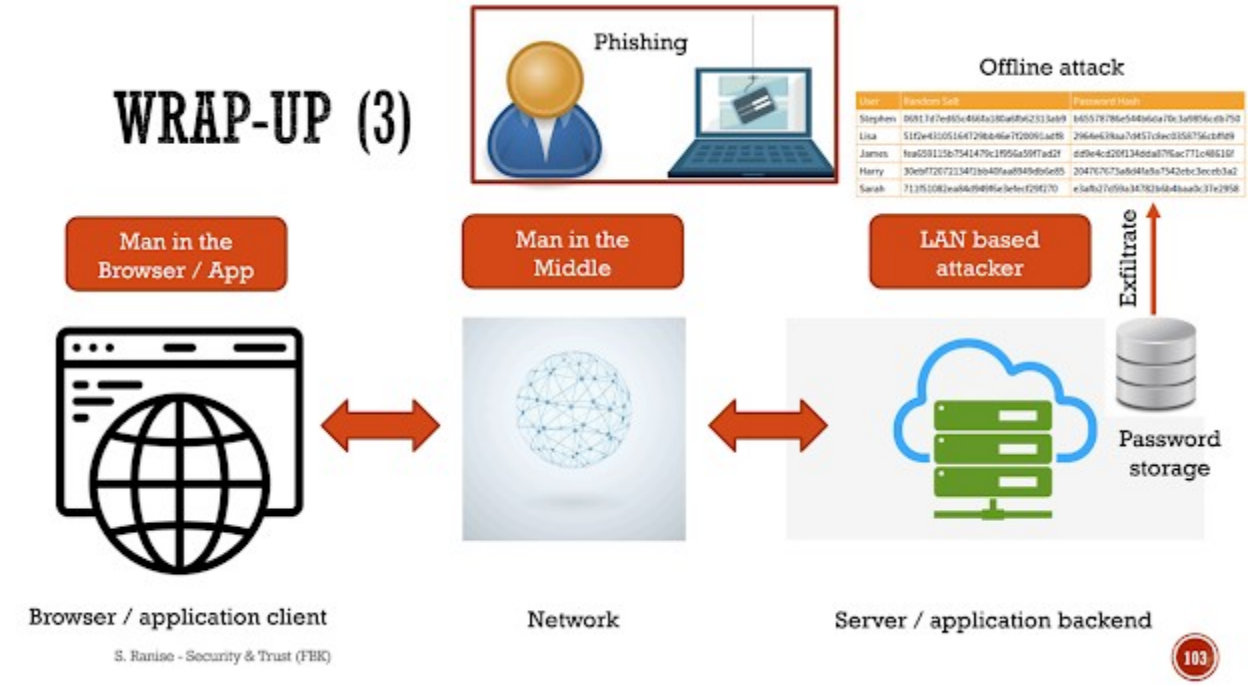
WRAP-UP (1)

Authentication can be seen as the front door of an application, service or ICT system…



- Authentication amounts to verifying the identity of a user, process, or device, often as a prerequisite to allowing access to resources in an information system
  - Here we focused on user authentication
- Passwords are one of the most widespread and accepted method for user authentication despite their shortcomings
- Adequate security mechanisms should be put in place to protect stored passwords
  - Hashing & salting, access control, …
- Authentication is only one phase of the identity management lifecycle
  - Also other phases (e.g., identity proofing) may have weaknesses
- Given the importance and difficulty of identity management, it is frequently outsourced to trusted third parties with advantages and disadvantages also from the viewpoint of security (and privacy)
  - This paves the way to Single Sign On (SSO) solutions





Hands on

In this part we're going to see an automatic tool for security analysis of MFA procedures.

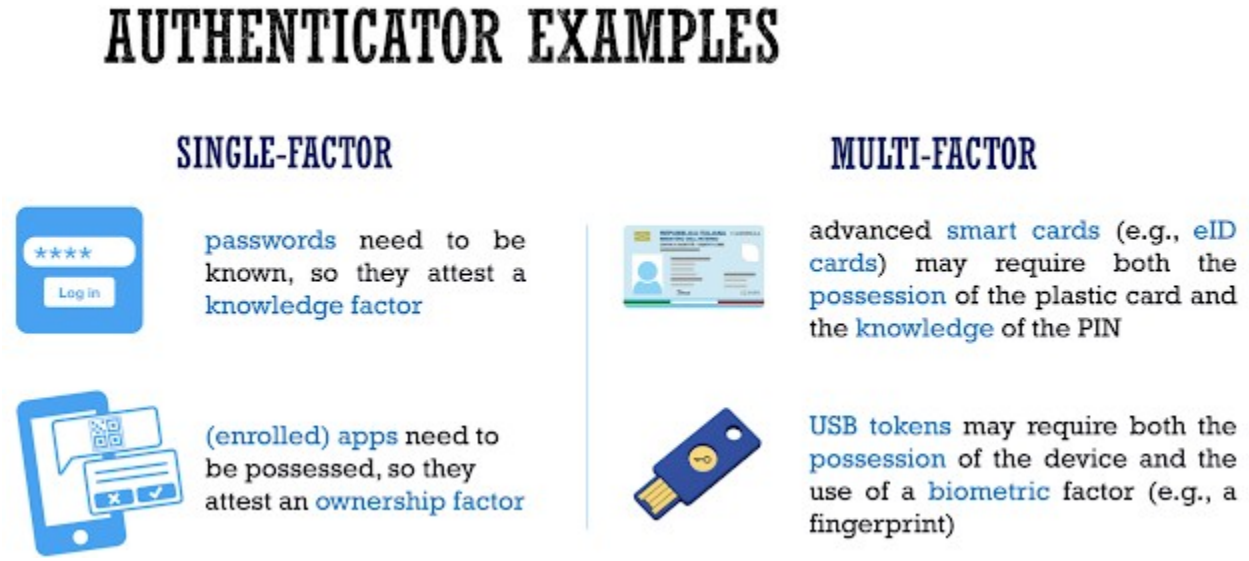
Let's first recap the problem of password-based authentication:

- **Guessing**
  This can be done by a **brute force attack** (try all possible combinations of valid symbols

up to a certain length) or by a **dictionary attack** (try a large number of commonly used usernames/passwords)

- **Credential stuffing**
  Attackers take billions of email addresses and corresponding cracked passwords from compromised databases and see how many of them work at other online services (Reuse of the same password for multiple services)
- **Phishing / Social Engineering**
  A user voluntarily sends the password over a channel, but is misled about the end point of the channel
- **Resetting technique**
- **Storing**
  Plaintext, without hash/salt, weak hash functions, etc...

A first possible mitigation to these problems is the use of MFA (Multi-Factor Authentication): Authentication process based on multi-factor **authenticators** that attest more than a single **authentication factor**.
Authentication factor: the three types of authentication factors are something you know, something you have, and something you are.
Authenticator: something the claimant possesses and controls that is used to authenticate the claimant's identity. Every authenticator attests one or more authentication factors.



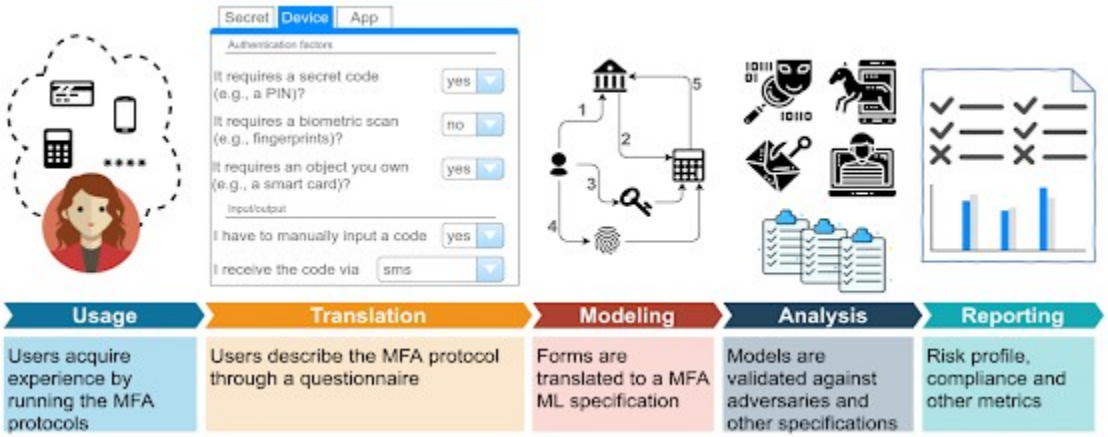
We've seen that there exists three levels of authentication assurance:

AAL1	Provides some assurance that the claimant controls an authenticator registered to the subscriver. AAL1 requires single-factor authentication using a wide range of available authentiction technologies.	
AAL2	Provides high confidence that the claimant controls authenticator(s) registered to the subscriver. Proof of possession and control of two different authentication factors is required through a secure authentication protocol.	
AAL3	Provides very high confidence that the claimant controls authenticator(s) registered to the subscriver. Authentication at AAL3 is based on proof of possession of a key through a crytographic protocol.	

Unfortunately, even MFA authentication can be vulnerable to attacks (but the risk is lower).
We observe that the attackers can collaborate together and break into different levels that we've seen before.

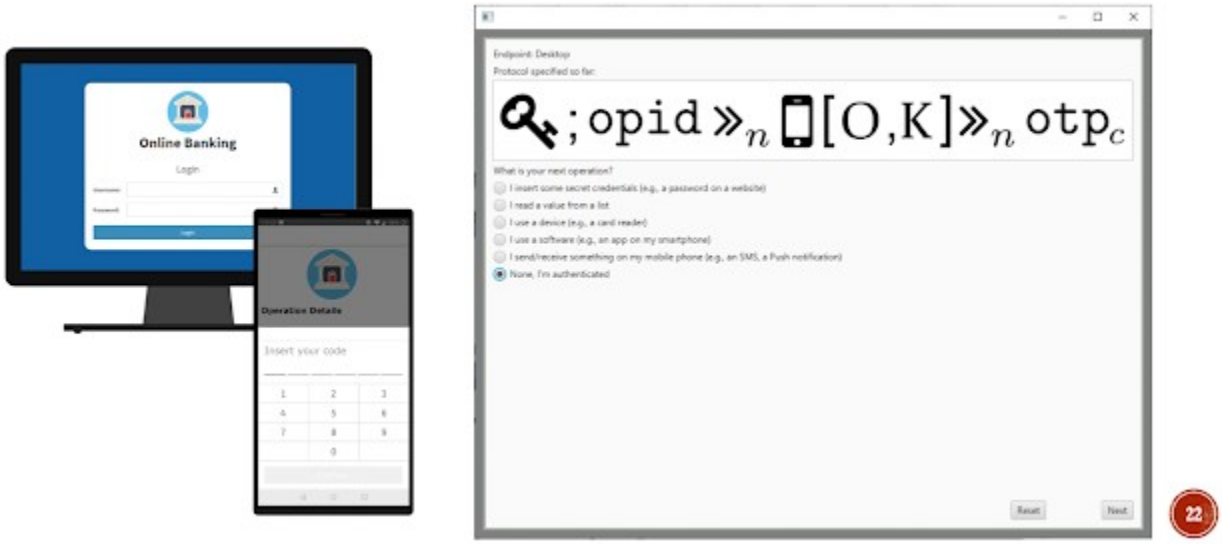
MuFASA

MuFASA stands for **Mu**lti-**F**actor **A**uthentication **S**pecification and **A**nalysis and it is an automatic tool for the security analysis of multi-factor authentication procedures.
This tool automatically tests the multi-factor authentication procedures.
To do this, the tool, starting from a high-level specification provided through a questionnaire, evaluates the security of the modelled protocol in terms of resistance against a set of potential attackers.
This is the flow:

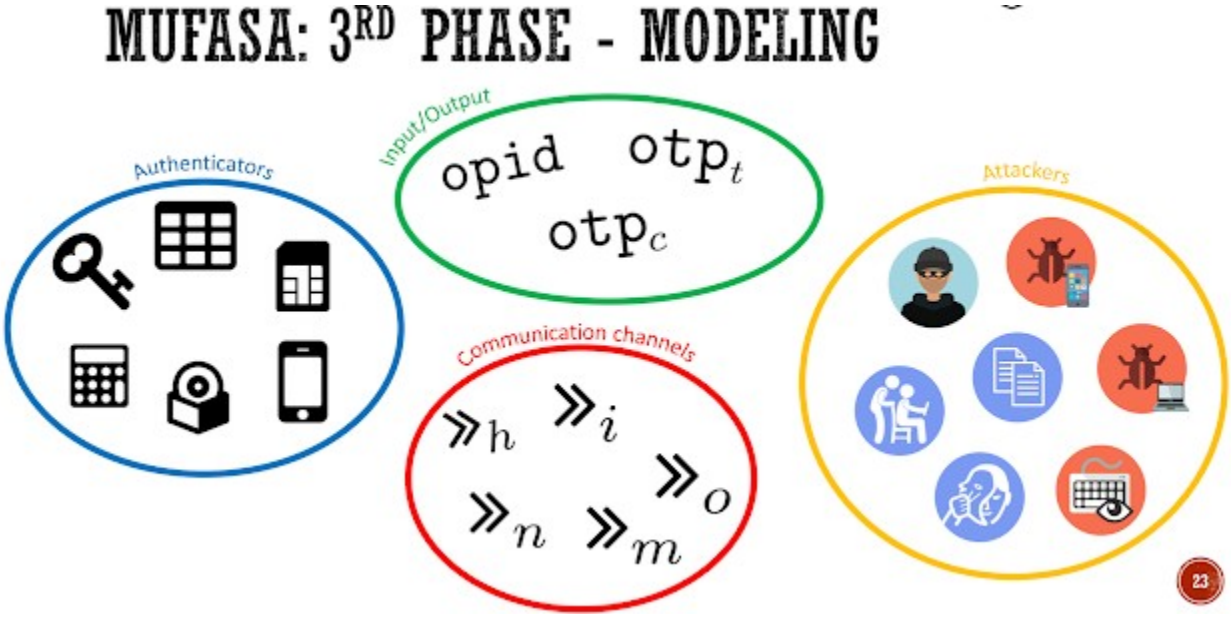


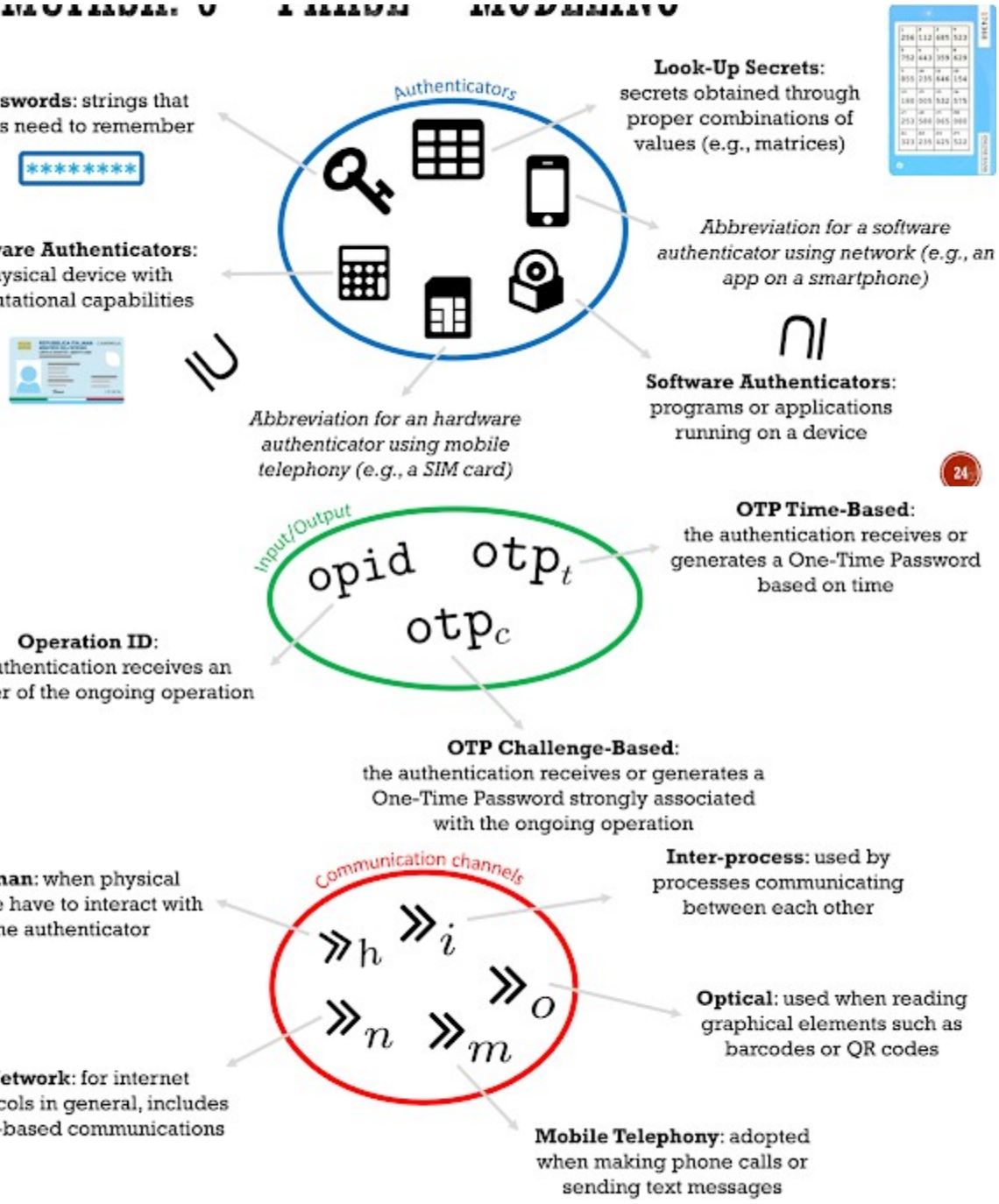
So the user first uses the service and knows about how it works. Then he inserts the data and the response in the tool and the tool will automatically evaluate the security.

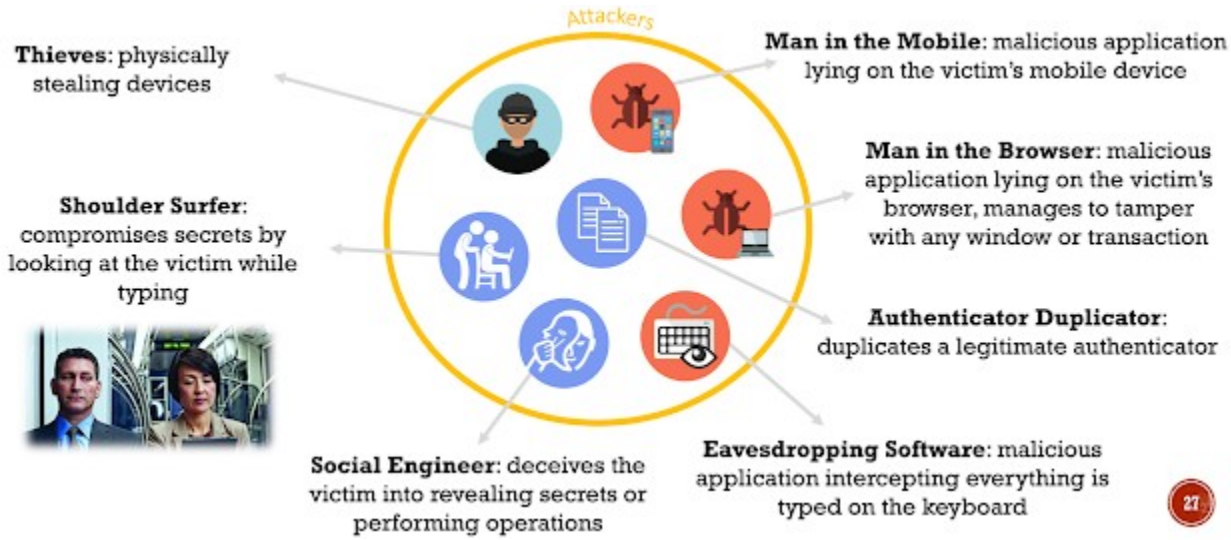
For example this is the translation for an online banking service, started on a computer, that uses second factor authentication with push notification on the smartphone and inserting a pin on the smartphone.
This is the translation:



This is the first step to define a **threat model**, because we specify the characteristics of the model that we want to analyze in terms of security.
These are the symbol used to define the authentication model:



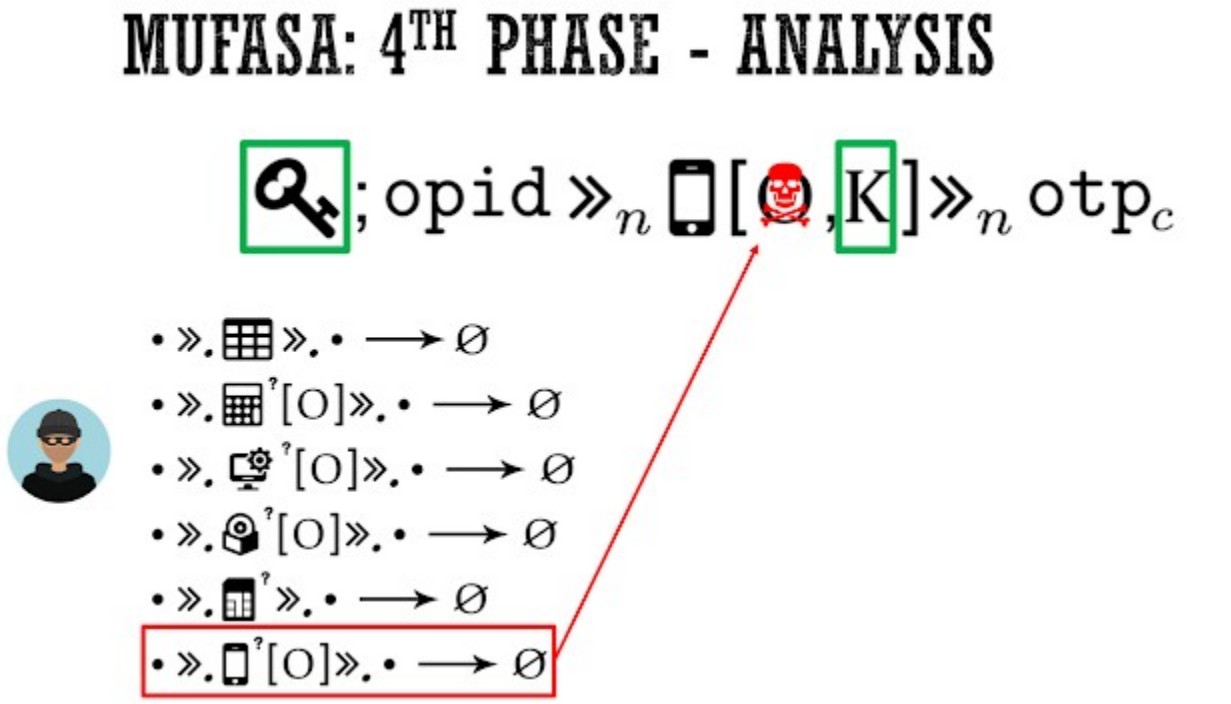

Authenticators
Passwords: strings that users need to remember
****
Look-Up Secrets: secrets obtained through proper combinations of values (e.g., matrices)
Hardware Authenticators: a physical device with computational capabilities
Abbreviation for a software authenticator using network (e.g., an app on a smartphone)
Software Authenticators: programs or applications running on a device
Abbreviation for an hardware authenticator using mobile telephony (e.g., a SIM card)
24
Input/Output
opid
otp_t
otp_c
OTP Time-Based: the authentication receives or generates a One-Time Password based on time
Operation ID: the authentication receives an identifier of the ongoing operation
OTP Challenge-Based: the authentication receives or generates a One-Time Password strongly associated with the ongoing operation
Communication channels
$\gg_h$
$\gg_i$
$\gg_o$
$\gg_n$
$\gg_m$
Human: when physical people have to interact with the authenticator
Inter-process: used by processes communicating between each other
Optical: used when reading graphical elements such as barcodes or QR codes
Network: for internet protocols in general, includes all IP-based communications
Mobile Telephony: adopted when making phone calls or sending text messages



With the modeling we obtain a representation of the system in this form:

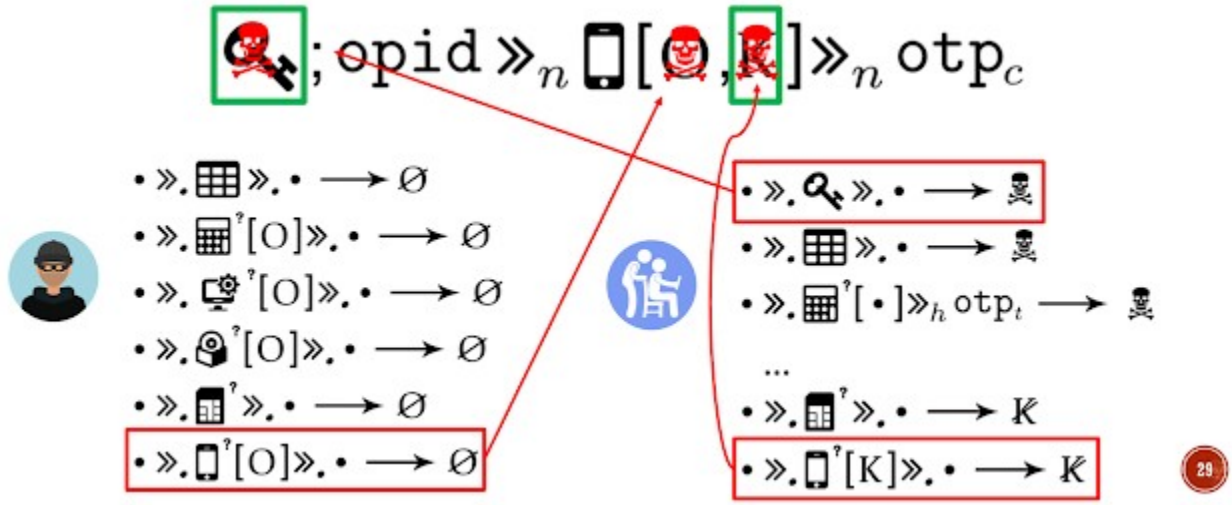
$$\text{🔑};\text{opid} \gg_n \text{📱}[\text{O},\text{K}] \gg_n \text{otp}_c$$

Then we have the analysis:



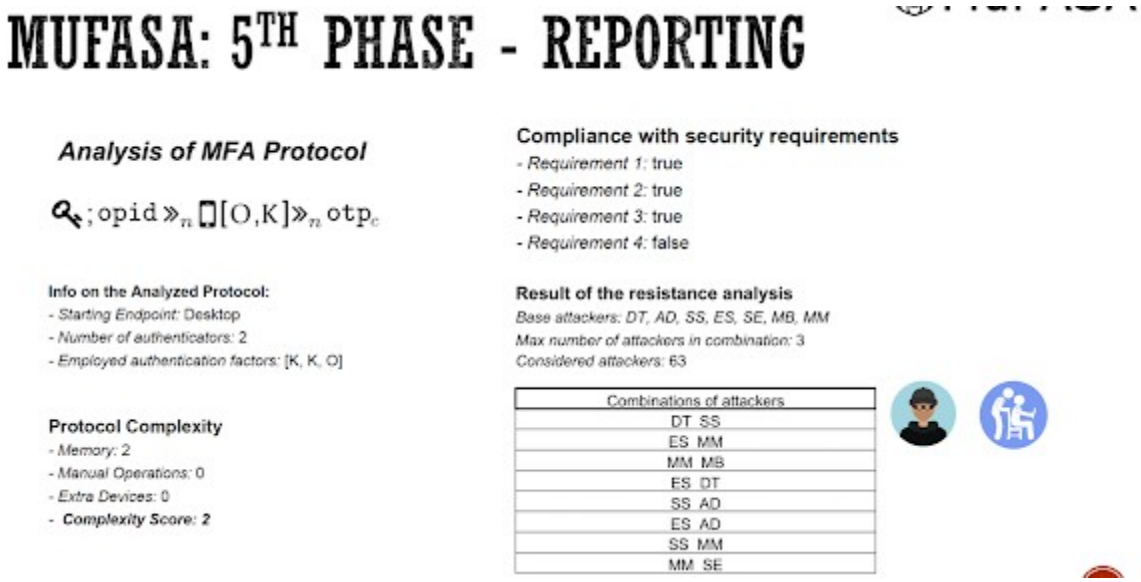
In this example we find that a thief's capability (the stealing of the smartphone) matches with our model, so we could have a threat. But the thief must also have the Knowledge factor, so we can say that a thief cannot compromise this protocol. (O,K means Ownership factor and Knowledge factor).

But if we combine a thief with a shoulder server, our system can be violated:



A combination of attackers becoming a threat if they can violate all the authentication factors.

The final phase is the reporting, were the possible threat and the combination of attackers are showed to the user:



At the beginning MuFASA was only able to make a **security analysis**, detecting the attackers that managed to compromise the protocol.
Lately it was extended to also make a **risk analysis**, evaluating the risks connected with the successful attackers detected. In this way we are able to find which attackers can do more damage to us and make a priority list for the attackers.

EVALUATING THE RISKS



OWASP Risk Rating Methodology

		Likelihood		
		Low	*Medium*	*High*
Impact	*Low*	Note	Low	Medium
	Medium	Low	Medium	High
	High	Medium	High	Critical

In this way we can classify the attackers we have found previously:

Analysis of MFA Protocol

$;\mathrm{opid} \gg_n [\mathrm{O,K}] \gg_n \mathrm{otp}_c$

Compliance with security requirements
- *Requirement 1:* true
- *Requirement 2:* true
- *Requirement 3:* true
- *Requirement 4:* false

Info on the Analyzed Protocol:
- *Starting Endpoint:* Desktop
- *Number of authenticators:* 2
- *Employed authentication factors:* [K, K, O]

Protocol Complexity
- *Memory:* 2
- *Manual Operations:* 0
- *Extra Devices:* 0
- ***Complexity Score: 2***

Result of the resistance analysis
Base attackers: DT, AD, SS, ES, SE, MB, MM
Max number of attackers in combination: 3
Considered attackers: 63

Combinations of attackers	Likelihood	Impact	Risk
DT_SS	MEDIUM	MEDIUM	MEDIUM
ES_MM	LOW	HIGH	MEDIUM
MM_MB	LOW	HIGH	MEDIUM
ES_DT	LOW	MEDIUM	LOW
SS_AD	LOW	MEDIUM	LOW
ES_AD	LOW	MEDIUM	LOW
SS_MM	LOW	MEDIUM	LOW
MM_SE	LOW	MEDIUM	LOW

For evaluating the risk, there are two different level:

- The **static** analysis
  Where the values used to compute the risk are static, thus they do not depend on the considered scenario.

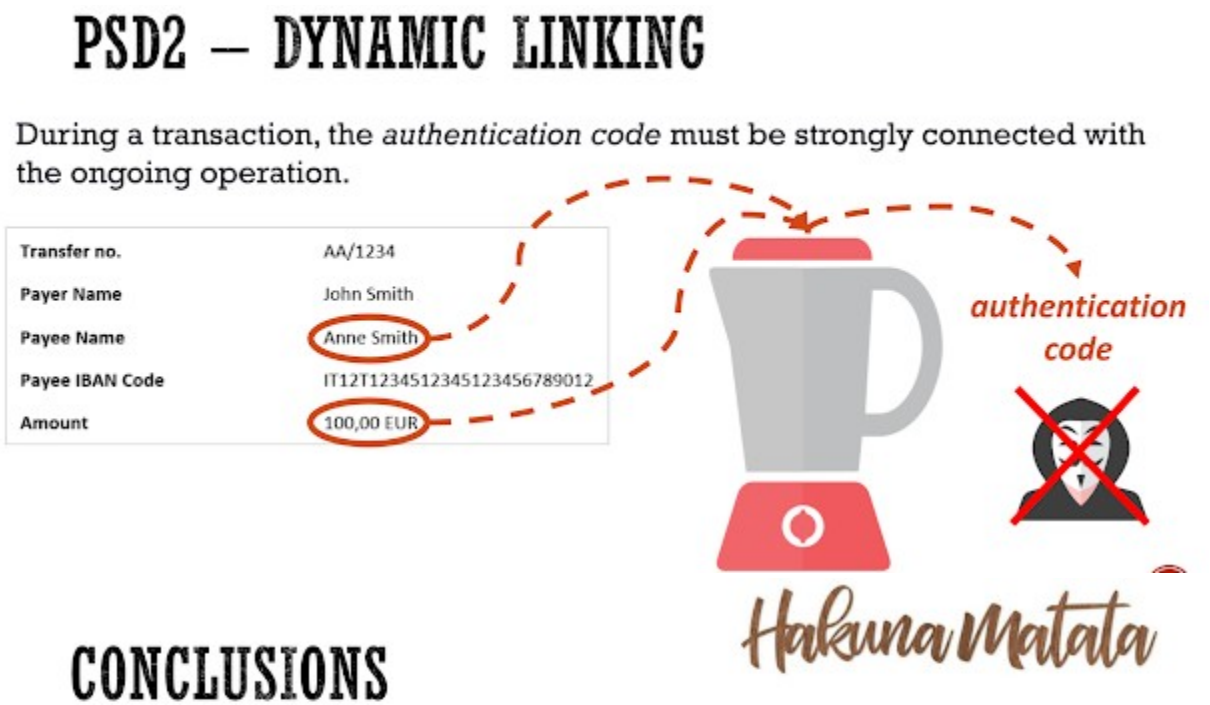
- The **flexible** analysis
  Where the values used to compute the risk start from static references, but then they are adjusted in order to fit as best as possible the considered scenario (this makes the analysis more precise).
  This can be done through a questionnaire.

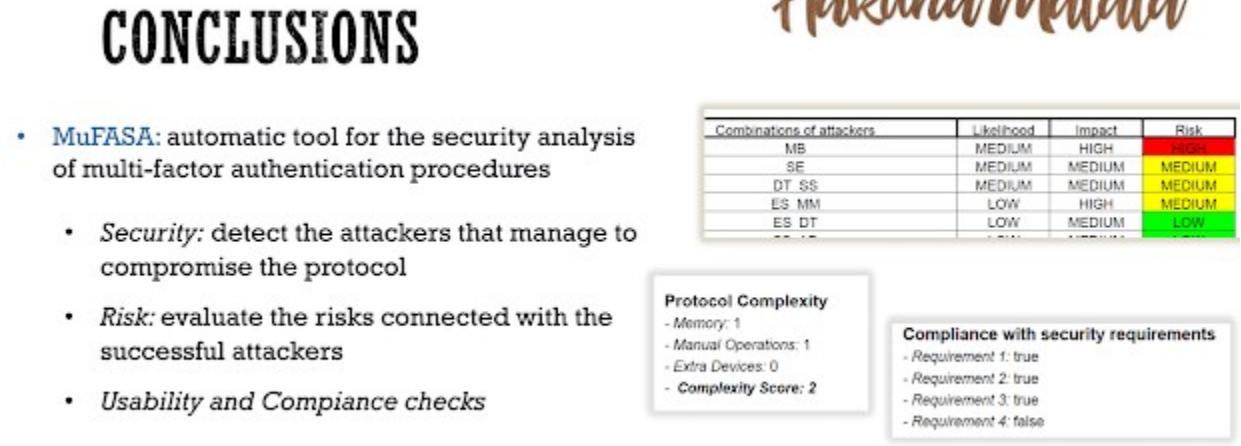
The only part that this program misses (that is also the most important in the Security world) is the security controls and the mitigation mechanisms.

Let's do a recap and some considerations on PSD2:









Cryptography

Introduction to Cryptography

A bit of history

Modern cryptography started with the italian Leon Battista Alberti who wrote the first book about a modern approach to cryptography in 1466. This book introduces the basics (building blocks and ideas) to design new cryptographic techniques.

Those who are charged with the highest affairs know by experience how important it is to have a very trustworthy person with whom to reveal projects and decisions of the most secret nature, without ever having reasons to regret it.[1] This is rare, because men so often tend to perfidy, and so secure systems called *ciphers* have been devised, which would be useful, except for those who manage, through art and ingenuity, to read and interpret them. I admit that those in power may be served by those who are expert in such operations, if by their means the schemes and machinations of enemies can be revealed, but in my opinion, it is even more advantageous to be able to communicate our intentions to another, no matter how far away, in a way that no other mortal except the intended recipient of the missive is able to read them.

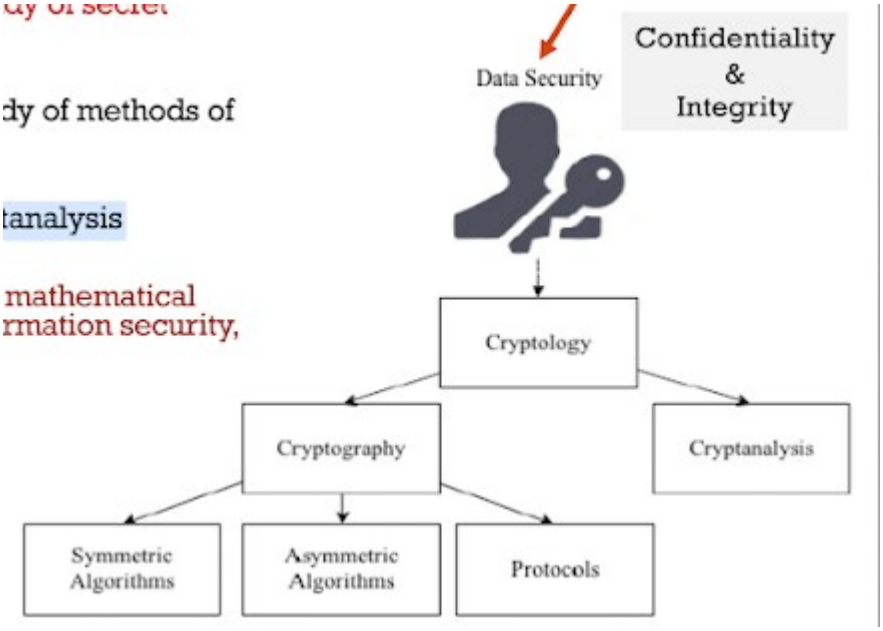
This present booklet of mine is a thorough treatment of both aspects of the question, paving the way for and directing the investigation into other people's secrets, as well as how, as we shall see, to protect secrets of your own.

The present circumstances persuaded me to send these notes of mine to you. Many friends who are devoted to you exhorted me to do so. I will be delighted if this work pleases you.[1]

Alan Turing (the person who invented the Turing machine) contributed to the second world war because he deciphered the secure communication of the German military throughout the war using the **enigma machine**.
Alan Turing and Leon Battista Alberti are considered the two fathers of modern cryptography.

Basic notions

Cryptography is a set of mechanisms and techniques that provide data security, so cryptography is the security mechanism which provides the security service data security.
Cryptography is the science and study of secret writing.
Cryptanalysis is the science and study of methods of breaking ciphers.
This distinction is only academic because in practice they are always considered together.
Cryptology is composed of cryptography and cryptanalysis.



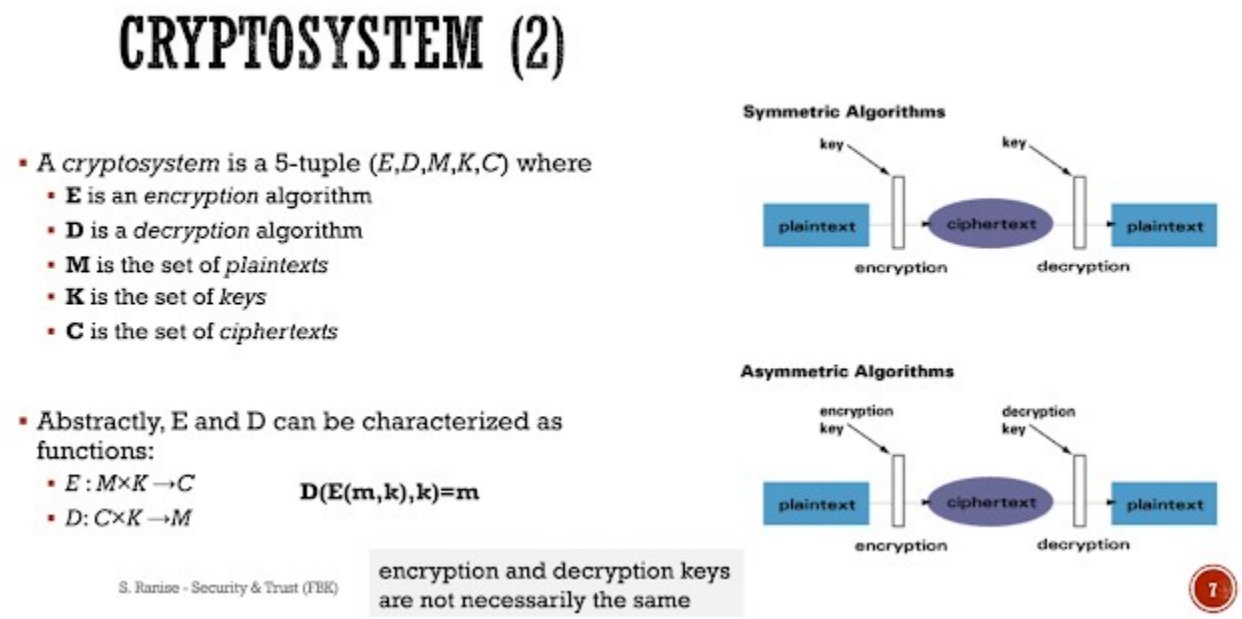
Symmetric cryptography: the same key is used for both encryption and decryption.
Asymmetric cryptography: two different keys (public and private) are used for encryption and decryption.

The **cryptographic protocols** are composed of many cryptographic primitives that combined together form a sort of secure channel/communication.

The two main goals of cryptography are **confidentiality** and **integrity**.
Cryptography is the study of mathematical techniques related to aspects of information security, such as (in addition to confidentiality and integrity) entity authentication and data origin authentication.

The main role of cryptography primitives is to transform the **plaintext** into a **ciphertext**.
So cryptography is a security mechanism that includes a set of techniques for scrambling or disguising data (to transform it into ciphertext) so that it is available only to someone who can restore the data to its original form (who will decipher the ciphertext).
The only way to recover a plaintext from a ciphertext without having the decryption key is to guess (if the cryptography is done correctly), so ciphertext looks like random garbage to anyone expect those that have the same key used to generate the ciphertext and the decryption algorithm.

A **cryptosystem** is a 5-tuple of set and functions:

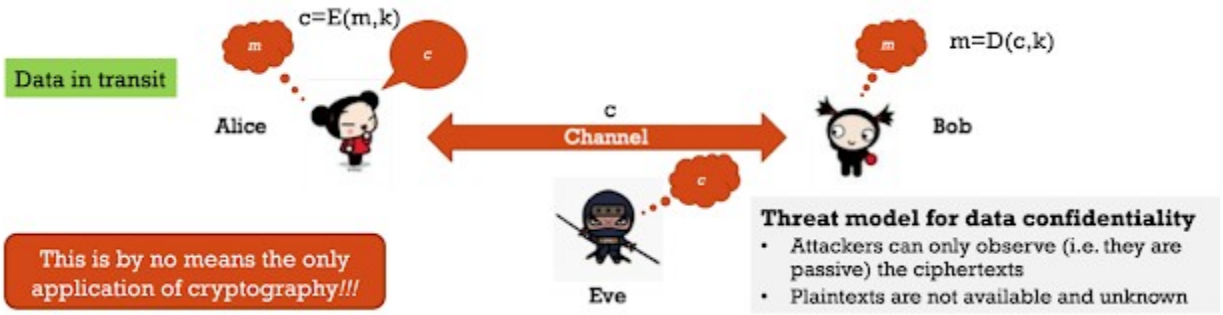


In current computer systems, cryptography provides a strong, economical basis for offering data security as a security service, i.e. a service for keeping data secret (confidential) and for verifying data integrity.

The typical application of cryptography is to obtain communication security, that is:

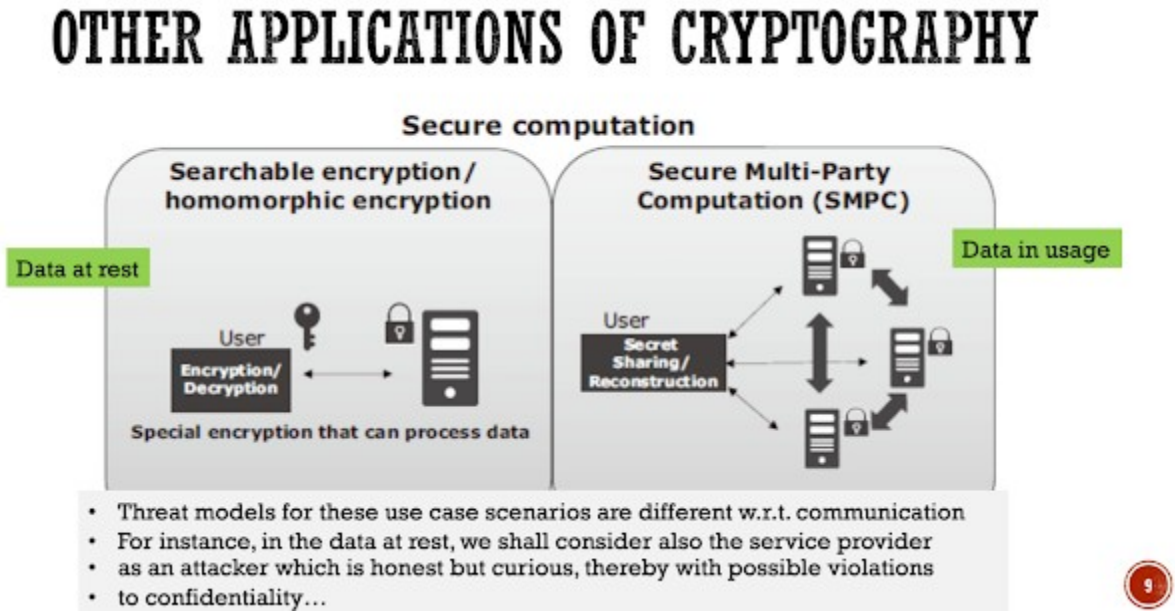
- **Data confidentiality**: encryption algorithms hide the content of messages
- **Data integrity**: integrity check (e.g., hash) functions provide the means to detect whether a document has been changed;
- **Data origin authentication**: message authentication codes (e.g., HMAC) or digital signature algorithms provide the means to verify the source and integrity of a message

This is an example of data confidentiality obtained with cryptography:



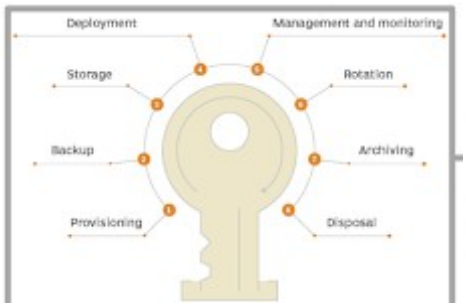
The key point of this procedure is that Bob knows how to decrypt the message, because he has the decryption key.
Obviously the key must be transmitted in a secure channel, because the key must be secret in order to maintain confidentiality.

Other usage of cryptography could be to “hide” the data stored in a database, or to hide data in usage (while computing something, for example if dataset comes from different organizations and no one wants the others to see its data):



The practice to encrypt data at rest is good because you have no assurance that the entity to which you share your data is safe (maybe an employee of that company will steal your data, or maybe the security mechanisms of that company aren’t secure).

Another important point is the non repudiation of a message sent (or action done).
The encryption/decryption system must have one important properties, the **Kirchoff principle**: Do not rely on the secrecy of algorithms; **the key should be the only secret that needs protection**.
So in general you can rely only on the secrecy of the key when you project/apply cryptography. So the functions E and D are better to be public, more precisely they should be public to obtain “help from people”.



- **provisioning** or generating keys for organizational needs;
- **backing up** keys that must be recoverable after system failures
- **storing** keys securely
- **deploying** keys to systems that need them
- **managing and monitoring** use of deployed keys
- **rotating** keys to prevent overexposing them to sensitive material
- **archiving** keys that are no longer being actively used
- **disposing** keys that are no longer required.

Projecting and implementing cryptographic primitives is a very difficult task to do, so in general (and it is very suggested) people use already made available libraries.

Resuming, a **key** is an input to a cryptographic algorithm used to obtain confidentiality, integrity, authenticity or other property over some data.
The security of the cryptosystem often depends on **keeping the key secret** to some parties.
The **keyspace** is the set of all possible keys. **Entropy** is a measure of the variance in keys (typically measured in bits).
For most cryptographic systems the cryptographic keys should be as random as possible. In other words, the keys should exhibit high entropy, i.e. the randomness collected by an operating system or application for use in cryptography.
Intuitively, randomness is the apparent or actual lack of pattern or predictability in events.

Keys are not always stored in a secure place, but there exists cases in which the key must be public (e.g. certificates).

One issue of keys is **distribution**
The two parties to an exchange must share the same key, and that key must be protected from access by others (frequent key changes are usually desirable to limit the amount of data compromised if an attacker learns the key).
We will see some techniques to transmit in a secure way the key over a secure channel.
The strength of any cryptographic system rests with the key distribution technique (because if an attacker knows the key, all the cryptographic techniques are useless).

- Example approaches for entities A and B:
  1. A key could be selected by A and physically delivered to B
  2. A third party could select the key and physically deliver it to A and B
  3. If A and B have previously and recently used a key, one party could transmit the new key to the other, encrypted using the old key
  4. If A and B each have an encrypted connection to a third party C, C could deliver a key on the encrypted links to A and B

Another issue of encryption schemes is that they have to be **computationally secure**. That means that an attacker can't guess the key in a reasonable amount of time.
An **encryption scheme is computationally secure** if the ciphertext generated by the scheme meets one or both of the following criteria:

- The cost of breaking the cipher exceeds the value of the encrypted information
- The time required to break the cipher exceeds the useful lifetime of the information

The problem is that the same cryptographic techniques that today allow us to guarantee confidentiality for a transaction for a certain amount of time (for example 48 hrs), in five years will be no longer the same, because computational power is going to grow exponentially and the cost is going to goes down.
It is very difficult to estimate the amount of effort required to cryptanalyze ciphertext successfully, but assuming there are no inherent mathematical weaknesses in the algorithm, then a brute-force approach is indicated, and so one can make some reasonable estimates about costs and time (on average in a brute force attack half of all possible keys must be tried to achieve success).

- E(k,P)=C
  - **Computing C from P is hard without k, computing C from P with k is easy**

- D(k,C)=P
  - **Computing P from C is hard, computing P from C with k is easy**

Informally, a **one-way function** might be described as a function for which evaluation in one direction is straightforward, while computation in the reverse direction is far more difficult (for example hash functions we've already seen).
A **trapdoor one-way function** is a one-way function with an additional requirement: computation in the reverse direction becomes straightforward when some additional (trapdoor) information is revealed (a cryptography function is a trapdoor one-way function with the key as a trapdoor).

So this is the main difference between hash functions and encryption/decryption functions:

- Hash functions are one-way functions and you have no way to obtain the original message starting from the digest
- Encryption/decryption functions are trapdoor one-way functions and you have no way to obtain the plaintext starting from the ciphertext, unless you have the secret key.

Hash functions shall be used when there is no need to recover the input from the output whereas encryption functions shall be used when there is a need to efficiently recover the input from the output provided that a key is available.

- **Hash function use case scenario: integrity**
  - compute the digest of a message and send along with the message
  - on the receiving side, digest can be recomputed using the same hash and result compared with the digest sent with the message
    - if equal, then message has not been tampered with
    - otherwise, violation of message integrity
    - confidentiality is not guaranteed as the message is sent in plaintext



- **Encryption function use case scenario: confidentiality**
  - assume sender and receiver share the same secret key
  - compute the ciphertext of a plaintext message with available key
  - send ciphertext
  - on the receiving side, ciphertext can be decrypted with the shared secret key
    - secrecy of message is preserved provided secret key is not disclosed
    - integrity is not guaranteed: if an attacker flip one bit of the ciphertext, the receiver cannot detect it



To end this section it is important to denote that cryptography cannot alone resolve all our security problems. In fact you have to use it carefully and cryptography transforms a problem into another (for example you have to add access control mechanisms in the computer system in order to protect the cryptographic keys).
Cryptography is a translation mechanism, because it converts a communication security problem into a key management problem.

Resuming, encryption is an algorithm used to make content unreadable by all but the intended receivers. The algorithm is public and the key is private (Kirchoff)

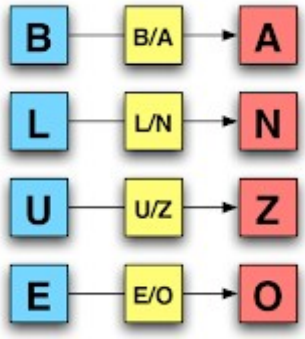
Substitution and transposition

Obviously in all the encryption methods, the fundamental requirement is that **no information shall be lost** (that means that all operations must be reversible).
The encryption can be done in various ways. Let's see two types of transformations:

- **Substitution**
  Each element in the plaintext (bit, letter, group of bits or letters) is mapped into another element (letters are replaced by other letters)
- **Transposition**
  Elements in the plaintext are rearranged (ame letters but arranged in a different order)

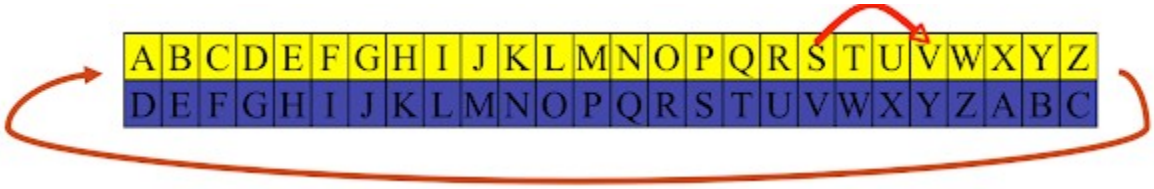
We described transformations over letters as they are more intuitive, however the same ideas apply to string of bits.
Most encryption systems nowadays use multiple stages of substitutions and transpositions (making the life of the attackers more complex).

A **substitution cipher** is a method of encrypting by which units of plaintext are replaced with ciphertext, according to a fixed system. The "units" may be single letters (the most common), pairs of letters, triplets of letters, mixtures of the above, and so forth.
The receiver deciphers the text by performing the inverse substitution.



This type of cipher has the issue of **frequency analysis** (based on the language).
This consists in calculating the frequency distribution of the letters in the ciphertext, i.e. count how many times each letter appears. Since English text has a very distinct distribution, this information can be used to crack codes (for example ‘e’ is the most common letter, ‘z’ the less common).

The easiest substitution cipher is **Caesar cipher**, where every character is replaced with the character that is k slots to the right.



In this cipher the key is the number of characters to shift the cipher alphabet (in the case above 3). This function is also called $ROTk$ (rotate by k).
Represent the $ROTk$ encryption function, $e(x)$, where x is the character we are encrypting, as:

$$e(x)=(x+k) \pmod{26}$$

where k is the key (the shift) applied to each letter.
The decryption function is:

$$d(x)=(x-k) \pmod{26}$$

Let’s see some cryptanalysis. The caesar cipher is probably the easiest of all ciphers to break. Since the shift has to be a number between 1 and 25 (0 or 26 would result in an unchanged plaintext), one can simply try each possibility and see which one results in a piece of readable text (brute force attack).
A more principled approach is the frequency analysis, so you can simply find the key that causes the ciphertext frequencies to match up closely with the natural English frequencies, then decrypt the text using that shift (key).

In order to mitigate these problems, **Vigenere cipher** was invented. It can be seen as a generalization of the Caesar cipher whereby several Caesar ciphers in sequence with different shift values are used.
As for the Caesar cipher, assume to associate the letters a, b, c, ..., z with the numbers 0, 1, 2, ..., 25.

In this case the key is a word. The word is written and repeated under the plaintext. To obtain the ciphertext you have to “sum” the letter of the plaintext with the letter under it of the key (that is applying a Caesar cipher based on the value of the letter).

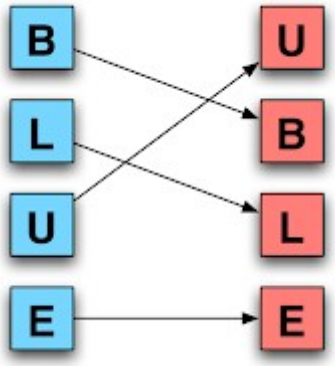
- Example

A	T	T	A	C	K	A	T	D	A	W	N	plaintext
L	E	M	O	N	L	E	M	O	N	L	E	keyword repeated to match plaintext length
L	X	F	O	P	V	E	F	R	N	H	R	ciphertext

This cipher is more secure than before because the frequency analysis is more difficult to do, given that each equal letter in the plaintext may become different letters in the ciphertext (in our example ‘A’ takes 4 different values).
The complexity of the frequency analysis on this cipher depends on the length of the keyword l, if the key has length l, then the key space has size of l^{26} .
With l higher, the complexity is higher, until it becomes impossible, because if l is equal to the length of the plaintext, then is impossible to determine the correct plaintext without the key.
In the case that l is equal to the length of the plaintext, the Vigenere cipher cannot be broken (if additionally the key is used only once).

So you have to share in a secure way a key that is as long as the plaintext every time, at this point why don’t directly share the plaintext in that way? So in practice the ideal Vigenere cipher doesn’t solve anything.

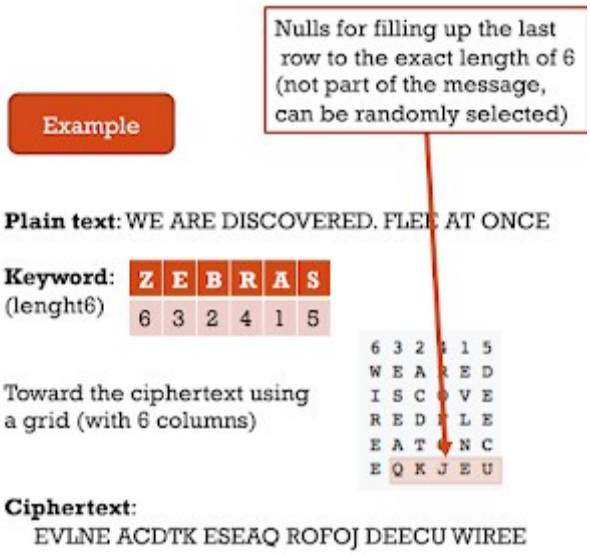
The other transformation is the **transposition cipher**.
In a transposition cipher, the units of the plaintext are rearranged in a different and usually quite complex order, but the units themselves are left unchanged. Several variants are possible.



The difference between the substitution cipher is that in a transposition cipher, the units of the plaintext are rearranged in a different and usually quite complex order, but the units themselves are left unchanged.

One type of transposition cipher is **columnar cipher**.
In a columnar transposition, the message is written out in rows of a fixed length, and then read out again column by column, and the columns are chosen in some scrambled order.
Both the **width of the rows** and the **permutation** of the columns are usually **defined by a keyword**.

For example, the keyword ZEBRAS is of length 6 (so the rows are of length 6), and the permutation is defined by the alphabetical order of the letters in the keyword. In this case, the order would be "6 3 2 4 1 5".



Modern encryption techniques

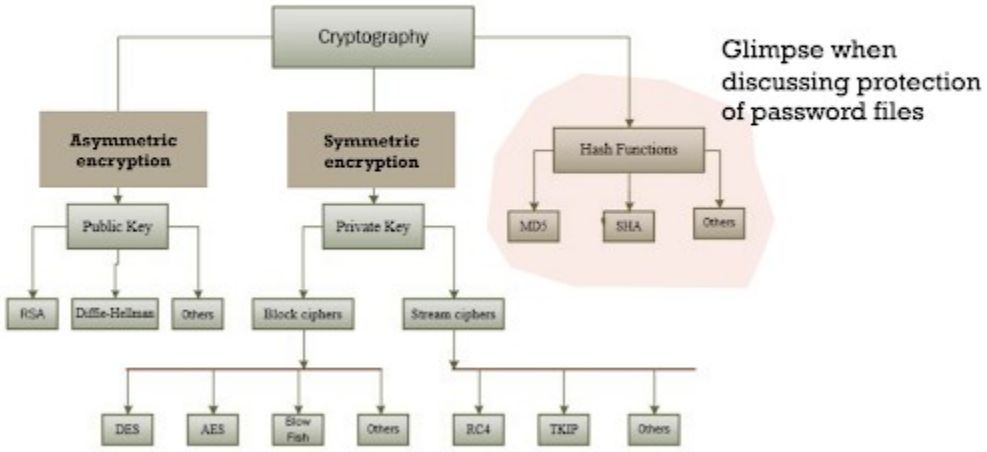
With the quick development of the computer, it became difficult to depend on the elementary classical technique as it is easy to decrypt the ciphertext.

So **modern encryption techniques** (new process depends on the algorithms, which use the bits instead of characters) enter:

- Stream encryption
- Block encryption
- Public-key encryption

Stream and block encryption are part of **symmetric key encryption** (which guarantees only confidentiality).
Public-key encryption is part of **asymmetric key encryption** (which can also guarantee integrity).

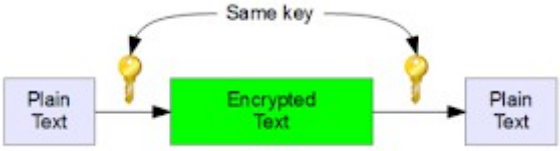
OVERVIEW OF MODERN CRYPTOGRAPHY



Symmetric key cryptography

The symmetric key encryption is based on **symmetric keys**.
Symmetric because a single key k is used for both encryption E and decryption D.
The main property is:

$$D(k, E(k, p)) = p$$



Management of keys determines who has access to encrypted data.

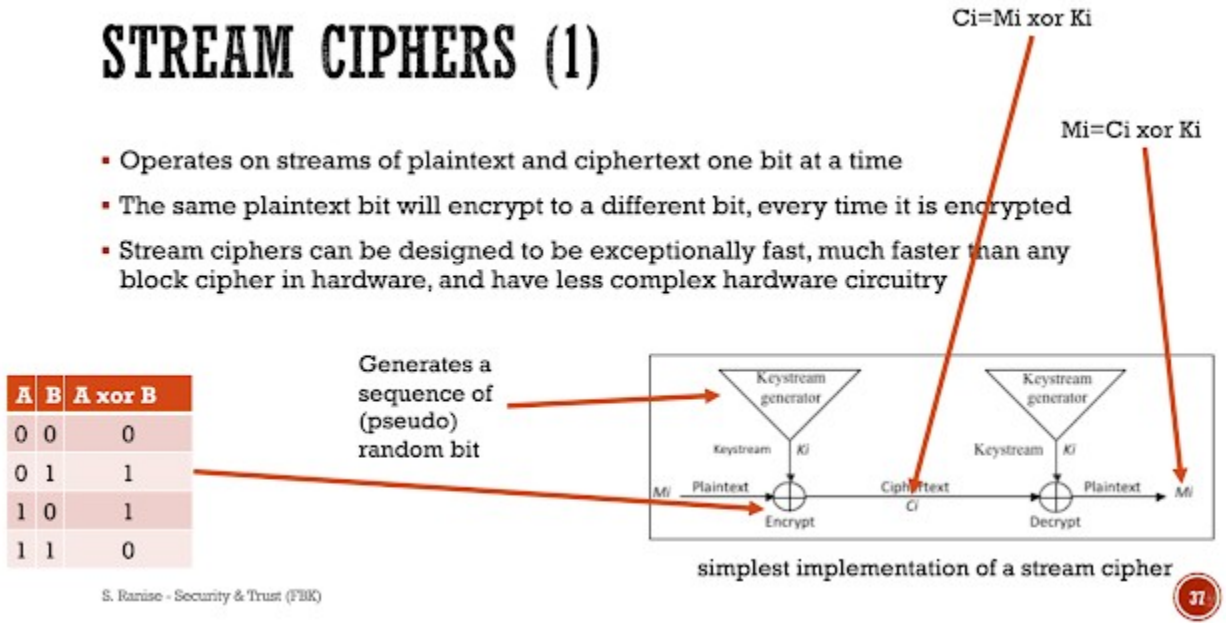
There exists two types of symmetric key cryptography:

- **Stream ciphers** (similar to the idea underline the Vigenere cipher)
  The input is a stream of bits.
  Encrypt sequences of “short” data blocks under a changing key stream.
  Security relies on the design of key stream generators. Encryption can be quite simple (e.g., XOR). Typical block length is 1 bit/byte.
- **Block ciphers** (combine transposition and substitution in a very complex way)
  The input is fixed block of the same length
  Encrypt sequences of “long” data blocks without changing the key
  Security relies on the design of encryption functions. Typical block length is 64/128 bits.
  Substitutions are called **S-boxes**: the aim is to “hide” the relationship between the key and the ciphertext, i.e. to obtain confusion
  Transposition are called **P-boxes**: the aim is to permute bits across S-boxes inputs, i.e. to obtain diffusion

Confusion means that each bit of the ciphertext should depend on several parts of the key.

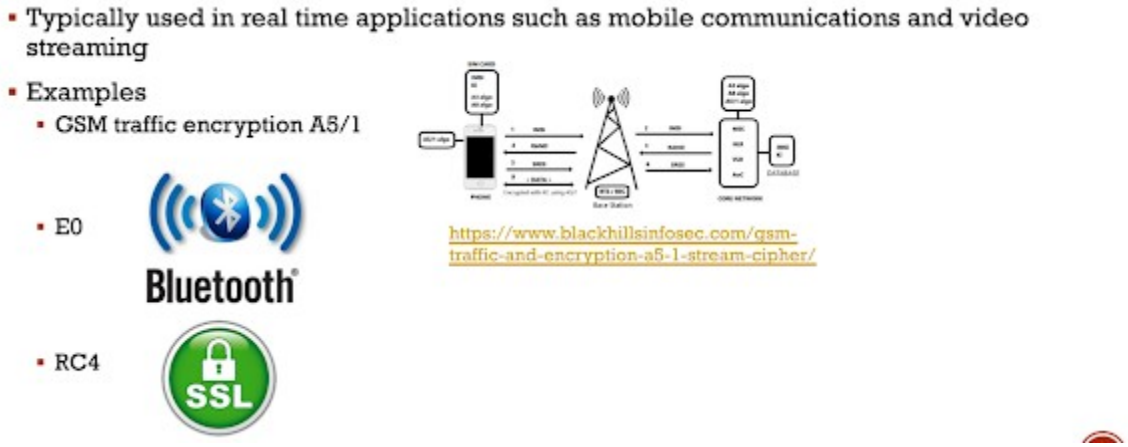
Diffusion means that if we change a single bit of the plaintext, then about half of the bits in the ciphertext should change, this is similar to the avalanche effect that we have discussed in conjunction with hash functions.

The **stream ciphers** operate on streams of plaintext and ciphertext one bit at a time. The same plaintext bit will encrypt to a different bit, every time it is encrypted. Stream ciphers can be designed to be exceptionally fast, much faster than any block cipher in hardware, and have less complex hardware circuitry.



In this cipher the decryption is very simple to implement, because it is exactly the same operation of encryption. The point here is how you can generate the same key that looks random, both in encryption and decryption. The generation of the key is **pseudo random**, because it looks random but you must be able to regenerate the same sequence of bits starting from the same sid.

Stream ciphers are applied when you have to be very very quick (for example real time application), since it is easy to implement in hardware.

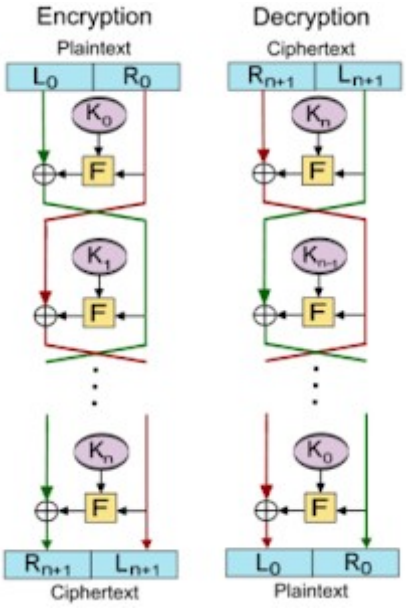


In **block ciphers**, a block of plaintext is treated as a whole and used to produce a ciphertext block of equal length (typical size of the block: between 64 and 256 bits).
A block cipher breaks (splits) the message M into successive blocks $M_1,\ M_2,\ M_3,\ldots,\ M_n$ and enciphers each M_i with the same key k (with the application of permutation and substitution in a complex way), obtaining blocks of ciphertext with the same size.
Data Encryption Standard (DES) and the Advanced Encryption Standard (AES) are well known examples of block cipher systems.

DES was designed by IBM in the 1970s and adopted by NIST in 1977 for commercial and unclassified government applications.
It is based on the **Feistel block-cipher** employing a 56-bit key that operates on 64-bit blocks.
It was designed specifically to yield fast hardware implementations and slow software implementations, although this latter point is not significant today since the speed of computer processors is several orders of magnitude faster.
The NSA also proposed a number of tweaks to DES that many thought were introduced in order to weaken the cipher, but analysis in the 1990s showed that the NSA suggestions actually strengthened DES.
DES was defined in American National Standard X3.92 and three Federal Information Processing Standards (FIPS), all withdrawn in 2005.

Feistel block-cipher (1973) is a deterministic algorithm that operates on blocks of bits with an unvarying transformation that is specified by a symmetric key.
The feistel block-cipher is composed of various rounds. In each round:

- The block of plain text to be encrypted is split into two equal-sized halves (L,R)
- The round function F (**substitution**) is applied to one half, using a subkey K. Then the output is XORed with the other half and saved instead of the other half.
- The two halves are then swapped



An important thing is that instead of using the same key in each round, a different subkey is derived from the main key (this is not shown in the figure).
The decryption has the same structure of the encryption, it is just sufficient to use the subkey in reverse order.

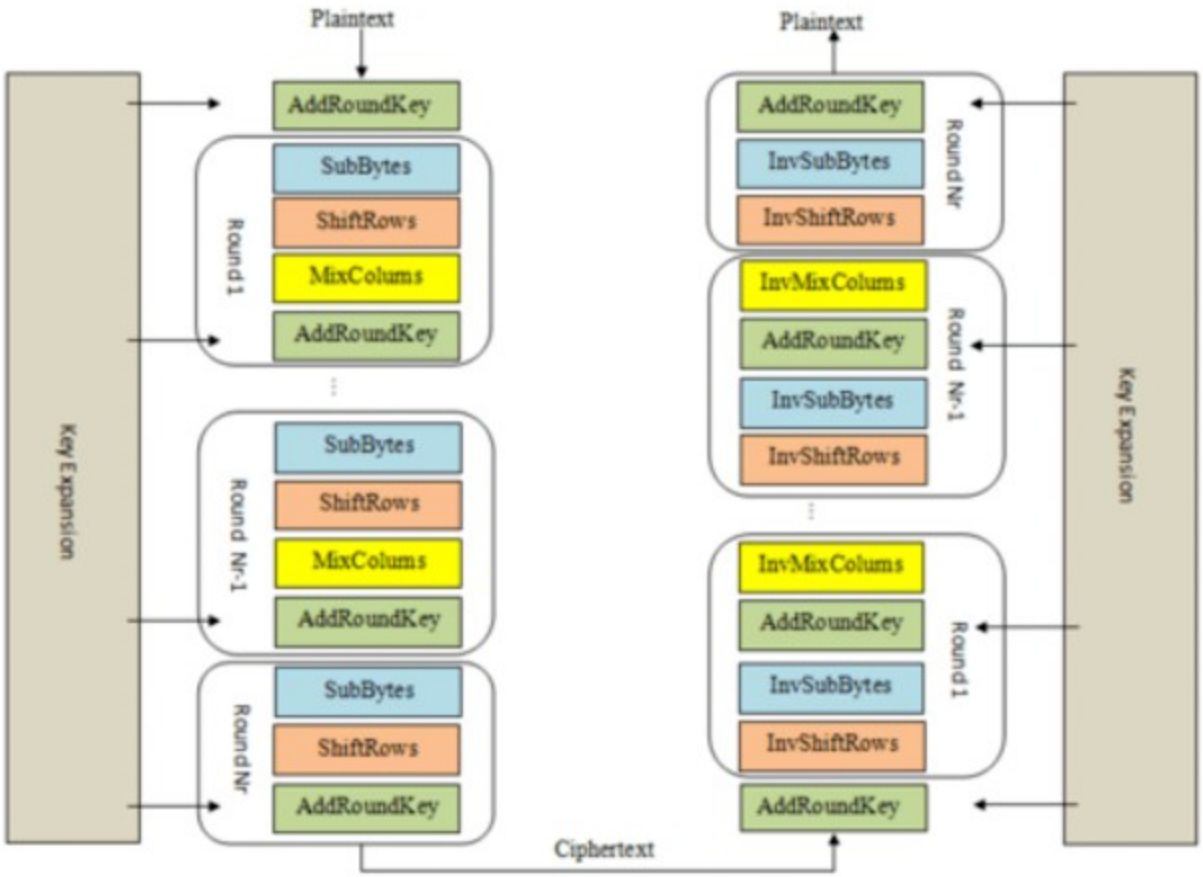
DES has 16 rounds and the blocks are of 64 bits.
But a certain point comes out that it is vulnerable. The easiest known way to build a practical DES Cracker is to have it try every key until it finds the right one and in 1998 the Electronic Frontier Foundation built a computer with special design to crack the key of a DES.

- Software parallelize searching for a DES key
  - A single DES-Cracker chip could find a key by searching for many years
  - A thousand DES-Cracker chips can solve the same problem in one thousandth of the time
  - The actual machine EFF built contains about 1,500 chips
- **It took** the EFF DES Cracker **<3 days to find a 56-bit key by searching** a total of 17,902,806,669,197,312 keys (**an average of 88 billion keys per second**)
- **Cost** of the project: **< $250,000** ($80,000 for design, integration, and test; $130,000 for hw)
- SW developed in 4-5 weeks; entire project completed within about eighteen months

So the use of DES started to be deprecated, because keys of 54 bits started to be not long enough.

In order to find a cryptography algorithm that is better than DES, in 1997, NIST initiated a public, 4.5 year process to develop a new secure cryptosystem for U.S. government applications (as opposed to the closed process in the adoption of DES 25 years earlier).
Result was the **Advanced Encryption Standard** (AES) that became the official successor to DES in December 2001.
AES uses a symmetric key crypto scheme called Rijndael, a block cipher designed by Belgian cryptographers Joan Daemen and Vincent Rijmen.
The algorithm can use a variable block length and key length; the latest specification allowed any combination of keys lengths of 128, 192, or 256 bits and blocks of length 128, 192, or 256 bits (lesson learnt from DES cracking, so when 128 will not be sufficient, we will pass to 192 etc). This is very important to extend the lifetime of this standard (the larger the keysize is, the better it is).

The encryption process is based on a series of table lookups and XOR operations which are very fast operations to perform on a computer.
Decryption of a ciphertext consists of conducting the encryption process in the reverse order. However, unlike for a Feistel Cipher, the encryption and decryption algorithms do have to be separately implemented, although they are very closely related.



AES is the cryptographic algorithm for use by U.S. government organizations to protect sensitive, unclassified information (commercial and other non-federal organizations are invited-but not required-to adopt and implement DES)
No one can be sure how long the AES-or any other cryptographic algorithm-will remain secure. However, NIST's DES was a U.S. government standard for approximately 20 years before it became practical to mount a brute force attack with specialized hardware. The AES supports significantly larger key sizes than what DES supports. Barring any attacks against AES that are faster than key exhaustion, and even with future advances in technology, AES has the potential to remain secure well beyond 20 years.
There are on the order of 10^{21} times more AES 128-bit keys than DES 56-bit keys

Asymmetric key cryptography

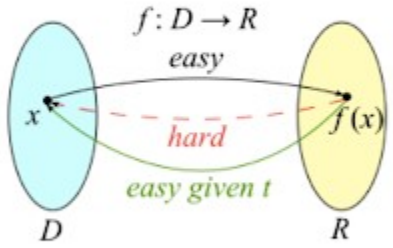
Asymmetric key cryptography (also called public key cryptography PKC) uses two different keys, one for encryption and one for decryption.
Asymmetric key cryptography is the most significant development in cryptography in the last 300-400 years.
The two keys are called **public key** and **private key**, and they can be used both for encryption and the other for decryption (the choice depends on the use case, one guarantees confidentiality and the other guarantees integrity).

This type of cryptography is the most significant development in cryptography in the last 300-400 years.
Two-key crypto system in which two parties could engage in a **secure communication over a non-secure communications channel without sharing a secret key**.

Obviously PCK functions are **trapdoor one-key functions**.

known, e.g., a suitable key. We call this kind of functions **trapdoor one-way functions**

- Examples
  - *Multiplication vs. factorization*
    - Given prime numbers, say 3 and 7, it is easy to calculate the product
    - Given number 21 that is a product of two primes, it is not easy to determine the prime factors
    - Problem becomes much harder if we start with primes that have, say, 400 digits or so, because the product will have ~800 digits
  - *Exponentiation vs. logarithms*
    - Take the number 3 to the 6th power; it is relatively easy to calculate $3^6 = 729$
    - Start with number 729; it is difficult to determine the two integers, x and y s.t. $\log_x 729 = y$
  - **With suitable parameters**, these problems are a basis for many cryptographic algorithms
  - However, **not all instances of these problems are difficult to solve**
- Example
  - 6,895,601 is the product of two prime numbers
  - What are those numbers?
  - A brute force solution is to try dividing 6,895,601 by several prime numbers until finding the answer
  - If one is told that 1,931 is one of the numbers, one can find the answer by computing
    - 6,895,601 ÷ 1,931 = 3,571
  - While computers can guess all of the possible answers within a second, by considering larger and larger primes, inverting similar functions becomes more and more computationally expensive
  - Indeed, the problem is **believed** to be difficult from a computational point of view
    - This means that there is no mathematical proof that it is indeed computationally hard



The basic idea of PKC is that we have two different keys, one public and one private, that are mathematically related, and we use one to encode and the other to decode.
The two keys are mathematically related, **although knowledge of one key does not allow someone to easily determine the other key**.

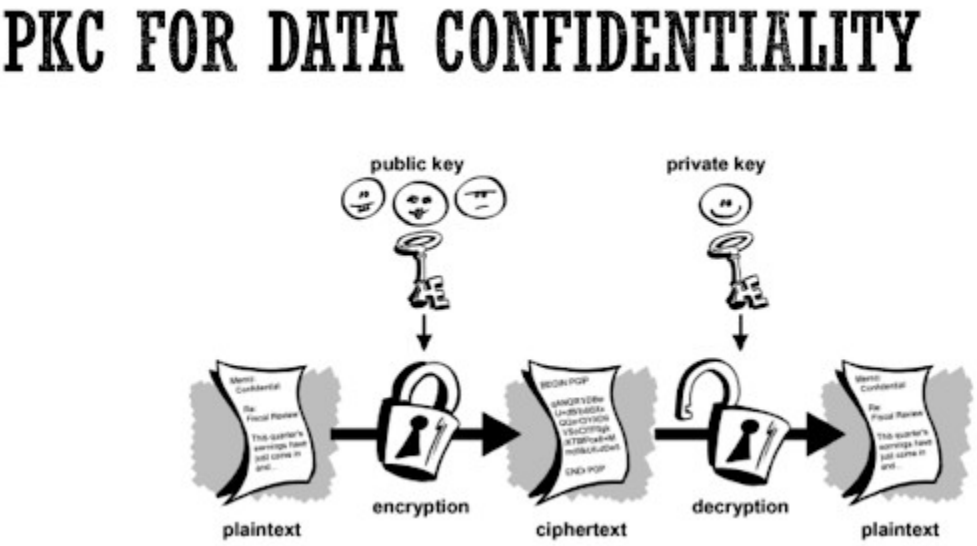


It does not matter which key is applied first, but that both keys are required for the process to work.

This method overcomes the problem of key distribution, since one of the keys is designated the public key and may be advertised as widely as the owner wants.
The other key is designated the private key and is never revealed to another party.

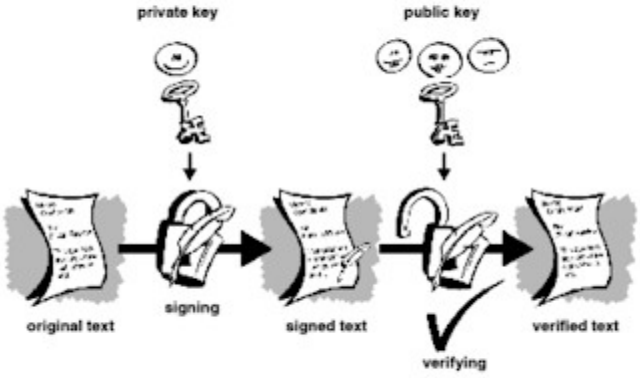
- How to send a message under this scheme: suppose Alice wants to send Bob a message
  - Alice encrypts some information using Bob's public key
  - Bob decrypts the ciphertext using his private key
- Method could be also used to prove who sent a message:
  - Alice could encrypt some plaintext with her private key
  - When Bob decrypts using Alice's public key, he knows that Alice sent the message (*authentication*) and Alice cannot deny having sent the message (*non-repudiation*)

So, if we use the public key to encrypt and the private key to decrypt, we provide **confidentiality**:



And if we use the private key to encrypt and the public key to decrypt, we provide **integrity**, **authentication** and **non-repudiation**:

PKC AND SIGNATURES



- Only sender could have signed
- Like handwritten signature… only more difficult to fake

A digital signature gives a recipient reason to believe that
- a message was created by a known sender (authentication),
- the sender cannot deny having sent the message (non-repudiation), and
- the message was not altered in transit (integrity)

S. Ranise - Security & Trust (FBK)

PKC is not easily implemented in hardware and it is usually implemented in software. Another point is that PKC cannot encrypt large messages (due to the cost of the algorithms).

Since the symmetric cryptography is easy to compute but we have the problem of sharing the key, and the asymmetric cryptography is good for small messages, we can combine this two techniques, sharing the key with asymmetric cryptography and then the messages with symmetric cryptography.

RSA (Rivest, Shamir, Adleman) is the first PKC algorithm, used in hundreds of software products and can be used for key exchange, digital signatures, or encryption of small blocks of data.
Mathematical "trick" of RSA: relatively easy to compute products compared to computing factorizations. RSA uses a variable size encryption block and a variable size key.

- Key-pair is derived from a very large number, n, that is the product of two prime numbers chosen according to special rules
  - Primes may be 100 or more digits in length each, yielding an n with roughly twice as many digits as the prime factors
  - **An attacker cannot determine the prime factors of n (and, therefore, the private key) from this information alone and that is what makes the RSA algorithm so secure**
  - The ability for computers to factor large numbers, and therefore attack RSA scheme, is rapidly improving and systems today can find the prime factors of numbers with more than 200 digits
  - Nevertheless, if a large number is created from two prime factors that are roughly the same size, there is no known factorization algorithm that will solve the problem in a reasonable amount of time; a 2005 test to factor a 200-digit number took 1.5 years and over 80 years of compute time

ON PKC AND FACTORISATION

addition	multiplication	factoring
Easy		Difficult
123 + 654 -------- 777	123 x 654 --------- 492 615 738 ----------- 80442	221 = ?x? 221/2 = 221/3 = 221/5 = 221/7 = 221/11 = 221/13 = 221 = 13 x 17

A **prime number** is a natural number greater than 1 that has no positive divisors other than 1 and itself.

Fundamental theorem of arithmetic: any integer greater than 1 is either a prime itself or can be expressed as a product of primes that is unique up to ordering.

Slow method of verifying primality of a number n: trial division

- Testing whether n is a multiple of any integer between 2 and $\sqrt{\square}$
- Why? If n is not prime, then there exist a and b s.t. $n = a \cdot b$ and so, obviously, a or b can be at most $\sqrt{\square}$
- Improvement: only prime numbers should be considered

Let's see how RSA works.

First of all you have to choice two large (a lot of digits) prime numbers p and q.

Then you multiplicate them and you obtain $N = p \cdot q$ (this is the **modulus**).

Then you choose a number e that is relatively prime to $(p-1)\cdot(q-1)$.

Then you find the number d that $e \cdot d = 1 \pmod{(p-1)\cdot(q-1)}$ (d is the inverse of $e \mod (p-1)\cdot(q-1)$)

You set the **public key** to (N, e) and the **private key** to d.

To encrypt plaintext M, having $0 < M < N$, you have to compute:

$$C = M^e \pmod{N}$$

To decrypt ciphertext C you have to compute:

$$M = C^d \pmod{N}$$

Since N and e are public, if attacker can factor N, he/she can use it to find d, since $e \cdot d = 1 \pmod{(p-1)\cdot(q-1)}$.

The “security” of this mechanisms is that only N and e is public, and to get d you have to compute the factorization of N, that is very complex to compute.

- For a positive integer n, two integers a and b are said to be *congruent modulo n*, if their difference $a - b$ is an integer multiple of n (i.e., if there is an integer k such that $a - b = kn$)
- In symbols, $a = b \bmod n$
- Ex: 38 = 14 mod 12 since 38 -14 = 2*12
- Two integers a and b are said to be **relatively prime** or **coprime** if the only positive integer (factor) that divides both of them is 1

Modular arithmetic and exponentiation

- Applying exponents in modular arithmetic can be done before or after simplifying:
  $a^k \bmod n = (a \bmod n)^k$
- Ex:
  - $6^2 \bmod 4 = 36 \bmod 4 = 0$
  - $6^2 \bmod 4 = (6 \bmod 4)^2 \bmod 4 = (2)^2 \bmod 4 = 4 \bmod 4 = 0$

57

Recall that
“multiplication of two large primes”
is *believed* to be a trapdoor one-way function

This is so because none has come up with a formal proof but repeated attempts of cryptographers failed to prove the contrary…

EXAMPLE

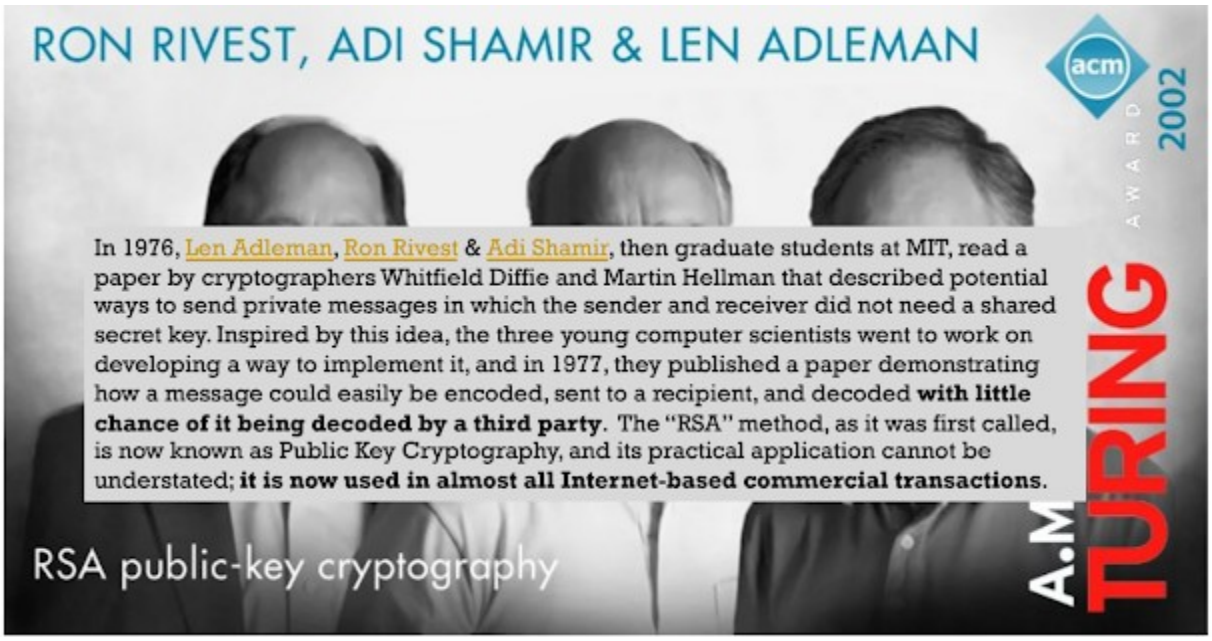
- Pick prime numbers $p=11$ and $q=3$
- Compute $N=p*q=11*3=33$
- Pick $e=3$ as it is relatively prime with both $p-1=10$ and $q-1=2$
- Compute d such that $e*d=1 \bmod ((p-1)*(q-1))$
  - $3*d = 1 \bmod (10*2)$
  - $3*d = 1 \bmod 20$
  - $3*d = 1 + 20*k$ for some k
  - Find d such that $3*d-1$ divides 20
  - By testing $d=1, 2, \ldots$, it is possible to find $d=7$ as $3*7-1=20$
- Set **public** key to $(N,e) = (33,3)$
- Set **private** key to $d=7$

$a = b \bmod c$ iff
there exists k such that
$a=b+c*k$
or, equivalently,
$a-b$ divides c

EXAMPLE (CONT’D)

- Consider plaintext message $M=7$
- Compute the ciphertext $C = M^e \bmod N = 7^3 \bmod 33 = 343 \bmod 33 = 13$
- Check decryption: $M = C^d \bmod N = 13^7 \bmod 33 = 62748517 \bmod 33 = 7$

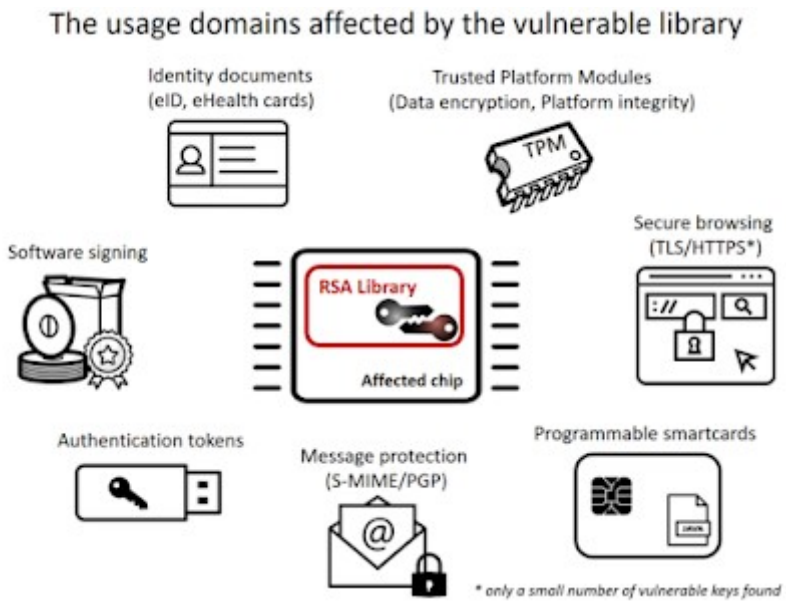
It is possible to avoid the computation of 13^7 by exploiting the following equality:
$a=b*c \bmod n=(b \bmod n)*(c \bmod n) \bmod n$



You have to be very careful in the generation of RSA keys.
For example, a security vulnerability was found in the implementation of RSA keypair generation in a cryptographic library used in a wide range of cryptographic chips produced by Infineon Technologies AG. The product is also integrated in authentication, signature and encryption tokens of other vendors and chips used for Trusted Boot of operating systems. The vulnerability is present in NIST FIPS 140-2 and CC EAL 5+ certified devices since at least the year 2012.

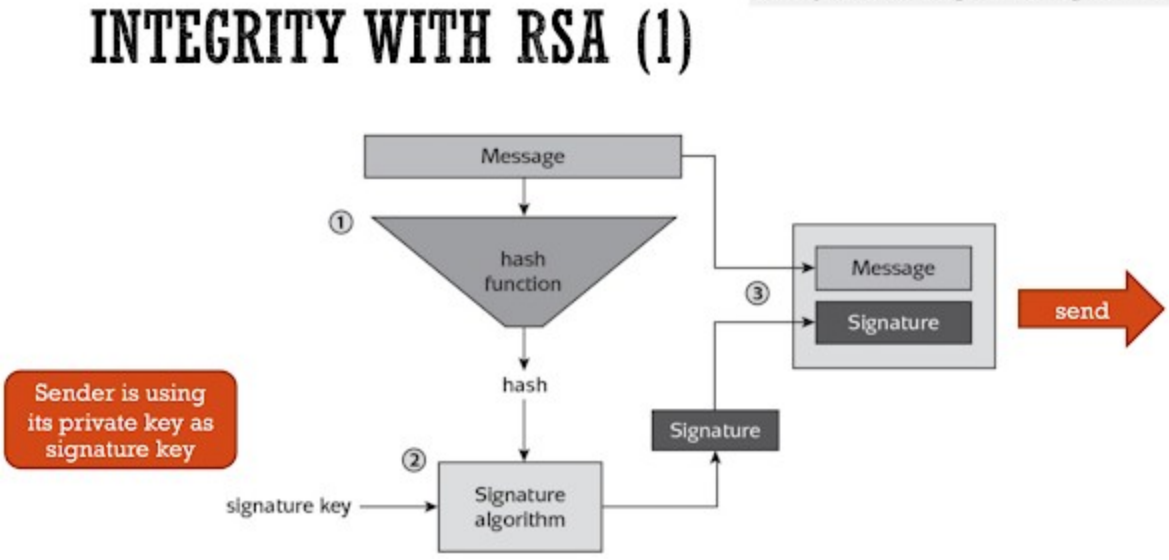
The algorithmic vulnerability is characterized by a specific structure of the generated RSA primes, which makes factorization of commonly used key lengths including 1024 and 2048 bits practically possible. Only the knowledge of a public key is necessary and no physical access to the vulnerable device is required. The vulnerability does NOT depend on a weak or a faulty random number generator. All RSA keys generated by a vulnerable chip are impacted. The attack was practically verified for several randomly selected 1024-bit RSA keys and for several selected 2048-bit keys.

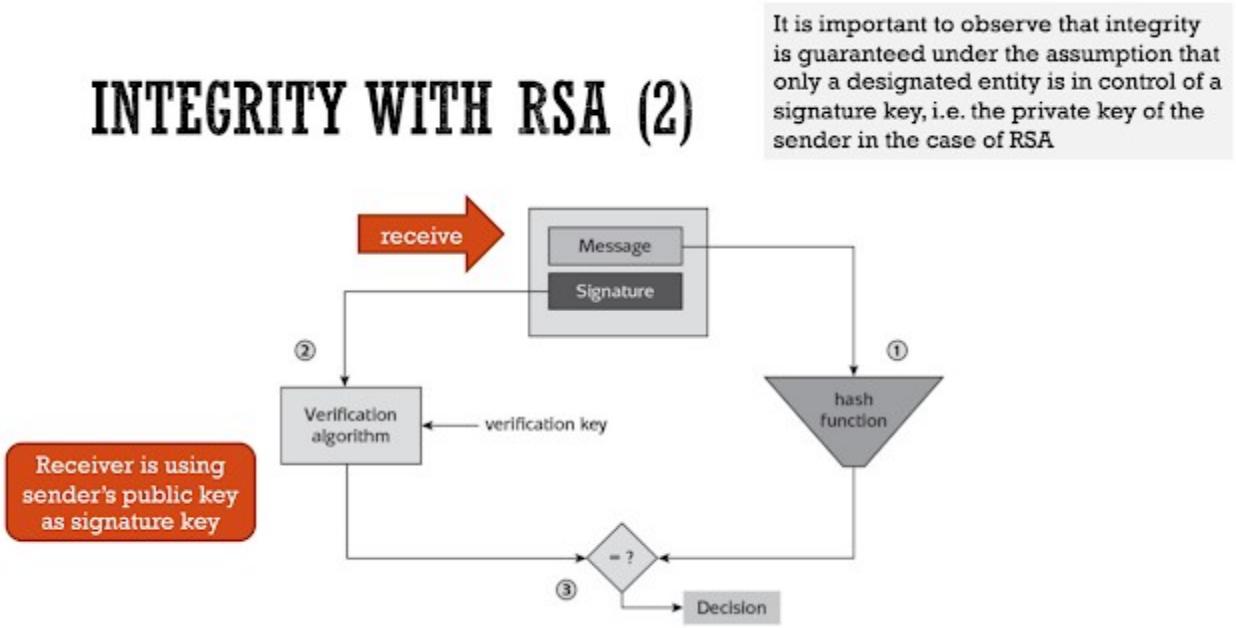
The impact was that a remote attacker can compute an RSA private key from the value of a public key.



Let’s see how integrity (and also authentication and non-repudation) can be implemented using RSA:

The hash function is used mainly because of efficiency reasons since RSA is relatively heavy from a computational point of view

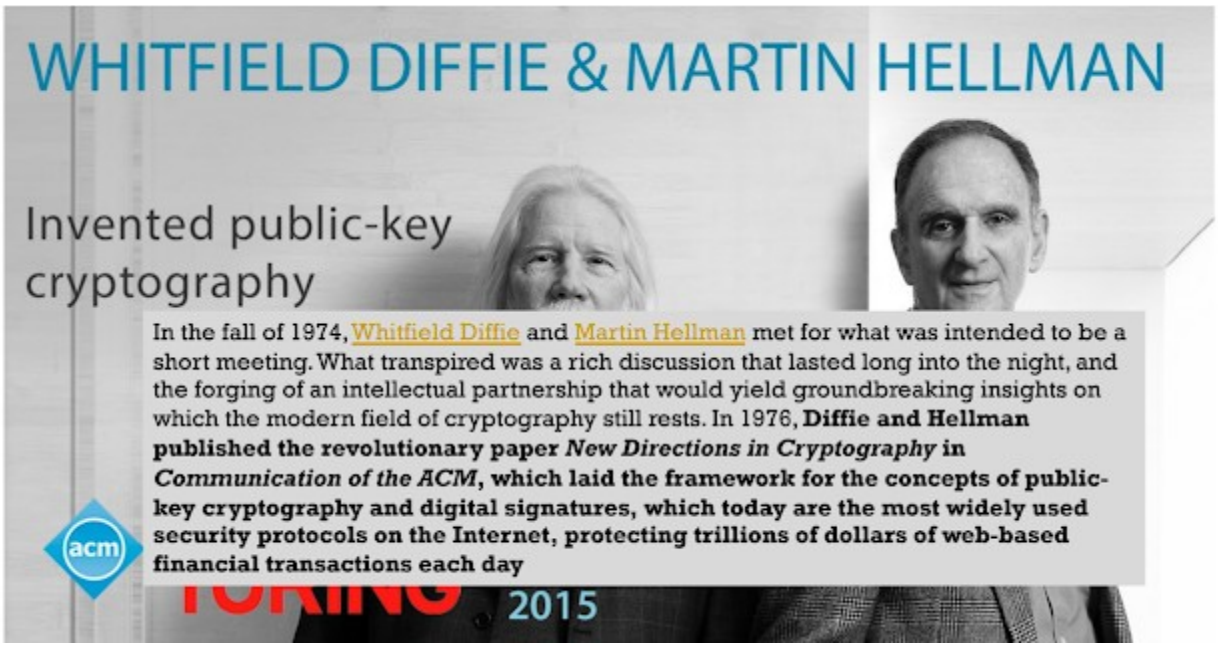




Resuming, public key cryptography is very very expensive in terms of computational cost.
So in practice it is impossible to add confidentiality (encrypting the message with the public key of the receiver) when you want to assure integrity.
The solution is to share only the symmetric key with public key cryptography and then create a secure channel over that symmetric key.
In this way you guarantee the best effort in communication.

RSA + symmetric cryptography is used in TLS. But at a certain point it was discovered that if for some reason one private key is stolen, then the attacker gets access to all the symmetric keys used by that user, and this is a very large problem.

So **DH** (Diffie-Hellman) was invented. After the RSA algorithm was published, Diffie and Hellman came up with their own algorithm.
DH is **used for secret-key key exchange only**, not for authentication or digital signatures.



The mathematical "trick" of DH is that it is relatively easy to compute exponents compared to computing discrete logarithms.

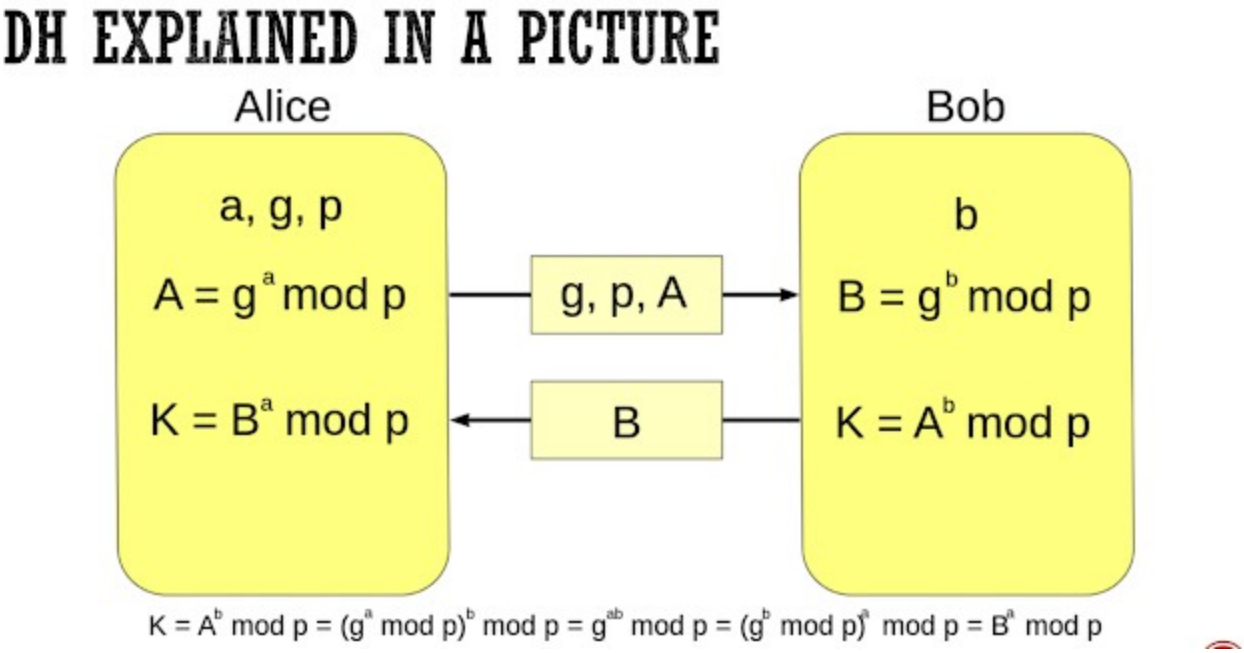
BASIC MATH NOTIONS

- **Finite field F** = GF(p) = Integers modulo prime p
  - GF stands for Galois Field
- Exponentiation $x \rightarrow g^x \pmod p$ is the one-way function
  - This is *computationally easy* to compute
- **Discrete Logarithm Problem (DLP)**
  - Given p, g, X, find the discrete logarithm x so that $X = g^x \bmod p$
  - This is a *computationally difficult* problem to solve for **carefully chosen p, g, X:**
    - p is a prime
    - g is a **generator**, i.e. for 0<X<p there is x s.t. X = $g^x \bmod p$
- Important property: exponentiation and modulus commute

DH allows two parties, say Alice and Bob, to generate a secret key. Exchange information over an unsecure communications channel to perform calculation but eavesdropper, Eve, cannot determine shared secret key based upon this information

So Alice choose a large random number a, that is the **private key of Alice**.
Bob choose a large random number b, that is the **private key of Bob**.
Then Alice computes $A = g^a \ mod \ p$ and sends $A, g \ \wedge \ p$ to Bob that computes $B = g^b \ mod \ p$.
Then Bob sends to Alice B and Alice computes $K_a = g^{a \cdot b} \ mod \ p = B^a \ mod \ p$ and Bob computes $K_b = g^{a \cdot b} \ mod \ p = A^b \ mod \ p$.
In this way an attacker who intercepts $A, g, \ p \ \wedge \ B$ cannot compute K. So a and b are kept secret while A and B are openly shared.

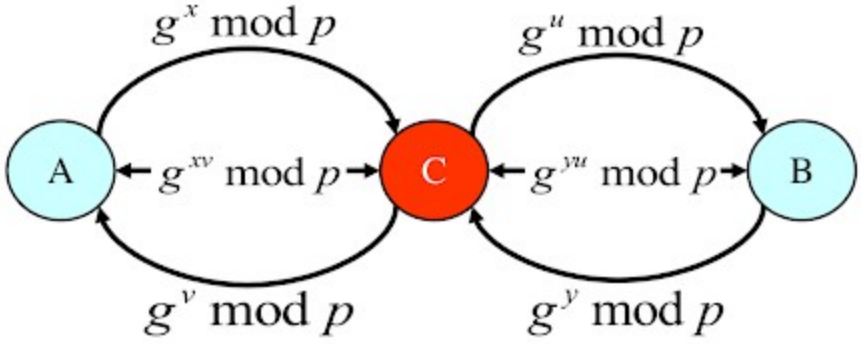
Based on their own private key and the public key learned from the other party, Alice and Bob have computed their secret keys, K_a and K_b, respectively, which are both equal to $g^{a \cdot b} \mod p$



In this way you never transmit the key but you only transmit partial information to calculate the key.

The security of DH depends on the difficulty of the DLP. The security of the DH protocol is not known to be equivalent to the DLP.
DH is a key agreement protocol.
Secrecy: assuming that DLP is difficult, an attacker observing the messages exchanged does not learn the key

One issue is that **DH does not provide authentication**, so parties do not know whom they are establishing a key with.
So an attacker C can mount a **MITM attack** (Man In The Middle):



Let's now denote the takeaways of this chapter:

- Avoid to design your own crypto algorithms (Amateur designs are usually broken quite easily)
- DES should no longer be used
  An extension of DES (Triple DES using 3 keys and invokes 3 times DES) still used in the financial sector
- Recommended key length for block cypher: at least 80/90 bits (use AES)
- No provable security for DES and AES
  Only resistant to known attacks (aiming to reduce the search space of a brute force attack)
- Don't forget key management!
  - How to share the key? Answer: key distribution techniques
  - How to keep the key secret? Answer: use access control techniques
  - With n participants, n^2 keys are needed
- PKC ≠ RSA: some specific RSA properties do not hold for any PKC system
- Provable security by reduction proofs to open problems: factoring or DLP
- Concrete security depends on the state of the art in solving those problems
- DH: only used for key exchange but problems with authenticity (beware of MITM attack!)
- Anthropomorphic metaphors may mislead: stories about Alice & Bob create the illusion that you talk about persons rather than keys

RECAP QUESTIONS

- What is a cryptosystem?
- Why key management is crucial for cryptography?
- What does it mean for a cryptographic technique to be computationally secure?
- What are substitution and transposition ciphers? Given an example for each one.
- What is symmetric cryptography?
- What are block and stream ciphers?
- Why is DES deprecated? Why is AES still used?

RECAP QUESTIONS-CONT'D

- What is asymmetric (or public key) cryptography?
- What are the main advantages and disadvantages of symmetric and asymmetric cruptography?
- What is RSA and for what it can be used?
- What is Diffie-Hellman and for what it can be used?
  - What is a possible attack to Diffie-Hellman?
- What is a one-way function?
  - What is the one-way function used in RSA?
  - What is the one-way function used in Diffie-Hellman?

RELATIVE SECURITY PER KEY SIZE

Method	Date	Symmetric	Factoring Modulus	Discrete Logarithm Key	Discrete Logarithm Group	Elliptic Curve	Hash
Lenstra / Verheul	2021	86	1937 1536	153	1937	163	172
Lenstra Updated	2021	82	1416 1614	164	1416	164	164
ECRYPT	2018 - 2028	128	3072	256	3072	256	256
NIST	2019 - 2030	112	2048	224	2048	224	224
ANSSI	2021 - 2030	128	2048	200	2048	256	256
NSA	-	256	3072	-	-	384	384
RFC3766	-	-	-	-	-	-	-
BSI	2020 - 2022	128	2000	250	2000	250	256

All key sizes are provided in bits. These are the minimal sizes for security.

Cryptography at work

Public Key Cryptography reloaded

Let's recap the pros and cons of PKC.
The pros are:

- Private key is only known by the owner (less risk)
- Confidentiality by encrypting with Receiver's Public key
- Non-Repudiation by encrypting with Sender's Private key

The cons are:

- Algorithms are 2–3 orders of magnitude slower than those for symmetric encryption (typically used in an initial phase of communication and then secret keys are generated for bulk encryption)
- How are Public keys made available to other people?

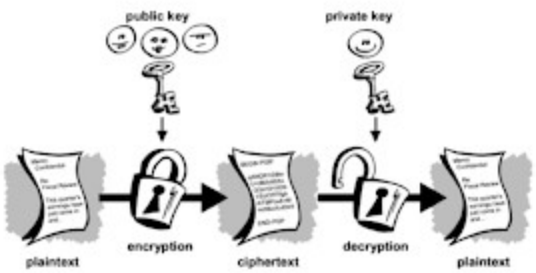
- There is still a problem of **Authentication**!!! (Who ensures that the owner of a key pair is really the person whose real life name is “Alice”?)

So we want to build up an infrastructure that allows us to share the public key and guarantee that that public key is associated with us.

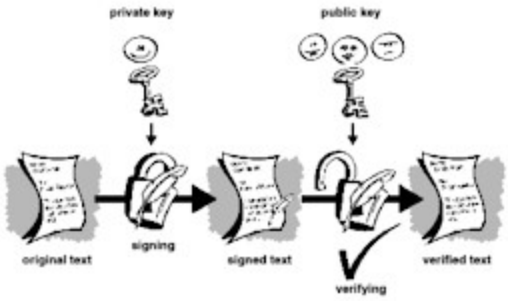
To provide integrity, authentication and non-repudiation **Digital signature** was invented.
Digital signature is a data item that **vouches the origin and the integrity** of a message.
The use of digital signature is this:

- The originator of a message uses the Private Key to sign the message and sends it together with its digital signature to a recipient
- The recipient uses the Public Key of the sender to verify the origin of the message and that it has not been tampered with while in transit

RECALL PKC

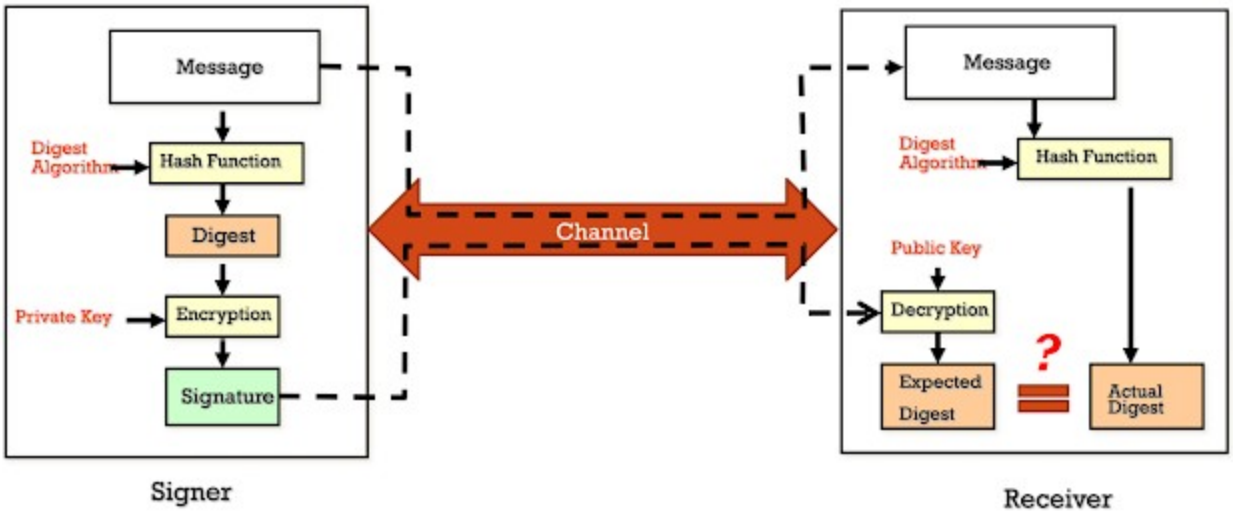


Confidentiality



Integrity

IN REALITY...

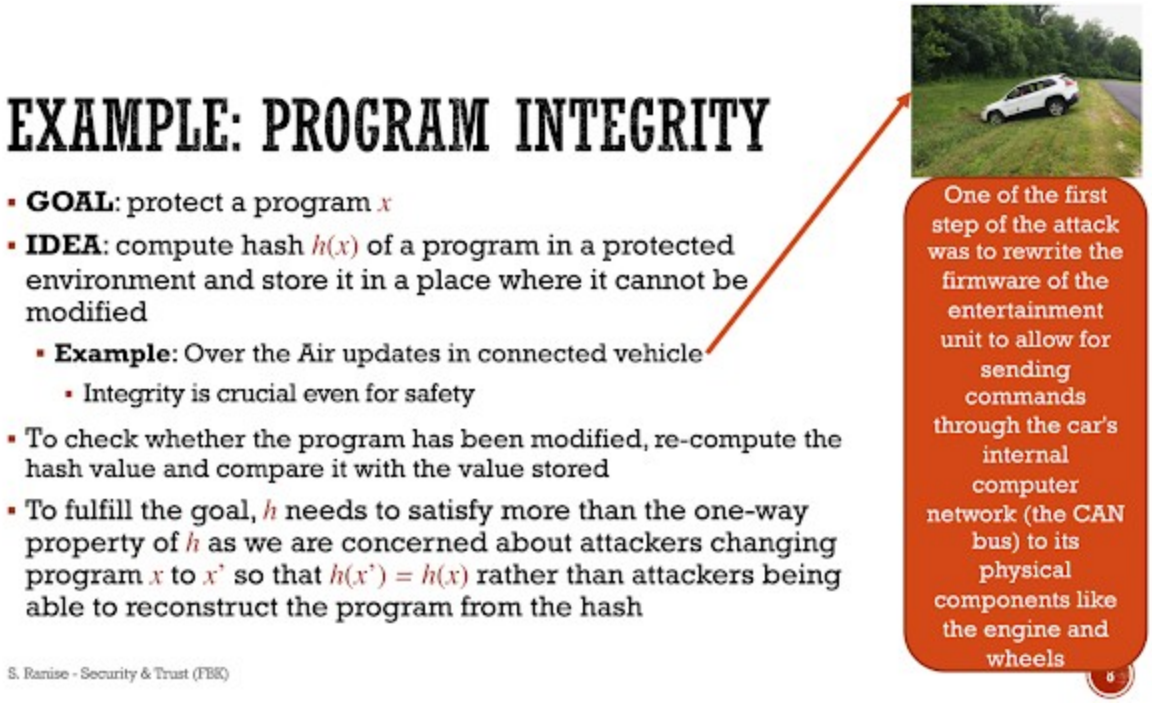


Signer

Receiver

First we use the hash function to greatly reduce the size of the message to a fixed size, in order to apply PKC on a small piece of information (since it is very costly computationally).

For example the authentication (and integrity) is fundamental in updating program “over the Air” (because if your update is changed by an attacker there could be big problems):



But digital signatures has still some problems:

- The usage of some hash functions is deprecated because they are considered weak. Even assuming hash functions are not a source of security issues...
- There is still a problem linked to the “Real Identity” of the signer (Why should I trust who the Sender claims to be?)

In other words, who guarantees that the one using the key is entitled to do so?

Public Key Infrastructure

The **public key infrastructure** is an attempt to solve the main problem underlying the digital signatures (the trusting of the possessor of the keys).

The **main requirement** on a public key is to be **authentically bound** to the identity of the party controlling the corresponding private key.
Not fulfilling this leads to impersonation attacks, for example:

- Party A might receive a digital signature on message M supposedly generated by party B using its private key but where the signature is generated by party C.
- Party A would verify the signature using C’s public key thinking it is using B’s key.

Another requirement is that parties relying on the correctness of the binding between public keys and corresponding party identities can be assured that the binding is still valid.

Satisfying these two requirements is typically done by setting up a **Public Key Infrastructure**.

For binding public keys and entities, the initial proposal is to have certificates as **bulletin boards**, where public keys and associated identity are listed.



Obviously this requires trust in the provider of the bulletin board service, and other problems are the scalability and the single point of failure.

A non-scalable and inflexible approach adopted in mobile and Internet of Things applications is to **hardcode the public key in the software** of the party.
Main security issues are to protect the stored public key from tampering attempts during software updates or substituting in the code repository of the software provider…

CWE™ (Common Weakness Enumeration) is a community-developed list of software and hardware weakness types. https://cwe.mitre.org/index.html

CWE-321: USE OF HARD-CODED CRYPTOGRAPHIC KEY

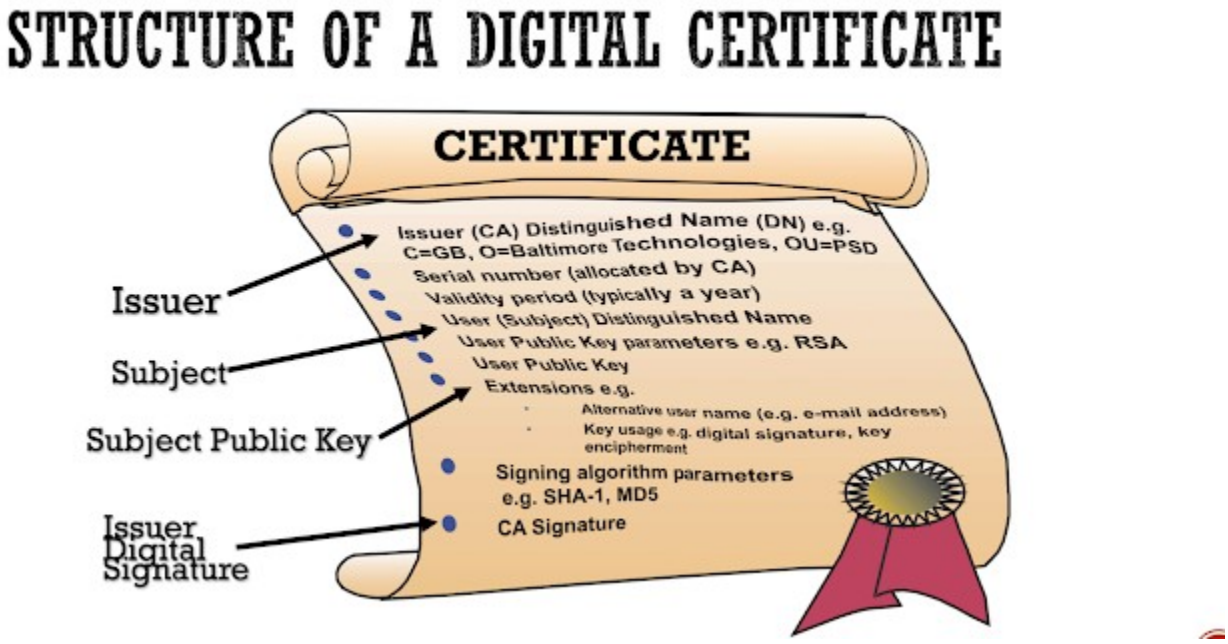
- The use of a hard-coded cryptographic key significantly increases the possibility that encrypted data may be recovered.
- Example

```
int VerifyAdmin(char *password) {
if (strcmp(password,"68af404b513073584c4b6f22b6c63e6b"))
{
   printf("Incorrect Password!\n");
   return(0);
}
printf("Entering Diagnostic Mode...\n");
return(1);
}
```

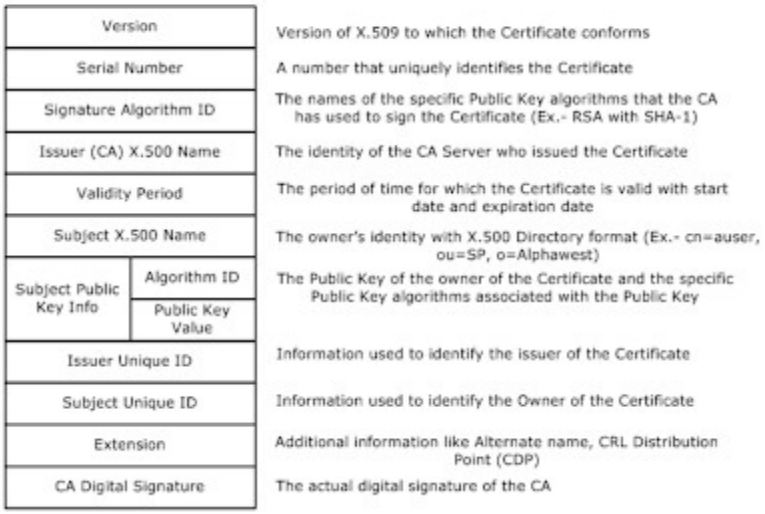
Widely adopted solution for Internet applications and services is the use of **digital certificates**, i.e. digital objects containing data including the identity and the public key that are signed by a **Trusted Third Party** (TTP) attesting the correctness of the binding between the identity and the public key.

Notice that **this moves the problem** of checking the correctness of the binding between identity and public key **to the distribution and keeping up to date the TTP's public key to check the authenticity of the certificate**.
This problem can be deferred several (but finitely many times) and thus building a certificate chain whose trustworthiness must be led back to a root certificate that must be self signed.

So a **digital certificate** is a binding between an entity's Public Key and one or more Attributes concerning its Identity (the entity can be a Person, a Hardware Component, a Service, etc.)
A Digital Certificate is issued (and signed) by someone. Usually the issuer is a Trusted Third Party (TTP) also called **Certificate Authority** (CA). Indeed, TTPs should be trusted, i.e. we should trust the digital certificates they are issuing.
So we transform the problem of binding the identity of an entity to a particular public key to the problem of understanding if the key used to sign that particular certificate is really associated with that particular Certificate Authority. So we need another certificate to do that. So we risk to enter into an infinite loop, and we have to avoid it.
In order to avoid it, typically, the public infrastructure that is used on the internet hardcodes the public key of trusted certification authority in the code of the browser (there are about 100 of them), or directly in the operating system.



X.509: A STANDARD FOR CERTIFICATES

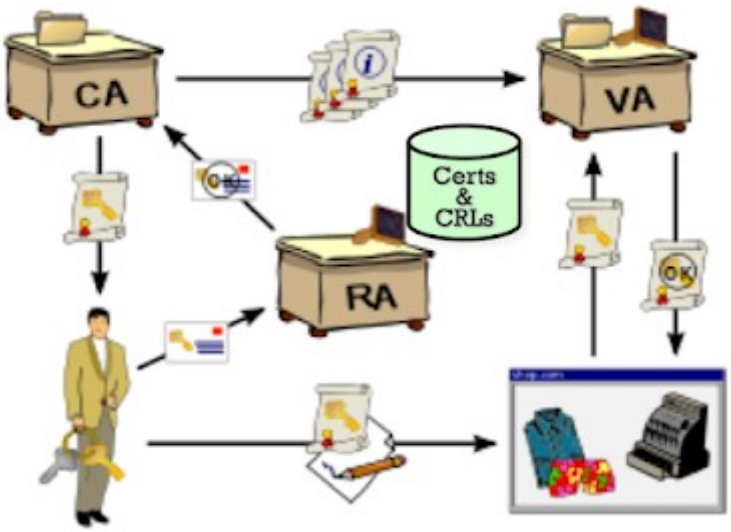


But this is the “final product” of digital certificates. We have some outstanding points:

- How are Digital Certificates Issued?
- Who is issuing them?
- Why should I Trust the Certificate Issuer?
- How can I check if a Certificate is valid?
- How can I revoke a Certificate?
- Who is revoking Certificates?

The answer to all these questions is **Public Key Infrastructure** (PKI).

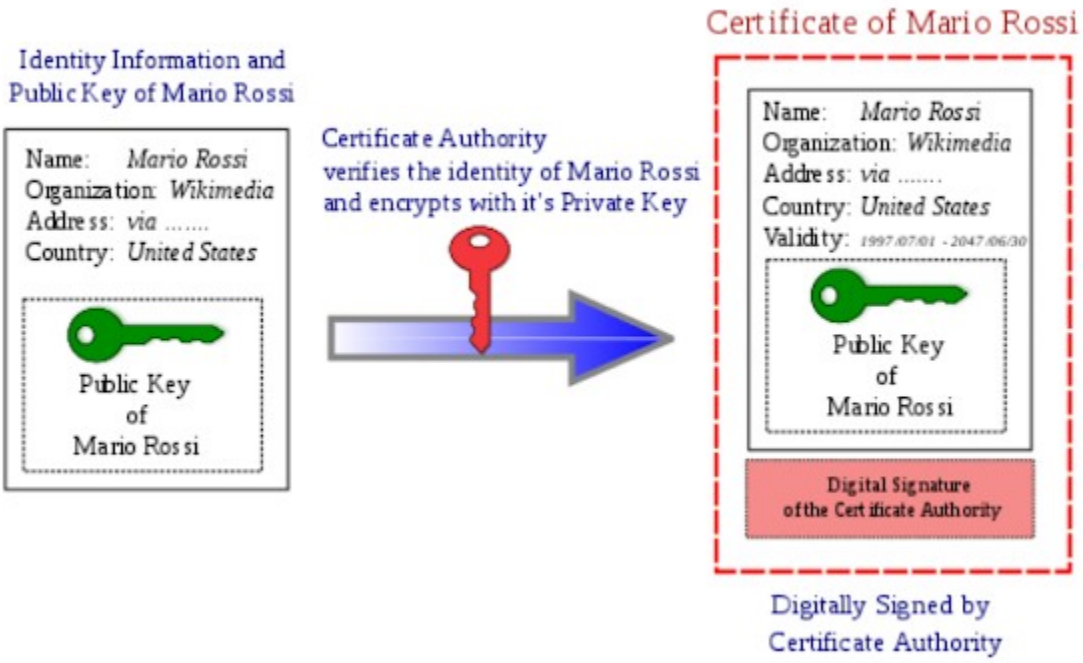
A **PKI** is an arrangement that binds public keys with respective identities of entities (like people and organizations). An entity must be uniquely identifiable within each CA domain on the basis of information about that entity.
The binding is established through a process of registration and issuance of certificates at and by a **Certificate Authority** (CA) (Automated or manual process depending on the level of assurance).
The PKI role that assures valid and correct registration is called a **Registration Authority** (RA). RA is responsible for accepting requests for digital certificates and authenticating the entity making the request.
A third-party **Validation Authority** (VA) can provide entity information on behalf of the CA.
Certificate distribution system = repository for certificates (e.g., LDAP) + Certificate Revocation List (CRL).
CRL is a sort of black list (if a web server distributes malware will be put in CRL).



Summary:

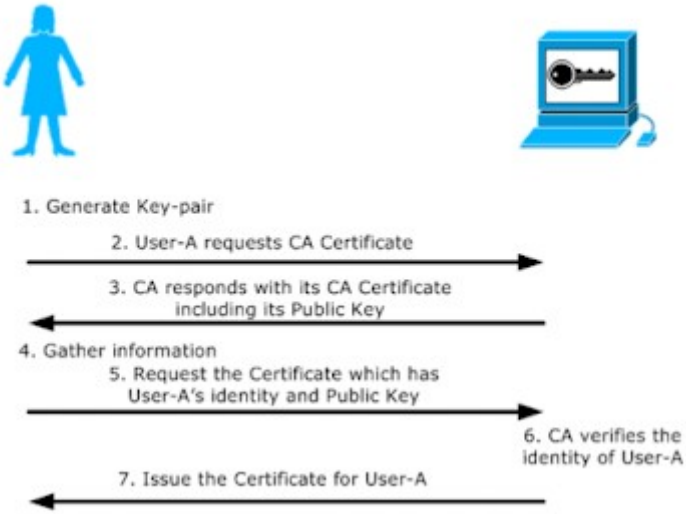
- PKI is the system that provides for a TTP to vouch for user identities and allows binding of public keys to subjects
- The root Certificate Authority (CA) is the pinnacle of trust and the most significant element in the CA hierarchy. The root CA authorizes subordinate CAs (The subordinate CA is responsible for issuing certificates)
- The Registration Authority (RA) verifies information in a certificate request. Certificates are not issued until the RA validates the information
- The Cryptographic Practices Statement (CPS) is a declaration of the security that the organization is implementing for all certificates issued by the CA holding the CPS. This statement tells potential partners or others relying on the security of the PKI system how well the security of the PKI system is being managed
- The Certificate Revocation List (CRL) is a published list of certificates that have been revoked (that are no longer valid)

So the CA will digital sign the certificates (that is the identity information and the public key of the entities):



The procedure of obtaining a certificate is this:

1. User-A generates a Public and Private key-pair (or is assigned a key-pair by some authority in their organization)
2. User-A first requests the certificate of the CA Server (to trust it)
3. The CA responds with its Certificate including its Public Key and its Digital Signature signed using its Private Key
4. User-A gathers all information required by the CA Server to obtain its certificate (e.g., User-A email address, fingerprints, etc.), that the CA needs to be certain that User-A claims to be who she is
5. User-A sends a certificate request to the CA consisting of her Public Key and additional information. This certificate request is signed by User-A's Private Key.
6. CA gets the certificate request, verifies User-A's identity and generates a certificate for User-A, binding his identity and his Public Key. The signature of CA verifies the authenticity of the Certificate
7. CA issues the certificate to User-A



Step 5 is also called the step in which a **Certificate Signing Request** (CSR) is generated. The CSR contains:

- Information identifying the requester which must be signed using the requester's private key (recall that the requester has preliminary generated a key pair)
- The public key chosen by the requester
- Additional proofs of identity required by the certificate authority (the certificate authority may contact the applicant for further information)
- A digital signature on the certification request information generated with the requester's private key

The signature by the requester prevents an entity from requesting a bogus certificate of someone else's public key (notice that the requester's private key is needed to produce, but it is not part of, the CSR).

The requirements on PKI are:

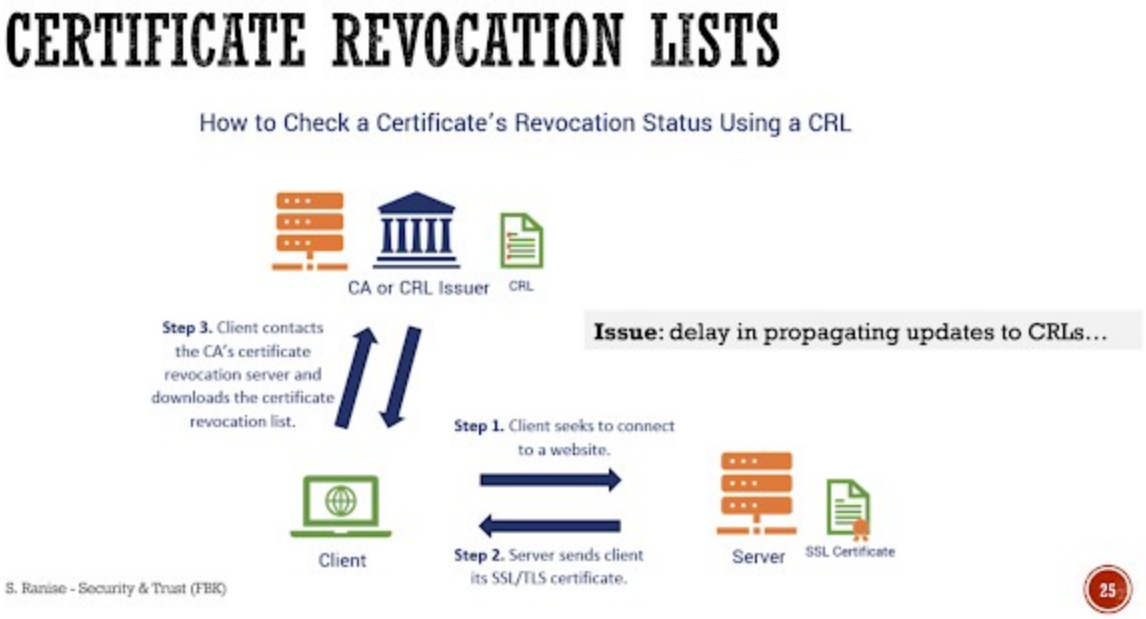
- Standardized naming mechanism for parties
- Parties should be able to prove to TTP that they own a given identity
- TTPs need to check that parties own a given identity
- TTPs must be trustworthy in issuing certificates only to the "right" parties

The last three points above are subject to law and regulations, since in the past, TTPs were involved in incidents resulting in certificates issued to the "wrong" parties (as a result, Google and other companies launched an initiative for building a platform for checking "certificate transparency" that is able to monitor and audit certificates).

- Relying parties shall be able to check the time validity window against available time sources to avoid that an attacker can use an expired certificate whose associated private key he or she has compromised.
- Relying parties need reliable and timely sources of information about the status of the certificates (is the certificate still valid or has it been revoked for some reason?)

Typically this is done by either **distributing lists of revoked certificates** (called Certificates Revocation Lists, CRLs) or the relying parties with the TTP that the certificate that it intends to use is still valid (this is done via the Online Certificate Status Protocol, OCSP).

CRLs perform the validity check locally (the client downloads the certificate revocation list) but there can be time windows in which the checks are not updated because of the synching time required to distribute updated versions.
OCSP requires high bandwidth availability because checks are not performed locally but the checks are always up to date.



The last (but not least) requirement is that software at relying party validating certificates shall work correctly. Given the complexity of the X.509 standard, this is far from obvious.
The RFC of X.509 standard is more than 150 pages long specifying complex data structures and encoding languages! Not surprisingly, several different bugs have been found in software implementations.

So, software implementing cryptographic primitives shall be updated according to the latest known vulnerabilities.
For example, vulnerabilities in SHA1 were found already in 2005 and cryptographers started to deprecate its usage.
In 2017, in fact, public collisions were made available and confirmed in 2019.
Despite the early warning more than 10 years before, only in 2017 web browsers stopped using SHA1 although certificates using SHA1 are still in usage today in payment systems!

SSL & TLS 1.2 and 1.3

The **Secure Socket Layer** (SSL) was developed by Netscape in the mid 1990s, with the purpose of guaranteeing secure communication between the client and the server on the web (SSLv2 now deprecated, SSLv3 still widely supported).

The **Transport Layer Security** (TLS) is the IETF-standardized version of SSL. TLS is now the standard for Internet security. The design is the same as the SSL protocol, but it uses different algorithms.
The primary goal of the TLS protocol is to provide privacy and data integrity between two communicating applications, in practice, it used to protect information transmitted between browsers and Web servers.

- TLS 1.0 = SSLv3 with minor tweaks, RFC 2246 (1999)
- TLS 1.1 = TLS 1.0 + tweaks, RFC 4346 (2006)
- TLS 1.2 = TLS 1.1 + more tweaks, RFC 5246 (2008)
- **TLS 1.3** = major changes + removed insecure features, RFC 8446 (2018)

Nowadays these protocols are implemented in every browser (TLS 1.3 is the most widely used). These protocols use the certificate mechanisms and the Public Key Infrastructure that we've seen before.

A practice example of the use of these protocols are the padlock near the URL in the browser.



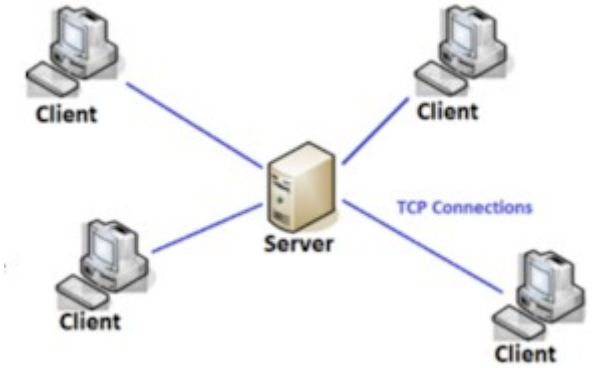
This padlock provides the user with identity of page origin and indicates to the user that page contents were not viewed or modified by a network attacker.
But notice that this doesn't imply immediately that that server is the server the user is willing really to contact (attacker has developed several different strategies to trick the user).
HTTPS provides:

- **authentication** of the website and associated web server with which one is communicating (with protection against man-in-the-middle attacks)
- bidirectional **encryption** of communications between a client and server, with protection against eavesdropping and tampering (or forging) with the contents of the communication

Nowadays there are automated ways to obtain a certificate (that are also free), and obviously also the verification and validation of the certificate is automated done by the browsers (but as we've already said, this not guarantees that the server is really the server that the user is willing to contact).

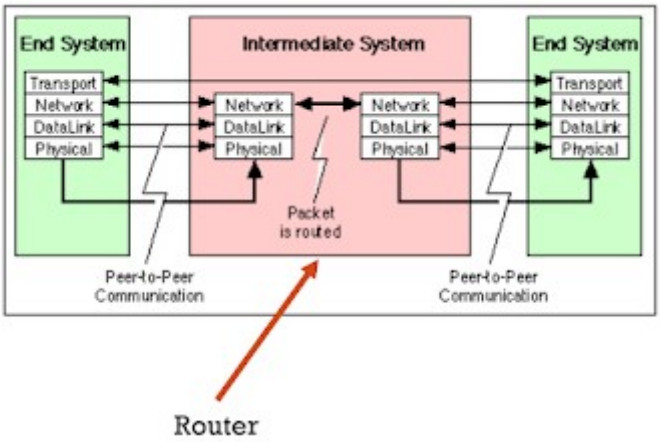
The TLS is implemented in a **client-server architecture** and runs over **TCP**.
A client-server architecture is an architecture of a computer network in which many clients request and receive service from a centralized server:

- Clients provide an interface to allow a computer user to request services of the server and to display the results the server returns
- Servers wait for requests to arrive from clients and then respond to them (servers typically provide a standardized transparent interface to clients so that these need not be aware of the specifics of the system that is providing the service)



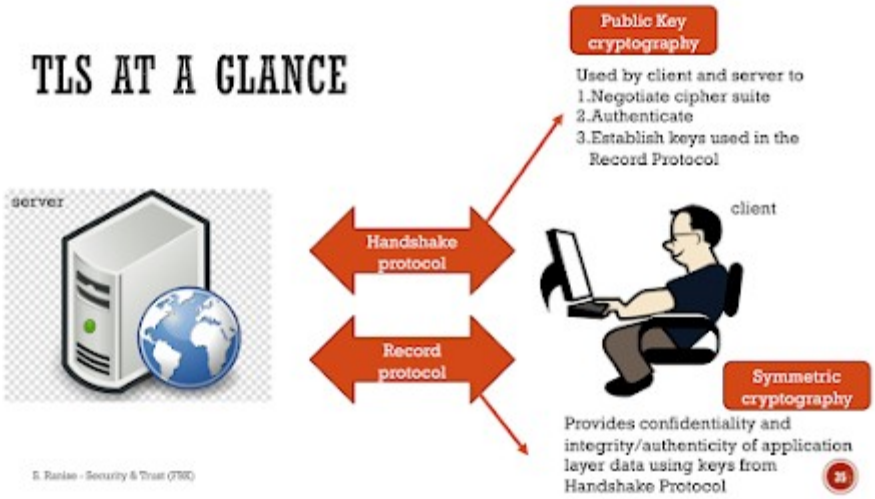
The **Transmission Control Protocol** (TCP) is a transport layer protocol which establishes a two-way connection between a server and a single client; it provides reliable byte stream transmission of data (in the sense of safety not security, i.e. it is able to handle accidental errors not those induced by an attacker with a strategy) with error checking and correction, and message acknowledgement.

A client or server application interacts directly with a transport layer protocol to establish communication and to send or receive information. The transport protocol then uses lower layer protocols to send or receive individual messages (a computer needs a complete stack of protocols to run either a client or a server).
The transport layer protocol provides transparent transfer of data between end systems (also called end points) using the services of the network layer (e.g., TCP/IP).



HTTP works without any cryptography and any integrity check, so anyone can read the messages and the data can be modified without users noticing. For this reason HTTP + TLS (HTTPS) started to be used.

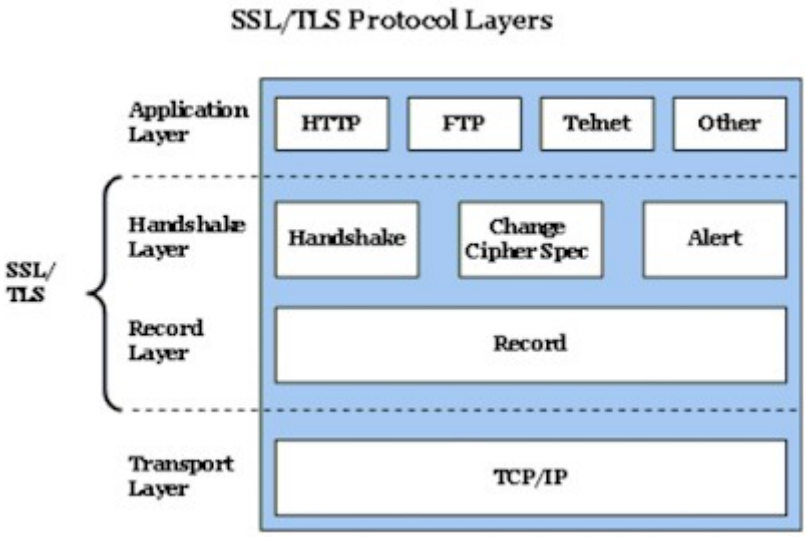
Let’s see of what is composed the TLS protocol:



So TLS consists of two main protocols:

- The **Handshake protocol** (Public Key Cryptography)
  Which use public-key cryptography to establish a shared secret key between the client and the server
- The **Record protocol** (Symmetric Cryptography)
  Which use the secret key established in the handshake protocol to protect communication between the client and the server

This division is made for the reason that Public Key Cryptography, as we've already seen, is inefficient for large data.



The Handshake protocol is used by the client and the server to:

- Negotiate version of the protocol and the set of cryptographic algorithms to be used (there exists interoperability between different implementations of the protocol)
- Authenticate client and server (optional) with the use of digital certificates to learn each other's public keys and verify each other's identity
- Use public keys to establish a shared secret

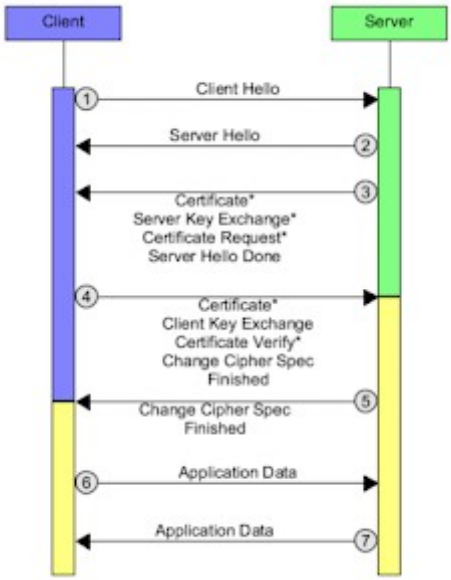
The Record protocol is used to provide Confidentiality and Integrity, using the shared key established in the handshake protocol.
On the send side the data is fragmented into blocks, then authentication and encryption primitives are applied to each block and finally the block is handed to TCP for transmission over the network.
On the receive side the blocks are decrypted, the integrity is verified, the blocks are reassembled and finally delivered to the upper-layer protocol.

Additional protocol used in TLS are:

- TLS Change Cipher Protocol
  It is used to change the encryption being used by the client and server
  It is normally used as part of the handshake process to switch to symmetric key encryption
- TLS Alert Protocol
  The primary use is to report the cause of failure (invalid messages received, messages that cannot be decrypted, connections closed)

TLS 1.2

Let's now see the Handshake protocol (that is how the client and the server agree on communicating in a secure way), step by step.



The **first step** is the Client Hello message, which contains:

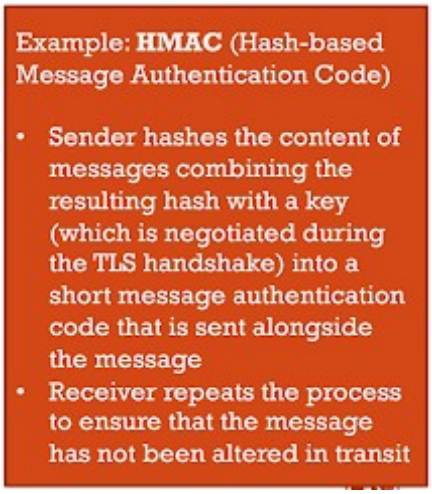
- The version of the protocol that the client wants to use (because often users don't update the browser, so the client won't have the latest version of the protocol)
- The list of supported ciphers suite

A cipher suite is a set of algorithms (cryptographic primitives) that help secure a network connection that uses TLS/SSL.
The cipher suite is a structure describing the algorithms that a machine supports in order for two machines to decide which algorithms to use to secure their connection.

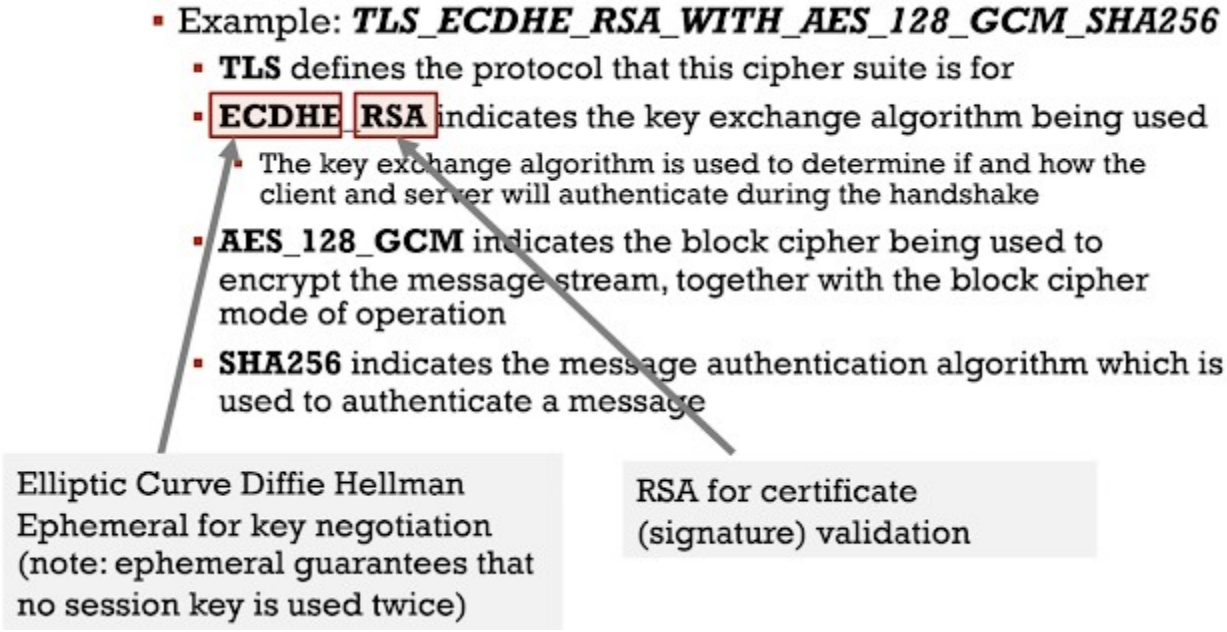
Usually it includes:

- a key exchange algorithm (asymmetric)
- a bulk encryption algorithm (symmetric)
- a message authentication code (MAC) algorithm

A message authentication code (MAC) is a short piece of information used to authenticate a message, i.e. to confirm that the message came from the stated sender (authenticity) and has not been tampered in transit (integrity).



Cipher suites can include signatures and an authentication algorithm to help authenticate the server or the client. Some cipher suites offer better security than others.
To share the cipher suite there exists a name convention:



We have to specify TLS because there exists other protocols that can be used (for example QUICK by google).
Ephemeral keys are important because if we use the same key for different sessions, an attacker that cracks that key can read all the data transmitted to each session with that key.

The **second step** is the Server Hello message, which contains:

- The chosen protocol (supported by both parties)
- The chosen cipher suite (supported by both parties)
- The session ID (a freshly-generated value that will identify the new session)

A session is a series of interactions between two communication end points that occur during the span of a single connection. Typically, one end point requests a connection with another specified endpoint and if that end point replies agreeing to the connection, the end points take turns exchanging commands and data ("talking to each other"). The session begins when the connection is established at both ends and terminates when the connection is ended.

The **third step** is a message send by the server that may contains:

- The Certificate of the server
  If the client requested an authenticated connection, the server must send a certificate (typically in X.509 format)
- Server Key Exchange
  A message that sets the premaster secret that will be later used to generate the Master Secret (the session key) in the case of DH (obviously we can use RSA).
- A certificate request (infrequent, typically only the server provides authentication with certificates)
  A non-anonymous server can send this message if it requires the client to be authenticated

The **fourth step** is a message send by the client that may contains:

- The Certificate of the client (if requested)
  If the client has received a CertificateRequest message, it must send a X.509 certificate
- Client Key Exchange
  A message that sets the premaster secret that will be later used to generate the Master Secret.
- Certificate Verify
  A message that provides explicit verification of the server certificate
- Tell the server that he has finished and he is ready

The pre-master key/secret is the value obtained from the key exchange (when using Diffie-Hellman the pre-master secret is $g^{ab} \mod p$).
Depending on the cryptographic parameters, the size of pre-master keys can vary. To simplify the generation of session keys, it is important to identify a fixed-length value from which to derive what we call master secret/key whose length is fixed and equal to 48 bytes.
The master secret is derived from the pre-master key by using a **Pseudo Random Function** (PRF), i.e. an efficient and deterministic function which returns a pseudorandom output indistinguishable from random sequences (main difference w.r.t. pseudorandom number generators is that they can accept any input data).

From the master secret both client and server derive multiple session keys by using again the PRF: half of the session keys is used for message integrity and the other half for message confidentiality.
Principle: do not use the same key for both encryption and signing/message authentication.

The **fifth step** is a message send by the server that may contains:

- Change Cipher Spec
  A message (consisting of a single byte of value 1) sent by both parties. Once received, the participant transitions to the agreed cipher suite.
- Finished
  Generated by hashing the entire handshake and sent by both parties. It is used to signal the completion of the algorithm.

The **sixth and seventh step** consists on the reception of the Change Cipher Spec message onward.
From that moment every message sent (both by server and client) will be encrypted using the shared key (the Record protocol started).

We've seen that in step 3 the TLS client verifies the server's digital certificate and in step 4 the TLS server verifies the client's certificate. But how SSL and TLS provide identification, authentication, confidentiality, and integrity?

Authentication:
For server authentication, the client uses the server's public key to encrypt the data that is used to compute the secret key. The server can generate the secret key only if it can decrypt that data with the correct private key.
For client authentication, the server uses the public key in the client certificate to decrypt the data the client sends during step 4 of the handshake. The exchange of finished messages that are encrypted with the secret key (steps 6 and 7 in the overview) confirms that authentication is complete.
The verification of the certificate is done in this manner:

1. The digital signature is checked
2. The certificate chain is checked in case of intermediate CA certificates
3. The expiry and activation dates and the validity period are checked
4. The revocation status of the certificate is checked

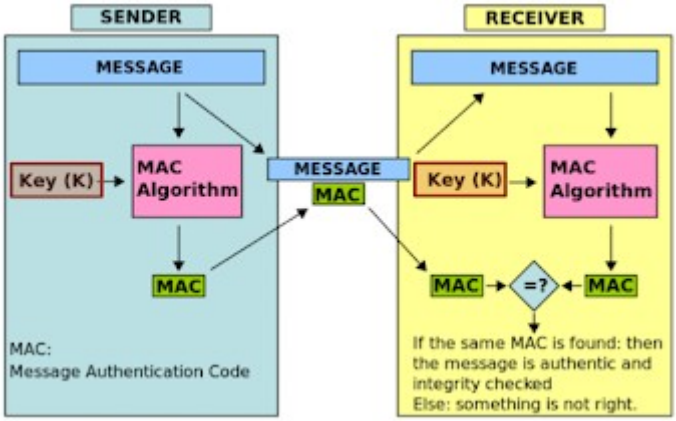
If any of the authentication steps fail, the handshake fails and the session terminates. The exchange of digital certificates during the SSL or TLS handshake is part of the authentication process.

Confidentiality:
SSL and TLS use a combination of symmetric and asymmetric encryption to ensure message privacy. During the TLS handshake, the TLS client and server agree on an encryption algorithm and a shared secret key to be used for one session only.

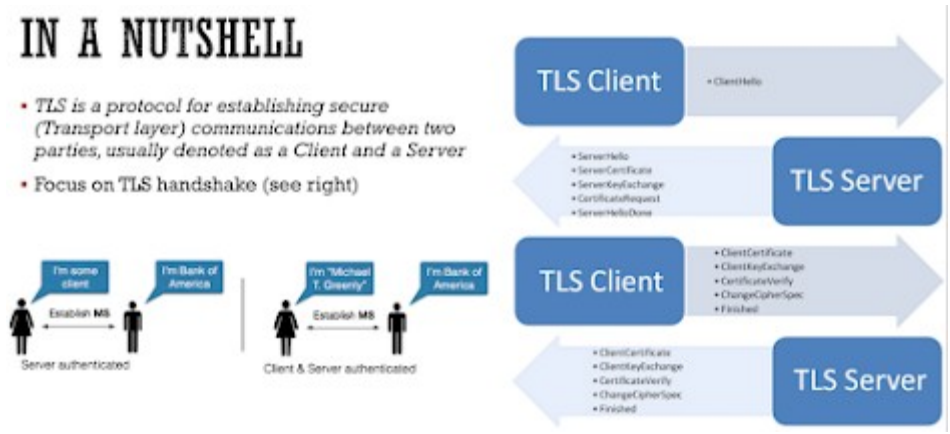
All messages transmitted between the SSL/TLS client and server are encrypted using that algorithm and key, ensuring that the message remains private even if it is intercepted.
SSL/TLS supports a wide range of cryptographic algorithms. Because SSL and TLS use asymmetric encryption when transporting the shared secret key, there is no key distribution problem.

Integrity:
Since symmetric cryptography guarantees only confidentiality, SSL and TLS provide data integrity by using a Message Authentication Code (MAC).
Use of TLS does ensure data integrity, provided that the CipherSpec in the channel definition uses a suitable hash or MAC algorithm.
Use of MD5 and SHA1 is strongly discouraged as this is now very old and no longer secure for most practical purposes.

A **message authentication code** (MAC) is a tag used to confirm that the message came from the stated sender (authenticity) and has not been tampered with (integrity). The MAC allows verifiers to detect any changes to the message content.
A MAC algorithm is a family of cryptographic functions (parameterized by a symmetric key) that can be used to provide data origin authentication, as well as data integrity, by producing a MAC on arbitrary messages.



MAC functions are similar to hash functions but have different security requirements.
MACs are similar to digital signatures but differ in that MACs are both generated and verified by using the same key.



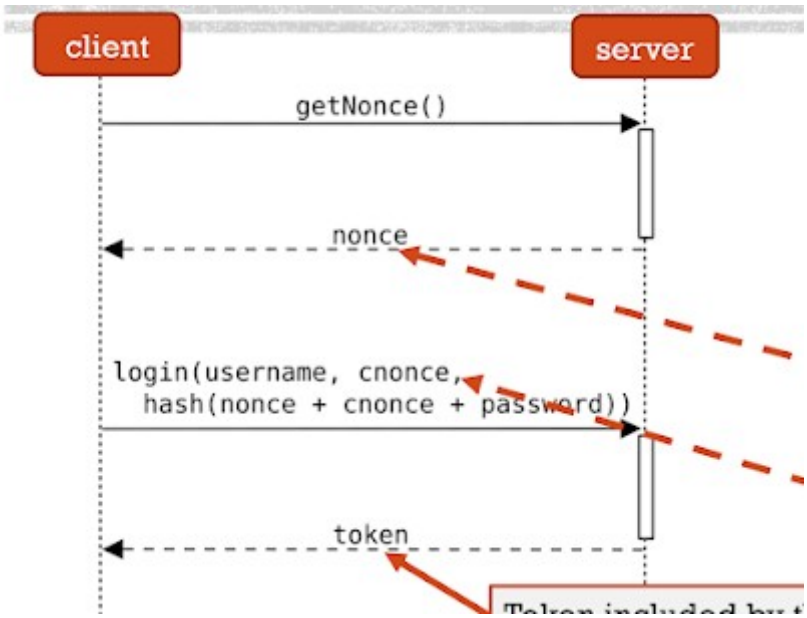
Let's talk about the vulnerabilities of TLS 1.2 (we can give only intuitions underlying the attacks as the technical details may be quite involved…):

- **Backward compatibility**
  The protocol still supports weak cipher suites and broken hash functions
- **Logical flaws**
  A set of logical loopholes can be used to "trick" both client and server
- **Implementation issues**
  Libraries (including the widely used Openssl) contain bugs and some of them can be exploited to mount attacks.
  These issues are caused by the human factor, because humans always make errors.

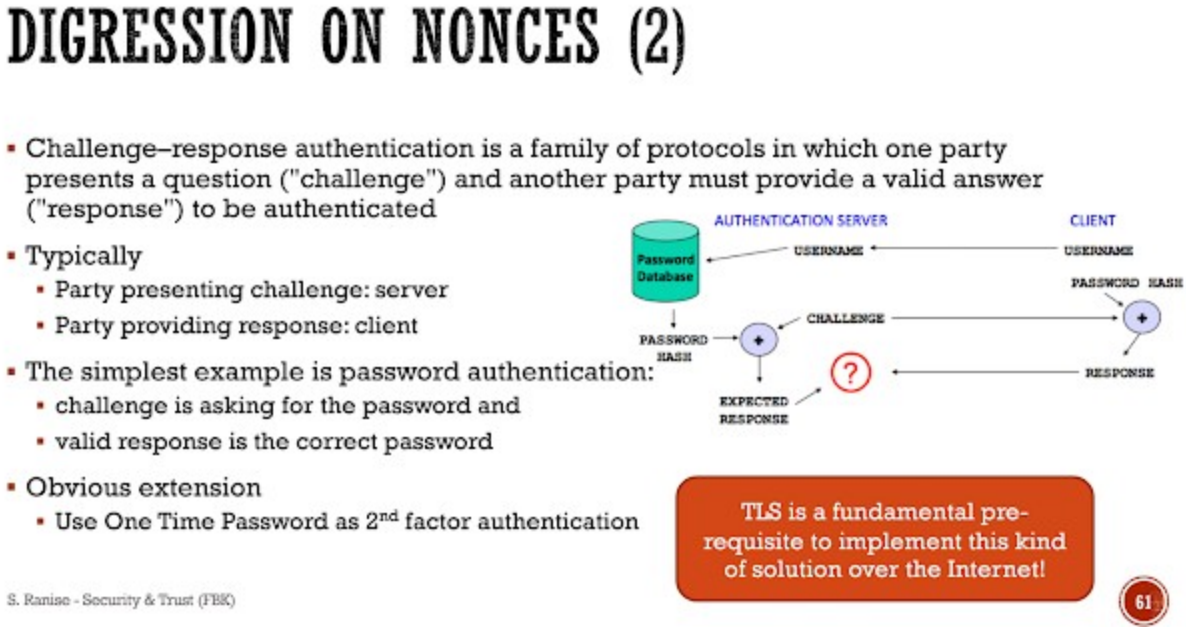
The TLS protocol is one of the most widely attacked, because if you find a bug and exploit a vulnerability, the impact is huge given that it is used everywhere.

During key establishment, replay attacks can be done.
ClientHello offers many different cipher suites, the choice of which to use is made by the server.
ClientHello and ServerHello also contain 32-byte nonces (28-byte random values + 4-byte time encoding).
A **nonce** is an arbitrary number that can be used just once in a cryptographic communication.
Typically, it is a (pseudo-)random number issued by one of the parties in a protocol to ensure that old communications cannot be reused in a replay attack.
Nonces are signed by the server in Diffie-Hellman based cipher suites and involved in key derivation. Nonce are important for security to prevent session replay attacks forcing reuse of session keys.
Nonces avoid attacks aiming to induce client and server to regenerate the same key of a previous session by replying the messages of the previous session containing the parameters to derive the session key using the Diffie-Hellman key exchange.

A typical use of nonces to ensure freshness in authentication protocols:

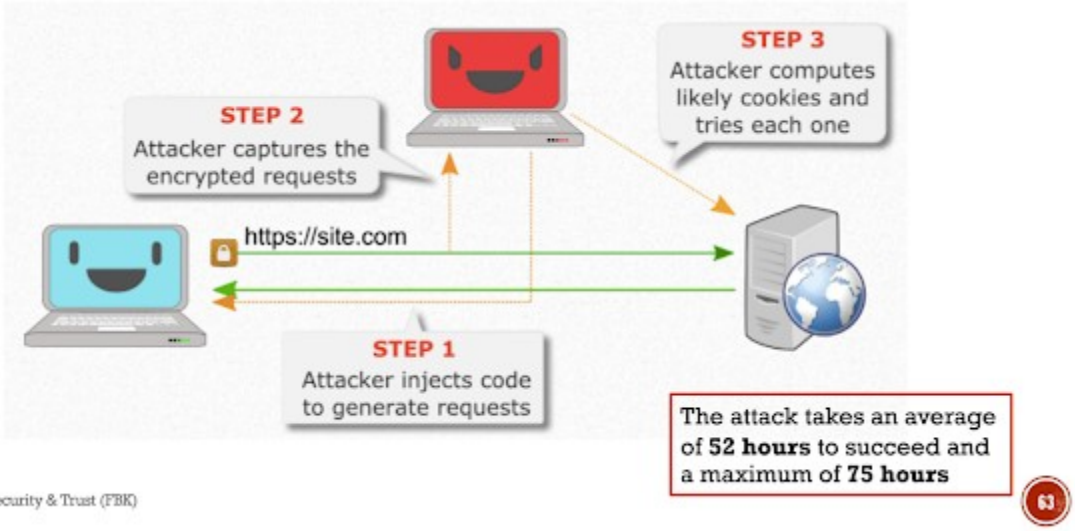


Without the server's nonce and client's cnonce an attacker could reuse a previously generated login(...) message to authenticate as the client



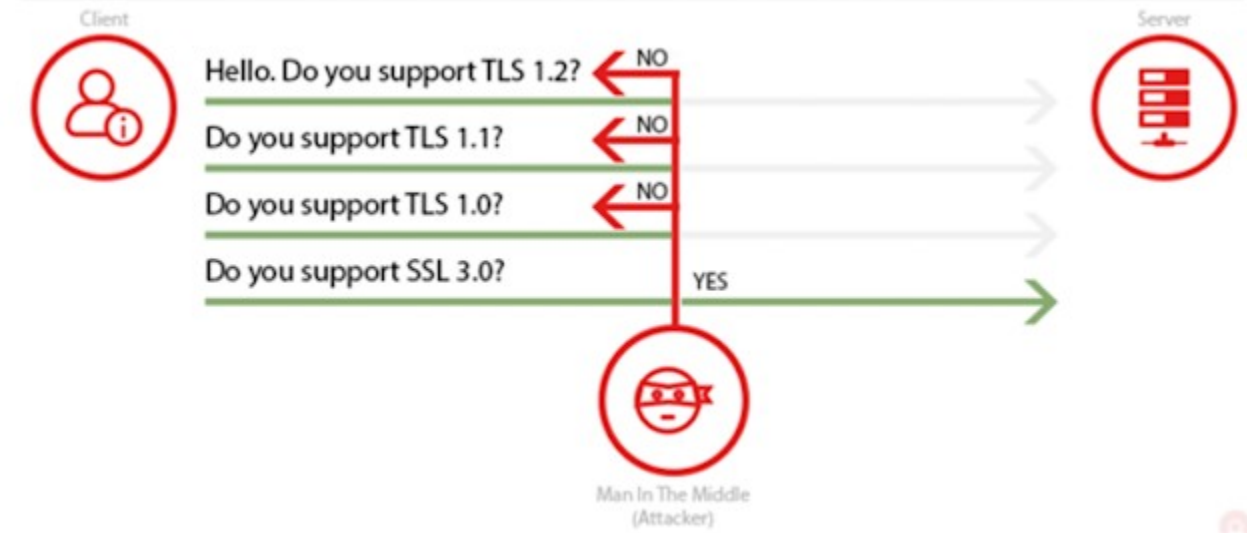
Let's talk about **RC4NOMORE** (RC4 Numerous Occurrence MOnitoring & Recovery Exploit). Remember that RC4 is a stream cipher whose software implementation is straightforward (it consists of shuffling the elements stored in an array of bytes) and it is very fast. It was, until 2017, one of the most widely used ciphers in deployments of TLS (record protocol).
In 2013, there was first evidence of the fact that RC4 was weak because an attacker can guess certain values of the key stream…
In 2015, the exploitation of such biases was shown possible in practice and in 2017 the RC4NOMORE attack showed the practicality of discovering repeated text in messages such as tokens used for authentication (these tokens are called cookies and are widely used in web

applications). Remembering how a stream cipher works, the problem here is that the random part isn't very random.
The idea is to use the weaknesses in the RC4 cipher to speed up the guessing of a token so that the attacker can authenticate as the victim.



So it is a sort of brute force attack where you make the client generate a lot of requests and analyze the responses you can compute likely cookies.
Comparison of ciphertext of the token with encrypted guess corresponds to feedback.
As a consequence of this attack, all major browsers deprecated the use of RC4 in TLS... nowadays, RC4 is almost never used in TLS connections.

Another attack which is quite likely used is **Poodle** (Padding Oracle On Downgraded Legacy Encryption) in 2014.



This attack aims to "downgrade" the protocol to a protocol that is well-known to be vulnerable.
The attack consists in:

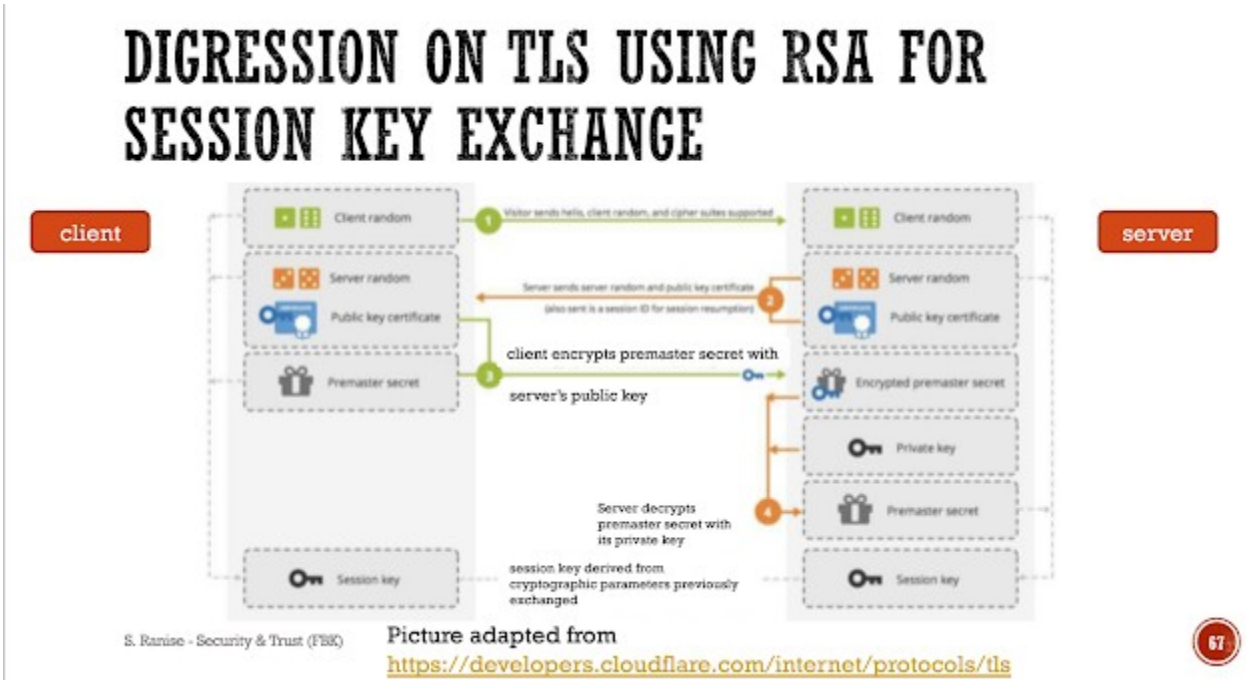
- The client initiates the handshake and sends a list of supported SSL/TLS versions

- An attacker intercepts the traffic, performing a man-in-the-middle (MITM) attack, and impersonates the server until the client agrees to downgrade the connection to SSL 3.0
- At this point, the attacker can exploit a weakness in one of the block ciphers (namely, the Cipher Block Chaining mode with padding) supported by SSL 3.0 to guess cookies and other sensitive information

This attack exploits two features:

- Some servers/clients still support SSL 3.0 for backward compatibility with legacy systems
- A vulnerability in SSL 3.0 related to block padding

One of the most difficult attacks to eradicate were **Bleichenbacher** attack and ROBOT (Return Of Bleichenbacher's Oracle Threat).
In 1998, Bleichenbacher discovered a devastating man-in-the-middle attack on SSL, the predecessor of TLS.
Specifically, Bleichenbacher presented an attack on RSA encryption exploiting the padding which in turn allows a man-in-the-middle adversary against SSL to recover the pre-master secret and thence the application keys (when RSA is used to exchange the session key and not Diffie-Hellman as we have seen before).
The weakness is that the server answers to ciphertext with wrong padding saying that this is the case while it answers positively when the padding is correct…an attacker can guess messages and as soon as the server says that one is correctly padded, he/she knows to be on the right track to guess the plaintext.
TLS incorporates an ad hoc fix to mitigate this attack, namely decryption failures are hidden from the attacker or, equivalently, the server answers correctly and incorrectly padded messages in the same way.



In 2017, we have witnessed the return after almost 20 years of the same type of vulnerabilities based on different server behaviors when RSA cryptography fails (ROBOT).
We just mention that the authors created a tool to systematically scan TLS servers and check their behavior in presence of RSA failures.
In addition, if the private key of the server is stolen, then the attacker can obtain all the session keys sent to that server.
The most important takeaways from this work is that the best mitigation is to deprecate the use of RSA for key exchange in TLS!

Another type of attack is **Heartbleeding** (2014).
It was found in the heartbeat extension of the popular OpenSSL library used to keep a connection alive as long as both parties are still there:

- The client sends a heartbeat message to the server with a payload that contains data and the size of the data (and padding)
- The server must respond with the same heartbeat request, containing the data and the size of data that the client sent

If the client sent false data length, the server would respond with the data received by the client and random data from its memory to meet the length requirements specified by the sender.
Random data from server memory can be the private key of the server, credentials, sensitive documents, credit card numbers, emails, ...



Let's reassume all these vulnerabilities:

- Over time, **weaknesses of some ciphers** may become known and they should be deprecated as soon as cryptographers say that they are no more safe to use since "attacks can only get better"
  Example: biases in the key stream generation of RC4 has lead to RC4NOMORE attack
- Sometimes the problem is not a weakness in the ciphers but **how the cipher is used** that make the resulting ciphertext amenable to so called covert channel attack, i.e. the normal behavior of a system may leak information about the data being processed in unintended/unexpected ways
  Example: a TLS server reacting in different ways to wrong or correct padding used in combination with RSA can reveal some information about the content of messages (in particular those parts that are repeated in every message such as authentication tokens) and is the basis of Bleichenbacher's attack and its variants such as ROBOT

- Sometimes the problem is **the combination of a weak cipher and a glitch in the logic** underlying the exchange of messages, i.e. in parts of the protocol
  Example: the possibility of inducing the selection, during renegotiation, of a cipher whose weaknesses an attacker knows how to exploit in order to extract plaintexts (e.g., authentication tokens) as it is the case in the POODLE attack exploiting weaknesses in block ciphers used in particular modes of operation

The mitigations to these vulnerabilities are:

- Once identified, the above vulnerabilities (as well as many others) were patched or mitigations were suggested
- Several problems can be avoided by proper configuration of the TLS server. However, configuring such servers is not easy, especially by IT professionals without proper training and expertise in security issues. To assist this task, automated tools are available that, given the URL of the TLS server, return a list of potential vulnerabilities

One such tool is TLSAssistant which besides identifying potential vulnerabilities can also provide suggestions on how to fix them or automatically produce fixes to the configuration.

TLS 1.3

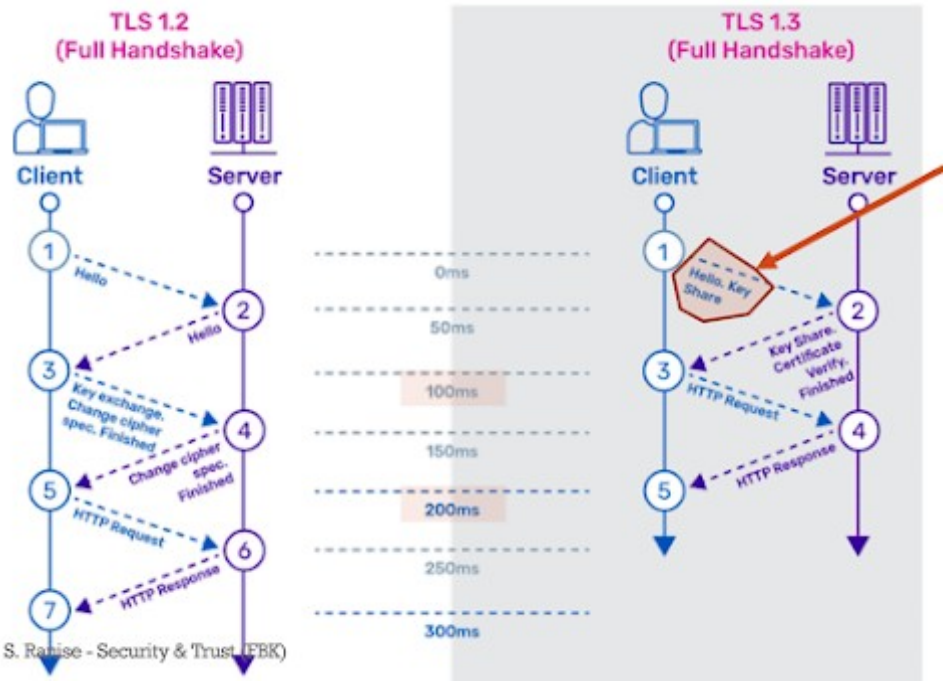
The most used version of TLS is 1.2, but the latest (and best talking about security and efficiency) is **TLS 1.3**.

The goals that be kept in mind during the design of this new version of TLS are:

- **Clean up**
  Remove unsafe or unused features
- **Security**
  Improve security with respect to modern techniques (including automated techniques in order to verify the security of the protocol)
- **Privacy**
  Encrypt more of the protocol
- **Performance**
  1-RTT and 0-RTT handshakes
- **Continuity**
  Backwards compatibility

Let’s see the main differences between TLS 1.2 and TLS 1.3.

In TLS 1.3 **the handshake is shorter** (meaning that require a smaller number of messages exchanged)



In TLS 1.2 there are more or less 5 messages exchanged before being able to exchange messages of application level.
TLS 1.3 requires only **3 messages** before the record protocol. This is made possible by merge step 1 and 3, so the client in the Hello message also includes all the cryptographic parameters. This is possible because in TLS 1.3 the only way to derive the session key is by using (a variant of) Diffie-Hellman (the ephemeral).
This reduces the number of messages to exchange during the handshake since there is no more need to agree on how to derive the session key.
This makes the protocol more secure against covert channel attacks such as those based on Bleichenbacher's weakness. More secure above means that Bleichenbacher's attacks are still possible even though TLS 1.3 does not support the weak key exchange mechanism based on RSA with padding. So improve security but doesn't totally eliminate some attacks that are well-known in TLS 1.2
Obviously also the step 2 and step 4 done by the server are merged.
Without the support of RSA we also avoid the problem of transmitting the key over the network.
So with this specification we improve both security and efficiency of the protocol.

TLS relies on key exchanges to establish a secure session. In earlier versions, keys could be exchanged during the handshake using one of two mechanisms: a static RSA key or a Diffie-Hellman key.
In TLS 1.3, RSA has been removed, along with all static (non-PFS) key exchanges, while retaining ephemeral Diffie-Hellman keys.
In addition to eliminating the security risk posed by a static key, which can compromise security if accessed illicitly, relying exclusively on the Diffie-Hellman family allows the client to send the

required cryptographic parameters for key generation already included in its “hello” (as we’ve already said).
By eliminating an entire round-trip on the handshake, this saves time and improves overall site performance, this modality is called “**one round-trip time**” (1-RTT)
In addition, when accessing a site that has been visited previously, a client can send data on the first message to the server by leveraging pre-shared keys (PSK) from the prior session, this modality is called “**zero round-trip time**” (0-RTT).
Zero round-trip time is an improvement in terms of efficiency but increases the risk of an attack (reusing the same key for more sessions is never a good idea).

Another difference between TLS 1.2 and TLS 1.3 is the number of cipher suites available.
In TLS 1.2 there were 319 possible cipher suites, and in TLS 1.3 this number has dropped dramatically to 5.

- **5 in TLS 1.3** against 319 in TLS 1.2

1. TLS_AES_128_GCM_SHA256
2. TLS_AES_256_GCM_SHA384
3. TLS_CHACHA20_POLY1305_SHA256
4. TLS_AES_128_CCM_SHA256
5. TLS_AES_128_CCM_8_SHA256

AES = Advanced Encryption Standard Block cipher: block size 128 bits, keys with various size
ChaCha20 = stream cipher
Poly1305 = Message Authentication Code
GCM = Galois Counter Mode
CCM = Counter with Cipher Block Chaining. Modes of operation for block cipher AES. They provide both data authenticity (integrity) and confidentiality
SHA = Secure Hash Algorithm. Family of cryptographic hash functions published by NIST and US FIPS. Generates digests of varying sizes

In TLS 1.2 and earlier versions, the use of ciphers with cryptographic weaknesses had posed potential security vulnerabilities.
TLS 1.3 includes support only for algorithms that currently have no known vulnerabilities, including any that **do not support Forward Secrecy**.

Forward secrecy (or perfect forward secrecy) is a feature of specific key agreement protocols that gives assurances that session keys will not be compromised even if long-term secrets used in the session key exchange are compromised (DH supports Forward Secrecy as we said).
For HTTPS the long-term secret is typically the private signing key of the server.
Forward secrecy **protects past sessions** against future compromises of keys or passwords.
By generating a unique session key for every session a user initiates, the compromise of a single session key will not affect any data other than that exchanged in the specific session protected by that particular key. This by itself is not sufficient for forward secrecy which additionally requires that a long-term secret compromise does not affect the security of past session keys.

The problem of the RSA method to pass keys is that a breach of the long term (private) keys would breach the keys previously used for secure communications. A leakage of the private key of the server would cause all the sessions which used this specific public key to be compromised.

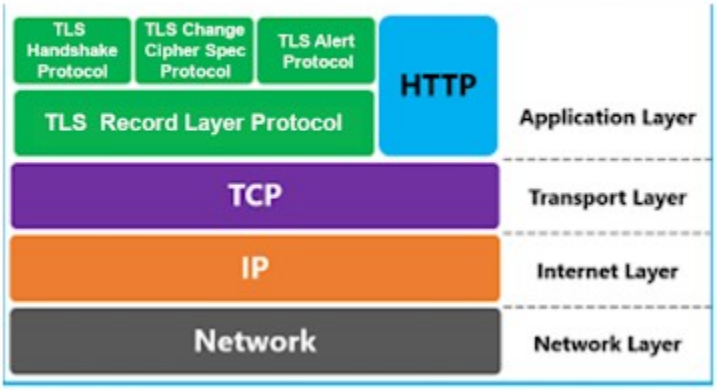
With some key exchange methods (e.g., the ”standard” Diffie-Hellman method), the same key will be generated if the same parameters are used on either side. This can cause problems as an intruder could guess the key
With ephemeral methods a different key is used for each connection, and, again, the leakage of any long-term would not cause all the associated session keys to be breached
The problem with the Diffie-Hellman method is that the keys are not ephemeral, so we should avoid it in generating keys. We need to use a suitable extension of the Diffie-Hellman method called **Ephemeral Diffie-Hellman**.

Ephemeral Diffie-Hellman (DHE in the context of TLS) differs from the standard (static) Diffie-Hellman (DH) in the way that static Diffie-Hellman key exchanges always use the same Diffie-Hellman private keys; so, each time the same parties do a DH key exchange, they end up with the same shared secret.
When a key exchange uses Ephemeral Diffie-Hellman a temporary DH key is generated for every connection and thus the same key is never used twice. Due to increasing concern about pervasive surveillance, key exchanges that provide Forward Secrecy are recommended.

Ephemeral Diffie-Hellman provides no authentication because the key is different every time (neither party can be sure that the key is from the intended party).
Within TLS, DHE should be combined with an additional authentication mechanism... this is done by using so-called Authenticated Encryption with associated data (AEAD) ciphers that we will not see in the course, it will be sufficient to know that AEAD ciphers are symmetric ciphers that can guarantee confidentiality together with authenticity and integrity.

We have only scratched the surface of the TLS specifications and security issues.
There are many more details about the parameters used during key exchange that we have omitted for the sake of brevity that play a key role in providing security.
There is a long history of attacks to TLS that we have omitted because they require the consideration of several different details that are impossible to consider in an introductory course such as this.

- A lot of security problems below TLS/SSL omitted such as
  - IP Security
  - Firewalls
  - IPsec/VPNs
  - Worms
  - DDOS
  - …



Obviously on the Internet there are a lot of other levels and concepts, but we won't see them in this course. But it is very important to mark the fact that doing security only between Transport Layer and Application Layer (like TLS) is useless if we don't consider all the other levels and concepts. TLS is only a part of Internet security.

RECAP QUESTIONS

- What are the advantages and disadvantages of Symmetric and Asymmetric Cryptography?
- How can PKC be used to sign messages?
  - What is the role of hash functions in this process?
- What is a digital certificate and what are its main components?
- What is the Public Key Infrastructure (PKI) and which are its main entities?
  - What is SSL/TLS? Where is it used?
  - How does the handshake protocol of TLS work?
    - What is the role of Diffie-Hellman technique?
    - Can RSA be used for negotiating a key during the handshake? What is the main drawback of this method with respect to Diffie-Hellman?
  - What is a MAC (Message Authentication Code)?
    - How is it different and similar to digital signatures?
  - What are the potential security problems of TLS?
    - How does TLS 1.3 attempt to address some of the security issues in previous versions of TLS?

Hands on

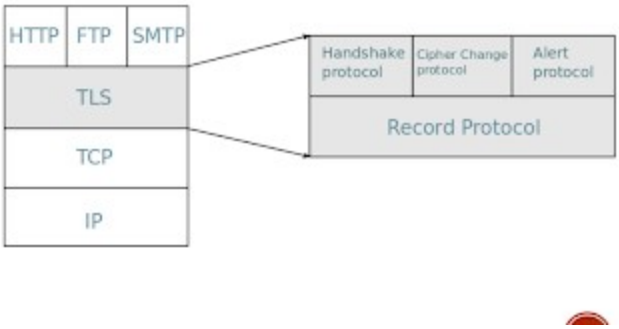
Recap

TLS is a mix of cryptographic protocol: the main are the handshake protocol and the record protocol.

TLS stands for Transport Layer Security. It provides:

- Authentication
- Confidentiality
- Integrity

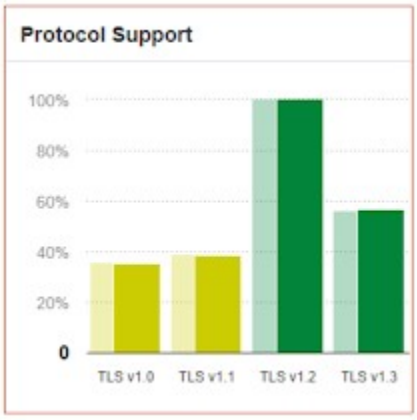




These are the four versions of TLS with their percentage of support by devices:

Four versions:

- v1.0 released in 1999 ~ 35.5%
- v1.1 released in 2006 ~ 38.7%
- v1.2 released in 2008 ~ **99.8**%
- v1.3 released in 2018 ~ 56.9%

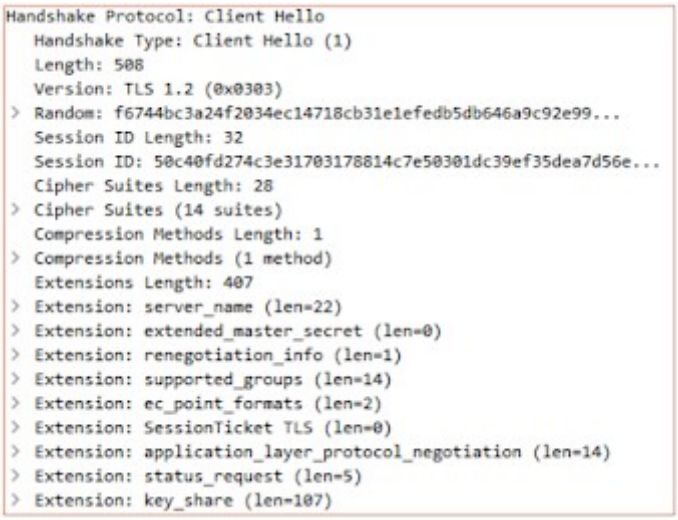


We will focus only on the first two steps of the handshake protocol, that are the **Client Hello** and the **Server Hello**.

The Client Hello message contains:

- The version of the protocol it wants to use. It **should** be the highest available
- A random value obtained by chaining the timestamp (32 bit in UNIX time) and a randomly generated nonce (28 bytes)
- A session identifier
- The list of **supported** cipher suites
- The list of **supported** compression methods
- The list of **requested** extensions (additional services the client wants the server to provide)

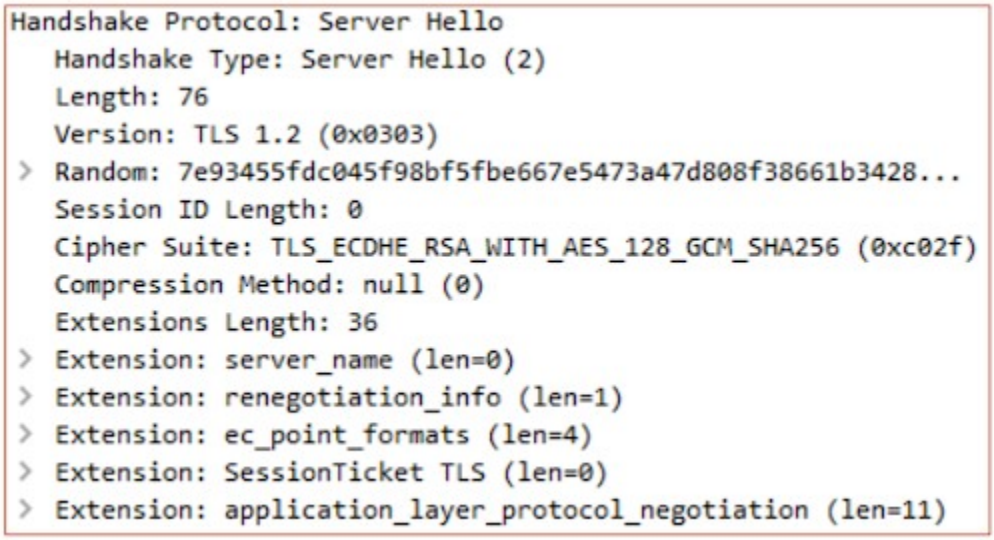
This is an example of a packet of Client Hello:



The Server Hello message contains:

- The **chosen** protocol (that is the highest version supported by both parties)
- A random value obtained by chaining the timestamp (32 bit in UNIX time) and a randomly generated nonce (28 bytes)
- A session identifier
- The **chosen** cipher suite
- The **chosen** compression method
- The list of **required** extensions

This is an example of a packet of Server Hello:



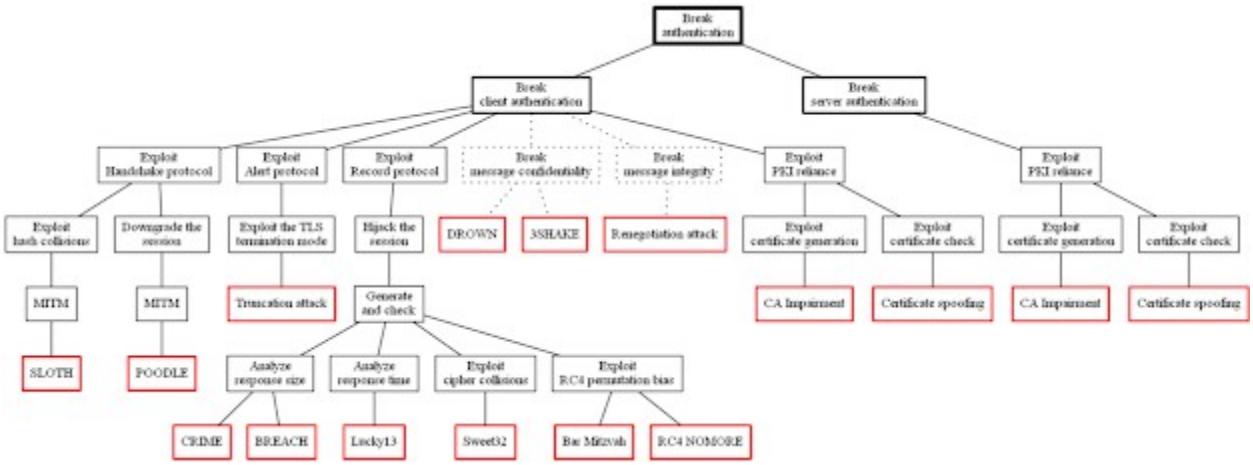
TLS vulnerabilities

When you use TLS you can't just "deploy" it, but you must:

- Choose the versions of TLS you want to offer (if a web server choose to offer only 1.2 and 1.3 for security reasons, a client with 1.1 couldn't connect to that website)
- Choose the set of available cipher(s)

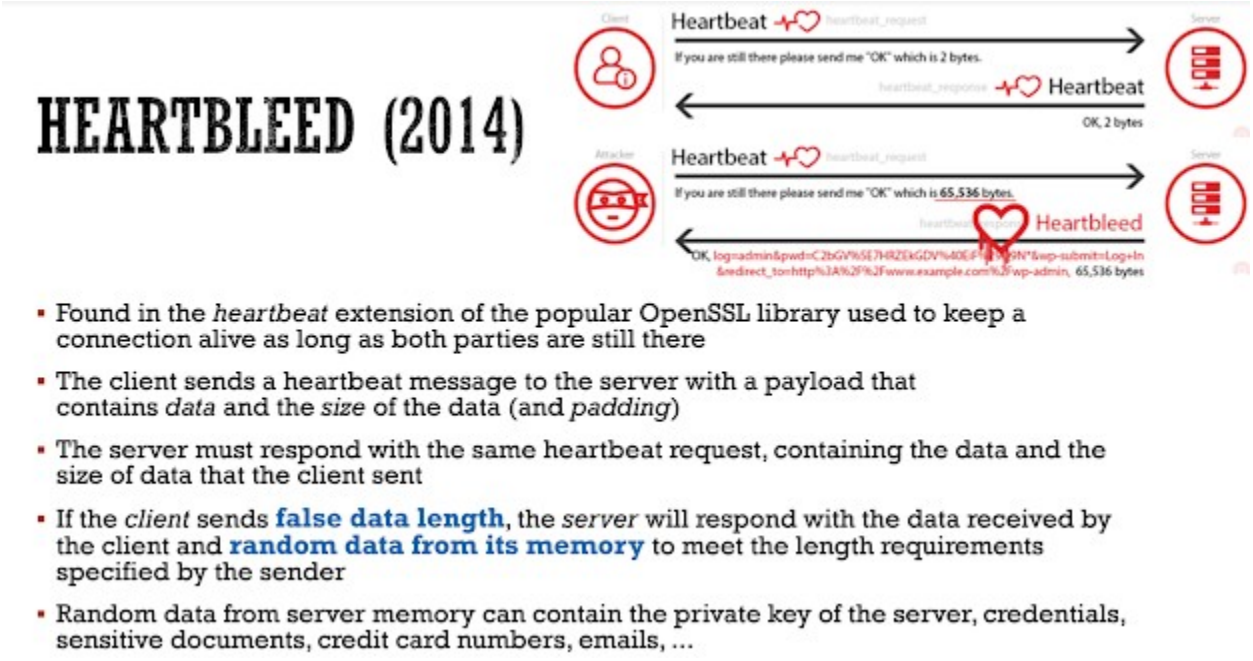
- Set a certificate issued by a trustworthy CA
- Cope with implementation issues (e.g., vulnerable libraries)
- More

If the configuration isn't done properly, these attack could be put in practice:



At the top we have the general attack and at the bottom we have the specific attacks.

Let's review the **heartbleed attack** (2014).



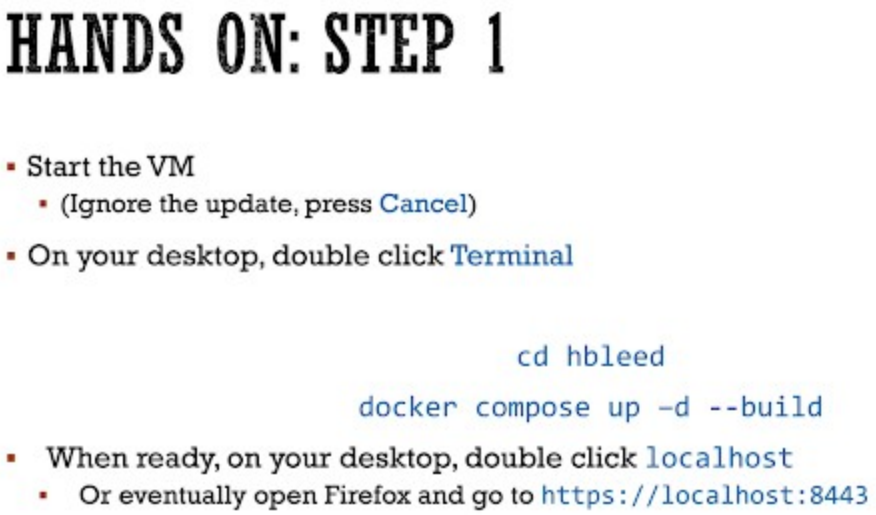
We may think "A single misconfiguration should not be that bad, right?"
The Heartbleed vulnerability, one of the most consequential since the advent of the commercial Internet, allowed attackers to remotely read protected memory from an estimated 24 to 55% of popular HTTPS sites.
Today 214.921 servers are still vulnerable to this type of attack.



The base score is the risk of this attack.

Hands-on session

The first step consists in emulate a web server on your pc:



Then you create an account (just to see something during the attack later):

HANDS ON: STEP 2

- Press: Sign Up
- Register an account...
  - **USE RANDOM CREDENTIALS**, but something you can remember
  - Generate or type in a password
    - Please, do not use your personal one(s).
- ... or let's spice things up
  - Let **your nearest friend register with his credentials on your VM** and then make your friend log out.
  - Again, **DO NOT PUT A REAL PASSWORD**, your friend will obtain them later.

```
cd hbleed
docker compose up -d --build
```

- When ready, on your desktop, double click localhost
  - Or eventually open Firefox and go to https://localhost:8443



Then you go to the terminal and you execute the heartbleeding attack:

HANDS ON: STEP 4

```
grep -a "<SOMETHING TO DISCOVER YOURSELF>" dump.bi
```

Hint: try by F12 the registration form OR check the image in this slide

- **Nothing found** even if you are sure of what you are searching?
  - Adjust the command:

```
python heartbleed-poc.py -p 8443 -n 1 localhost
```

`-n 1` means only one heartbeat.
You want to get more memory if you are not seeing the results.
Try with `-n 10` or `-n 100`

and run again the grep command:

```
grep -a "<SOMETHING TO DISCOVER YOURSELF>" dump.bin
```

And you will have this result (searching “email” as a keyword):

- Finally, we have some results!

```
tlsa@tlsa-virtualbox:~/hbleed/heartbleed-poc$ grep -a "email" dump.bin
username=matteo&email=mrizzi%40fbk.eu&password_1=Jzpqg9akDFmT5kE&password_2=Jzpqg9akDFmT5kE&
```

For a fast way

```
grep -a "email" dump.bin
```

For the long, but "graphical" way

```
hexyl dump.bin | less -R
```

For all interesting results

```
pcregrep -ao "[A-Za-z0-9_-]+=[0-9a-zA-Z]+" dump.bin
```

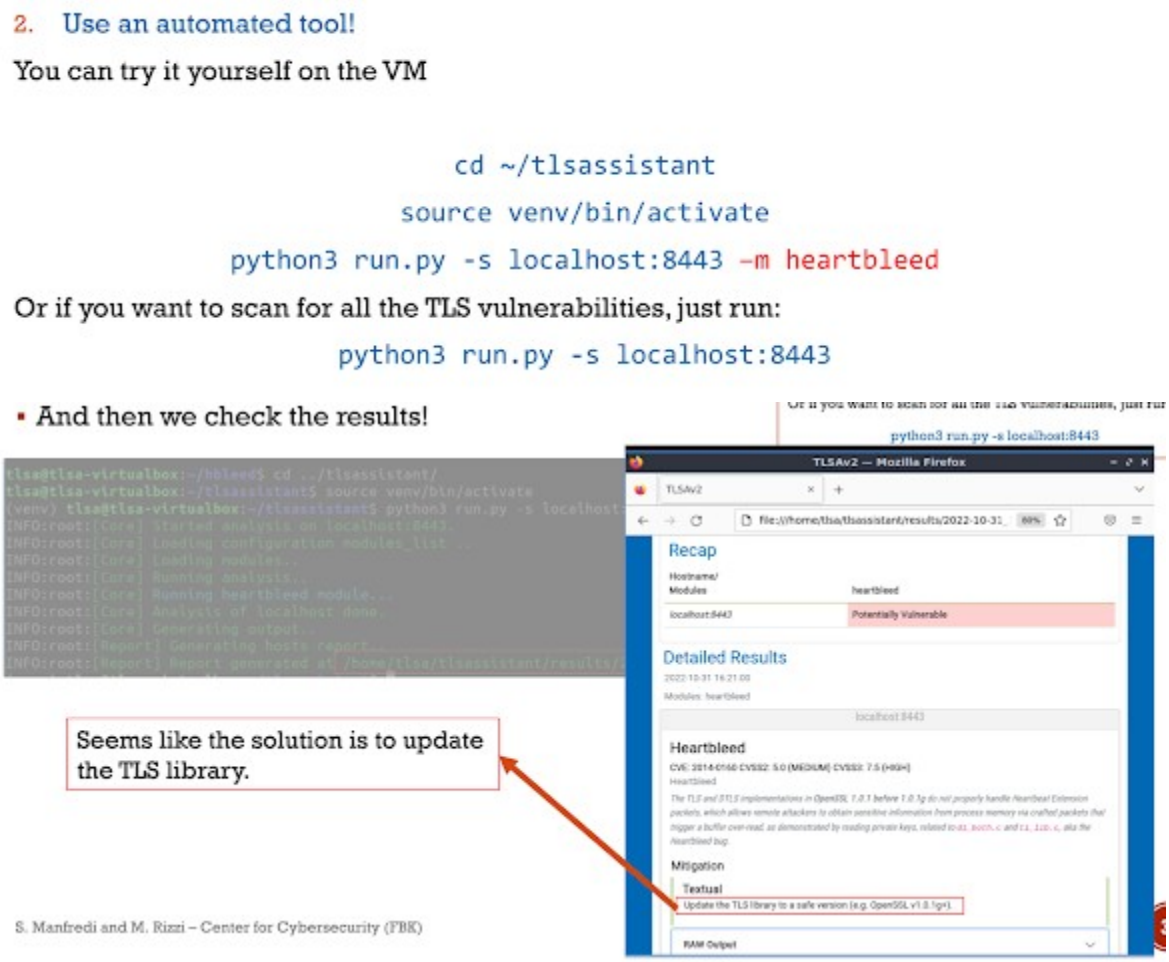
The recap is that we execute one script on a vulnerable endpoint, we noticed that we had to set enough heartbeats to get more data and we used this to get **private information**!

Once identified, the above vulnerabilities (as well as many others) were patched, or mitigations were suggested.
Several problems can be avoided by proper configuration of the TLS server, or by issuing updates on the cryptographic library.

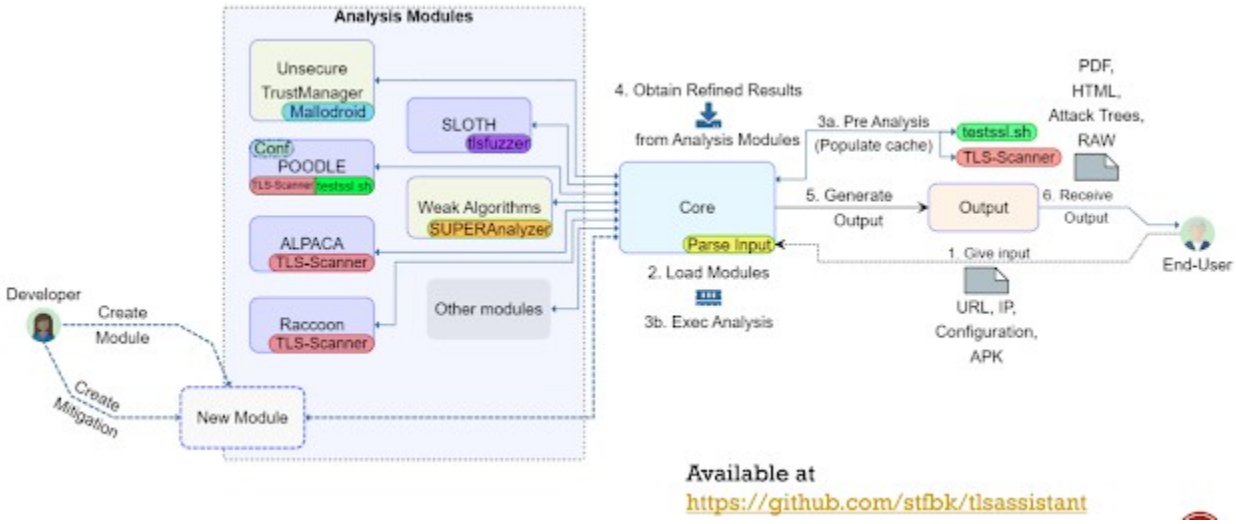
The issue is that nobody really learns each solution for each context (there exists very many attacks).
A way to perform a mitigation is to google the vulnerability and apply the solution, but often you must search through multiple articles and you may perform operations that are unnecessary for the web server. And you must repeat this for each vulnerability!!

Since the vulnerabilities can be detected just sending the Client Hello and analyzing the Server Hello, **automated tools** were created.

For example FBK developed TLS Assistant:



Tools like TLS Assistant are very important because they help administrators of web servers that aren't very expert in network security to find the vulnerabilities and the mitigation to apply to their web server.



Authentication II

In the previous chapter of Authentication we discussed only **computer security** talking about authentication.
In this chapter we will see how it is possible to authenticate users over the network and we will talk about the fact that the services are lazy and don't want to lose time and resources on digital identity, so they rely on identity providers. This practice is done also to concentrate the budget on the core business of the company, and the identity providers will do the same providing a secure service (focusing only on that).
In this way the life of users is simplified because they will always have the same layout and procedures to encounter and they must remember only one password.
On the other hand, this introduces a single point of failure.

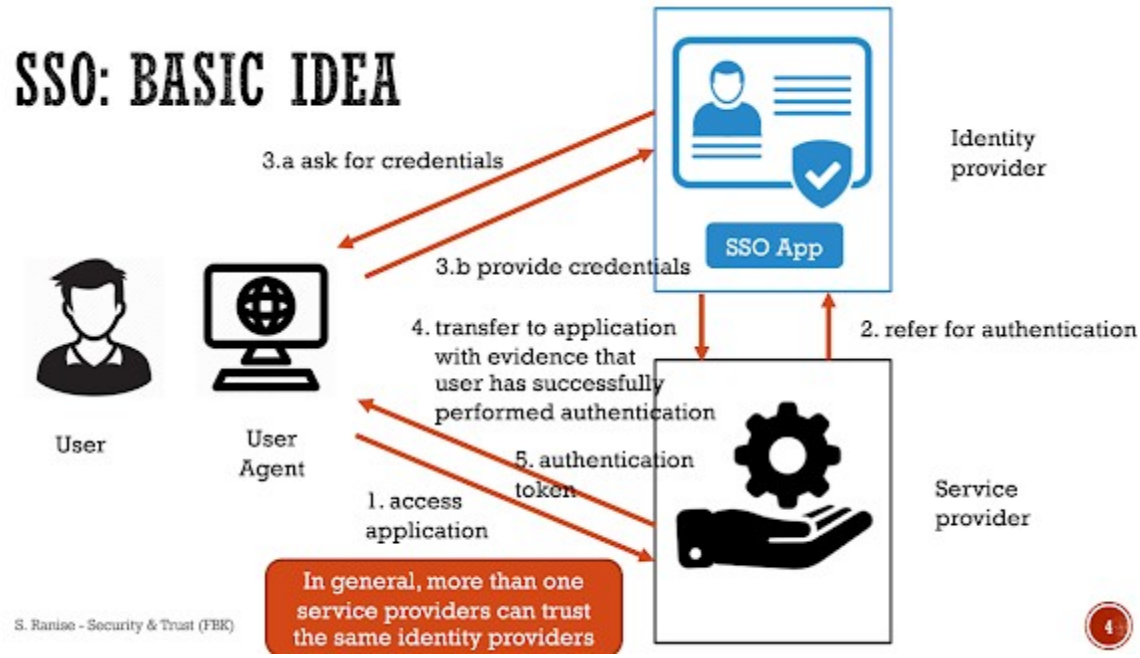
Single Sign On

The SSO (for the user) consists of outsourcing authentication to trusted 3rd party identity providers.

As we've already discussed, multiple systems typically require multiple sign-on dialogues, resulting in headache for administration and users (managing digital identity, multiple sets of credentials, presenting credentials multiple times).

A **security domain** is an application or collection of applications trusting a common security token for authentication, authorization or session management.
A **security token** is issued to users after they have actively authenticated with their identifiers and credentials (e.g., passwords or other authentication factors) to the security domain.
The problem is that the more security domains are, the more sign-ons are required.

So the basic idea is to implement more services under a single security domain:



One important thing is that all this data is shared over the network, so all this connection must be secured (using TLS protocol).

Some important **properties** of SSO system are:

- Credentials never leave the authentication domain (that is composed by user agent and identity provider)
- Service providers (affiliated domains) have to trust the authentication domain
- Authentication transfer has to be protected
    - Replay prevention on authentication tokens (with nonces as a freshness parameter to limit the reuse and lifetime of the authentication tokens/assertions)
    - Interception/masquerade attacks (need to add cryptographic signatures to authentication assertions)

    If an attacker impersonates the identity provider, the attacker can fake a login.

So, based on the trusting relationship between Service Provider and Identity Provider, the Service Provider receives a message from the Identity Provider which confirms the authentication on the user and he must check the signature in order to be sure that the message arrives from a trusted Identity Provider.
In the same way the Identity Provider will provide its service (that is make the authentication) only to the service provider that is willing to receive and get support from it.

Security Assertion Markup Language (SAML)

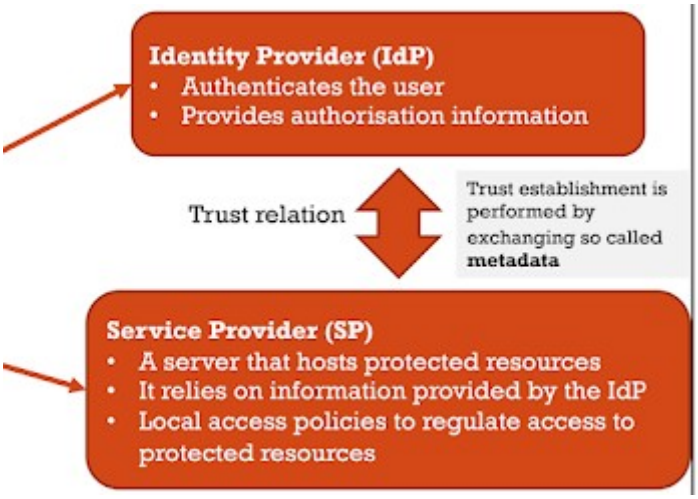
One of the main standards that perform SSO is SAML.
SAML is a security service for authentication that can be configured to fit the goals of several different use case scenarios. SAML 1.0 defined in 2002, SAML 2.0 defined in 2005.
SAML is a common **language** and **flow** between systems that want to provide an SSO experience to users.

It was the answer to the lack of standards and interoperable solutions for exchanging authentication and authorization information across security domains (it is heavily based on mechanisms implemented in browsers e.g., redirections).

SAML is a data format used to exchange both **authentication data** and **authorization data**.
In fact, a SAML message can contain, in addition to authentication data (the user is correctly authenticated), other attributes or information associated with that particular user, for example the level of the user.
It is XML based.
XML is a software- and hardware-independent framework for storing and transporting data. XML is a markup language much like HTML: XML was designed to carry data with focus on what data is, HTML was designed to display data with focus on how data looks, XML tags are not predefined like HTML tags are. XML was designed to store and transport data. XML was designed to be self-descriptive (whatever this means).

SAML is an open standard from OASIS (Organization for the Advancement of Structured Information Standards).

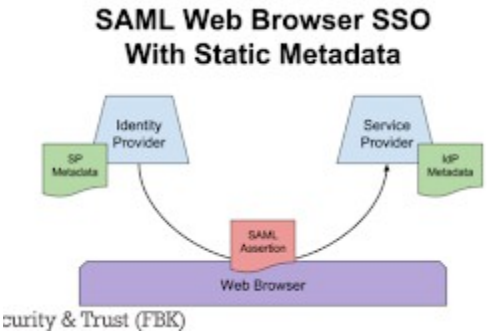
The concept of **SAML federation** is simply the identification of Identity Providers (IdP) and Service Providers (SP) that trust each other to perform authentication service.

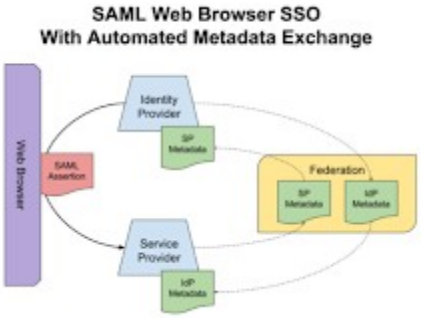


So the federation is a group of entities that share a common policy, and it is managed as a single entity.
The authentication is always between a User and an IdP. The federation establishes only the initial trust among the resources.

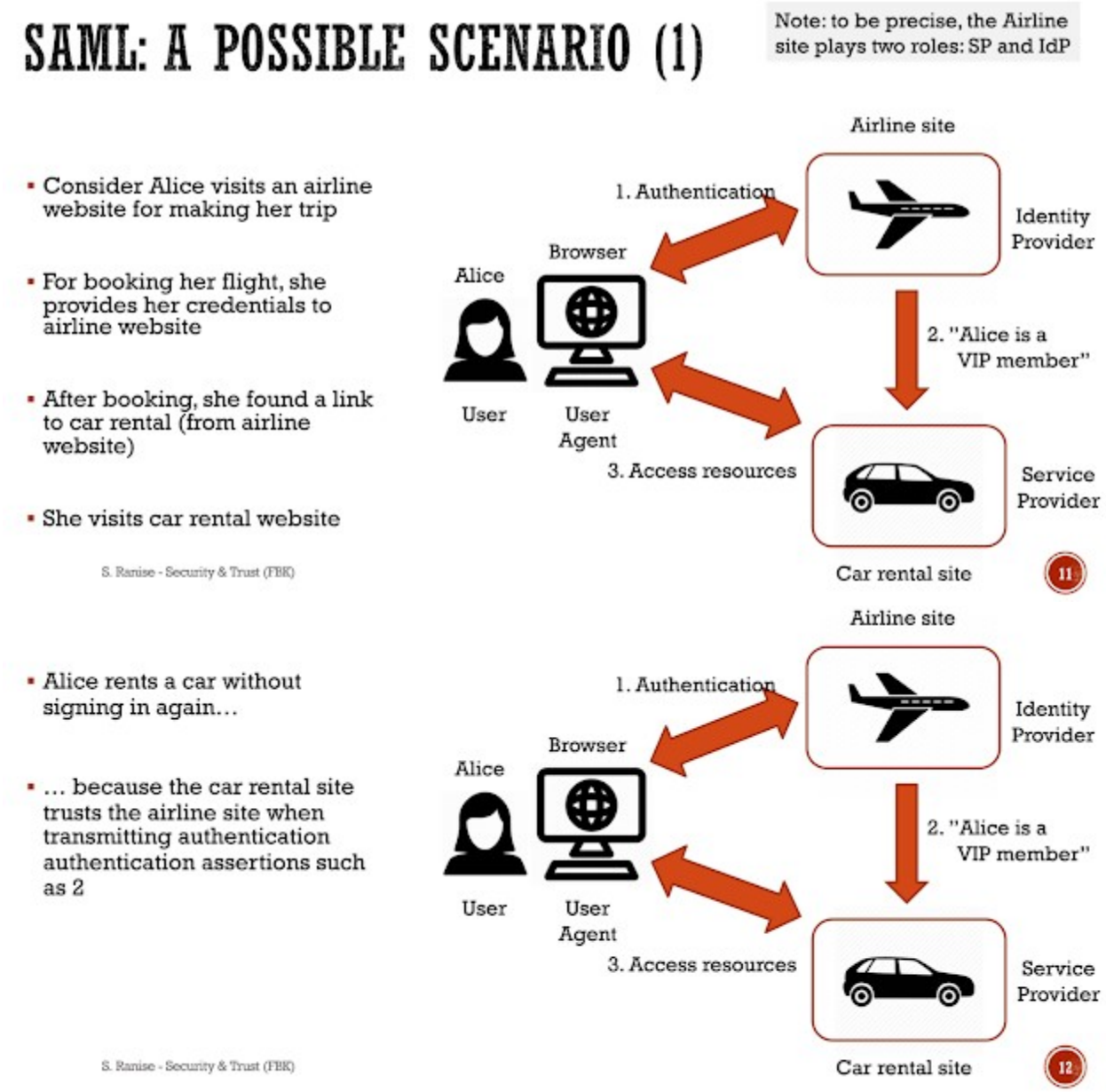
IdP and SPs share metadata in whatever form and by whatever means possible.
At least the following metadata must be shared:

- Entity ID (globally-unique identifier included in every message issued by the entity)
- Cryptographic keys
  For authentication purposes, a SAML message may be digitally signed by the issuer and the content of the message can be protected by a public encryption key belonging to the ultimate receiver. Indeed, trusted public keys must be shared in advance.



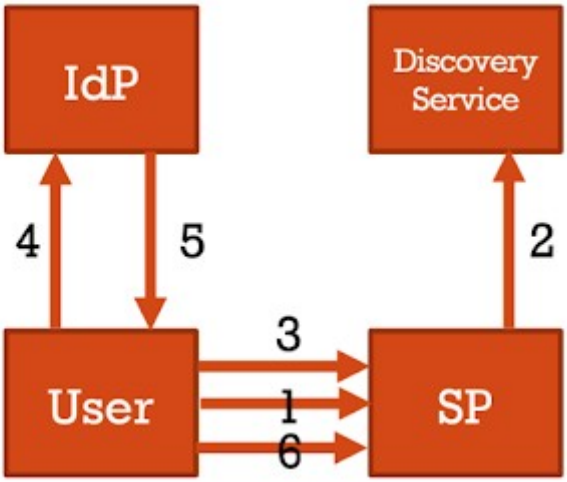


Let's see an example of the advantages of SSO:



In practice, the flow of SAML Authentication is this:

1. A user want to access an SP
2. The user is redirected to a Discovery Service (it can be an external service or embedded in the SP) that allow the user to choose the IdP
3. The user goes back to the SP with the ID of his/her own IdP
4. The user is redirected to the IdP
5. Authentication is performed
6. The user goes back to the SP with the authentication

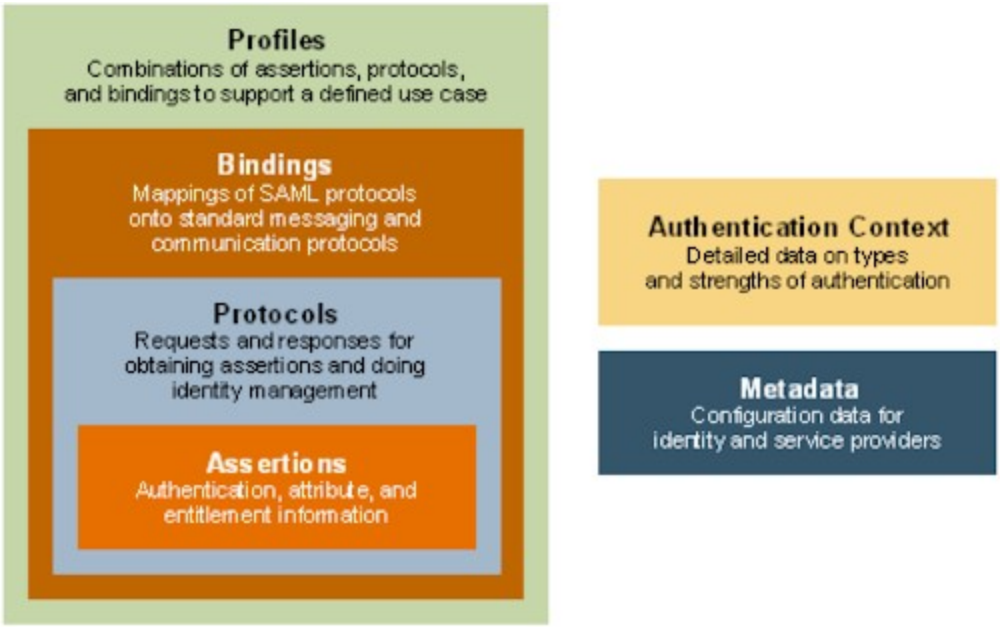


All the steps above are associated with a SAML assertion (in XML format)

Some remarks to do on SAML are:

- SAML Authentication was mainly designed for Web Services (but it is possible to support other type of resources either web based or native applications)
- Resources can support multiple SAML **profiles** (the profile identifies the exchange protocol and the message format)
- Most widely used profile for web applications is redirect (the browser shows a page with a Javascript performing the redirect)

These are the main components of SAML:



The **Authentication Context** is basically how a user authenticates at an Identity Provider.
The Identity Provider includes the authentication context in an assertion at the request of a Service Provider or based on configuration at the Identity Provider.
A Service Provider can require information about the authentication process to establish a level of confidence in the assertion before granting access to resources.
This is done because Service Providers can decide which level of assurance is required to access specific protected resources, based on some assessment of “value” or “risk”.

A SAML **metadata** document describes a SAML deployment such as a SAML identity provider or a SAML service provider. Deployments share metadata to establish a baseline of trust and interoperability.
Minimum set of metadata to be shared:

- Entity ID (a globally unique identifier used in software configurations, relying-party databases, and client-side cookies, every SAML protocol message contains the entity ID of the issuer)
- Cryptographic Keys
- Protocol Endpoints (bindings and URLs)

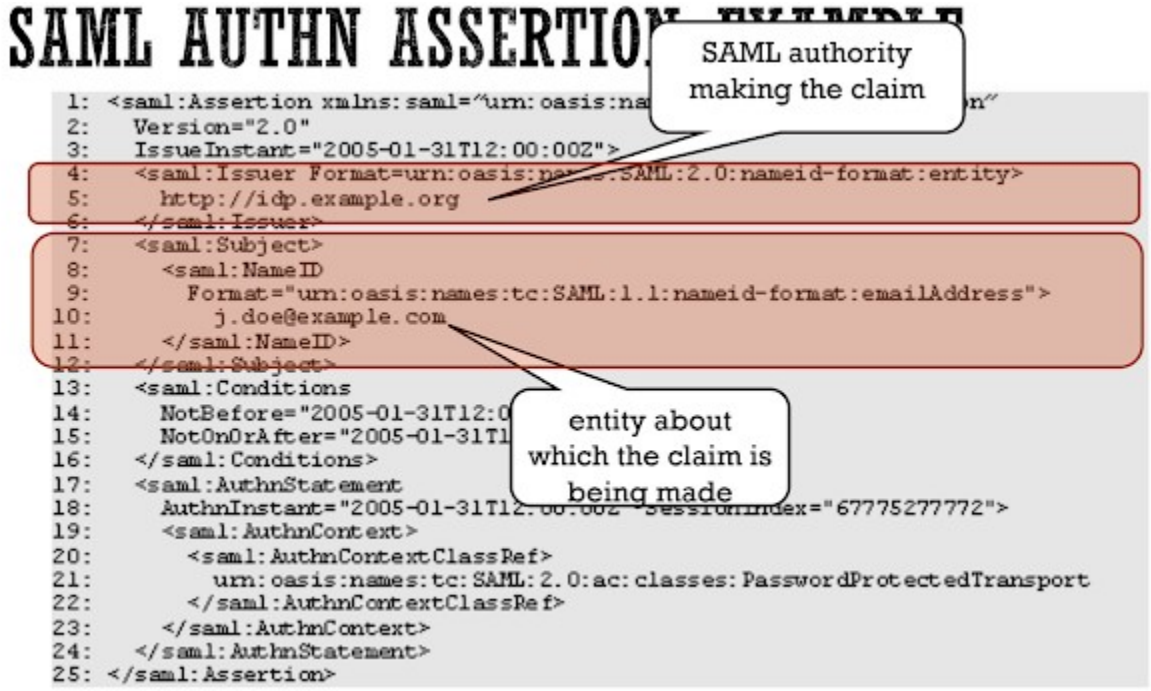
For authentication purposes, a SAML message may be digitally signed by the issuer (so trusted public keys must be shared in advance, for both signing and encryption).
Once the message is signed and encrypted, the issuer sends the message to a trusted protocol endpoint, the location of which must be known in advance. Upon receipt, the message receiver decrypts the message (using its own private decryption key) and verifies the signature (using a trusted public key in metadata) before mapping the entity ID in the message to a trusted partner.
This scenario requires each party to know the other in advance. To establish a baseline of trust, parties share metadata with each other.
So some metadata must be shared at priory, before any authentication procedure may happen.
But the standard how this exchange of metadata is supposed to happen.

Initially, this may be as simple as sharing information via email. Over time, as the number of SAML partners grows, the natural tendency is to automate the metadata sharing process. An implementation that supports SAML Web Browser SSO requires a schema-valid SAML metadata file for each SAML partner.

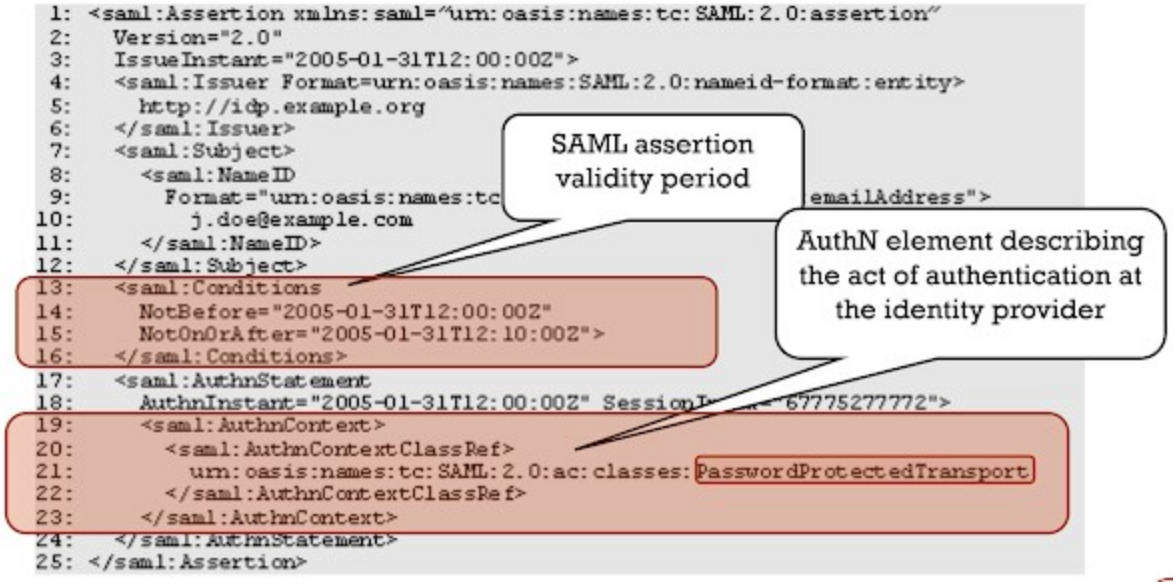
SAML manipulates **assertions** that are in a particular format.
An assertion is a set of statements (claims) made by a SAML authority (asserting party).
Assertion can be seen as the unit of information exchanged in SAML.
There are three types of assertions:

- **Authentication assertion**
  Issued by a party that authenticates users.
  It describes who issued the assertion, the Authenticated Subject, the Validity period and other authentication-related information
- **Attribute assertion**
  It defines specific details about the Subject (Example: ‘Alice’ has ‘VIP member’ status)
- **Authorization assertion**
  It defines something the Subject is entitled to do (Example: ‘Alice’ is permitted to rent a car when on a business trip)

Here’s an example of an authentication assertion:



SAML AUTHN ASSERTION: EXAMPLE



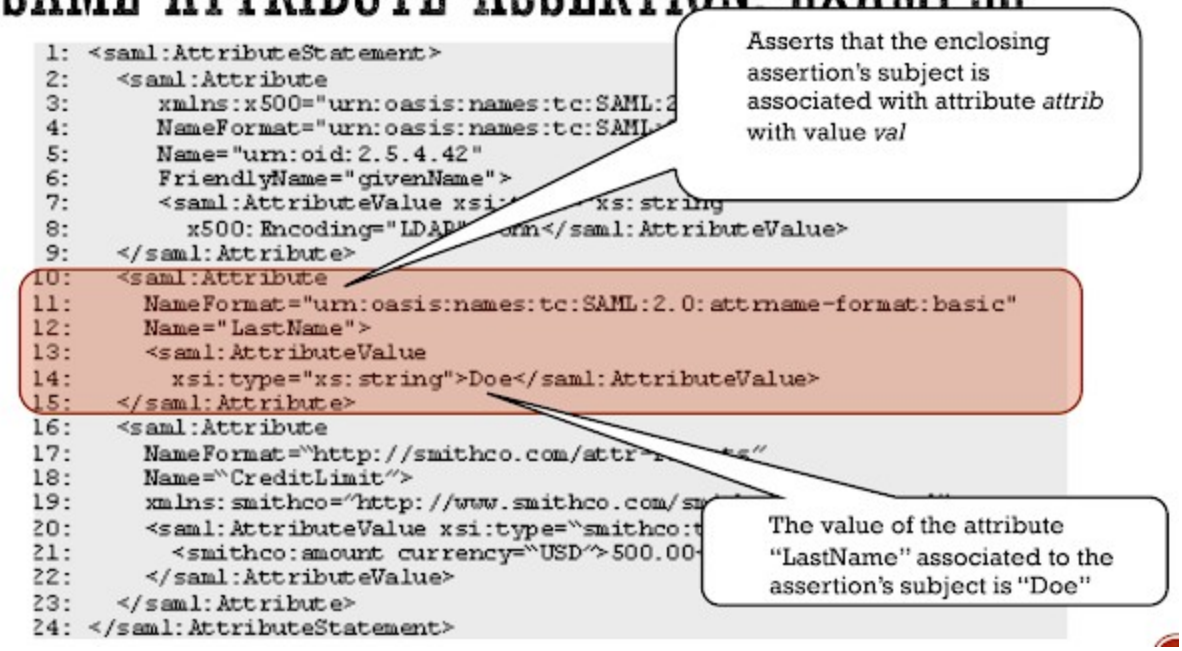

```
<saml:Assertion xmlns:saml="urn:oasis:names:tc:SAML:2.0:assertion"
  Version="2.0"
  IssueInstant="2005-01-31T12:00:00Z">
  <saml:Issuer Format=urn:oasis:names:SAML:2.0:nameid-format:entity>
    http://idp.example.org
  </saml:Issuer>
  <saml:Subject>
    <saml:NameID
      Format="urn:oasis:names:tc                    emailAddress">
        j.doe@example.com
    </saml:NameID>
  </saml:Subject>
  <saml:Conditions
    NotBefore="2005-01-31T12:00:00Z"
    NotOnOrAfter="2005-01-31T12:10:00Z">
  </saml:Conditions>
  <saml:AuthnStatement
    AuthnInstant="2005-01-31T12:00:00Z" SessionI      ="6775277772">
    <saml:AuthnContext>
      <saml:AuthnContextClassRef>
        urn:oasis:names:tc:SAML:2.0:ac:classes:PasswordProtectedTransport
      </saml:AuthnContextClassRef>
    </saml:AuthnContext>
  </saml:AuthnStatement>
</saml:Assertion>
```

The time window (the validity period) is useful because the user doesn't need to re enter every time the credentials in that time window.
Notice that:

- Authentication itself is not part of SAML (there are only information about the authentication, like the method, and the outcome)
- Statement refers to an authentication act that took place at a prior time

Here's an example of an attribute assertion:

SAML ATTRIBUTE ASSERTION: EXAMPLE




```
<saml:AttributeStatement>
  <saml:Attribute
    xmlns:x500="urn:oasis:names:tc:SAML:2
    NameFormat="urn:oasis:names:tc:SAML
    Name="urn:oid:2.5.4.42"
    FriendlyName="givenName">
    <saml:AttributeValue xsi:          xs:string
      x500:Encoding="LDAP"          rm</saml:AttributeValue>
  </saml:Attribute>
  <saml:Attribute
    NameFormat="urn:oasis:names:tc:SAML:2.0:attrname-format:basic"
    Name="LastName">
    <saml:AttributeValue
      xsi:type="xs:string">Doe</saml:AttributeValue>
  </saml:Attribute>
  <saml:Attribute
    NameFormat="http://smithco.com/attr
    Name="CreditLimit">
    xmlns:smithco="http://www.smithco.com/s
    <saml:AttributeValue xsi:type="smithco:t
      <smithco:amount currency="USD">500.00
    </saml:AttributeValue>
  </saml:Attribute>
</saml:AttributeStatement>
```

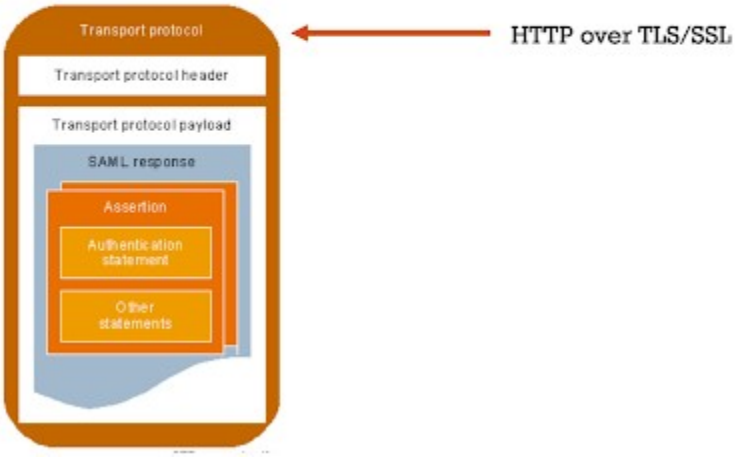
The assertions are exchanged in a particular order between the various entities involved in the system by using **protocols**.
A protocol is a set of messages with a particular format exchanged between entities to reach certain goals (in SAML the main goal is to authenticate users).
We notice that protocols only specify what to do (perform the authentication), because SAML protocols are agnostic about how to implement the authentication procedures.
So SAML provides a set of specifications to define the assurance level (trust level) which the user is authenticated with.
Protocols define the flow of assertion query and request (for obtaining SAML assertions) (for example: The SP authenticates the user through an IdP assertion).
Important remark: The behavior of a protocol is typically independent of how it is to be implemented.
A typical representation of a protocol is making a vertical line for each entity and an horizontal line for each message that goes from one entity to another, specifying the order and time of each message.

Protocols are in an abstract form, because they only give an idea of what kind of actions or steps need to be taken in order to achieve a certain goal.
So **bindings** are used in order to describe how the abstract specifications given by the protocols can be implemented on actual technology (for example HTTPS).
This separation is very important because if a new technology arrives, you can simply add a new binding in order to explain how you can implement the protocol that is specified abstractly.
So SAML binding is the mechanism to transport these messages.
Some types of bindings are:

- SAML URI
- SAML SOAP
- HTTP redirect
- HTTP POST
- HTTP artifact

HTTP redirect enables SAML protocol messages to be transmitted within URL parameters and it enables SAML requestors and responders to communicate by using an HTTP user agent as an intermediary.
The intermediary might be necessary if the communicating entities do not have a direct path of communication. The intermediary might also be necessary if the responder requires interaction with a user agent, such as an authentication agent.
HTTP redirect is sometimes called browser redirect in single sign-on operations.
HTTP is an application layer protocol designed to transfer information between networked devices and runs on top of other layers of the network protocol stack. A typical flow over HTTP involves a client machine making a request to a server, which then sends a response message.

HOW ASSERTIONS ARE EXCHANGED



SAML 2.0 **profiles** combine protocols, assertions, and bindings to create a federation and enable federated single sign-on.
Types of profiles:

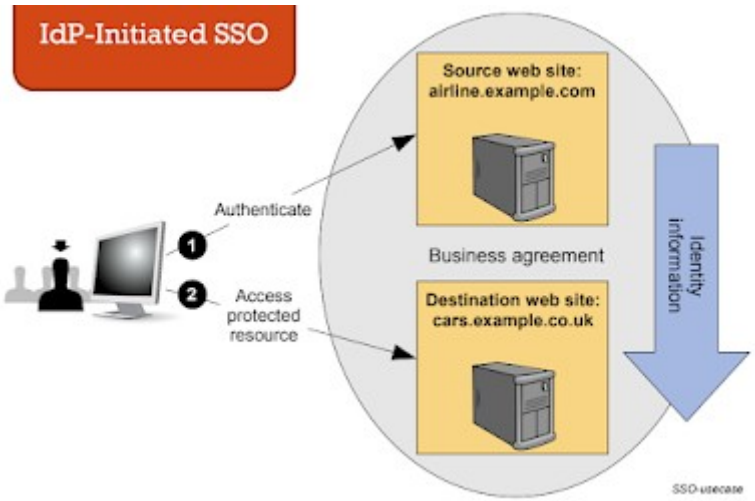
- **Web browser SSO**
  Web browser single sign-on profile provides options regarding the initiation of the message flow and the transport of the messages:
  Flow initiation: The message flow can be initiated from the identity provider or the service provider
  Bindings: HTTP redirect, HTTP POST, HTTP artifact
- **Single Logout**
  Single Logout profile is used to terminate all the login sessions currently active for a specified user within the federation
  A user who achieves single sign-on to a federation establishes sessions with more than one participant in the federation
  The sessions are managed by a session authority, which in many cases is an identity provider
  When the user wants to end sessions with all session participants, the session authority can use the single logout profile to globally terminate all active sessions
- **Artifact resolution**

We will analyze two particular profiles:

- IdP-Initiated SSO
  Where the user first visitate the Identity Provider and then the Service Provider
- SP-Initiated SSO
  Where the user first visitate the Service Provider and then the Identity Provider

A possible scenario for an IdP-Initiated SSO is:

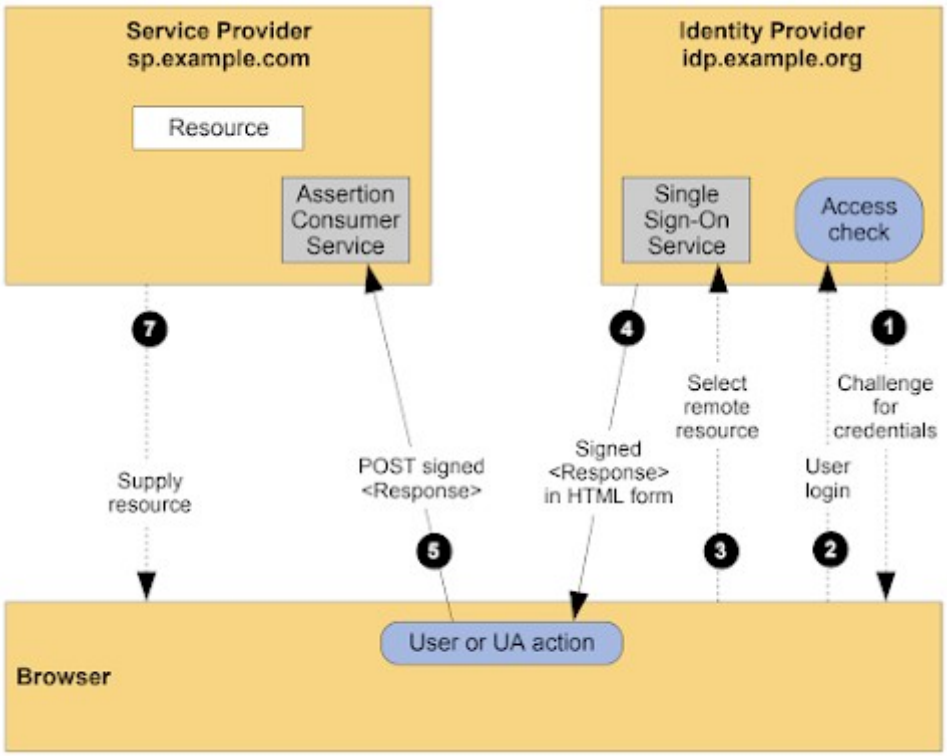
1. A user has a login session (that is, a security context) on a web site (airline.example.com) and is accessing resources on that site.
2. At some point, either explicitly or transparently, he is directed over to a partner's web site (cars.example.co.uk) (we assume that a federated identity for the user has been previously established between airline.example.com and cars.example.co.uk)
3. The identity provider site (airline.example.com) asserts to the service provider site (cars.example.co.uk) that the user is known (by referring to the user by their federated identity), has authenticated to it, and has certain identity attributes (e.g. has a “Gold membership”).
4. Since cars.example.co.uk trusts airline.example.com, it trusts that the user is valid and properly authenticated and thus creates a local session for the user



There are three entities:

- The user agent (typically browser)
- The Identity Provider
- The Service Provider

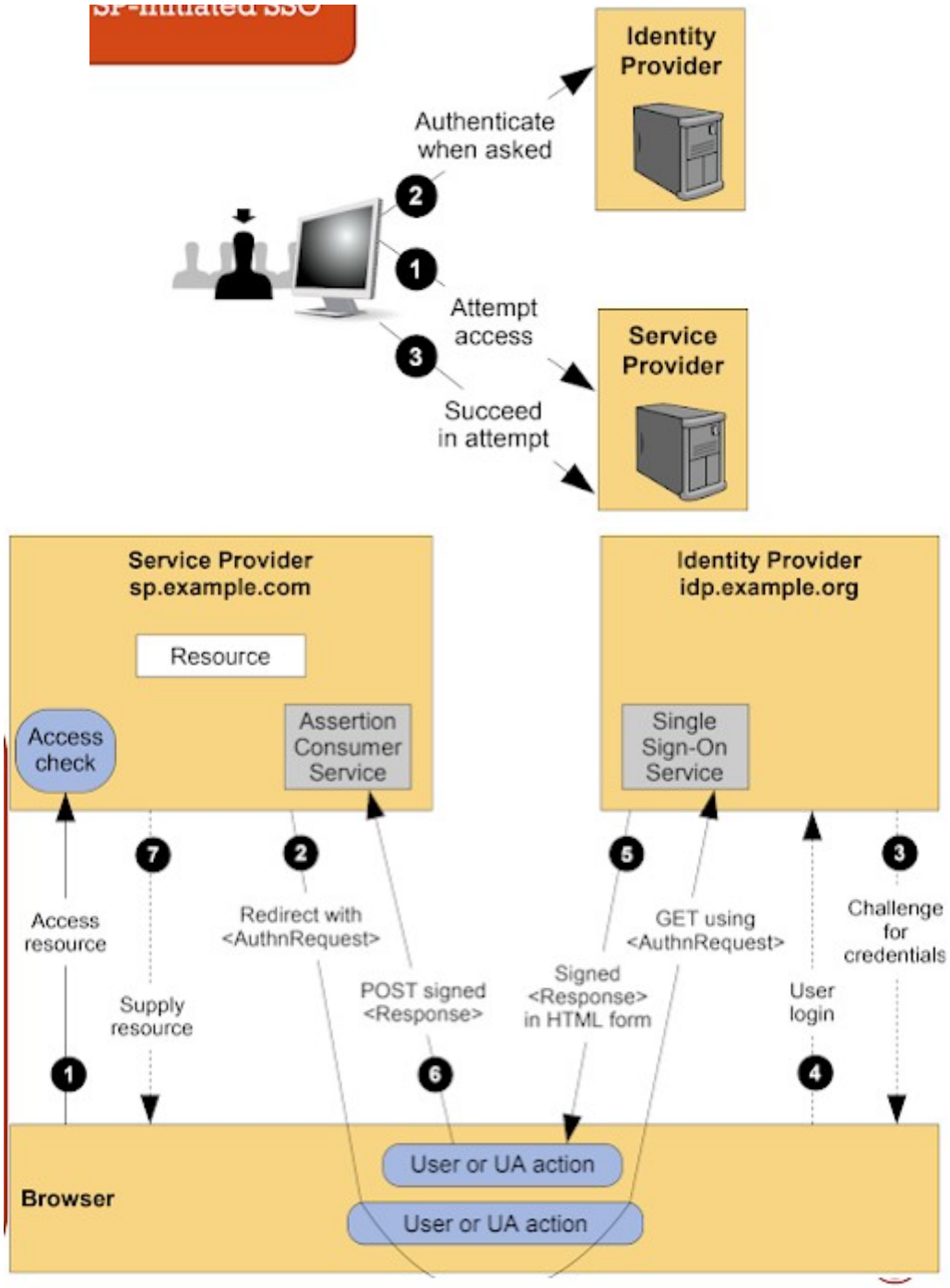
In this case some kind of assertion is passed from the Identity Provider to the Service Provider through the User Agent.



The flow is this:

1. User is challenged to supply credentials to the IdP site
2. User provides valid credentials and a local logon security context is created
3. User selects a menu option or link on the IdP to request access to an SP web site that trust that particular IdP; this causes the IdP's SSO Service to be called
4. SSO Service builds a SAML assertion representing the user's logon security context and redirects the user agent to the SP.
5. Browser issues an HTTP POST request to send a form to the SP's Assertion Consumer Service (that will check the signature on the assertion and the validity)
6. Final access check to allow/deny user access to resource

A more common scenario starts with a user visiting an SP site, possibly first accessing resources that require no special authentication or authorization.
When they subsequently attempt to access a protected resource at the SP, the SP will send the user to the IdP with an authentication request in order to have the user log in.
Once logged in, the IdP can produce an assertion that can be used by the SP to validate the user's access rights to the protected resource.



The flow is:

1. User attempts to access a resource on SP
2. SP sends a redirect response (signed in order that the IdP can check if it is an associated SP)
3. SSO Service determines whether user has an existing logon security context at the identity provider; if not, IdP interacts with user to provide valid credentials
4. User provides valid credentials and a local logon security context is created

5. IdP Single Sign-On Service builds a SAML assertion representing the user's logon security context (with his signature)
6. The browser issues an HTTP POST request to send the form to the SP's Assertion Consumer Service
7. Final access check to allow/deny user access to resource

Having seen how SAML works, let’s make some security considerations.
The standard doesn’t define a particular way to organize the metadata and exchange the metadata.
Since everyone can choose different options, people make a lot of wrong choices, and several many security issues may result.

First of all you need to set up some appropriate techniques to secure the exchange of messages. This requires TLS as minimum and possibly one additional layer of cryptographic techniques, in order to guarantee both confidentiality and integrity.

Just providing assertions from an asserting party to a relying party may not be adequate to ensure a secure system (what prevents a “man-in-the-middle” attack that might grab assertions to be illicitly “replayed” at a later date?).
SAML defines a number of security mechanisms to detect and protect against such attacks. Primary mechanism is for the relying party and asserting party to have a pre-existing trust relationship which typically relies on a Public Key Infrastructure (PKI). While use of a PKI is not mandated by SAML, it is recommended.

Use of particular security mechanisms are described for each SAML binding in the standard. General recommendations are the following:

- Where message integrity & confidentiality are required, then SSL/TLS is recommended
- When a relying party requests an assertion from an asserting party, bi-lateral authentication is required and the use of SSL/TLS using mutual authentication is recommended
- When a response message containing an assertion is delivered to a relying party via a user's web browser (for example using the HTTP POST binding), then to ensure message integrity, it is mandated that the response message be digitally signed using XML Signature

A set of problems that can be encountered using the SAML standard (that are inside the assertions) are:

- Message expiration
  SAML messages should contain a timestamp of when the request was issued, when it expires or both.
  If the SAML message never expires or if the expiration is not verified, there is a greater risk of a message falling into the hands of an attacker.
  Check the message for timestamps

- Message replay
  Assertions should contain a unique ID that is only accepted once by the application
- SAML from Different Recipient
  An application should only accept a SAML message intended for the SP application.
  If the application does not perform this check, it may accept a SAML message generated from authenticating to another application and allow an attacker into the application as the user from the other application
- XML External Entity (XXE)
  A SAML message is just a user-provided XML message that is processed by the Service Provider
  Check all standard XML attack vectors such as XXE that forces to parse malicious data

All of these precautions must be done by the implementer since they are not part of the SAML standard.

The last important thing to observe on SAML, is that there is a dramatic lack of privacy.
Privacy refers to both:

- a user's ability to control how their identity data is shared & used
- mechanisms that inhibit their actions at multiple service providers from being inappropriately correlated

An attacker that infiltrates an identity provider can view exactly what a specific user does online, analyzing the assertions.
One important thing about privacy is that an attacker should not be able to associate a certain action to a certain user. So instead of using a global identifier, we use a random string associated with the user in a secure way in our system.

SAML has a number of mechanisms that support deployment in privacy:

- **Persistent pseudonyms** established between an identity and a service provider. Pseudonyms do not themselves enable inappropriate correlation between service providers (as would be possible if the identity provider asserted the same identifier for a user to every service provider, a so-called global identifier)
- **One-time/transient identifiers**
  This ensure that every time a certain user accesses a given service provider through a SSO operation from an identity provider, that service provider will be unable to recognize them as the same individual as might have previously visited
- **Authentication Context**
  This allows a user to be authenticated at a sufficient (but not more than necessary) assurance level, appropriate to the resource they may be attempting to access at some service provider.

SAML allows the claimed fact of a user consenting to certain operations to be expressed between providers. How, when or where such consent is obtained is out of scope for SAML

National Identity Infrastructures

Let’s see the application of the SAML standard.

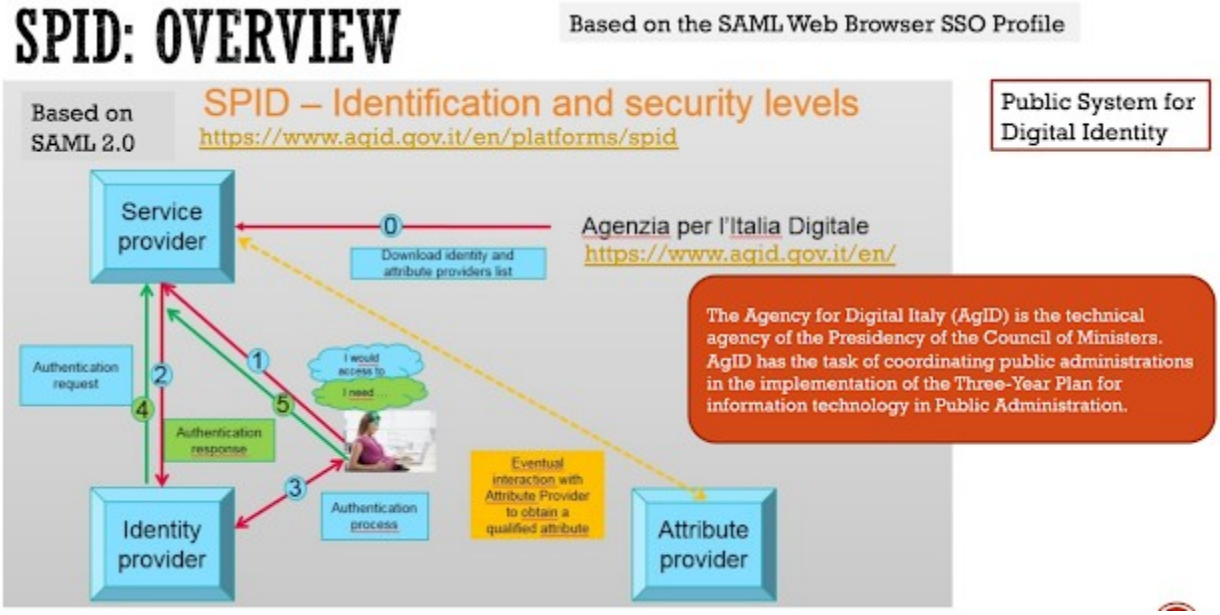
Some examples of National Identity Infrastructures are the italian ones:

- SPID = Sistema Pubblico Identità Digitale (Public System for Digital Identity)
  It is based on a particular profile of SAML 2.0: the SP-Initiated SSO
  This system concentrates all the attributes that appartain to an user in one (or few) place.
  This system usually has level 1 or 2 of assurance.
- CIE 3.0 = Carta d'Identità Elettronica 3.0 (Electronic Identity Card)
  It provides the user with a physical Card with a chip. This provides the user with NFC protocol plus cryptographic capabilities. So the credentials are allocated in our pockets.
  This system has level 3 of assurance.

The public administration services chose this approach due to the many advantages that it brought.

SPID

The idea underlying SPID is basically the idea underlying the SAML standard.
There are two main stakeholders: the IdP and the SP.



An example of Attribute Provider can be Motorizzazione Civile that provides (with an Attribute Assertion) the number of the car license.
When you use a SPID Identity Provider, it will ask for your confirmation of the attribute that will be sent to the Service Provider (typically the fiscal code, the email etc).

The Identity Provider at the start were few (Poste Italiane is one of the firsts and most important), nowadays there are more than 10 Identity Providers.

AgID (Agenzia per l'Italia Digitale) is the company in charge of managing and creating this system of Digital Identity in Italy. It provides the definitions, procedures and operational rules to set up the trust relationship between Service Provider and Identity Provider, and it contains the

metadata necessary for this scope. So it needs to update each new Identity Provider and Service Provider which joins SPID.

Repository of all the information related to the entities adhering to the SPID and represents the evidence of the so-called circle of trust established therein.
The relationship of trust on which the federation established in SPID is based is achieved through the intermediation of the Agency, third party guarantor, through the process of accreditation of digital identity providers, the attribute authorities and service providers.
Adhesion to SPID constitutes the establishment of a relationship of trust with all existing members accredited by the Agency, based on the sharing of the standard security levels of assurance declared and guaranteed by SPID.
Adhesion to the trust agreement among member entities is demonstrated by the presence of such entities in the SPID Register managed by the Agency.

The **federation registry** contains the list of entities that have passed the accreditation process and are therefore part of the SPID federation.
For each entity the registry contains an entry called AuthorityInfo consisting of:

- SAML identifier of the entity
- name of the subject to which the federation entity refers
- type of entity (Identity Provider, Attribute Authority, Service Provider)
- URL of the metadata provider service
- List of qualified attributes which can be certified by an Attribute Authority

The federation registry is populated by AgID following stipulation of the agreements and updated by said Agency during the activities related to management of the agreements and the supervision of the parties of the SPID circuit.

Here we are starting to see a new dimension of cybersecurity. Which is not only technical security but also compliance with laws and legal regulations. This topic is also important in the financial sector. It also requires integrity and confidentiality.

For this purpose, a correct process of identification and then authentication is fundamental. This is the preliminary step to obtain any kind of security assurance for accessing the service providers of these contexts.

Identity providers are private and each one provides a certain assurance level.
Legal provisions sometime get in the way (SSO seems impossible when considering higher assurance levels).
User identification for enrollment is delicate (Sometime ago, an Italian journalist “hacked” the procedure and successfully obtained the digital identity of a colleague…).

CIE 3.0

The CIE (Carta d’Identità Elettronica) stored personal information:

- Name
- Surname

- Place and date of birth
- Residency
- Holder's picture
- Two fingerprints (one of each hand), only if the applicant is aged 12 or over (can be accessed only by law enforcement agencies)
- Different validity periods (10 years for adults aged 18 and above, 5 years for minors aged 3–18, 3 years for children aged up to 3)

The CIE has the capability of using NFC (Near Field Communication) which is a set of communication protocols for communication between two electronic devices over a distance of at most 4 cm. It provides a low-speed connection with simple setup.
In this way NFC devices can act as electronic identity documents and keycards.

It obviously also has cryptographic properties.

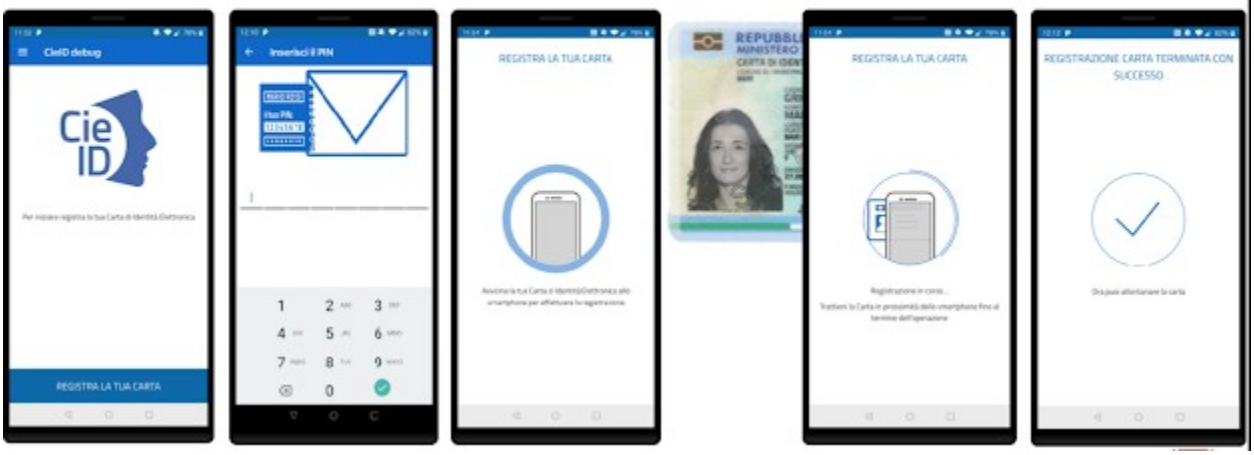
- AES (192, 256 bits) and Triple DES (112 bits)
- SHA-2 or SHA-1
- Diffie-Hellman 2048 bits and Elliptic Curve Diffie-Hellman 192 bits
- RSA 2048 bits (v1.5)
- X.509 certificate with personal data signed by official CA
- …

Inside the CIA there is a X.509 certificate with all the data that is about the citizen plus the public key associated with that particular citizen when the card was created. Obviously here also is the signature of the certification authority.

CIE can be used in place of SPID in public administration services:



CIE REGISTRATION ON CIEID APP

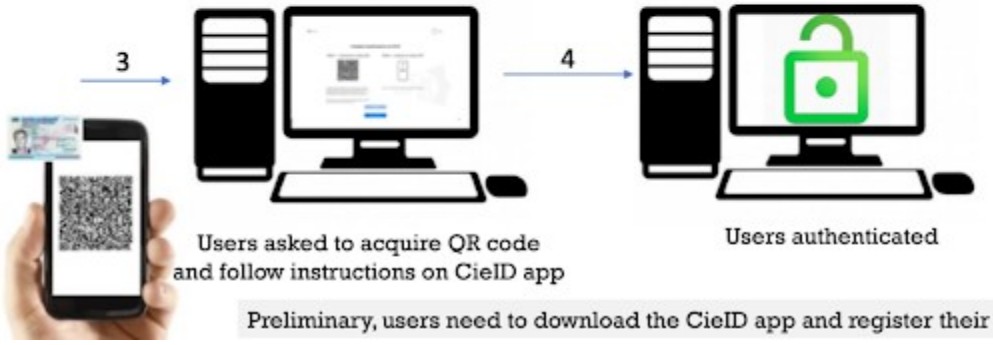




The SP shows the button "Entra con CIE"

Users warned that they will need CIE and smartphone

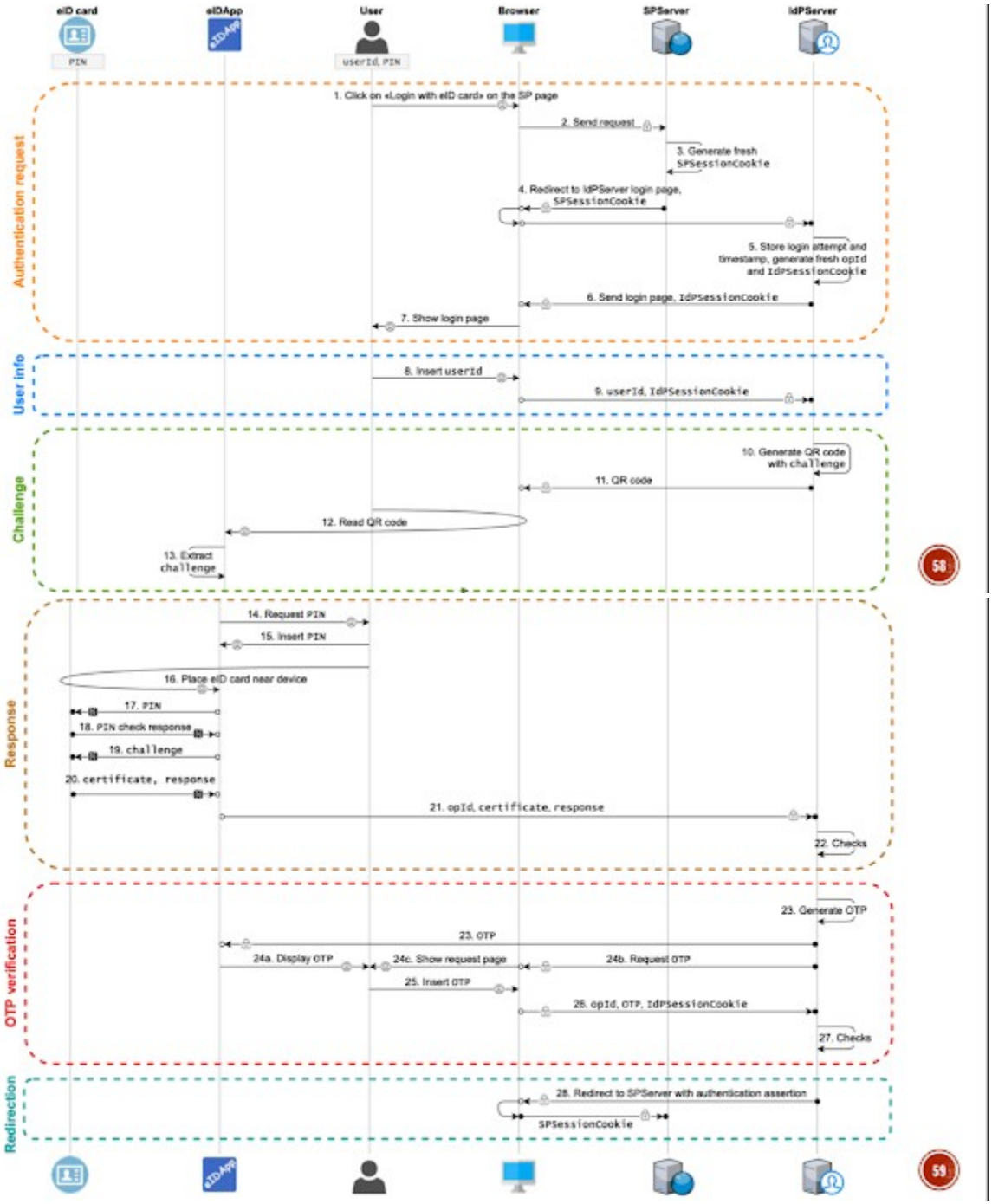
Users asked to enter the number shown on the CIE



Users asked to acquire QR code and follow instructions on CieID app

Users authenticated

Preliminary, users need to download the CieID app and register their cards

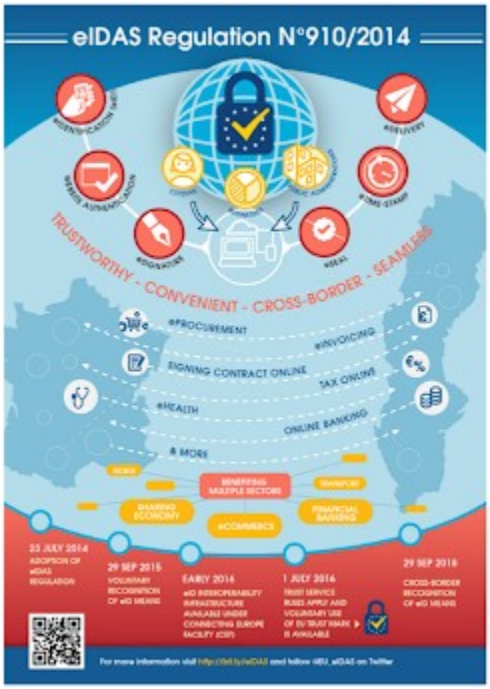


Due to legal requirements, CIE must introduce a username and password login, becoming similar to SPID and enlarging the attack surface.

European Identity Infrastructure

All the National Identity Infrastructures are “orchestrated” by this European Entity Infrastructure called eID.AS

It is a framework for the portability of the various National Identity Infrastructures and also provides regulations for digital signatures.
The idea is to have a unique single digital identity marketplace.

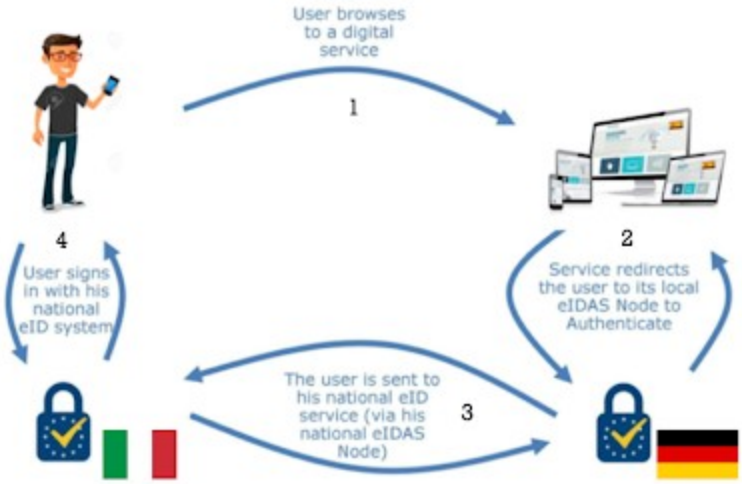


SPID is interoperable with eIDAS!

CIE is also interoperable with eIDAS!

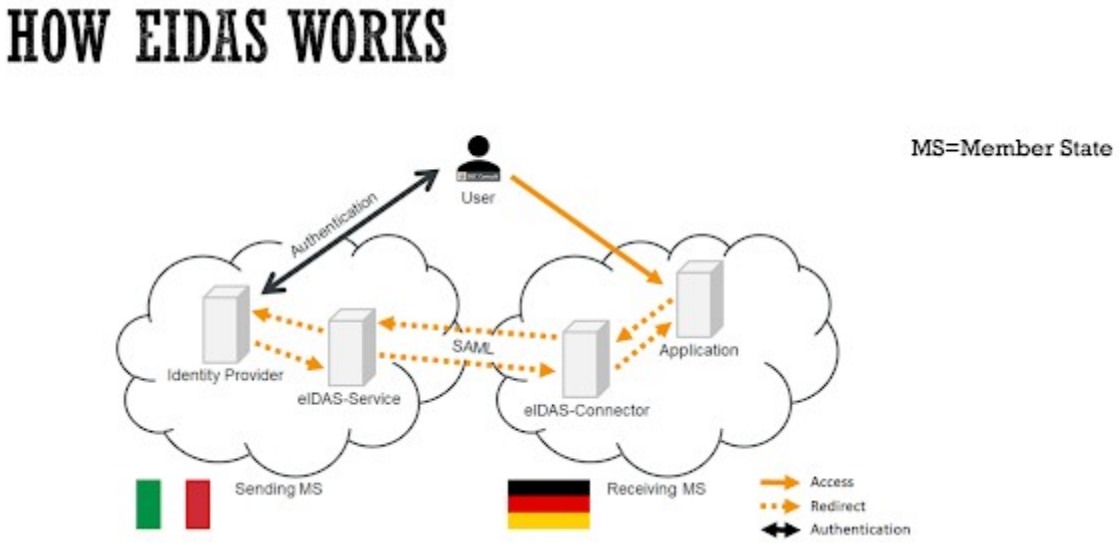
So the idea is that any citizen of any european country can access to any service of another european country with his National Digital Identity:

EXAMPLE: OPENING BANK ACCOUNT



So if an Italian citizen wants to authenticate against a German online service, first the German eIDAS-Node (eIDAS-Connector) is directed by the web application to initiate the authentication process.
It sends a request to the Italian eIDAS-Node (eIDAS-Service).

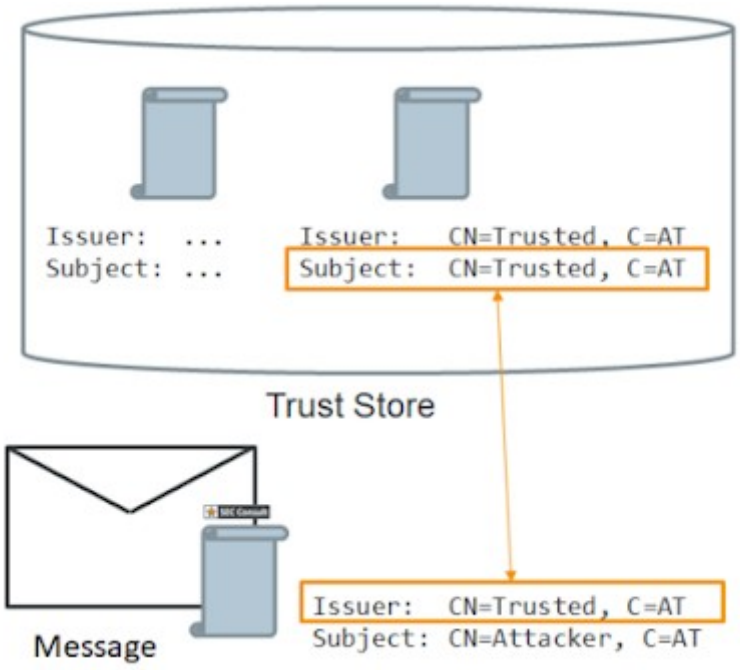
The Italian eIDAS-Node forwards the user to a system that is equipped to authenticate the Italian citizen using the national eID scheme.
After authentication, the German eIDAS-Connector receives the citizen's information which it forwards to the web application.
eIDAS relies on SAML for communication between the eIDAS-Connector and eIDAS-Service (called eIDAS-Nodes).



It's interesting to tell a little bit the story about a vulnerability found in 2019.
As each Member State must provide its own eIDAS-Node, the European Commission has provided the eIDAS-Node Integration Package, which can be used as a basis for implementing such a service (focus on the SAML parsing code).
SEC Consult found a vulnerability that basically allowed attackers to bypass the signature verification, allowing them to send any SAML message to an affected eIDAS-Node. So there was a wrong implementation in the signing verification process.
The attacker could therefore, e.g., send a manipulated SAML response to an eIDAS-Connector to authenticate as anybody.
The vulnerable code was used to verify the trust of the certificate the SAML response was signed with:

1. The certificate is accepted if it is in the local trust store
2. Otherwise, the issuer certificate of the entity certificate is retrieved from either the local trust store or from the supplemental certificates in the SAML message
3. If a trust path can be established between the issuer certificate and a certificate in the trust store, the entity certificate is accepted

In step 2 **the application does not verify whether the entity certificate has been correctly signed by the issuer certificate** (other checks, such as whether the basic constraints of the issuer certificate allow it to act as a certificate issuer are not verified).
An attacker can therefore sign a manipulated SAML response with a forged certificate.
The certificate must contain an Issuer DN that matches the subject of a certificate in the trust store.



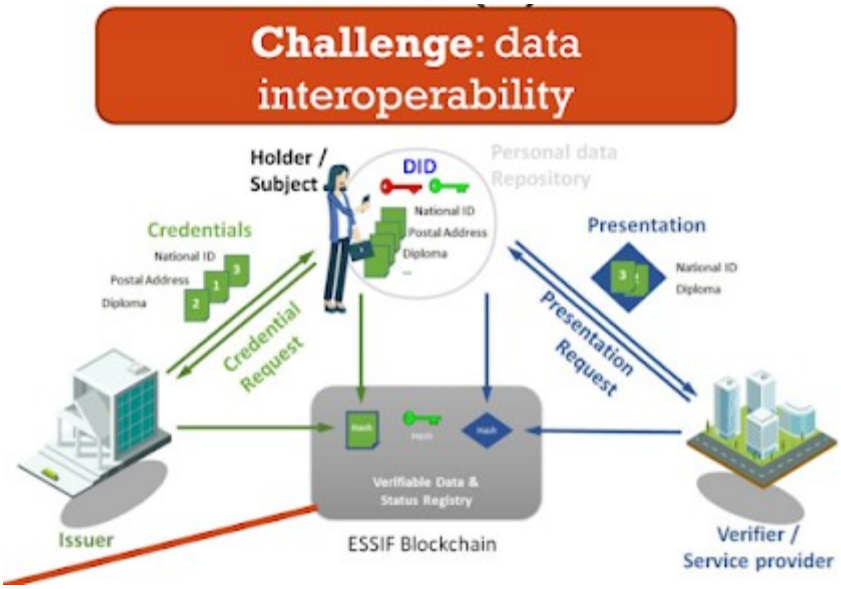
Recently eID.AS has been going through a revision that will become 2.0.
The new version of eID.AS will focus on **digital wallets**.
A personal digital identity wallet is a user-controlled app empowering the citizens to provide proofs of their identity or attributes so that the citizen is able to selectively share personal information with services by fishing (verifiable) credentials from the wallets.
So the idea is to put all the digital identity information in one place, using blockchain and decentralization.
An example is that citizens will use a mobile application that allows them to handle credentials and attribute's users.



So the goal is to no longer need a central trusted entity as before, but decentralize the trust and implement the trust by using algorithms (like blockchain or PKI).

The new systems will be based on:

- **Self-sovereign identity**
  Users control the verifiable credentials that they hold and their consent is required to use those credentials
- **Decentralized Identifiers**
  Identify subjects that the controller decides that they identify
- **Verifiable credentials**
  Digital equivalent of physical credentials such as credit cards, passports, driving licenses, qualifications and awards



This new system will be called ESSIF (European Self-Sovereign Identity Framework) and it will be compatible with eID.AS.

Important things to takeaways from this chapter are:

- SAML allows service providers to outsource identity management and focus on their core business (Reduced administration burden; Improved interoperability, usability, security and privacy)
- SAML profiles are useful use case scenarios
  Web SSO most widely adopted
  SSO has increasing importance and is gaining wider and wider adoption
- SAML is ideal starting point to build infrastructures for digital identity management
  Key enablers in an increasing digital world (SPID, eIDAS)
  First line of defense against attackers

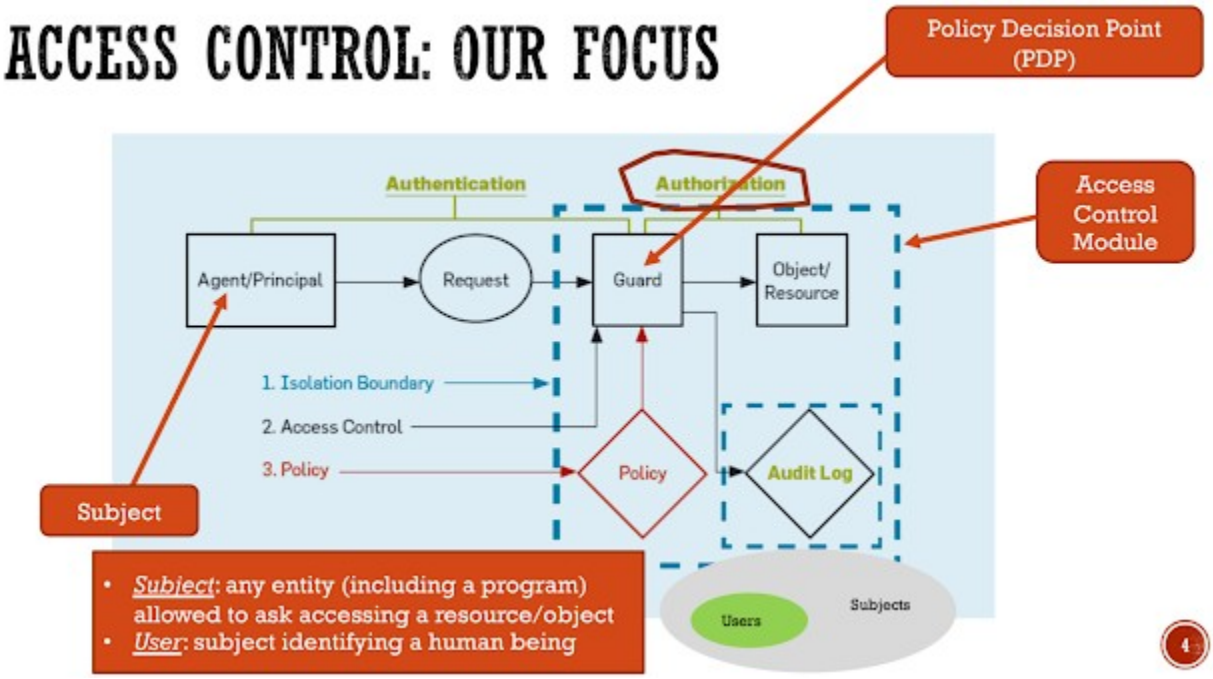
RECAP QUESTIONS

- What are the goals of SAML?
- What is the structure of a SAML assertion?
- What is a SAML profile? Give an example of SAML profile.
- What is the difference between IdP-Initiated and SP-Initiated Web SSO?
- What is the flow of an IdP-Initiated Web SSO?
- What is the flow of an SP-Initiated Web SSO?
- What are the main security concerns underlying the deployment of SAML? What are the main mitigations measures?
- What is SPID? What is eIDAS? Is there a relationship between the two?
- Give an example of scenario in which eIDAS is useful.

Access Control I

What is Access Control

Another phase of the digital identity lifecycle is the **authorization**.
The main argument of this chapter is that users are not equal in the point of view of a system, each user authenticated can do some things and can't do others. So we talk about privileges and permissions.
Access Control is a security service that supports the authorization procedures. In general Access Control is the security service to which people pay less attention.
The general architecture of Access Control is this:



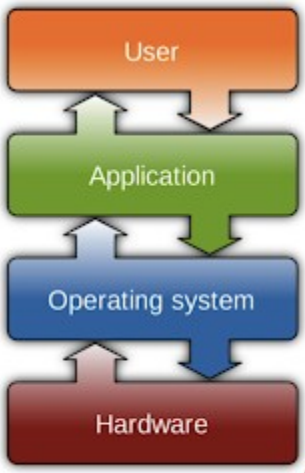
As we can see, authentication is a kind of preliminary step of authorization.
The blue dashed lines are **Isolation Boundaries**, which are particular mechanisms that guarantee that all authorization requests are intercepted by the Guard (pass through the guard).

- The outer one prevents the by-passing of the Guard: all authorization requests go through the Guard, are evaluated according to the policy, and in case they are granted, access is passed to the object or resource
- The outer boundary can be by-passed only by privilege users (such as administrators) that need to install/update policies or perform other routine administrative operations (e.g., updating the software implementing the Guard)
- The inner boundary guarantees the integrity of the audit log so that this can be inspected to understand what happened and why (i.e. to whom access was granted and for which reason)
- Contrary to the outer boundary, the inner boundary cannot be by-passed even by privilege users (such as administrators) as this would give rise to possible insider attacks (to the integrity of the log) that would make auditing meaningless

The **Guard** is a sort of engine which verifies and applies the Policy to the requests.
So the resources are protected by these Isolation Boundaries. Administrators can bypass the Guards to access and modify the Policy.

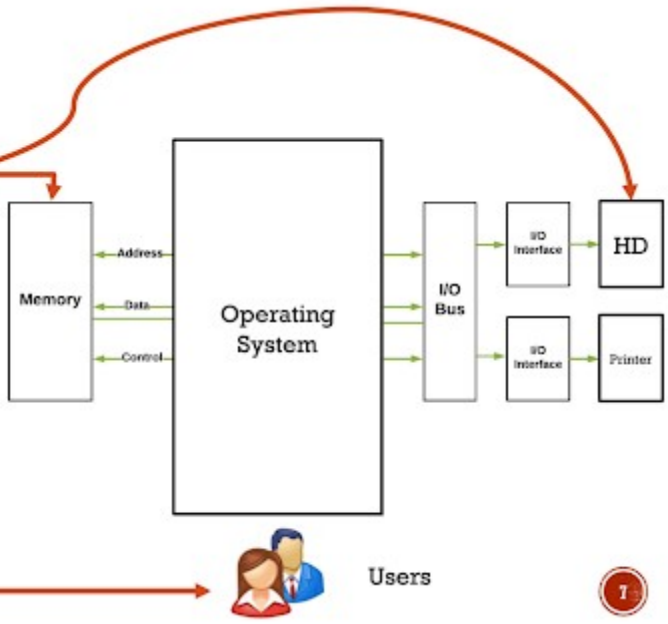
AN EXAMPLE: OPERATING SYSTEM (1)

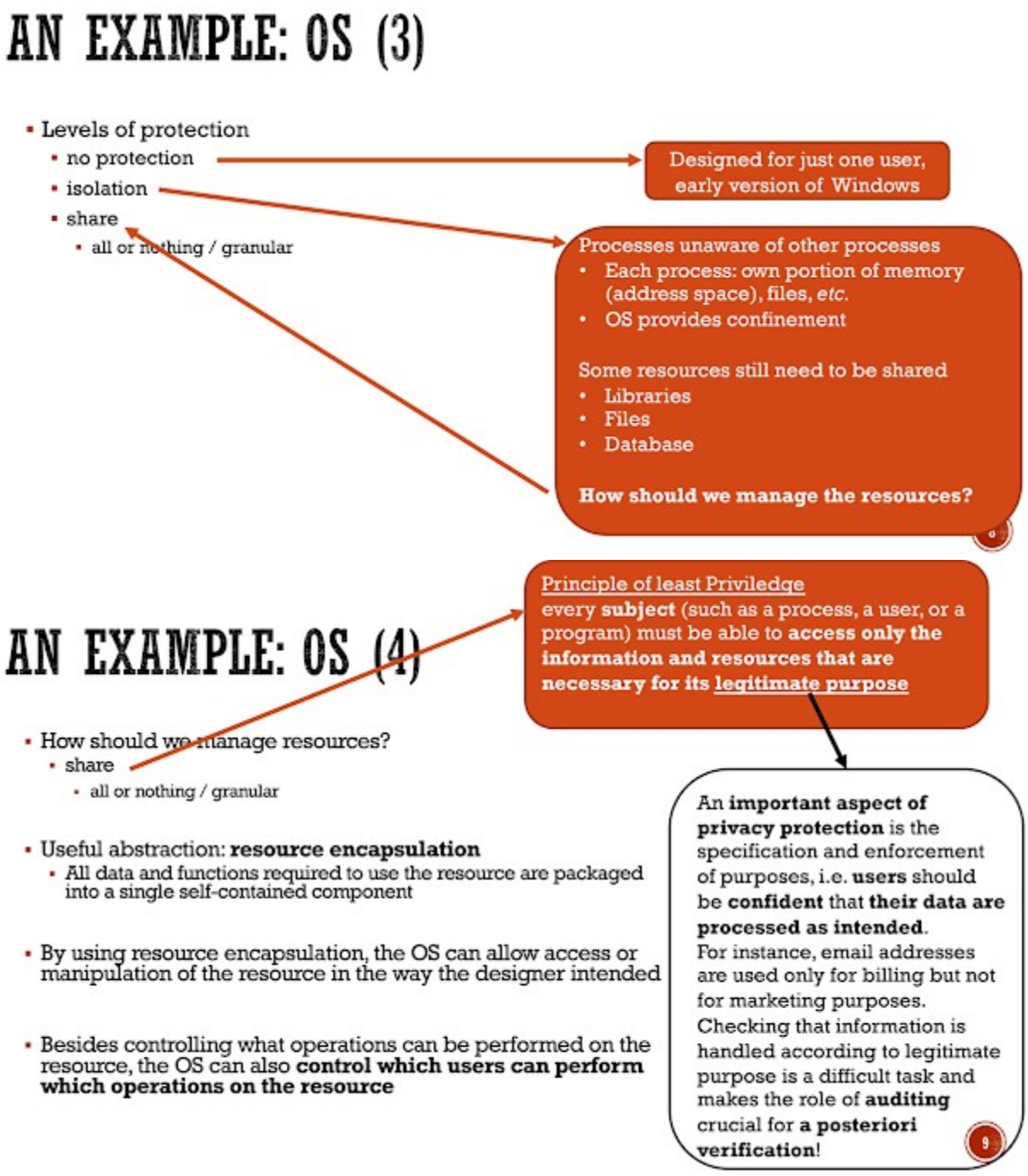
- An Operating System (OS) is a software that manages HW and SW **resources** and provides
  - common services for computer programs and
  - users with the capability of communicating with their computers
- A **multi-user** OS allows for multiple users to use the same computer at the same time and/or different times
- A **multi-tasking** OS is able to allow multiple software processes to run at the same time



AN EXAMPLE: OS (2)

- OS must protect users from each other
  - memory protection
  - file protection
  - general control and access to objects
  - user authentication
- The fundamental tradeoff of OS security
  - *Sharing* (desirable)
  - *Protection* (difficult)





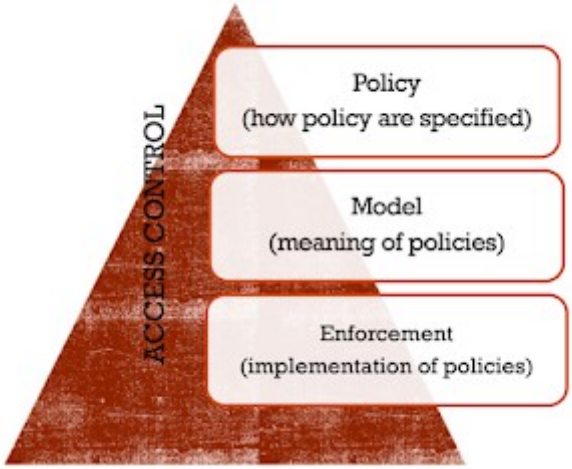
So **access control** is the process of **mediating requests** to resources and data of a system and **determining whether a request should be granted or denied**.
Having a subject s that want to perform an action a to a resource r, the flow of this process is:

- Access request (s, a, r) is sent to access control module
- The access control module returns grant/deny
- If answer = grant, then the system allows s to perform a on r.
  Else s is informed that cannot perform a on r

The policies (which “protects” the resources) depend on several different factors, first of all the context. The conditions of the policies usually depend on the three elements that we’ve seen before, but sometimes they don’t depend on them (time and date, location, etc).
Policy is the main device with which you define the goals and the rules to implement specific security properties.

In a structured approach to Access Control mechanisms we have this view:

- **Policy**
  Rules that control what actions, subjects may perform on resources (or objects) containing information, written down in a specific policy language
- **Model**
  Formal (mathematical) representation of the policy and its working (how to interpret the policy)
- **Enforcement**
  Low level (software or hardware) functions that implement the controls imposed by the policy and formally stated in the model



As always, the division of concerns brings a lot of advantages:

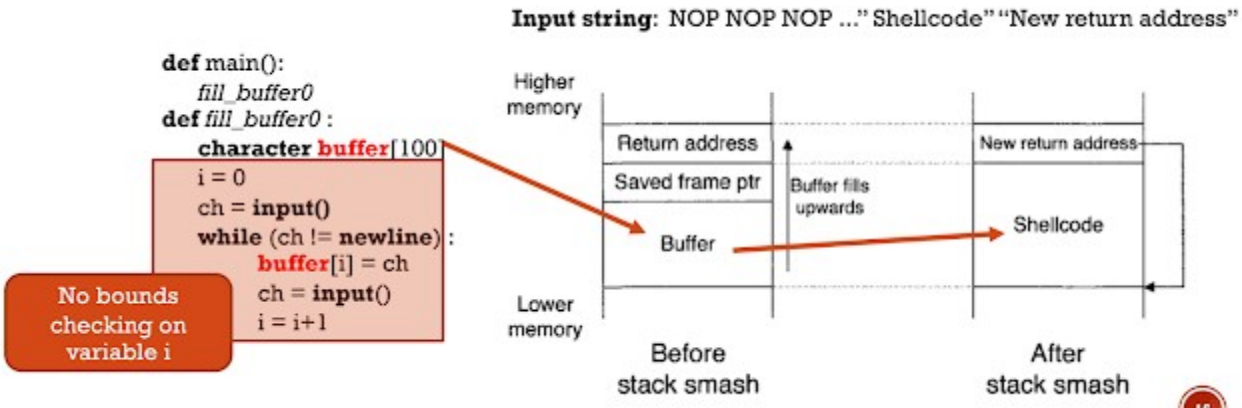
- When reasoning about policies, one can forget about their enforcement, just assume that the latter works correctly
- When policy and model are defined as mathematical objects, the meaning of policies is independent of a particular implementation

Obviously something can always go wrong, so let’s see an example of enforcement which went wrong:

- Example from the past: Unix **finger** command (user information lookup program)
  - A widespread implementation would accept an **argument of any length, although only 256 bytes had been allocated** for this argument by the program
  - The result was that when an **attacker used the command with a longer argument, the trailing bytes of the argument ended up being executed by the CPU**
  - In 1988, large numbers of Unix computers were brought down simultaneously by **Morris worm**, which used the finger vulnerability and thus brought **memory-overwriting attacks** to the notice of the mass media

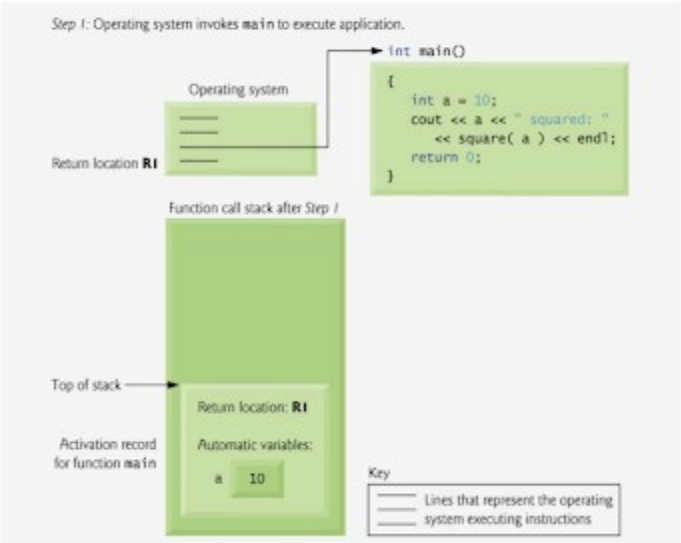
- Programmers are often careless about checking the size of arguments, still now and **holes continue to be found!**

Smashing the stack



16

FUNCTION CALL AND THE STACK

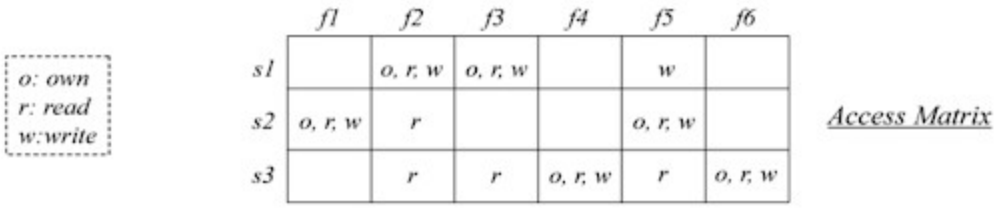


17

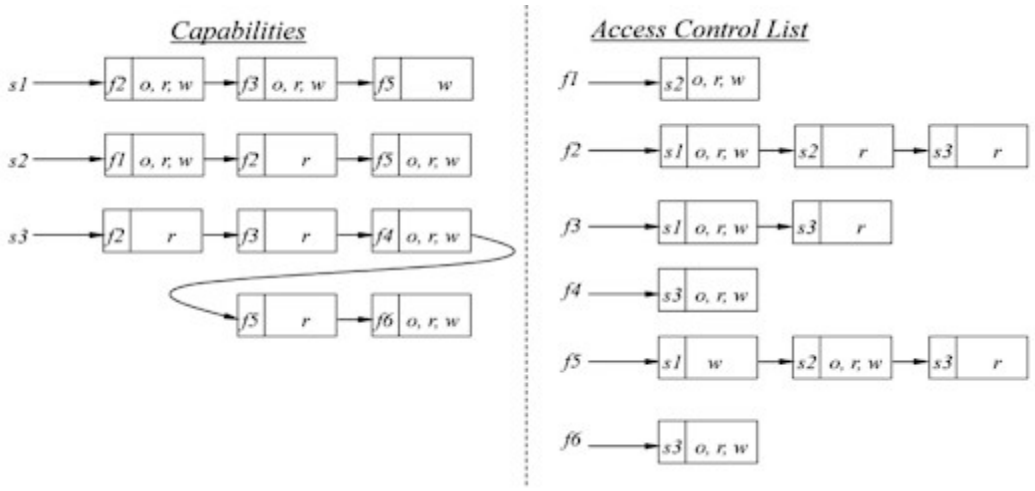
We will see which kind of models have been defined in order to express more and more conditions of the access control policies.

Access Control Matrix

This model is used in Operating Systems.
The meaning of policies of the form “a subject can perform some actions on an object” can be given in terms of the **Access Control Matrix** (ACM).
Given all subjects and objects, the ACM enumerates what actions are allowed for each subject/object pair.



Typically when you have an o (that means that you own the resources) you can both read and write it. There exists also x, that means execute.
The problem of this model is the loss of space, because usually you have a very large matrix which is sparse.
So, in order to optimize the space we can use lists to enforce this model. So we use the **list of capabilities of users** (that is extend the matrix along the rows) or the (most used) **access control list of resources** (that is extend the matrix along the columns):



These mechanisms require a little bit more access time but very less space, so they are a good tradeoff.
Any operating system uses Access Control List because it is particularly easy and convenient to use with existing operating system mechanisms, and also the Isolation Boundaries of Access Control List is easy to implement.
A very important thing to denote is that the Access Control List can be managed by the System, and whenever a subject wants to make an action to a resource, it simply checks the Access Control List of that particular resource. So the information is stored in the system (inside the Isolation Boundaries).
But in the case of Capabilities, the information is outside the system (because it is associated with subjects). So each subject should provide some evidence that he’s entitled to access a

particular resource. Obviously to send his list, he must use some cryptographic techniques to be trusted and perform integrity. So capabilities are more difficult to implement.

Obviously if the number of users is very large, then the lists will be very long.

Let's see the practical use in operating systems.
Operating systems has the goals of:

- Prevent malicious misuse of the system by users or programs
- Ensure that each shared resource is used only in accordance with system policies

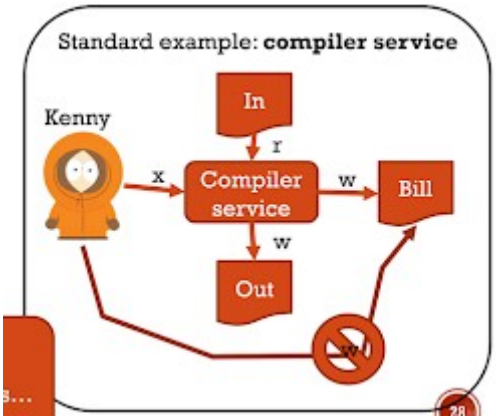
They follow the **principle of least privilege**, that is: programs, users, and systems shall be given just enough privileges to perform their tasks
The **key idea** is to ensure that failures do the least amount of harm and allow the least of harm to be done.
For example: each user is given their own account, and has only enough privilege to modify their own files. The System Administrator should also have an ordinary account, and reserve use of the root account for only those tasks which need the root privileges. The root account should not be used for normal day to day activities.

A computer can be viewed as a collection of processes and objects (hardware or software).
The **need to know principle** says that a process should only have access to those objects it needs to accomplish its task and only for the operations for which it needs access and only during the time frame when it needs access.
A **domain of protection** is the set of objects and the access rights, i.e. the ability to execute certain operations on selected objects.
Formally, a domain of protection is a set of pairs of the following form

$$< object , ¿ set ¿ ¿ >$$

- Domains of protection can be implemented as users, processes, or procedures
  - Example
    - In Unix/Linux, each user corresponds to a domain
    - A domain defines the access rights of a user
    - Changing domains involves changing user identifiers
    - **Programs running on behalf of a user inherit the user identifier and thus are associated to the same domain of protection**
    - **When an executable is given permissions to access certain (e.g., shared) resources that the user executing the program cannot access, then security issues may arise as we will see next…**

We said that Access Control Lists are preferred to Capabilities because they are simpler to implement and to obtain, but let's now see a use case in which Capabilities is better in terms of security.
Let's take in consideration a compiler service, which takes in input the names of the source file to be compiled and an output file.
The compiler service compiles the source file, writes the compiled binary to the output file and, on the side, writes a billing file that records how much the caller must pay for using the compiler.
The caller tricks the compiler by passing to it the name of the billing file in place of the output file, which causes the compiler to **overwrite the billing file** with a binary, thus destroying the billing file's integrity.



This type of attack is called **Confused deputy**, it consists in a privilege escalation attack where the adversary who does not have direct access to some sensitive resource, indirectly writes the resource by confusing a subject (called deputy) who can access the resource.

The compiler to execute with In as the source file and $Bill$ as the output file needs to perform the following authorization queries:

- $(Kenny,\ x,\ Compiler)$
- ¿
- $(Compiler,\ w,\ Bill)$

In ACLs, to guarantee the correct functioning of the compiler service, we assume the following ACLs:

- $Compiler \rightarrow [Kenny \mid x]$
- ¿ $\rightarrow [Kenny \mid o, r, w]$
- $Bill \rightarrow [Compiler \mid w]$

Here the authorization queries all are granted. The confusion arises by the fact that the Compiler is delegated:

- by Kenny to read its source file In
- by the OS to write the billing file $Bill$

During execution, the compiler cannot distinguish the source from which each right derives from and uses them both.

In capabilities, to guarantee the correct functioning of the compiler service, we assume the following capabilities:

- $Kenny \rightarrow [Compiler \mid x] \rightarrow [¿ \mid r]$
- $Compiler \rightarrow [Bill \mid w]$

There is no more confusion as Kenny needs to explicitly pass its capabilities to the Compiler:

- it can do this for reading the file In
- it cannot do it for writing to the billing file $Bill$

In conclusion, capabilities are better talking about security, because they avoid this kind of problem.

D(iscretionary) and M(andatory) AC

This model is used in Military Systems.
We have not considered something that is very important, that is who is defining the policies (and consequently who is defining the ACLs and/or the capabilities).
So we need to understand who can create what, and we have two solutions:

- **Discretionary** (DAC)
  Subjects can give rights to other subjects
  It is a democratic and flexible system but this flexibility can cause problems
- **Mandatory** (MAC)
  System enforces mandatory rules
  It is the opposite of democratic because there is a single entity that decides all, so it is restrictive, but have problems of single point of failure (if the entity that distributes the permission isn't available, then nobody can access the resources)

In **DAC** the owner of a resource decides how it can be shared. So there's NO central entity deciding who can do what on which resources. Owners can choose to give read/write/... access to other users on the resources they own.
It is typically used in operating systems (in fact it is usually enforced by ACLs). To simplify administration, users can be put in **groups**.

Groups are usually created by large organizations to put together employees according to their profiles and form these small numbers of categories. (Example: a bank might have 40 or 50 such categories: teller, chief teller, branch accountant, branch manager, and so on).
The remaining people (such as the security manager, and chief foreign exchange dealer,...), who need to have their access rights defined individually, may amount to only a few dozen people.
These observations can be exploited to simplify AC by identifying a small number of predefined groups to which (almost) each employee can be assigned. A group may appear in access control matrices or ACLs. Checking that a subject has a permission amounts to finding a group to which the user belongs and that appears in an ACL associated with that permission on a given resource.

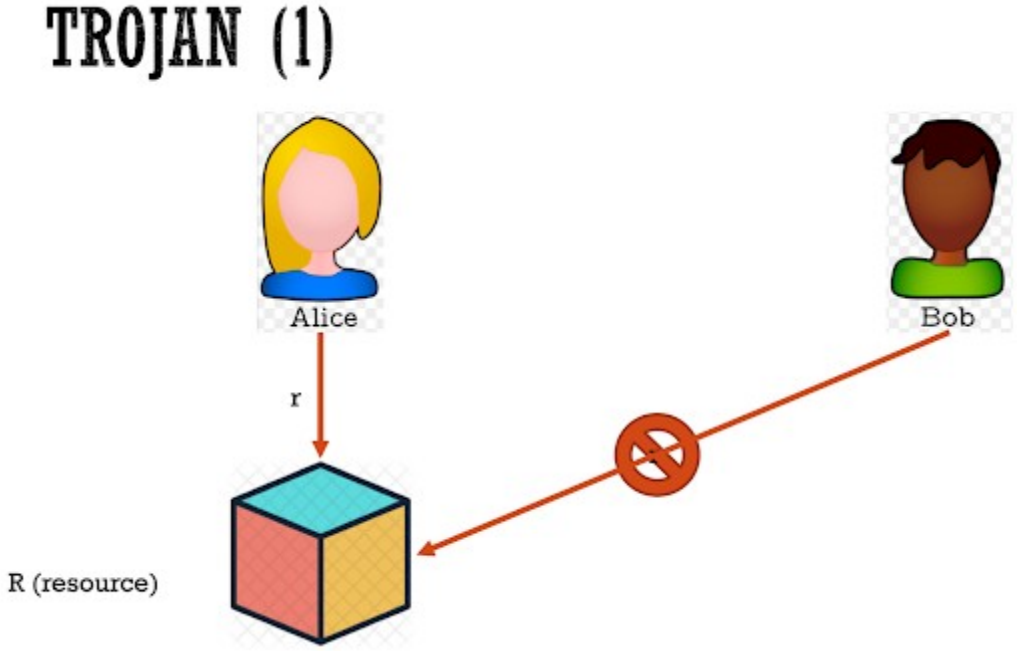
The pros of DAC are:

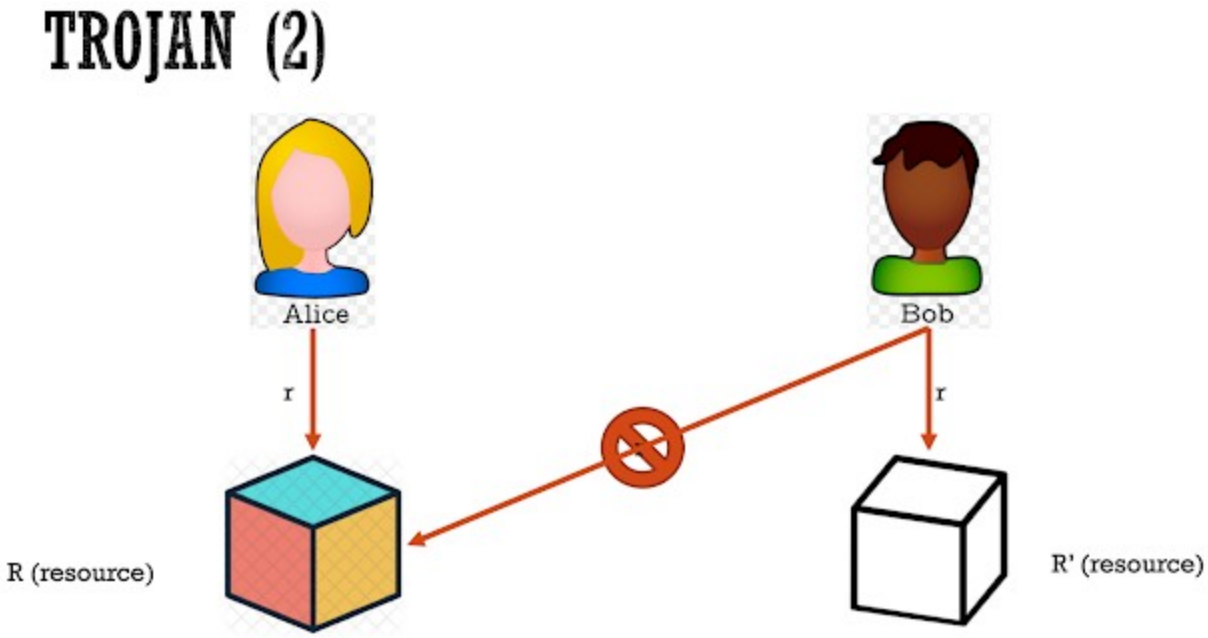
- The flexibility (that is the main reason of its popularity in OSes)
- The implementation (well understood)
- Intuitive

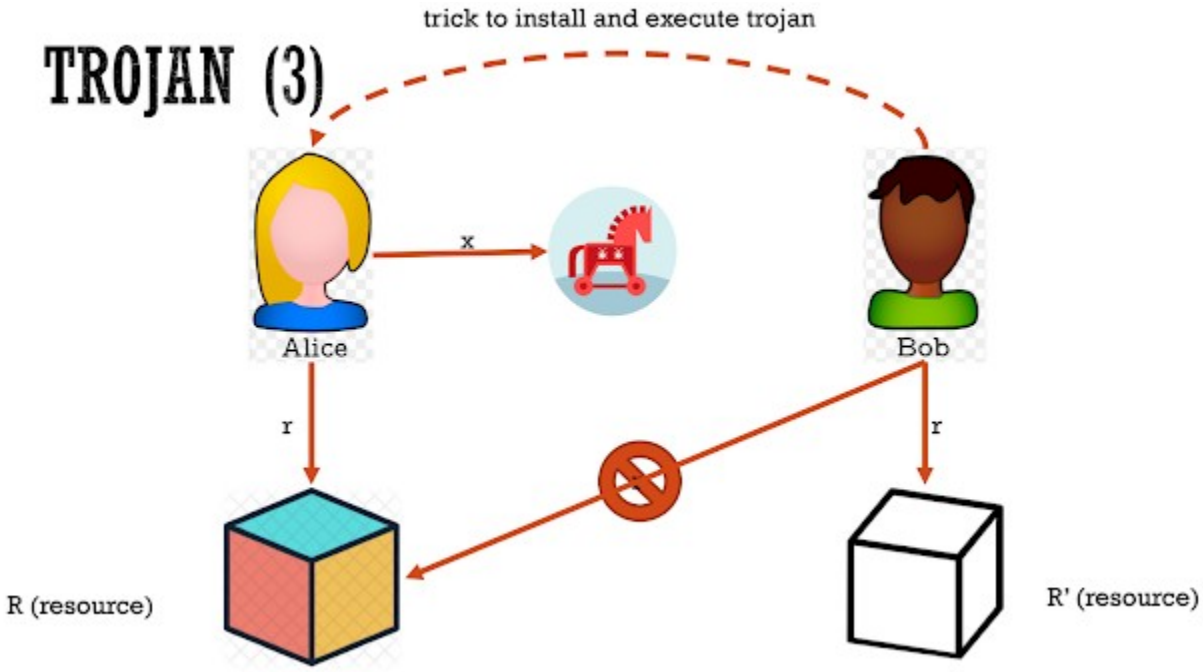
The cons are:

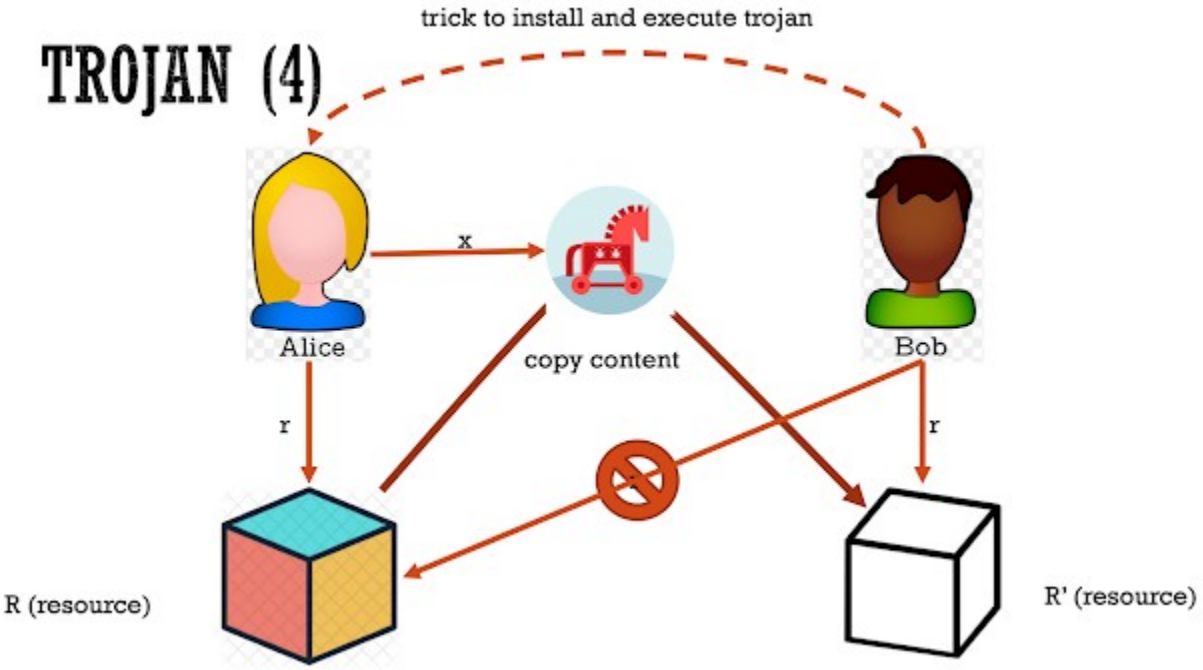
- It is subjective
  It rely on owner's judgement to modify the protection state of a resource
- It works if users make no mistakes (impossible): losing control of the situation is very easy!
- It is vulnerable to trojans (information leakage)

Trojan: any malicious computer program which misleads users of its true intent. They are generally spread by some form of social engineering, for example where a user is duped into executing an e-mail attachment disguised to be unsuspicious.









In the case of **MAC** the users operate in an organization, and it is the organization that decides how data should be shared. So there is a central entity deciding who can do what on which resources.
So DAC doesn't consider secrecy but gives importance to availability and sharing (used in operating systems), MAC is the opposite (used in military context).
For example hospitals own patients records and limit their sharing.

MAC is usually enforced by using security labels. This leads to **multi-level security models**.

An example is the protection of confidentiality in a military setting, where we have information at various sensitivity levels and users having various degrees of trustworthiness.
So the goal is to define policy to **prevent the release of sensitive information** (e.g., war plans) **to untrusted users** (e.g., janitors), with also the principle of "**need-to-know**".
But there are also cases in which even a user with a certain degree of trustworthiness doesn't need a particular information, and so it can't access it.

To categorize information with reference to sensitivity, information is compartmentalized into separate containers (resources such as documents, folders, or files) labeled according to their sensitivity label $L = (S, N)$, where:

- S takes values over a linearly ordered set (such as Top Secret >= Secret >= Confidential >= Unclassified)
- N is a set of "need-to-know" categories from an unordered set expressing membership within some interest group (e.g., Crypto, Nuclear, Janitorial, Personnel, etc).
  - Examples
    - label **(Secret, {Nuclear, Crypto})** indicates a resource containing sensitive information related to the categories "Nuclear" and "Crypto"
    - label **(TopSecret, {Crypto})** indicates a resource containing very sensitive information related to the category "Crypto"

The sensitivity label is associated with the resource at the creation time by resource originator according to some criteria. It is relatively easy to decide the interest group of a resource, but it is slightly more complex to give the sensitivity level, because it can be subjective.
When a document contains both sensitive and non-sensitive information, the originator needs to use the highest appropriate level.
When some conditions change, e.g. time, it may happen that the associated security label should be downgraded (de-classified) as the content has become less sensitive. So we need to introduce a mechanism to downgrade the sensitivity level of a resource.

Then, after associating sensitivity labels to resources, we must establish which users are authorized to access which resources.
So we assign clearances (or authorization levels), which have the same structure of the sensitivity labels associated to resources, i.e. each user is associated to a clearance $C = (S, N)$:

- S is a hierarchical security level indicating the degree of trustworthiness to which the user has been vetted
- N is a set of "need-to-know categories" indicating domains of interest in which the user is authorized to operate

Need-to-know categories reflect the Principle of Least Privilege: even within a given security level (such as Top Secret) not everyone needs to know everything.

The access control policies are defined by using sensitivity labels and clearances. There are two rules that allow us to achieve confidentiality (but not integrity).
The **No Read Up Property**:

- Subject s can read resource r if (S_s, N_s) dominates (S_r, N_r)

- Subject s asking to read the content of a resource r must show that its clearance dominates the sensitivity label of the resource

The **No Write Down Property**:

- Subject s can write to resource r if (S_r, N_r) dominates (S_s, N_s)
- Subject s asking to write to a resource r must show that its clearance is dominated the sensitivity label of the resource

(S_1, N_1) dominates (S_2, N_2) if:

- $S_1 \geq S_2$
- $N_1 \supseteq N_2$

Examples:

- (Secret, {Crypto}) dominates (Confidential, {Crypto}) since
  - Secret >= Confidential
  - {Crypto} ⊇ {Crypto}
- (Secret, {Crypto, Nuclear}) does not dominate (Top Secret, {Crypto}) since
  - Secret />= Top Secret despite the fact that {Crypto, Nuclear} ⊇ {Crypto}

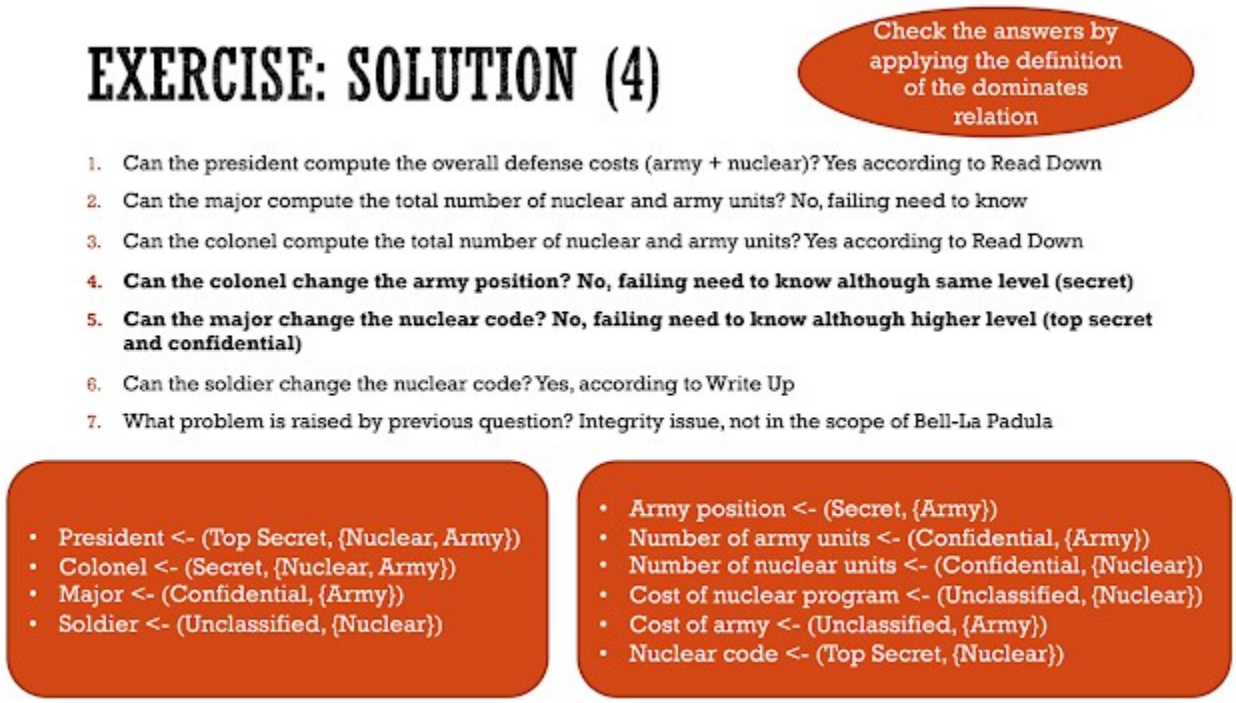
The No Write Down Property sounds strange but aims to prevent that a subject with access to a Top Secret file may copy the information into an Unclassified file. But in this way we don't guarantee integrity (we can define additional rules that are complementary to this to guarantee integrity).

An implicit assumption is the **Tranquillity Principle**, which prevents the ability to change security labels arbitrarily as this can subvert security.
Without the Tranquility Principle a subject s_1 with highest clearance can read a resource r_1 with highest sensitivity label, then s_1 change his clearance level to the lowest and can write to resource r_2 with lowest sensitivity label the content of resource r_1.
While both the basic properties of MLS are satisfied, confidentiality is not preserved as now s_2 (with lowest clearance) can read resource r_2 that also contain the information from r_1.

Now Read Up, No Write Down and the Tranquility Principle are at the heart of the **Bell-La Padula security model** introduced in 1973. Despite its age, it is a cornerstone of modern computer security and widely used in military applications.

Bell-La Padula security model has a shortcoming: A subject with the lowest clearance can change the content of a resource with the highest sensitivity label (for example a corporal with no clearance overwrite the war plan)
In 1977 Biba model was introduced, based on integrity labels (similar mathematical model, different meaning):

- Integrity label of a resource characterizes the degree of “trustworthiness” of the information contained in that resource
- Integrity label of a subject measures the confidence one places in its ability to produce or handle information



The pros of MAC are:

- It is not vulnerable to trojans because of the No Write Down property
- It is rigid (it is easy to keep the situation under control!)

The cons are:

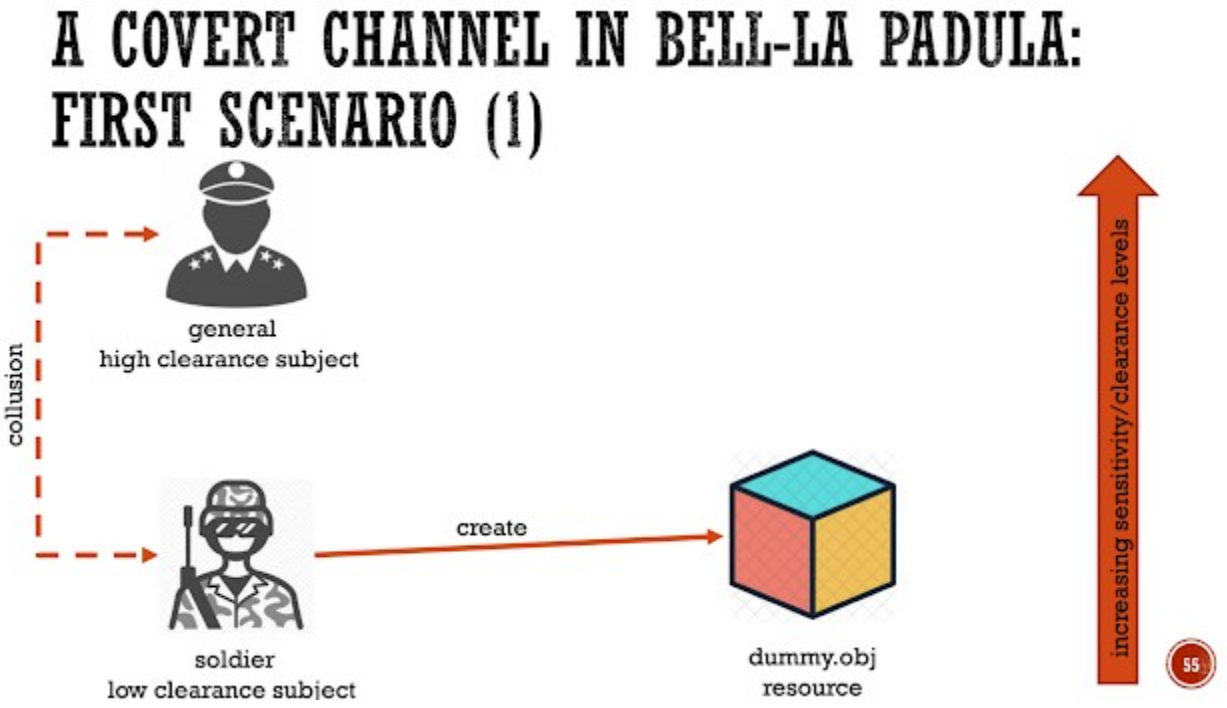
- It is rigid (may hinder business continuity)
- The information leakage is still possible by covert channel

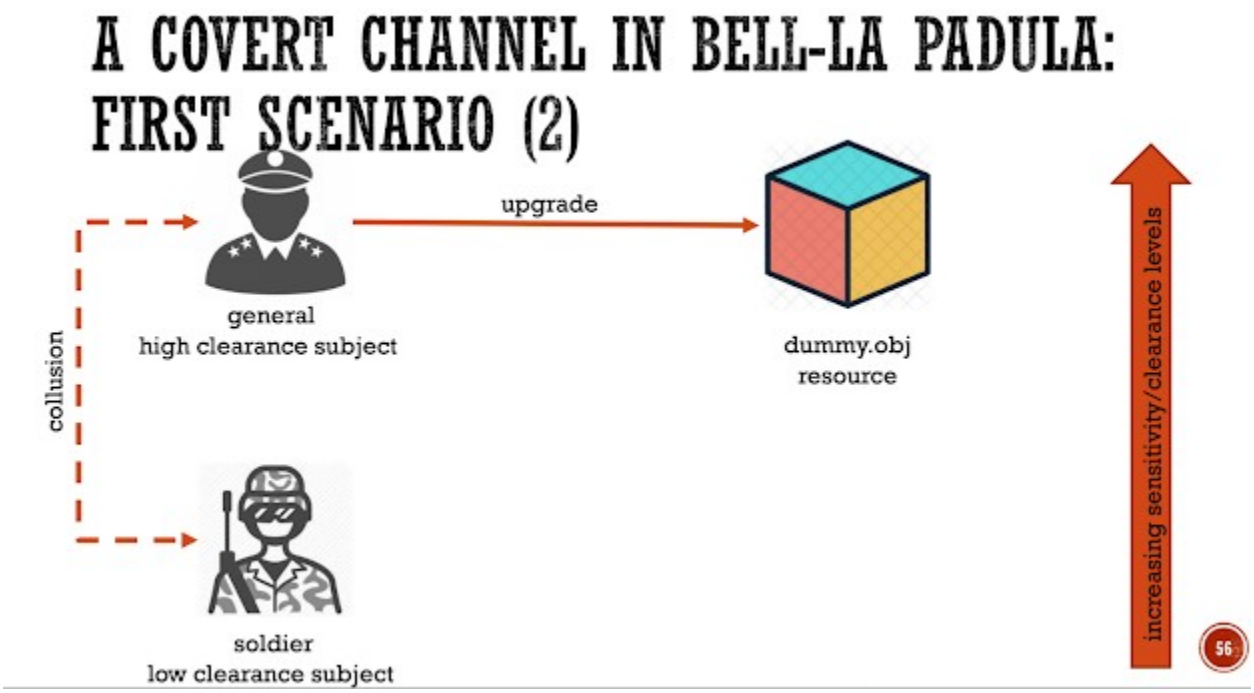
A **covert channel** is a path for the flow of information between subjects within a system, utilizing system resources that were not designed to be used for inter-subject communication.

Let’s see an example of covert channel in Bell-La Padula:

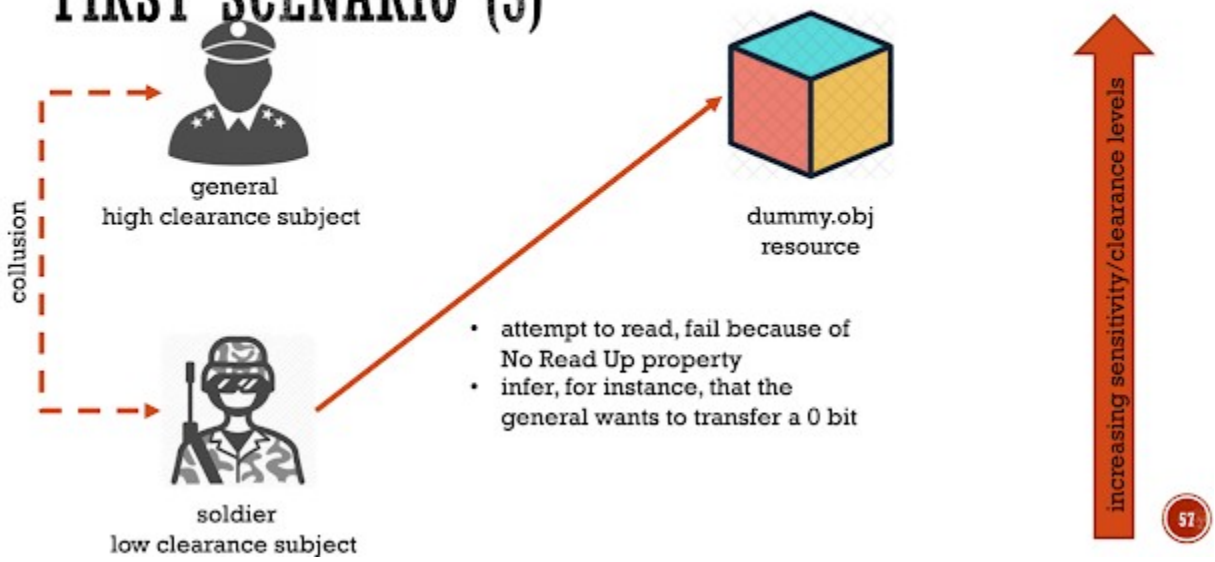
- A low level subject makes an object “dummy.obj” at its own level
- Its high level accomplice either upgrades the security level of dummy.obj to high or leaves it unchanged
- Later, the low level subject tries to read dummy.obj:
    - Failure means dummy.obj has be upgraded
    - Success means dummy.obj has not been changed
- On the base of this response, **one bit of information has flown from high to low**

So, the high level accomplice can send one bit of information to the low level subject.
If they can repeat this multiple times, the high level accomplice can send any amount of information to the low level accomplice!

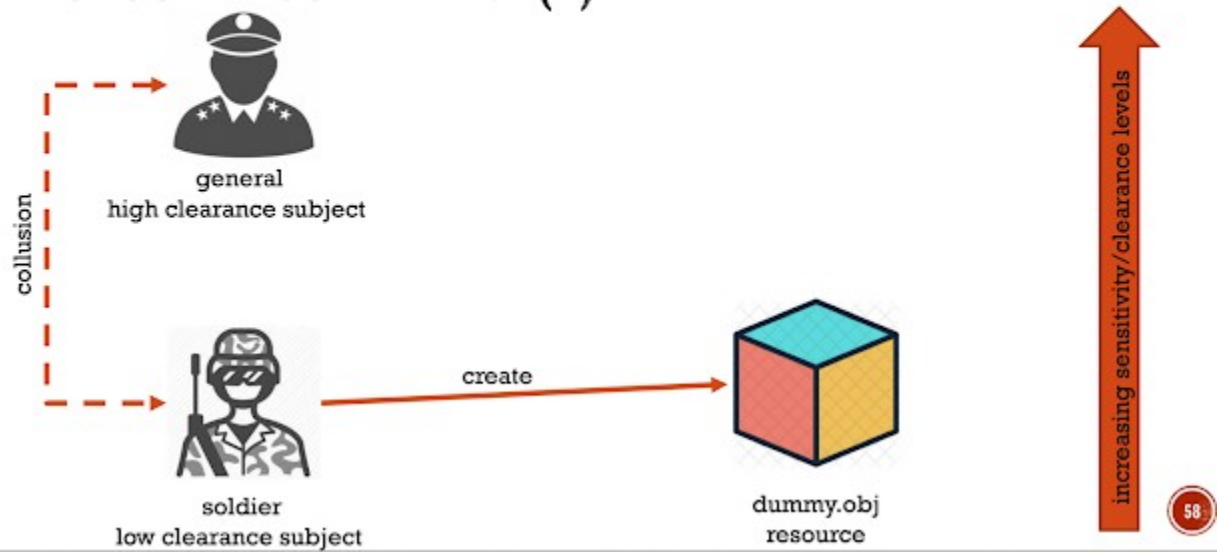


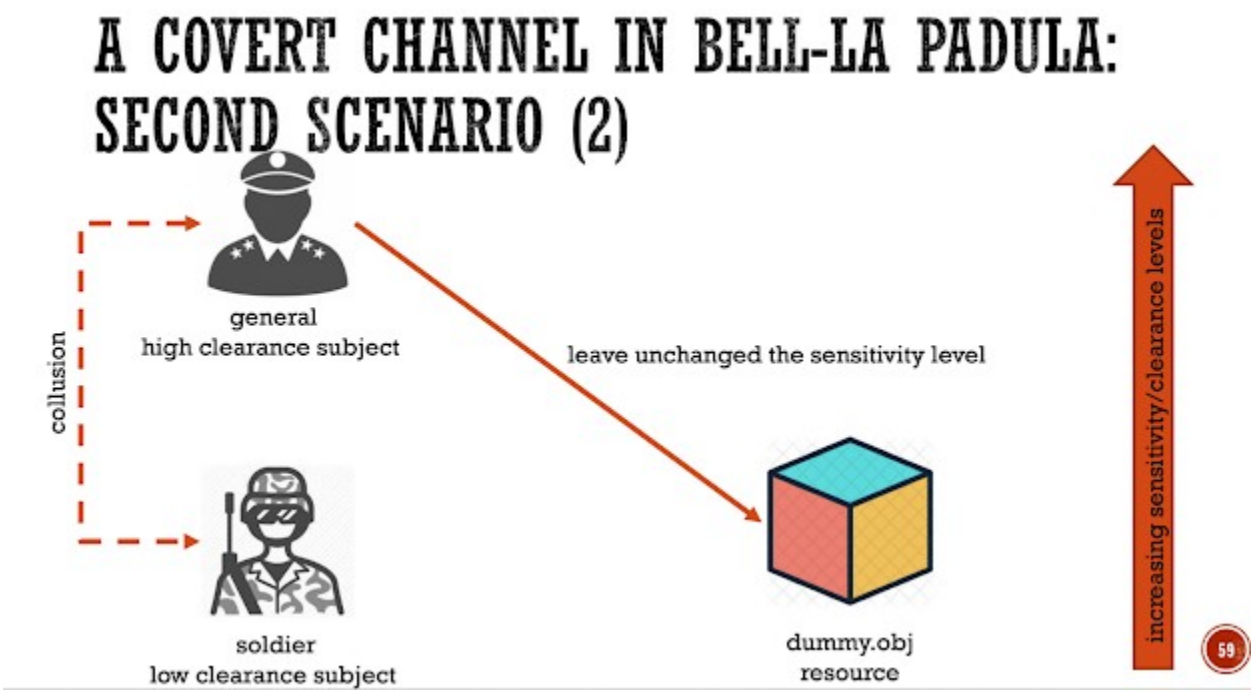


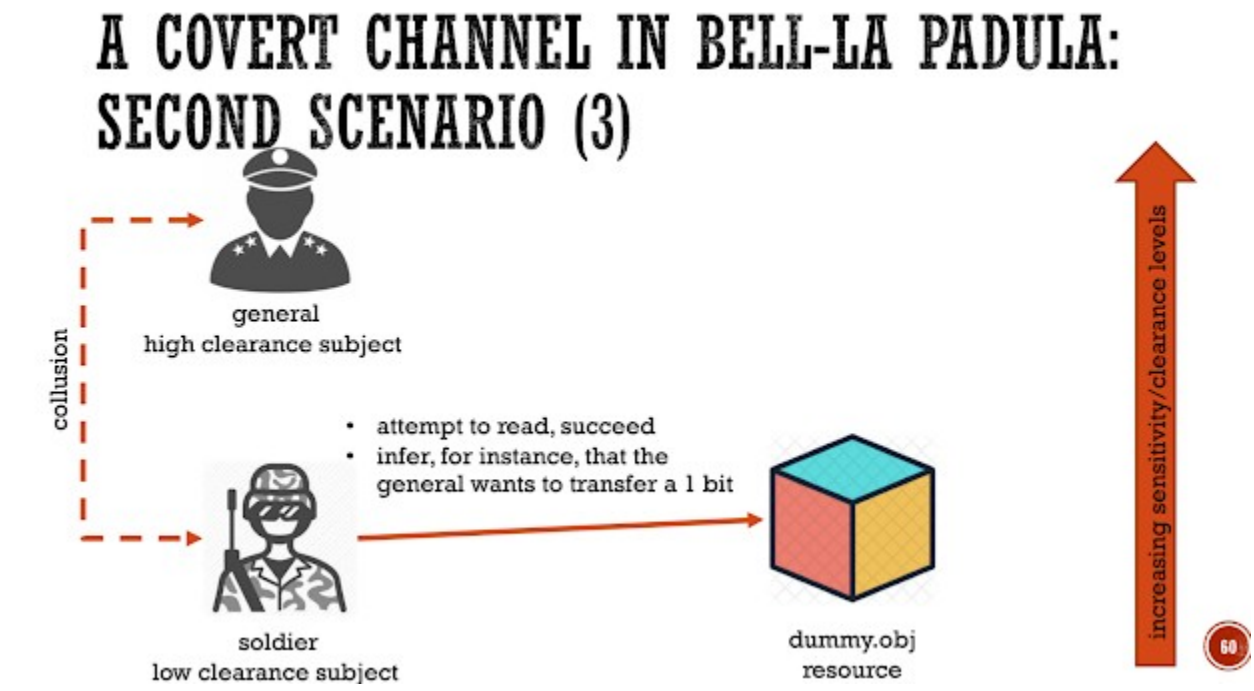
A COVERT CHANNEL IN BELL-LA PADULA: FIRST SCENARIO (3)



A COVERT CHANNEL IN BELL-LA PADULA: SECOND SCENARIO (1)







Role Based Access Control (RBAC)

This model is used in Enterprise Systems.

Let's first define a **permission**, that is an abstraction of $(action, resource)$ that can be assigned to more subjects. This is useful because often there are more subjects with the same capabilities.

The idea is to group together all the subjects which have the same permissions, defining **roles**.

EX: AT A SMALL UNIVERSITY

- Users: Alice, Bob, Charlie, David, Eve, Fred, Greg
- Permissions: GrantTenure, AssignGrades, ReceiveBenefits, **UseGym**, Register4Courses
  - A **permission** is an abstraction of (action,resource)
- Policy rules:
  - Alice has permissions: GrantTenure, AssignGrades, *ReceiveBenefits*, **UseGym**
  - Bob and Charlie have permissions: GrantTenure, AssignGrades, **UseGym**
  - David has permissions: AssignHWScores, Register4Courses, **UseGym**
  - Eve has permissions: *ReceiveBenefits*, **UseGym**
  - Fred has permissions: Register4Courses, **UseGym**
  - Greg has permission: **UseGym**

EX: AT A SMALL UNIVERSITY HOW TO TAKE AC DECISIONS?

- Look at the permissions
- It is possible to group them according to the profile of users or the set of functionalities users need to carry out their work!

User	Permission	
Alice	GrantTenure	Promotion Committee Member
Alice	AssignGrades	
Alice	*ReceiveBenefits*	
Alice	UseGym	
Bob	GrantTenure	Faculty Member
Bob	AssignGrades	
Bob	UseGym	
Charlie	GrantTenure	Faculty Member
Charlie	AssignGrades	
Charlie	UseGym	
David	AssignHWScores	Teaching Assistant
David	Register4Courses	
David	UseGym	
Eve	*ReceiveBenefits*	University Employee
Eve	UseGym	
Fred	Register4Courses	Student
Fred	UseGym	
Greg	UseGym	University Member

EX: AT A SMALL UNIVERSITY HOW TO TAKE AC DECISIONS?

- When a user is promoted or demoted (i.e. changes job and thus changes the set of functions needed to carry out its work)…
- … Administrators need to update the table very carefully for each permission!

User	Permission	
Alice	GrantTenure	Promotion Committee Member
Alice	AssignGrades	
Alice	*ReceiveBenefits*	
Alice	UseGym	
Bob	GrantTenure	Faculty Member
Bob	AssignGrades	
Bob	UseGym	
Charlie	GrantTenure	Faculty Member
Charlie	AssignGrades	
Charlie	UseGym	
David	AssignHWScores	Teaching Assistant
David	Register4Courses	
David	UseGym	
Eve	*ReceiveBenefits*	University Employee
Eve	UseGym	
Fred	Register4Courses	Student
Fred	UseGym	
Greg	UseGym	University Member

EX: AT A SMALL UNIVERSITY A 2ND LOOK AT THE POLICY

1. Alice is a member of the **Promotion Committee** and has permissions GrantTenure, AssignGrades, *ReceiveBenefits*, **UseGym**
2. Bob and Charlie are **Faculty Members** and have permissions GrantTenure, AssignGrades, **UseGym**
3. David is a **Teaching Assistant** and has permissions AssignHWScores, Register4Courses, **UseGym**
4. Eve is a **University Employee** and has permissions *ReceiveBenefits*, **UseGym**
5. Fred is a **Student** and has permissions Register4Courses, **UseGym**
6. Greg is a **University Member** and has permission **UseGym**

- Words in boldface identify **roles** to which both *users* and *permissions* are associated

EX: AT A SMALL UNIVERSITY
HOW TO TAKE AC DECISIONS AGAIN?

User Assignment (UA)

User	Role
Alice	**PCMember**
Bob	**Faculty**
Charlie	**Faculty**
David	**TA**
David	**Student**
Eve	**UEmployee**
Fred	**Student**
Greg	**UMember**

Permission Assignment (PA)

Role	Permission
PCMember	GrantTenure
PCMember	AssignGrades
PCMember	*ReceiveBenefits*
PCMember	UseGym
Faculty	AssignGrades
Faculty	GrantTenure
Faculty	UseGym
TA	AssignHWScores
TA	Register4Courses
TA	UseGym
UEmployee	*ReceiveBenefits*
UEmployee	UseGym
Student	Register4Courses
Student	UseGym
UMember	UseGym

User u has permission p iff there exists a role r s.t. $(u, r) \in UA$ and $(r, p) \in PA$

ADMIN IS SIMPLIFIED!

User Assignment (UA)

User	Role
Alice	**PCMember**
Bob	**Faculty**
Charlie	**Faculty**
David	**TA**
David	**Student**
Eve	**UEmployee**
Fred	**Student**
Greg	**UMember**

Reconsider the situation in which Alice resigns from being member of the Promotion Committee and becomes a Faculty Member

1. Delete the association (Alice, **PCMember**) from UA
2. Add the association (Alice, **Faculty**) to UA
3. PA is left unchanged!

Can we do better?

So it is easier for the administrator and the roles reflect the real organization of the company.

Now we can observe that some roles are subsets of other roles:

EX: AT A SMALL UNIVERSITY
A 3RD LOOK AT THE POLICY

Permission Assignment (PA)

Role	Permission
PCMember	GrantTenure
PCMember	AssignGrades
PCMember	*ReceiveBenefits*
PCMember	UseGym
Faculty	AssignGrades
Faculty	GrantTenure
Faculty	UseGym
TA	AssignHWScores
TA	Register4Courses
TA	UseGym
UEmployee	*ReceiveBenefits*
UEmployee	UseGym
Student	Register4Courses
Student	UseGym
UMember	UseGym

Set of permissions associated to **PCMember** is a superset of Set of permissions associated to **Faculty**

Set of permissions associated to **TA** is a superset of Set of permissions associated to **Student**

There are other relationships, find them out!

67

So we can create a **role hierarchy** (with the scope of denoting which role is a "subset" of another) and we can conserve a more compact version of the Permission Assignment relation.

EX: AT A SMALL UNIVERSITY
HOW TO TAKE AC DECISIONS YET AGAIN?

User Assignment (UA)

User	Role
Alice	**PCMember**
Bob	**Faculty**
Charlie	**Faculty**
David	**TA**
David	**Student**
Eve	**UEmployee**
Fred	**Student**
Greg	**UMember**

Permission Assignment (PA)

Role	Permission
PCMember	*ReceiveBenefits*
Faculty	AssignGrades
TA	AssignHWScores
UEmployee	*ReceiveBenefits*
Student	Register4Courses
UMember	UseGym
Faculty	GrantTenure
UEmployee	*ReceiveBenefits*

Role hierarchy ($\succeq$)

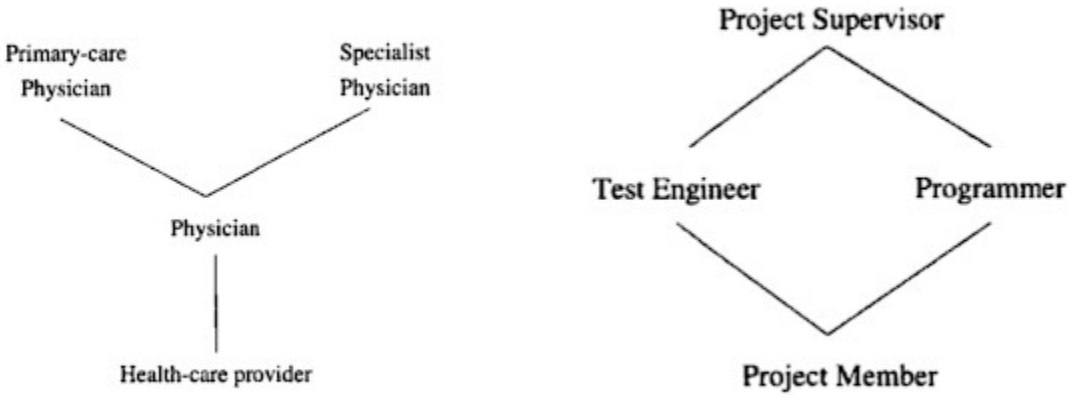
Least reflexive, antisymmetric, and transitive relation containing (**PCMember**, **Faculty**), (**Faculty**, **UEmployee**), (**TA**, **Student**), (**Student**, **UMember**), i.e.



Hasse diagram of partial order $\succeq$

User u has permission p iff there exist roles r, r' s.t. $(u, r') \in UA$ and $r' \succeq r$ and $(r, p) \in PA$

ROLE HIERARCHY: EXAMPLES

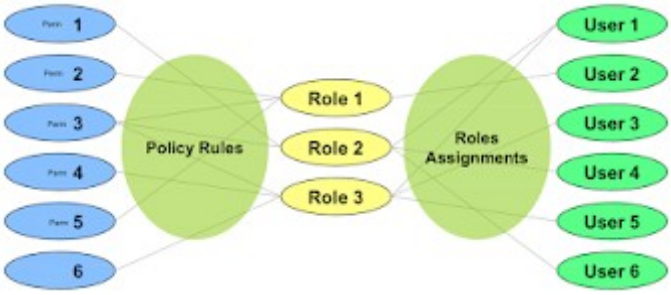


By convention more powerful (or senior) roles are shown toward the top of these diagrams, and less powerful (or junior) roles toward the bottom

This type of diagram is called **Hasse diagram** and it is a type of mathematical diagram used to represent a finite partially ordered set, in the form of a drawing of its transitive reduction.
The **transitive reduction** of a directed graph D is another directed graph with the same vertices and as few edges as possible, such that if there is a (directed) path from vertex v to vertex w in D, then there is also such a path in the reduction.

So, in a **Role Based Access Control** the permissions are assigned to roles rather than to individual users and users are assigned to roles rather than directly to permissions.
This level of indirection facilitates user-permission management and provides additional security benefits.

A role is a job function within the context of an organization, with some associated semantics regarding the authority and responsibility conferred on the subjects to whom the role is assigned. Roles allow to manage permissions indirectly through roles.



The difference between roles and groups is that groups are a collection of users, roles instead are a collection of users plus a collection of permissions (sometimes roles are defined simply as a collection of permissions).

Let's say something more on the various elements of this system.

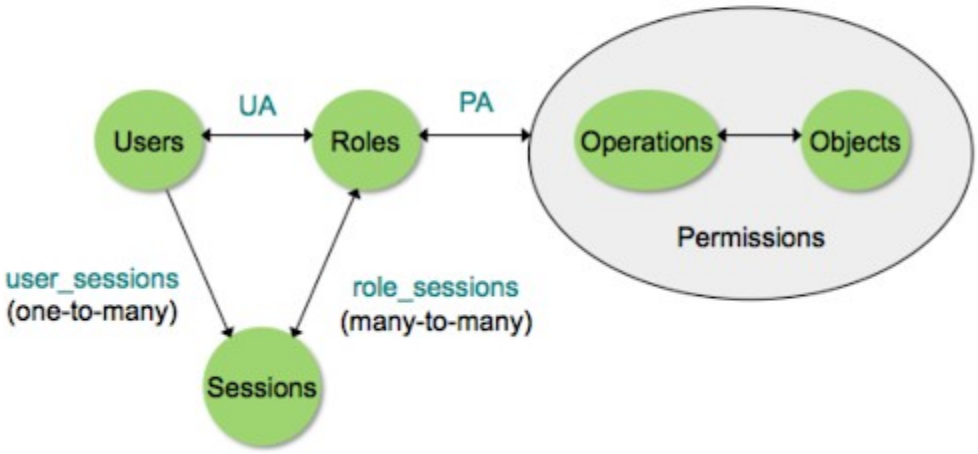
The **users** (subjects, principals) are human beings or other active agents (as programs, applications, ...). Each individual should be known as exactly one user (that's why authentication is a prerequisite!).

Talking about **user-role assignment**, a user can be a member of many roles and each role can have many users as members.

Talking about the **permission-role assignment**, a permission can be a member of many roles and each role can have many associated permissions.

Let's see a more precise model of this system:

CORE RBAC

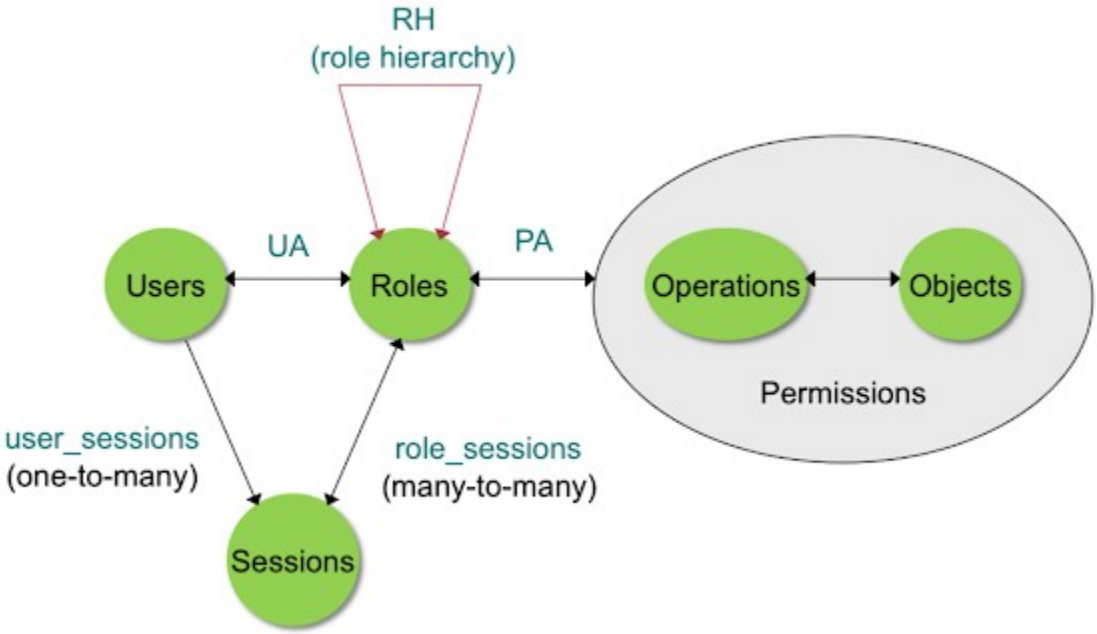


The upper part of this diagram (Users, Roles, Operations and Objects) formalizes the notions that we've already discussed.
A user can invoke multiple **sessions**. In each session a user can invoke any subset of roles that the user is a member of . Sessions are useful because a user doesn't always need all the roles that are associated with him, and this helps in implementing the Principle of least Privilege (and improves security).

Let's denote the set of users, roles, permissions, sessions with U, R, P and S respectively.
The many-to-many user-role assignment binary relation UA is a subset of the cartesian product $U \times R$.
The many-to-many permission-role assignment binary relation PA is a subset of the cartesian product $P \times R$.
The function user $S \rightarrow U$ maps each session s to a user u. Function user does not change during the session's lifetime.

The function roles $S \rightarrow 2^R$ maps each session s to a set of roles such that $\{r \mid (user, r) \in UA\} \supseteq roles$. Function roles may change during the session's lifetime since the user may decide to activate or deactivate some of the roles he/she is associated with in the relation UA.

The function permissions $S \rightarrow 2^P$ maps each session s to a set of permissions, that is the union of the set $\{p \mid (p, r) \in PA\}$ for each r in roles.



Let's add hierarchy to the core RBAC model.

The role hierarchy RH is a subset of $R \times R$ and it represents a partial order on R. (r, r') in RH also written as $r \geq r'$.

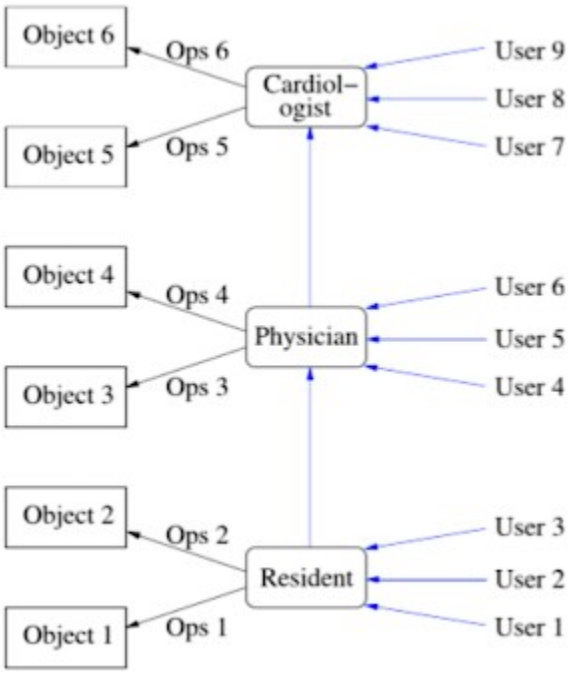
With role hierarchy we have to change the definitions of the functions roles and permissions.

The function roles $S \rightarrow 2^R$ maps each session s to a set of roles such that $\{r \mid there\ \exists\ r'\ such\ that\ r' \geq r\ \wedge\ (user, r') \in UA\} \supseteq roles$.

The function permissions $S \rightarrow 2^P$ maps each session s to a set of permissions, that is the union of the set $\{p \mid there\ \exists\ r'\ such\ that\ r' \geq r\ \wedge\ (p, r') \in PA\}$ for each r in roles.

EXERCISE

- Consider the RBAC model depicted on the right
- Write the UA relation
- Write the PA relation
- Assume the set S = {s1,s2} of sessions and define
  - users(s1) = User5 and
  - users(s2) = User9
- Write
  - roles(s1)
  - permissions(s2)



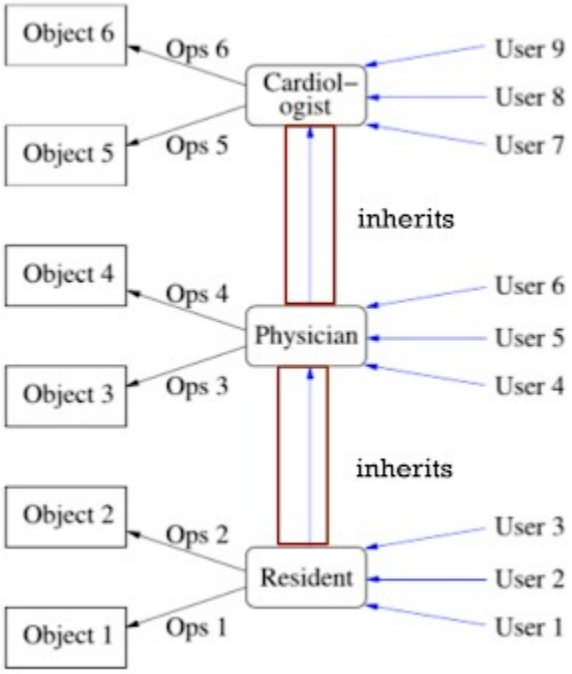
EXERCISE

- Draw the access matrix that corresponds to the RBAC model depicted on the right
- Hint

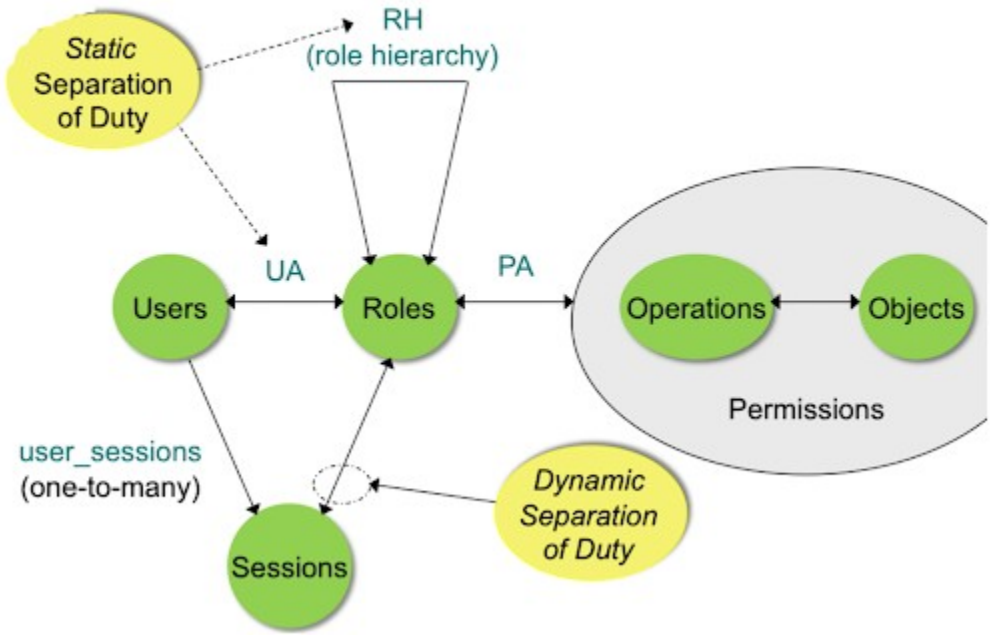
- Cardiologist >= Physician
- Physician >= Resident

Equivalently

- Permissions associated to Resident are inherited by role Physician
- Permissions associated to Physician are inherited by role Cardiologist



CONSTRAINED RBAC



Another possible extension of the RBAC model is the **constrained RBAC**, which introduces additional constraints that could be useful.
In certain situations, you need to guarantee some kind of additional requirements that cannot be specified by the basic RBAC model. In particular these additional requirements are called **separation of duty constraints**.
An example of separation of duty constraints is in a bank loan, where it is required that the employee that makes the frontend part is a different employee from who has to take the decision on the loan. This is done to avoid that the employee makes some agreements with the customer.
The separation of duty constraints can be:

- Static
  Which impose restrictions on user-role assignment relation (if the role hierarchy is present it has to be taken into account).
  These types of constraints are inflexible.
- Dynamic
  Which impose restrictions on the session and they are applied only on the active roles.
  These types of constraints are more flexible.

The pros of RBAC are:

- Easy to grasp the idea of roles
- Easy to manage in principle
  - Roles decouple digital identities from permissions
  - Simply assign roles to a new subject, instead of deciding on access for each resource
  - Easy to revoke authorization for a subject by removing roles

- Easy to tell through roles which permissions a subject has, and why

The cons are:

- Difficult to decide on the granularity of roles
  For example, should we create separate roles for modifying client information and for deleting a client, or not? Or is authorization too broad / still up-to-date for all subjects having this role)
- Role meaning is fuzzy
  For example, the position of employee in a company may be different from RBAC roles. Source of misunderstanding between admins and managers, for instance who assign them.

RECAP QUESTIONS

- What is access control and what is its basic architecture?
- What is an access control matrix? ACLs? Capabilities?
- What is DAC?
- What is MAC? Define No Read Up and the No Write Down principles.
- What are the differences, advantages and disadvantages of DAC and MAC?
- What is RBAC? How does it simplify administration?

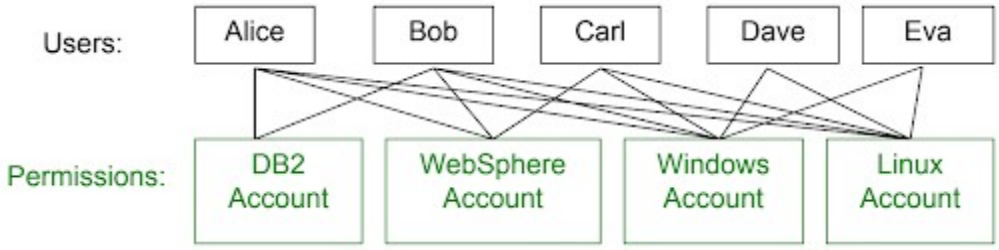
RECAP QUESTIONS

- Define the Principle of Least Priviledge
- What is a confused deputy?
- What is a trojan?
- What is a covert channel? How can a covert channel be created in MAC?
- How access control can mitigate command injection attacks?

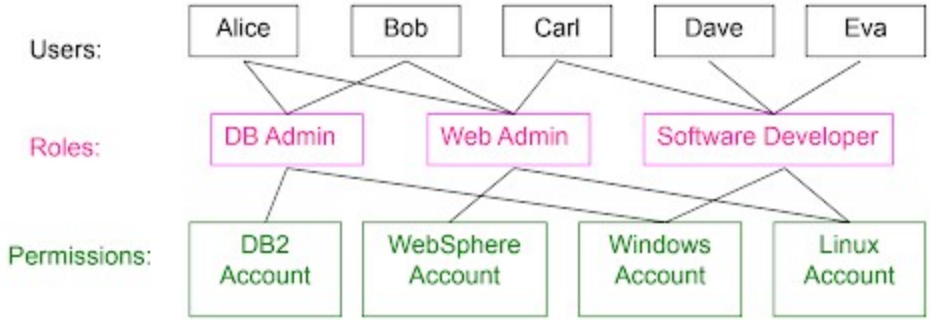
Access Control II

Now let's see a model that is a refinement of the ones that we've seen, which subsumes and extends them.
This model is called **ABAC** (Attribute Based Access Control) Model.

Let's first see the problems of the models that we've seen so far.
In **DAC and MAC** models, for a large number of subjects & objects, the number of authorizations can become very large. Another problem for dynamic user population is that the number of grant & revoke operations to be performed can become very difficult to manage.



With **RBAC** we have done better, because organizations operate based on roles and roles add a useful level of indirection (isolating/splitting the problem).
As we've seen, RBAC assigns permissions to roles in the organization, rather than directly to users. With roles, there are fewer relationships to manage (possibly from $O(m+n)$ to $O(m \cdot n)$ where $m = \#\ of\ \ users$ and $n = \#\ of\ \ permissions$).



But on the other hand, roles may not be enough for easily expressing authorization conditions.
For example conditions depending on:

- Additional attributes in the profile of a subject?
- Additional attributes of a resource? (for example meta-data of files)
- Environment attributes (for example time and location to specify that an employee can access a resource only in his working hours and in the building of the company)

All these conditions aren't supported in the models that we've seen.
To meet these requirements, RBAC has been extended in several deployments.
It is possible that a mixture of MLS and RBAC is needed in some situations; their combination is not obvious.

ABAC

The **ABAC** (Attribute Based Access Control) defines authorizations that express conditions on properties of both the resource and the subject. Each resource has an attribute (e.g., the subject that created it) and a single rule states ownership privileges for the creators.

The strengths of this model are:

- Flexibility and expressive power
- Possibility to combine different patterns of authorization conditions in a natural way
- Possibility to consider authorization conditions depending on environment attributes

ABAC can combine and express all the models that we've seen before.

So, the ABAC control access can be based on 3 different attribute types:

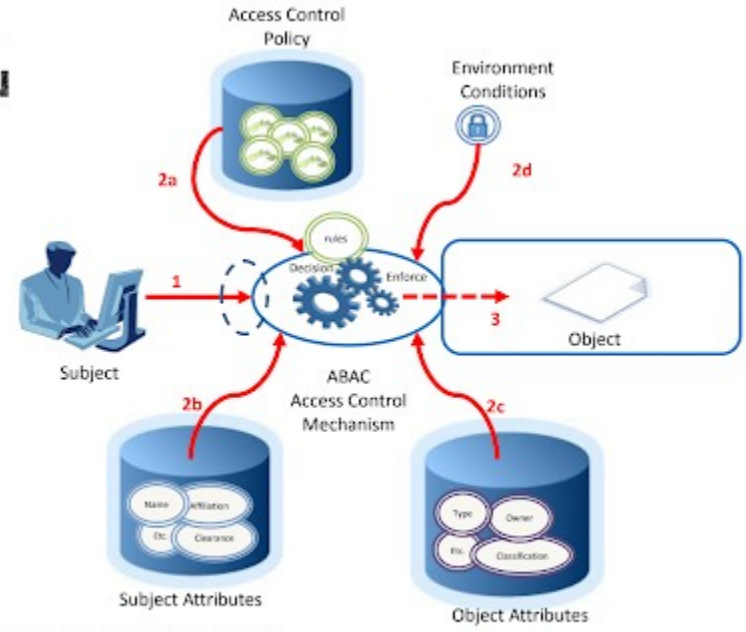
- User attributes
- Attributes associated with the resource to be accessed
- Environmental conditions

Any available attribute can be used by itself or in combination with another to define the right authorization condition for controlling access to a resource.

- Example
  - Allowing only users who are type=employees and have department=HR to access the HR/Payroll system and only during business hours within the same time zone as the company

The difference between RBAC and ABAC is that RBAC is for **coarse-grain** AC (for example giving all teachers access to Google), while ABAC is for **fine-grain** AC (for example giving teachers access to Google if they are at School X and teach Grade Y).

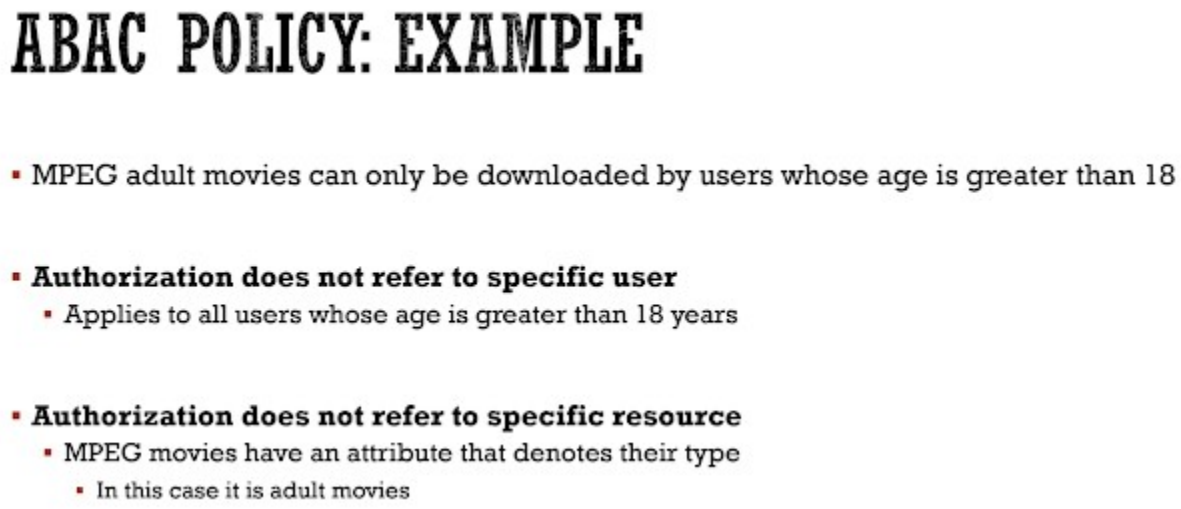
THE ABAC MODEL



1. Subject requests access to object
2. Access Control Mechanism evaluates a) Rules, b) Subject Attributes, c) Object Attributes, and d) Environment Conditions to compute a decision
3. Subject is given access to object if authorized

So, **subjects and objects are characterized and associated with sets of attributes**. These attributes are features which are relevant for taking an authorization decision. We can view a role in the RBAC model as a sort of an attribute.
Also the environment conditions are associated with attributes.
Authorization is expressed as conditions on these attributes.

An **ABAC policy** is a set of rules that govern allowable behavior within an organization, based on the privileges of subjects and how resources or objects are to be protected under which environment conditions. It is typically written from the perspective of the object that needs protection and the privileges available to subjects.
Privileges represent the authorized behavior of a subject and are defined by an authority and embodied in a policy.
So an authorization request (that is always the same thing: subject, action and resource) must "respect" the policy, which will obtain all the attributes that it needs from subject, resource and environment to evaluate the request.

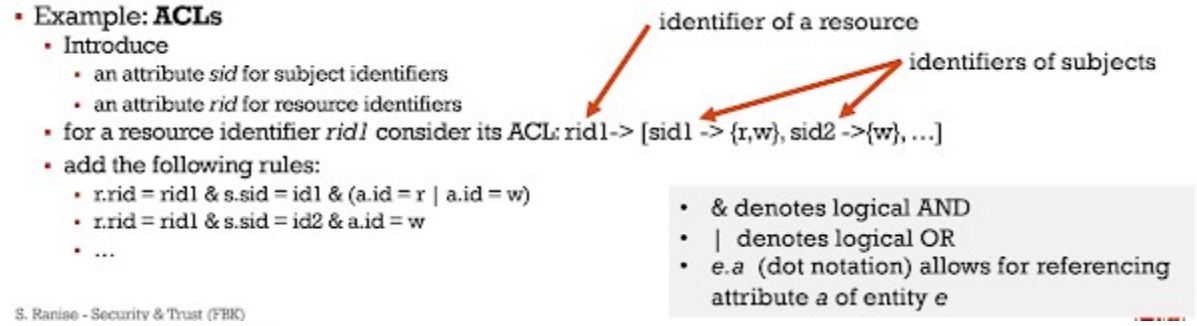


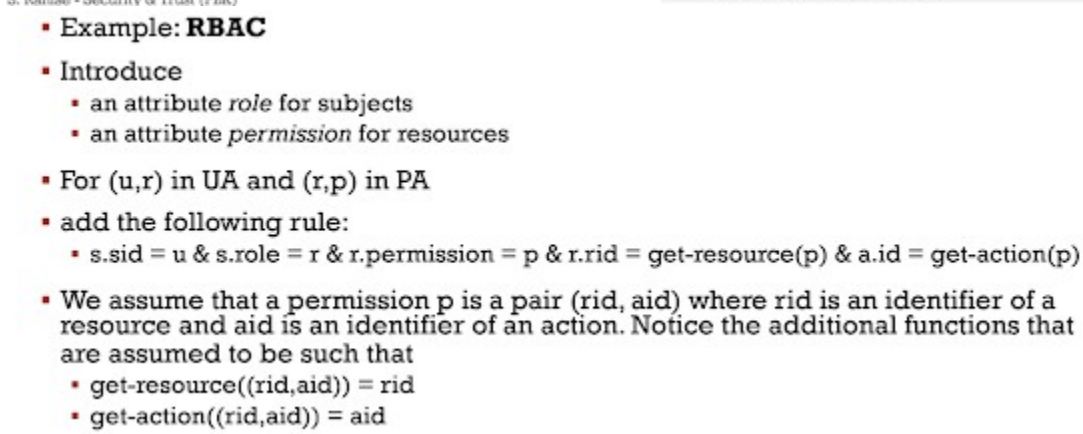
A subject is an active entity that causes information to flow among objects or changes the system state.
Subject attributes define the identity and characteristics of the subject (e.g. name, organization, job title, role, etc.)

An object (or resource) is a passive information system-related entity containing or receiving information.
Object attributes can be leveraged to make access control decisions (e.g. title, author, date of creation, size, categories of content, etc.)

Environment attributes describe the operational, technical, and even situational environment or context in which the information access occurs (e.g. current date, current virus/hackers activities, network security level; in general any feature not associated with a resource or subject).
These attributes have been largely ignored in most access control policies.

As we've already said, **ABAC can mimic all previous AC models**.
To see how this is possible, it is sufficient to observe that it is sufficient to introduce appropriate attributes for subjects and resources (and possibly appropriate function or predicates) so as to be able to express the conditions for granting or denying access of the corresponding AC model.





XACML

XACML (eXtensible Access Control Markup Language) is an OASIS standard that supports the idea of ABAC with a concrete syntax and semantics. But XACML is more than a language for ABAC policies.
It was developed for collaborative environments (data sharing across different organizational domains). XACML is extensible and is an XML encoded language.

XACML can specify access control policies, access control requests, and access control decisions and contains more than policy specification language.
The components of XACML are:

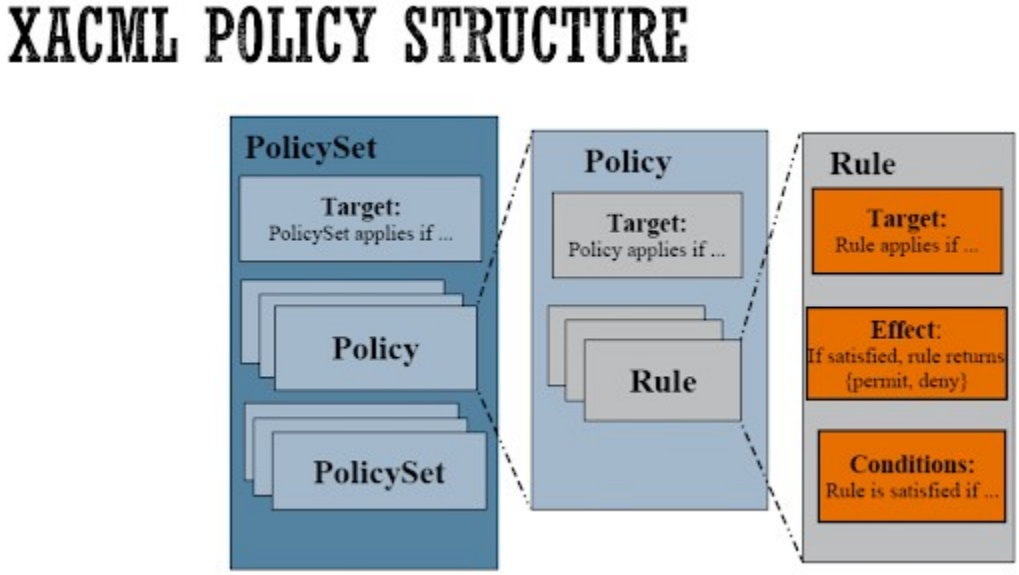
- **XACML policy language**
  To specify access control rules and algorithms for combining policies
- **XACML request/response protocol**
  Which is used to query a decision engine that evaluates user access requests against policies

- **XACML reference architecture**
  For deployment of software modules to house policies and attributes and compute and enforce access control decisions

Let's see some vocabulary:

- Resource: Data or system component needing protection
- Subject: An actor who requests access to specific resources
- Action: An operation on a resource
- Environment: Properties not belonging to resources, subjects, or actions that are important for the authorization decision
- Attributes: Characteristics of the resource, subject, action, or the environment
- Target: Defines conditions that determine whether policy applies to the request

A **XACML access request** consists of attributes of subject, resource, action, and environment. XACML attributes are name-value pairs (e.g. Role = "Doctor", ObjectAttr = "Medical Record"). Attributes are stored in a Policy Information Point (PIP) and retrieved at the time of decision making.



The XACML policies are structured as **PolicySets**. PolicySets consist of Policies and may include other PolicySets.

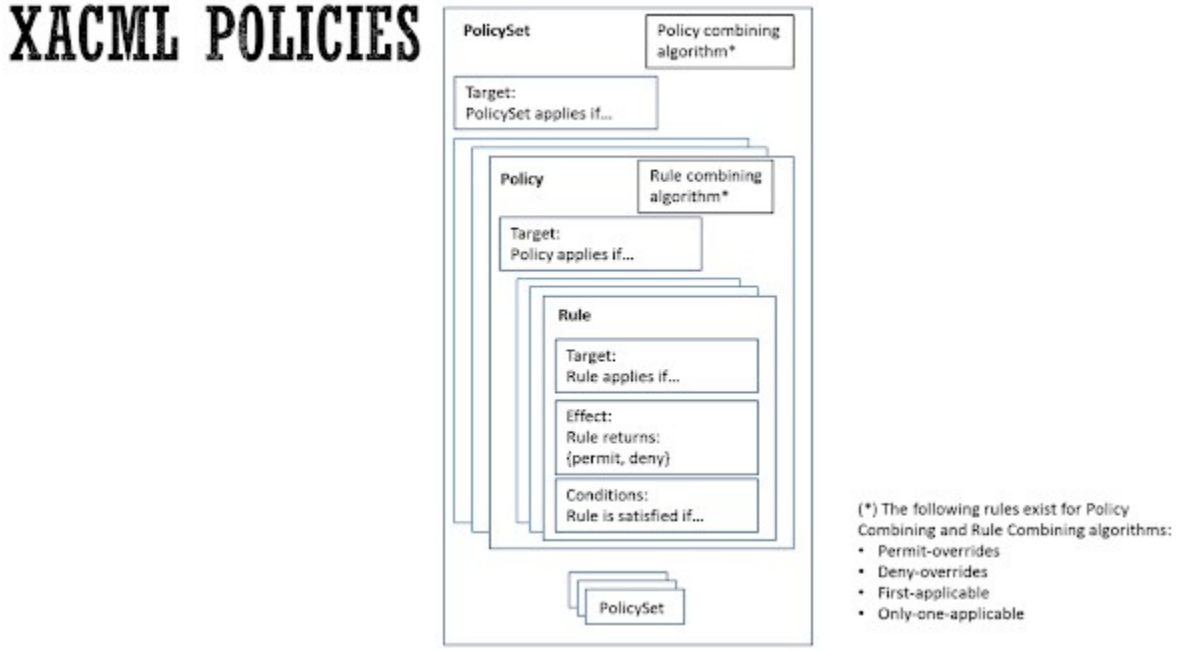
The **policies** are composed of Rules. Policies are helpful to group rules which belong to the same context (for example if a user belongs to a department, there will be considered only the rules which belong to that department).
Target (which are in PolicySet, Policy and Rule) defines a Boolean condition that is very quickly evaluated

- If true, the request gets evaluated by a PDP
- If false, the decision is Not Applicable

Targets are helpful because they minimize the PolicySets, Policy, and Rules that must be examined.

Rules (which are the basic building blocks of XACML policy specification language) have a set of Boolean **conditions**. Rules evaluate to true or false or indeterminate. Policy can have multiple rules. Rules can be combined by **rule combining algorithms**.

The previous models we've seen follow the closed world assumption: What is not known to be true, is false, so that absence of information is interpreted as negative information.
In this case this is not followed, in fact you can specify both the granted access or the denied access.
So we can have situations that aren't neither allowed nor denied, and this allows us to be more precise in different use case scenarios.



This division and abstraction is helpful because it is very flexible and it can adapt well in each use case scenario. The two upper levels of abstraction are useful only to organize the rules, since the rules are the basic building blocks which implement the ideas.

Let's now see in more detail the **rule combining algorithms**.
These combination algorithms specify how to behave in front of an authorization request which are allowed from one rule but denied from another one. So rules combining algorithms specify how to handle these situations of conflict.
There are 12 rule combining algorithms available but we will only see 4:

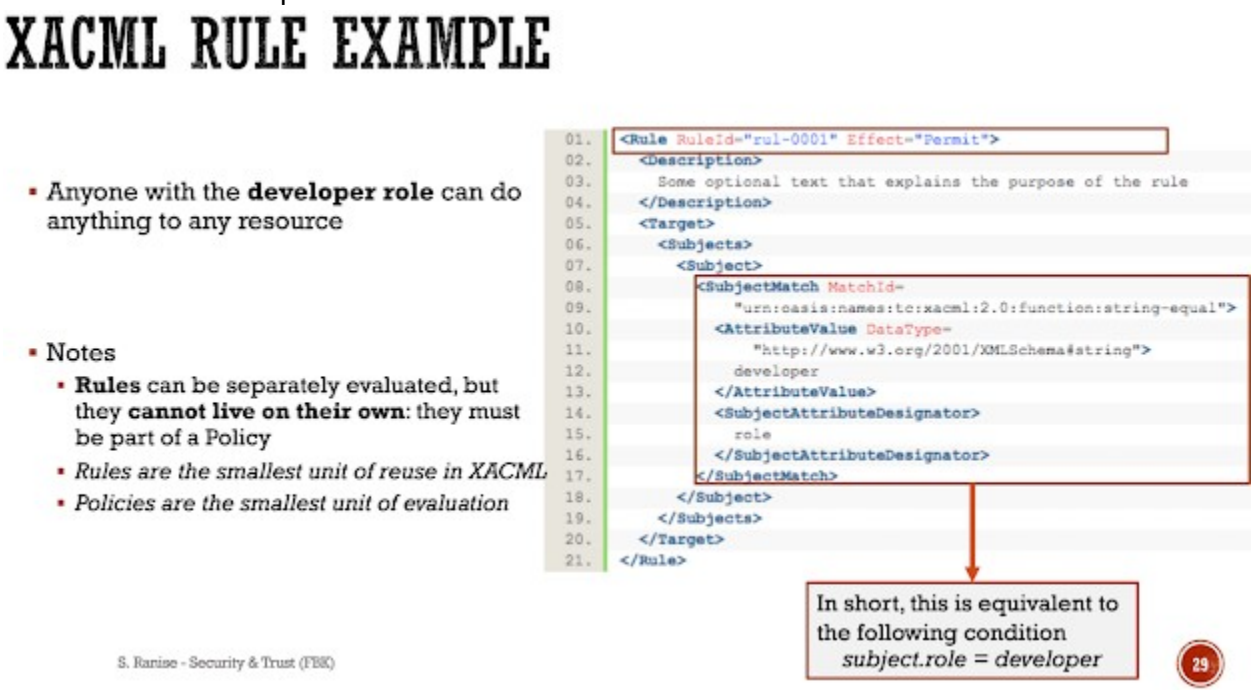
- **Deny overrides**
  Consist to make AND operations on the various effects. In this situation a denied access will override all the permitted accesses.
  This is more secure and inflexible, we want to be cautious and deny access possibly causing a bit of problems wrt business continuity
- **Permit overrides**
  Consist to make OR operations on the various effects. In this situation a permitted

access will override all the denied accesses.
This is less secure but is more flexible, we want to guarantee business continuity and possibly be less secure

- **First applicable**
  The result is the result of the first decision, so it is completely based on the order
- **Only one applicable**
  The result is “defined” only if there is exactly one rule applicable. Iif more than one decision applies, then the result is Indeterminate

XACML includes the concept of **obligations**.
Obligation describes what must be carried out before or after an access request is approved and denied (e.g if Alice is denied access to Document X, email her manager that Alice tried to access document X).

Let’s see some examples:



XACML POLICY EXAMPLE

- Previous rule wrapped in a policy

- Notes
  - The **RuleCombiningAlgId** attribute on the Policy identifies the algorithm that combines Effects from multiple Rules into a single result

```
<Policy PolicyId="pol-0001" RuleCombiningAlgId=
    "urn:oasis:names:tc:xacml:1.0:rule-combining-algorithm:deny-overrides">
  <Description>
    Some optional text that explains the purpose of the policy
  </Description>
  <Target>
    <Subjects>
      <Subject>
        <SubjectMatch MatchId=
            "urn:oasis:names:tc:xacml:2.0:function:string-equal">
          <AttributeValue DataType=
              "http://www.w3.org/2001/XMLSchema#string">
            developer
          </AttributeValue>
          <SubjectAttributeDesignator>
            role
          </SubjectAttributeDesignator>
        </SubjectMatch>
      </Subject>
    </Subjects>
  </Target>
  <Rule RuleId="rul-0001" Effect="Permit"/>
</Policy>
```


XACML POLICYSET EXAMPLE

- Previous policy wrapped in a policy set

- Notes
  - A **Policy Set** contains a Target, a Policy-Combining Algorithm, a set of Policies, and some Obligations
  - The **Policy-Combining Algorithm** specifies the procedure by which the results of evaluating the component Policies are combined
  - A Policy Set can reuse not just Policies, but also entire Policy Sets

```
<PolicySet PolicySetId="pls-0001" PolicyCombiningAlgId=
    "urn:oasis:names:tc:xacml:1.0:policy-combining-algorithm:deny-overrides">
  <Description>
    Some optional text that explains the purpose of the policy set
  </Description>
  <Target>
    <Subjects>
      <Subject>
        <SubjectMatch MatchId=
            "urn:oasis:names:tc:xacml:2.0:function:string-equal">
          <AttributeValue DataType="http://www.w3.org/2001/XMLSchema#string">
            developer
          </AttributeValue>
          <SubjectAttributeDesignator>
            role
          </SubjectAttributeDesignator>
        </SubjectMatch>
      </Subject>
    </Subjects>
  </Target>
  <PolicyIdReference>
    pol-0001
  </PolicyIdReference>
  <Policy PolicyId="pol-0001" RuleCombiningAlgId=
      "urn:oasis:names:tc:xacml:1.0:rule-combining-algorithm:deny-overrides">
    <Target/>
    <Rule RuleId="rul-0001" Effect="Permit"/>
  </Policy>
</PolicySet>
```

XACML REQUEST EXAMPLE

- Subject has an Attribute named role, that our example Rule refers to using the SubjectAttributeDesignator element
- Notes
  - **Request** consists of attributes for the Subject(s), Resource, Action, and Environment
  - **Attributes** are named, so that Rules can refer to them, and strongly typed, so that the PDP can do proper comparisons.
  - **Environment** Attributes allow different access decisions to be made, for example depending on whether the Subject is in a public place or within the secure confines of his office
  - PDP will match the Target of the Policy Sets, Policies, and Rules against the Request to see whether they are applicable

```
<Request>
  <Subject>
    <Attribute AttributeId="role"
        DataType="http://www.w3.org/2001/XMLSchema#string">
      developer
    </Attribute>
  </Subject>
  <Resource>
    <ResourceContent>
      <!--
        In case the Resource is an XML document, it can be inlined
        here, so that Rules can depend on the contents.
      -->
      <example/>
    </ResourceContent>
    <Attribute DataType="http://www.w3.org/2001/XMLSchema#string"
        AttributeId="urn:oasis:names:tc:xacml:1.0:resource:resource-id">
      /some/example.xml
    </Attribute>
  </Resource>
  <Action>
    <Attribute DataType="http://www.w3.org/2001/XMLSchema#string"
        AttributeId="urn:oasis:names:tc:xacml:1.0:action:action-id">
      retrieve.contents
    </Attribute>
  </Action>
  <Environment>
    <Attribute DataType="http://www.w3.org/2001/XMLSchema#dateTime"
        AttributeId="urn:oasis:names:tc:xacml:1.0:environment:current-dateTime">
      2010-06-14T11:12:03Z
    </Attribute>
  </Environment>
</Request>
```

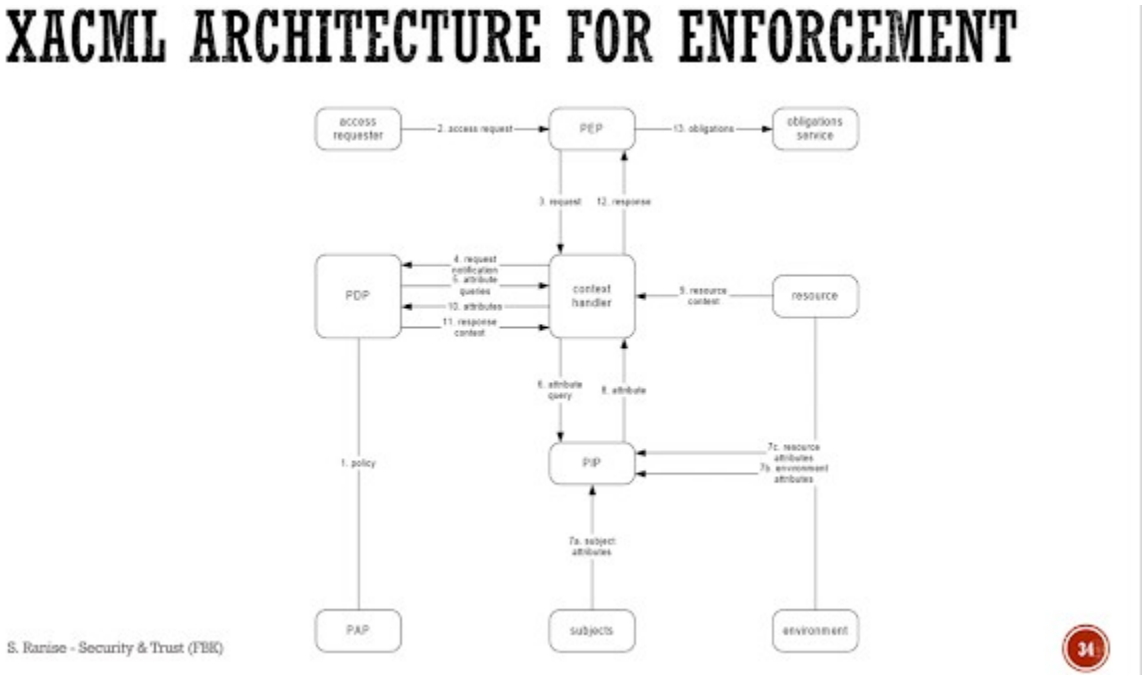

In short, this is equivalent to the following authorization request
(subject.role = developer, resource.resource-id=/some/example.xml, action.action-id=retrieve.contents, environment.current-dateTime=2010-06-14T11:12:03Z)

XACML RESPONSE EXAMPLE

- When the PDP has reached a verdict on whether the Subject may perform the Action on the Resource in the given Environment, it creates a Response object
- The Context Handler returns the Response to the PEP, which enforces the decision
- The Result may also include Obligations that the PEP must fulfill
  - Ex: filter out credit card information before returning the Resource content to the Subject
  - Ex: write an audit record
- PEP must either honor all of these Obligations or treat the decision as denied.

```
<Response>
  <Result ResourceId="/some/example.xml">
    <Decision>Permit</Decision>
  </Result>
</Response>
```

So let's now see the last component of XACML, the **architecture**, where all the aspects that we've seen are going to be specified.
Let's see an architecture that must be read from top to bottom:



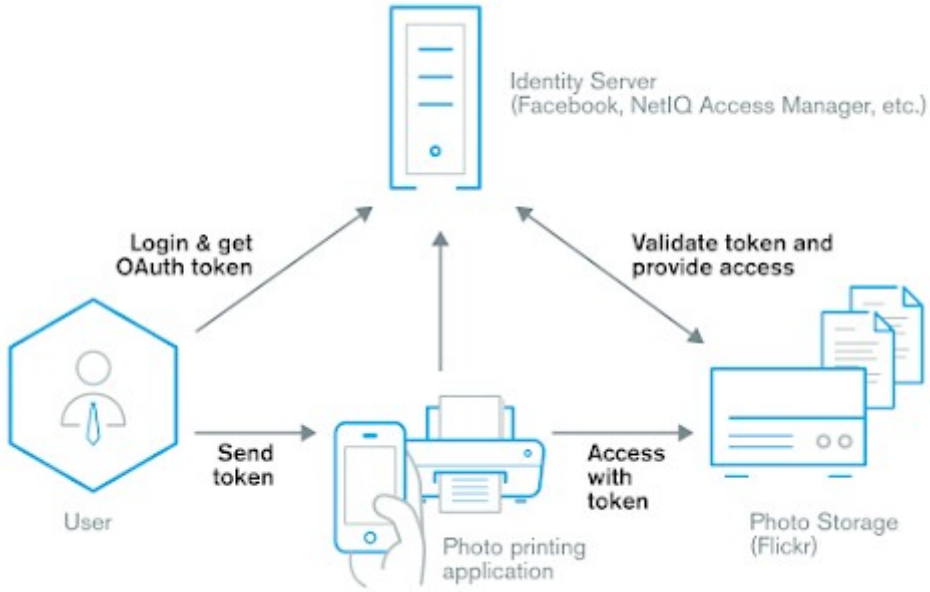
We make the usual assumption that the subject has already been authenticated.
The access request is intercepted by **PEP** (Policy Enforcement Point), which is the interface between the outside world and the Isolation Boundary, protecting the resources. PEP Performs access control by making decision requests and enforcing authorization decisions.
When intercepted, the access request is sent to the **context handler**, which translates from the native standard of authorization request to XACML. The context handler also converts the Authorization decisions in the XACML canonical form to the native format.
Then the request is sent to the **PDP** (Policy Decision Point) which receives and examines the request, retrieves applicable policies, evaluates the applicable policy and returns the authorization decision to PEP. To do this, the PDP has to ask the context handler to retrieve the attribute it needs.
The module that mediates the exchange of attributes is **PIP** (Policy Information Point) and serves as the source of attribute values, or the data required for policy evaluation.
The policies that are stored in PDP are created by **PAP** (Policy Administration Point), which precisely creates security policies and stores these policies in the repository (this is the first thing that has to be done).
The **obligations service** module simply manages the obligations, that is the actions to make after the authorization decision has been taken.

OAuth 2.0

Now we are going to see something that is very related to Access Control, but it is also related to Authentication protocols.
We are going to see the protocol of a particular model of Access Control, which is very widely used, that is **OAuth 2.0**. This protocol implements a mechanism called delegation (can be seen as a discretionary access control model). The user can delegate permissions of access to his resources to a third party application.

An analogy that can be done is the **valet key**.
Many cars today come with a valet key, which is a special key you give the parking attendant and unlike your regular key, will not allow the car to drive more than a mile or two. Some valet keys will not open the trunk, while others will block access to your onboard cell phone address book.
Regardless of what restrictions the valet key imposes, the idea is clear: **you give someone limited access** to your car **with a special key**, while using your regular key to unlock everything.

A practical example of OAuth 2.0 is a photo lab printing your online photos.
A straightforward implementation may request to provide your username and password to the other site. But when you agree to share your secret credentials, not only do you expose your password to someone else, you also give them full access to do as they wish (they can do anything they want, even change your password and lock you out).
This is the problem OAuth solves, because **it allows you (users) to grant access to your private resources on one site (which is called the Service Provider), to another site (called Consumer/Client)**.



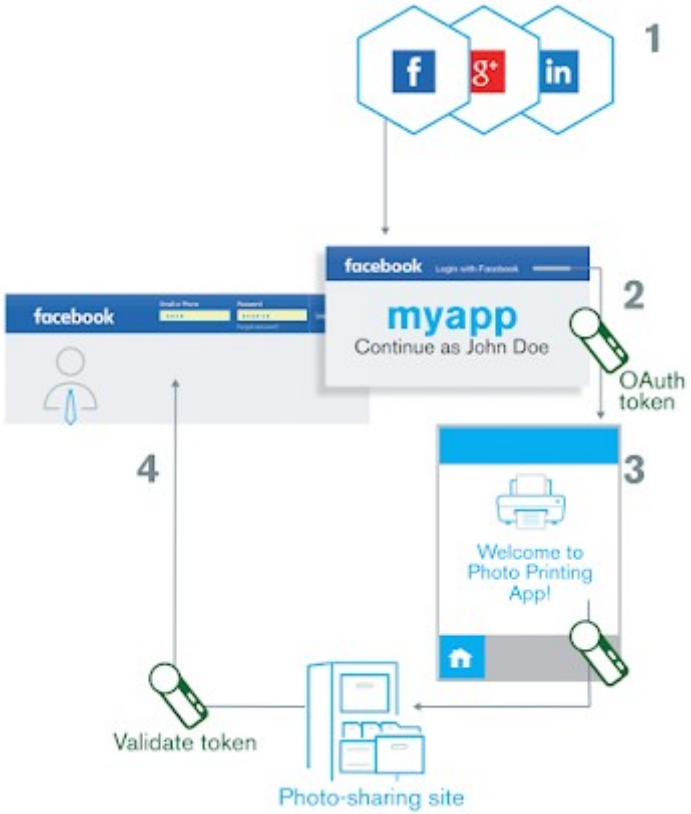
So in the digital world, the valet key is a token. Anyone presenting that token, will be able to access the selected resources without any kind of authentication. So we need to pay a lot of attention to whom we are sending the token.

The flow of OAuth 2.0 is:

1. **Authentication**
   The user logs into the social site; not part of OAuth 2.0 protocol. The focus of OAuth 2 is only authorization and leaves authentication to the application.
   OAuth 2 simply requires the user to be authenticated and does not dictate how to do this. Extensions to Oauth 2 such as OpenID Connect (OIDC) can be useful for

authentication.
The authentication also includes the authentication of the device/tool used by the user in order to interact with the system.

2. **User consent**
   OAuth 2.0 allows users to decide what can be shared with 3rd-party apps. An OAuth token will be created based on the rights to which the token user consents
3. **Get OAuth Token**
   The third-party printing app receives an OAuth bearer token from the social site. This token includes details about the access rights of the token bearer. The application can then use this token to access the photos on behalf of this user
4. **Access Resource**
   The printing app can now access the resource server (Flickr) using the token to get the user's data (in this case photos)



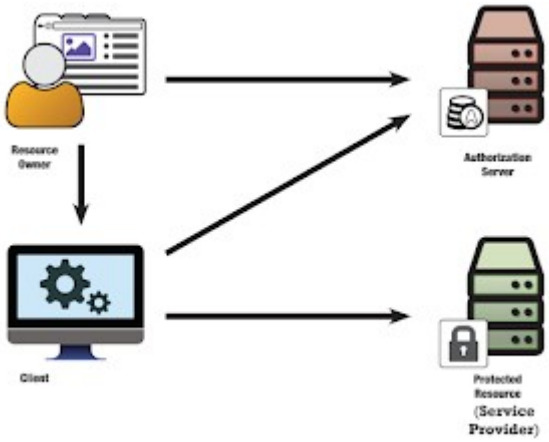
The definition of OAuth 2.0 from RFC6794 (IETF) is that the OAuth 2.0 authorization framework enables a third-party application to obtain limited access to an HTTP service either on behalf of a resource owner by orchestrating an approval interaction between the resource owner and the HTTP service.

Let's see the involved entities in OAuth 2.0:

- The **resource owner** (usually a person)
  Who can access to certain resources and can delegate access to resources

- The **protected resource**
  Which is protected for its owner by the service provider. The service provider shares resources on owner's request
- The **client** (app)
  Which wish to access protected resources. It acts on owner's behalf
- The **authorization server**
  Which generates tokens for the client, authenticate resource owners and clients and manage authorizations

In many cases the protected resource and the authorization server "live" on the same server.



So the OAuth token represents delegated authorities (From the resource owner to the client for the protected resource). The OAuth token is issued by the Authorization server and it is used by the client.
Token format is opaque to the client but not to AS and RS. How can AS and RS understand what is inside a token?

- Database lookup
- Put info in the token (JWT)
- RS query the AS

The core protocol OAuth 2.0 is defined only for HTTP and relies on TLS for securing messages (so we can guarantee Integrity and Confidentiality).
It isn't an authentication protocol but only relies on authentication in several places (client authentication to token endpoint, resource owner authentication at authorization endpoint).
But **authentication protocols can be built using OAuth** (OpenID Connect).
In OAuth 2.0 there is no authorization processing. Tokens can represent scopes and other authorization information and processing of this information is up to the resource server.
There is no token format, in fact the token is opaque to the client and the resource owner(they don't need to "understand" the token). Token needs to be issued by the authorization server and understood by the resource server, but they're free to use whatever format they want. JSON Web Tokens (JWT) provide a useful common format.
It is important to denote that OAuth 2.0 is not a single protocol, but it is a framework consisting of several flows.

Let’s make a disgression on JWT. JWT is the de facto standard for OAuth 2.0 access token (OAuth Standard does not impose any particular format). JWT defines a container to transport data between interested parties.
JWT consists of 3 components:

- Header (identifies algorithm used to sign)

```
{
 "alg" :  RS256 ,
 "typ" : "JWT"
}

base64url encoded string: eyBhbGcgOiBIUzI1NiwgdHlwIDogSldUIH0K
```

- Payload (information actually used for access control)

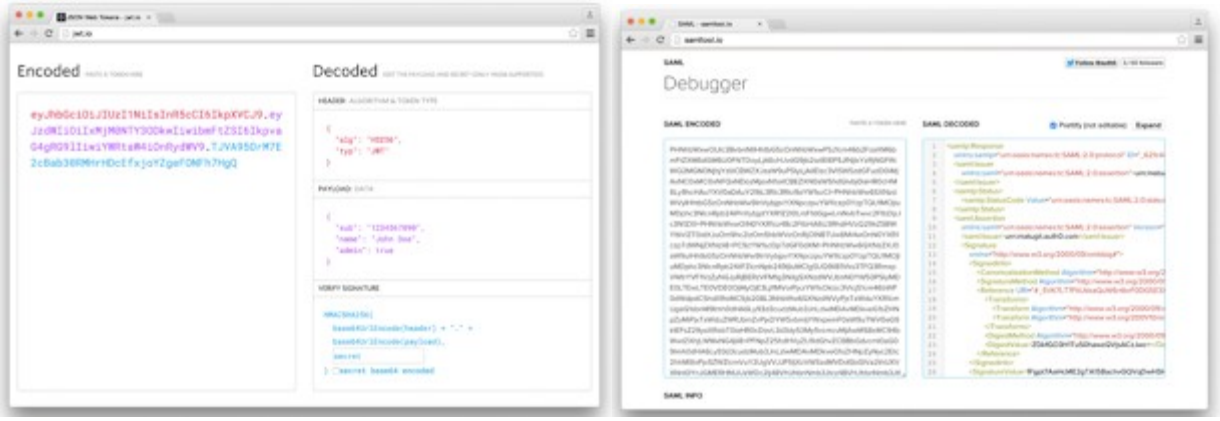
```
{
 "user_name" : "admin",
}

base64url encoded string: eyB1c2VyX25hbWUgOiBhZG1pbiB9Cg
```

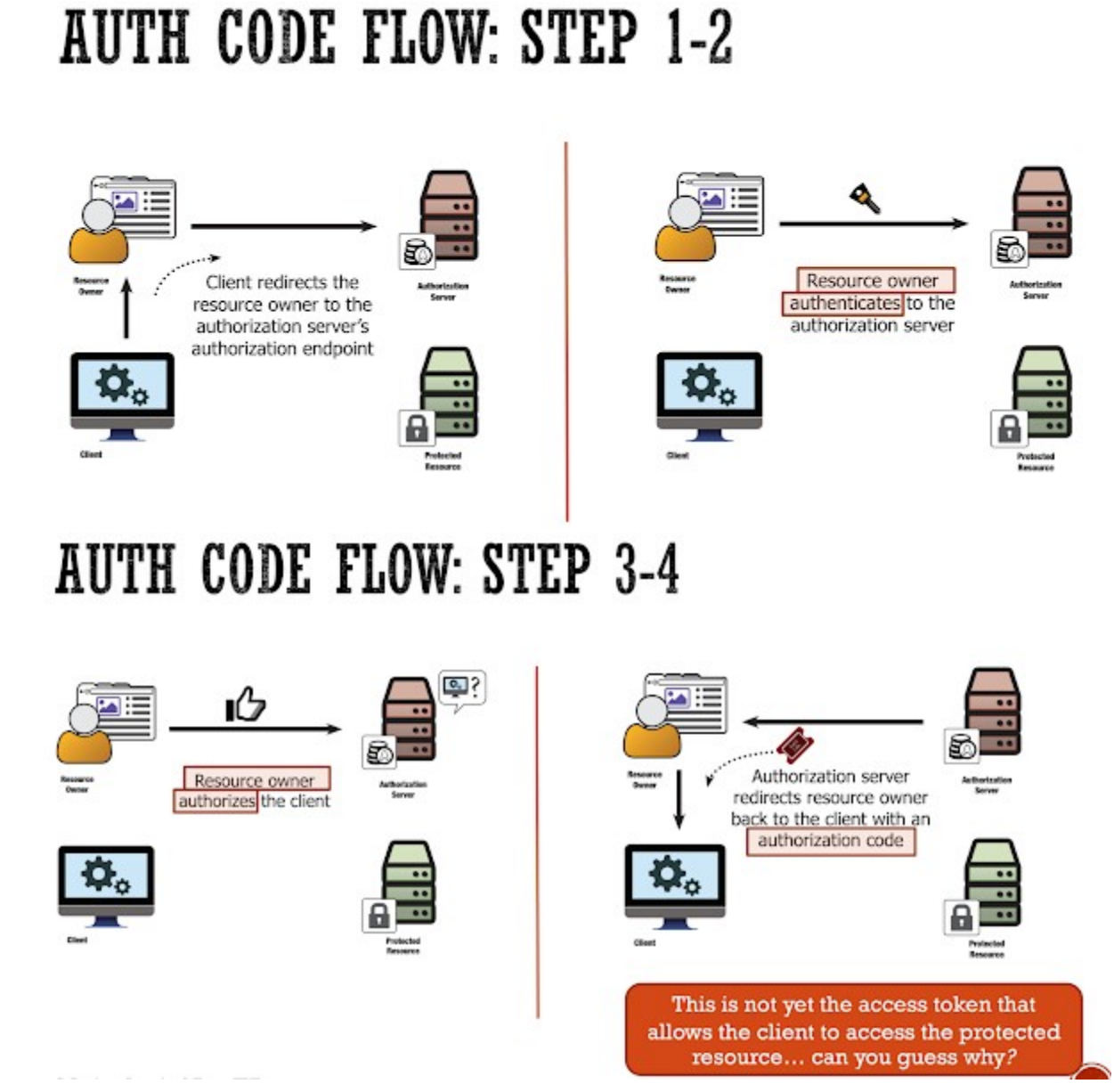
- Signature
  Used to validate that the token has not been tampered with (without the signature anyone could create a bogus token). It is calculated by concatenating the header with the payload, then signing with the algorithm specified in the header (e.g., RS256)

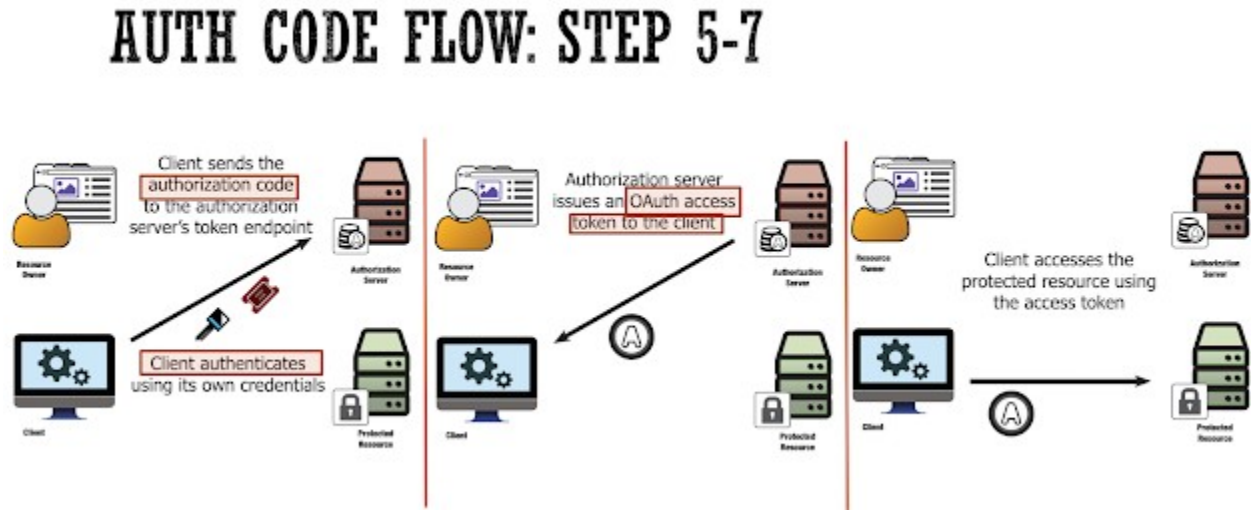
RS256 = RSA Signature with SHA-256 asymmetric algorithm using a public/private key pair.
Base64 is a way to encode binary data into an ASCII character set known to pretty much every computer system, in order to transmit the data without loss or modification of the contents itself.
Base64url encoding is basically base64 encoding except for the use of non-reserved URL characters (e.g., – is used instead of + and _ is used instead of /) and omits the padding characters.
Every JWT is encoded in base64url.

JSON is less verbose than XML, when it is encoded its size is also smaller, making JWT more compact than SAML. This makes JWT a good choice to be passed in HTML and HTTP environments.
Both JWT tokens and SAML responses can use a public/private key pair in the form of a X.509 certificate for signing, but signing XML with XML Digital Signature without introducing obscure security holes is very difficult when compared to the simplicity of signing JSON.
JSON parsers are common in most programming languages because they map directly to objects; XML doesn't have a natural document-to-object mapping.
JWT is used at Internet scale. This highlights the ease of client-side processing of the JSON Web token on multiple platforms, especially mobile.



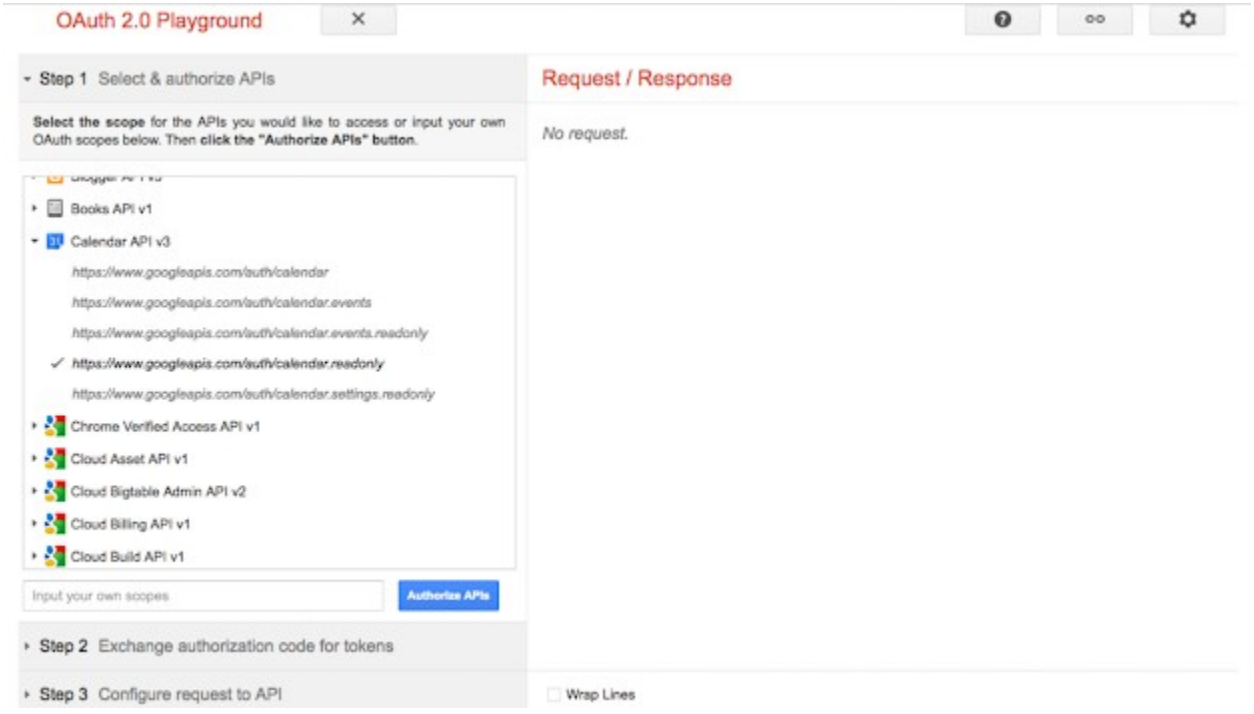
Let's now see the most widely used flow underline this protocol:

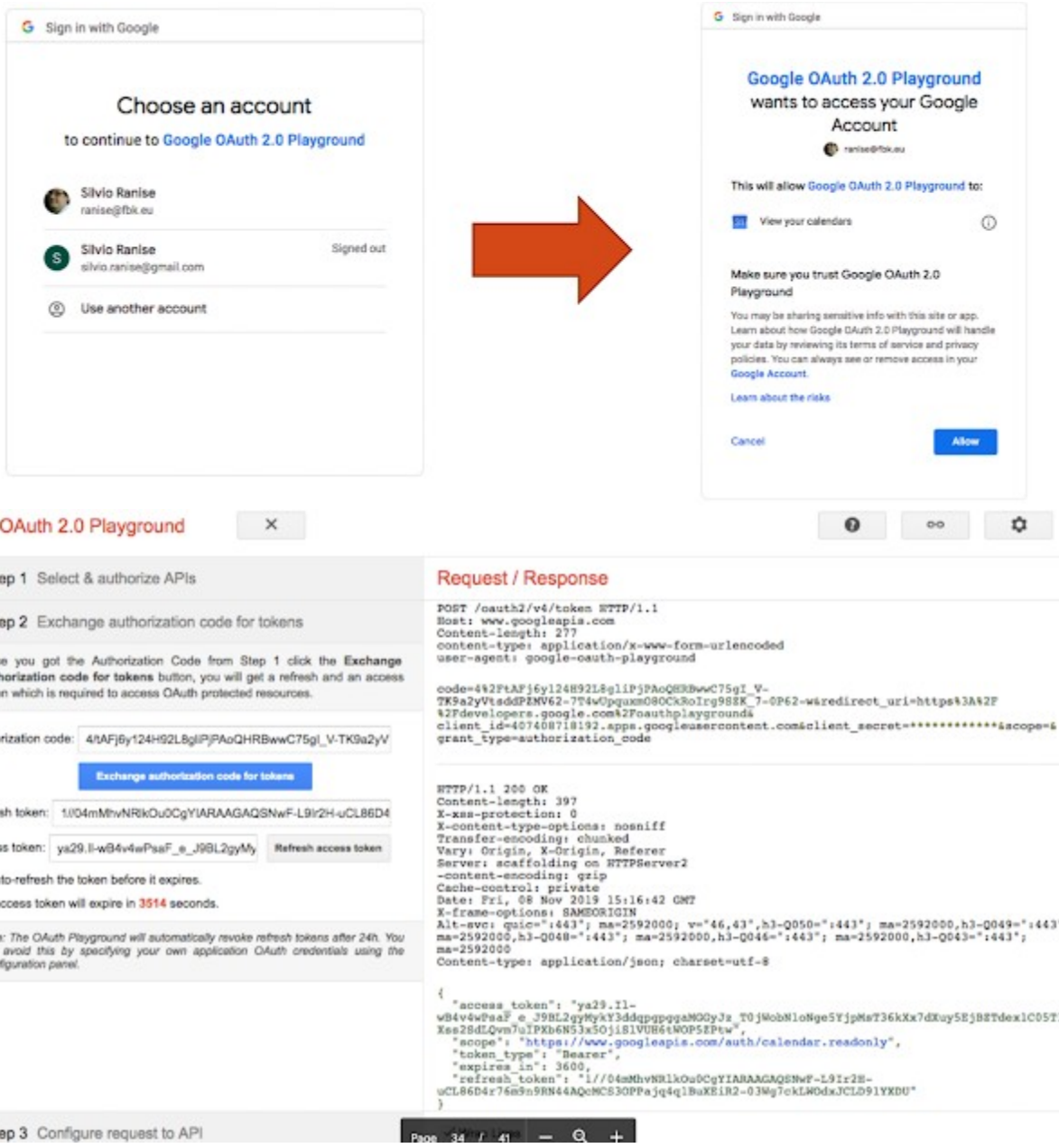


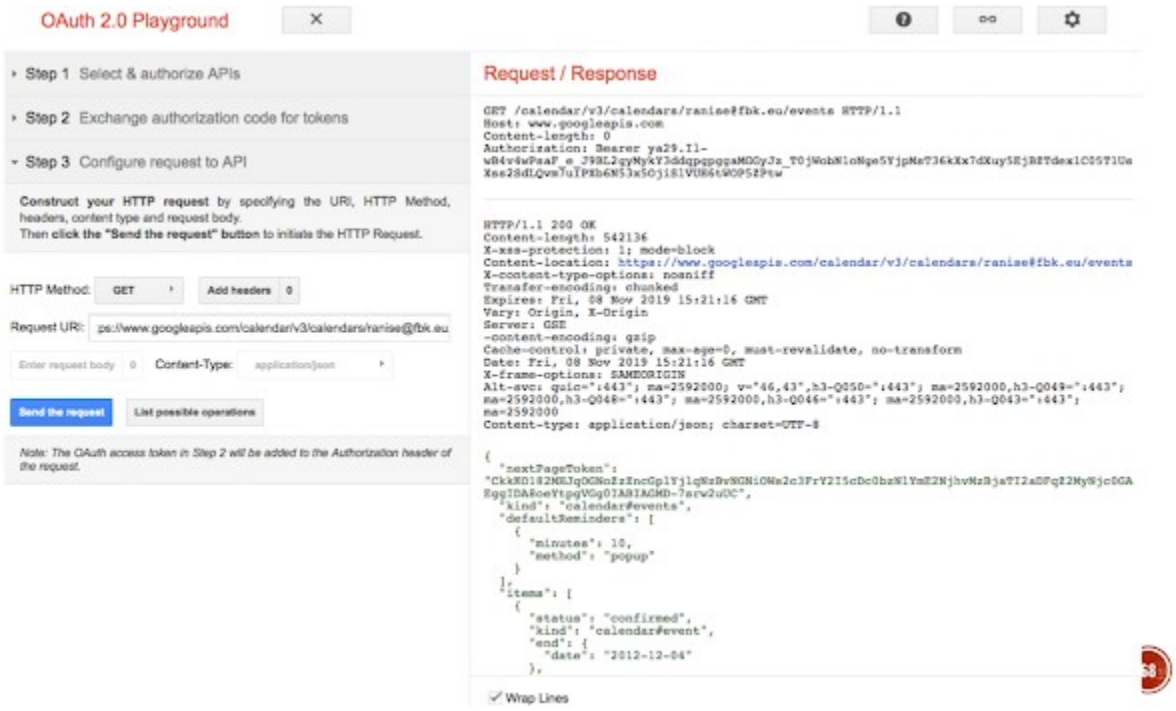


The **authorization code** is different from the access token, because it is a preliminary code that the resource owner will forward to the client. Then the client will directly authenticate (using also this authorization code) and ask the authentication server for the access token, which will be directly sended to the client.
This preliminary step is needed because the authorization code travels in two channels (Authentication Server -> User and then User -> Client), increasing the risk that it can be stolen. In this way also the client has to somehow be authenticated by the authorization server, which can also check if the client can be trusted.
The client then uses the access token to retrieve the resource from the Protected Resource Server, which checks the validity of the access token with the signature made by the Authorization Server.

You can play with this kind of protocol because Google provides a playground:




Sign in with Google
Choose an account
to continue to Google OAuth 2.0 Playground
Silvio Ranise
ranise@fbk.eu
Silvio Ranise
silvio.ranise@gmail.com
Signed out
Use another account
Sign in with Google
Google OAuth 2.0 Playground
wants to access your Google Account
ranise@fbk.eu
This will allow Google OAuth 2.0 Playground to:
View your calendars
Make sure you trust Google OAuth 2.0 Playground
You may be sharing sensitive info with this site or app. Learn about how Google OAuth 2.0 Playground will handle your data by reviewing its terms of service and privacy policies. You can always see or remove access in your Google Account.
Learn about the risks
Cancel
Allow
OAuth 2.0 Playground
Step 1 Select & authorize APIs
Step 2 Exchange authorization code for tokens
Once you got the Authorization Code from Step 1 click the Exchange authorization code for tokens button, you will get a refresh and an access token which is required to access OAuth protected resources.
Authorization code: 4/tAFj6y124H92L8gliPjPAoQHRBwwC75gI_V-TK9a2yV
Exchange authorization code for tokens
Refresh token: 1//04mMhvNRlkOu0CgYIARAAGAQSNwF-L9Ir2H-uCL86D4
Access token: ya29.Il-wB4v4wPsaF_e_J9BL2gyMy
Refresh access token
Auto-refresh the token before it expires.
The access token will expire in 3514 seconds.
Note: The OAuth Playground will automatically revoke refresh tokens after 24h. You can avoid this by specifying your own application OAuth credentials using the Configuration panel.
Step 3 Configure request to API
Request / Response
POST /oauth2/v4/token HTTP/1.1
Host: www.googleapis.com
Content-length: 277
content-type: application/x-www-form-urlencoded
user-agent: google-oauth-playground
code=4%2FtAFj6y124H92L8gliPjPAoQHRBwwC75gI_V-TK9a2yV&redirect_uri=https%3A%2F%2Fdevelopers.google.com%2Foauthplayground&client_id=407408718192.apps.googleusercontent.com&client_secret=**********&scope=&grant_type=authorization_code
HTTP/1.1 200 OK
Content-length: 397
X-xss-protection: 0
X-content-type-options: nosniff
Transfer-encoding: chunked
Vary: Origin, X-Origin, Referer
Server: scaffolding on HTTPServer2
-content-encoding: gzip
Cache-control: private
Date: Fri, 08 Nov 2019 15:16:42 GMT
X-frame-options: SAMEORIGIN
Alt-svc: quic=":443"; ma=2592000; v="46,43",h3-Q050=":443"; ma=2592000,h3-Q049=":443"; ma=2592000,h3-Q048=":443"; ma=2592000,h3-Q046=":443"; ma=2592000,h3-Q043=":443"; ma=2592000
Content-type: application/json; charset=utf-8
{
"access_token": "ya29.Il-wB4v4wPsaF_e_J9BL2gyMykY3ddqpgpgqaMOGyJz_T0jWobNloNge5YjpMsT36kXx7dXuy5EjBETdexlC05TlUsXss28dLQvm7ulPXb6N53x5OjiSlVUH6tWOP5EPtw",
"scope": "https://www.googleapis.com/auth/calendar.readonly",
"token_type": "Bearer",
"expires_in": 3600,
"refresh_token": "1//04mMhvNRlkOu0CgYIARAAGAQSNwF-L9Ir2H-uCL86D4r76m9n9RN44AQcMC5JOPPajq4qlBuXEiR2-03Wg7ckLWOdxJCLD91YXDU"
}
Page 34 / 41



The key points of this process are:

1. User need to log into the Provider's OAuth service when redirected
2. Give explicit consent to limited access, to create a string that represent what the token can do
    - Type of action: Read, Write, Delete
    - Type of resource: Photos, Metadata, Profile
    - Time of access: user is offline, limited number of accesses
3. Protected Resource validates access token (i.e. requested access) when getting it from the Client (need of suitable cryptographic mechanisms to protect integrity and avoid forging!)

Since the access token has a validity period, after which it expires, and when it expires the user must re-authenticate and redefine the permissions in order to get another token, **refresh tokens** are used. Refresh token is created with the first access token and it's just sent by the client to the authorization server (along with client credentials) to get a new valid access token with the same type of permissions of the first token. In this way the resource owner isn't required to be online during the usage of the protocol.

Recalling capabilities, OAuth tokens are **similar** to them because they can be transferred to other subjects so that permissions are delegated and permissions are decoupled from the identities of subjects.

At a certain point people started to "abuse" the use of this protocol, because they use it to perform authentication.

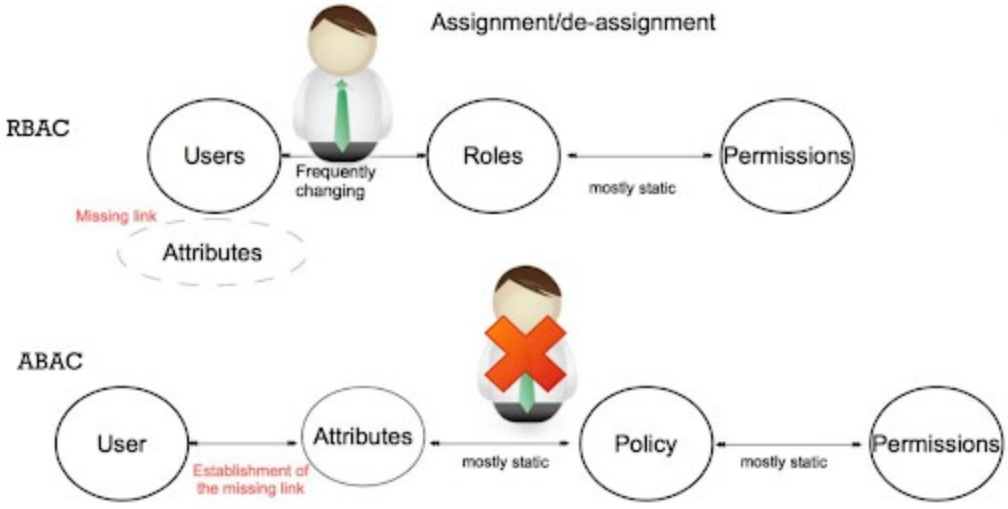
The idea is that the protected resource that has been shared is the credentials of the user. The users started to do this because this protocol is much simpler in implementation than SAML (we've also seen the difference in size of tokens).
In this way you obtain a sort of SSO experience, because you enter your credentials just once and you can access multiple times to multiple sites presenting the refresh tokens. But this sort of "session" will last forever if the refresh token doesn't have an expiration date.
Furthermore, the client can be attacked and then the attacker can authenticate as the user.
This is a bad idea, in fact the standard of OAuth 2.0 says: "Authenticating resource owners to clients is out of scope for this specification".
This is a bad idea for two reasons:

- The assumption that possession of a valid access token is enough to prove that a user is authenticated is true only in some cases (when the access token was freshly minted)
- There are other ways to obtain a valid access tokens than authenticating resource owner (e.g. using the refresh token)

It is important to remember that authentication is about the user and their presence with the application.

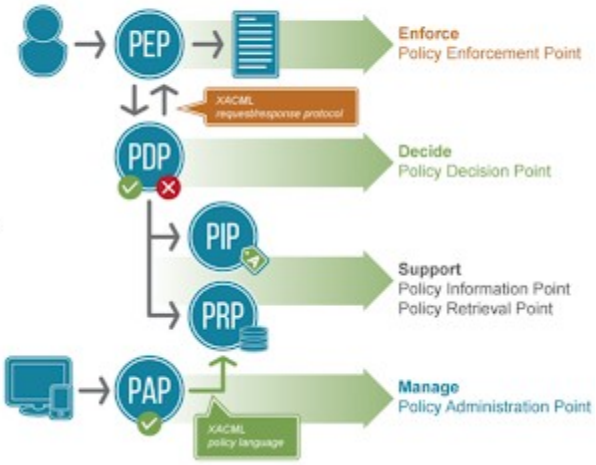
This leads to the definition of another standard, that is **OpenID Connect**, which performs authentication on top of OAuth 2.0. It is an open standard published in early 2014 that defines an interoperable way to use OAuth 2.0 to perform user authentication.
Instead of building a different protocol to each potential identity provider, an application can speak one protocol to as many providers as they want to work with.
OpenID Connect manages to avoid many of the pitfalls discussed above by adding several key components to the OAuth base…
For example, recently AgID has been pushing an alternative implementation of SPID based on OpenID Connect. At the moment the two standards coexist.
As we said, SAML works on redirects of browsers, while OpenID Connect doesn't rely on this mechanism, making it more easy to implement to some native applications.

TAKEAWAYS: ABAC



TAKEAWAYS: XACML

- XACML is an OASIS standard
  - It has even a RBAC profile (for both core and hierarchical)
- A **standardized** approach to authorization
  - Authorization rules no more embedded in the SW of individual systems
  - Focus is on corporate policies rather than the technicalities of SW environments
- An **externalized** approach to authorization
  - PDP offers authorization as a service in the infrastructure
  - Authorization algorithms can be removed from the application logic of individual information systems, which will then query the PDP via their own Policy Enforcement Points (PEP)
- An **attribute** and **policy based** approach to authorization
  - XACML policies introduce abstract logic to replace previous static assignments of user permissions
  - Instead of "Bob can access document X" a rule "any user belonging to company X with security clearance equal to or higher than the security classification of a document should be granted access to that document"
  - To determine whether Bob should be granted access to document X, his security clearance as well as the document classification needs to be gathered. These descriptive pieces of information are called **attributes**



TAKEAWAYS: OAUTH 2.0

- Key mechanism to delegate permissions without distributing credentials
- Recall to use TLS to secure communications
- It can be used as the starting point for authentication protocols
  - OpenIdConnect

RECAP QUESTIONS

- What is the problem solved by OAuth? Which entities are involved in Oauth?
- What is an OAuth token? Is it opaque for which entity involved in OAuth?
- What is ABAC? What is an ABAC policy? What are the advantages of ABAC over RBAC?
- What is XACML? What is a XACML target, effect, condition, rule policy, and policy set? What are the XACML policy combining algorithms?
- Describe the XACML architecture
- What is the problem that the Oauth 2.0 standard solve?
- Describe the Authorization Code flow
- Explain why an access token is not immediately created for the client and instead an authorization code is created
- Describe the structure of JWT
- Why is a bad idea to use OAuth 2.0 for authentication? What should you use instead?

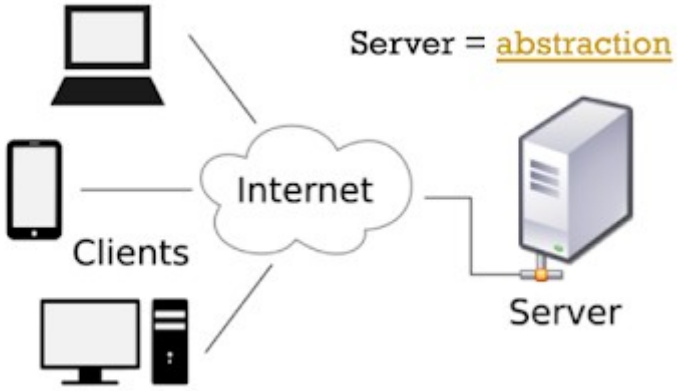
Web app and IoT Security

Until now we've seen a set of mechanisms that put together form a layered security service that performs authentication and authorization.

Now we are going to see what happens after you've been authenticated and authorized on a service online.
An example of IoT (Internet of Things) security is with healthcare devices, that have to share data with other devices and we don't want other people to get this data. Digital Identity is also important because you have to associate all this information to the right user in a secure way.
It is very important that Confidentiality, Integrity and Availability of these applications are guaranteed, because in certain situations they can save the lives of users. They also need to share data in real time.
The integration of these devices is increasing, and they are called **phygital devices** (physical + digital).

Web applications

When we talk about web applications, we basically talk about a client-server application.



The client is not concerned with how the server performs while fulfilling the request and delivering the response, **it only needs to understand the response** based on the well-known application protocol, i.e. the content and the formatting of the data for the requested service.
The server is an abstraction of computer resources.
Clients and servers exchange messages in a request–response messaging pattern:

- client sends a request
- server returns a response

To communicate, client and server must satisfy a communication protocol, that is:

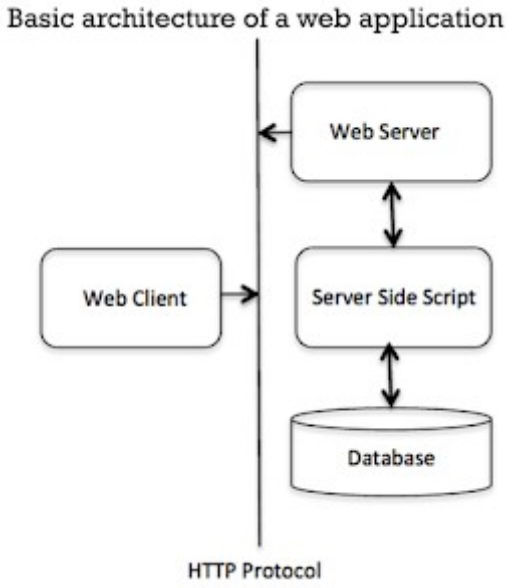
- have common language
- follow rules

All client-server protocols operate in the application layer.

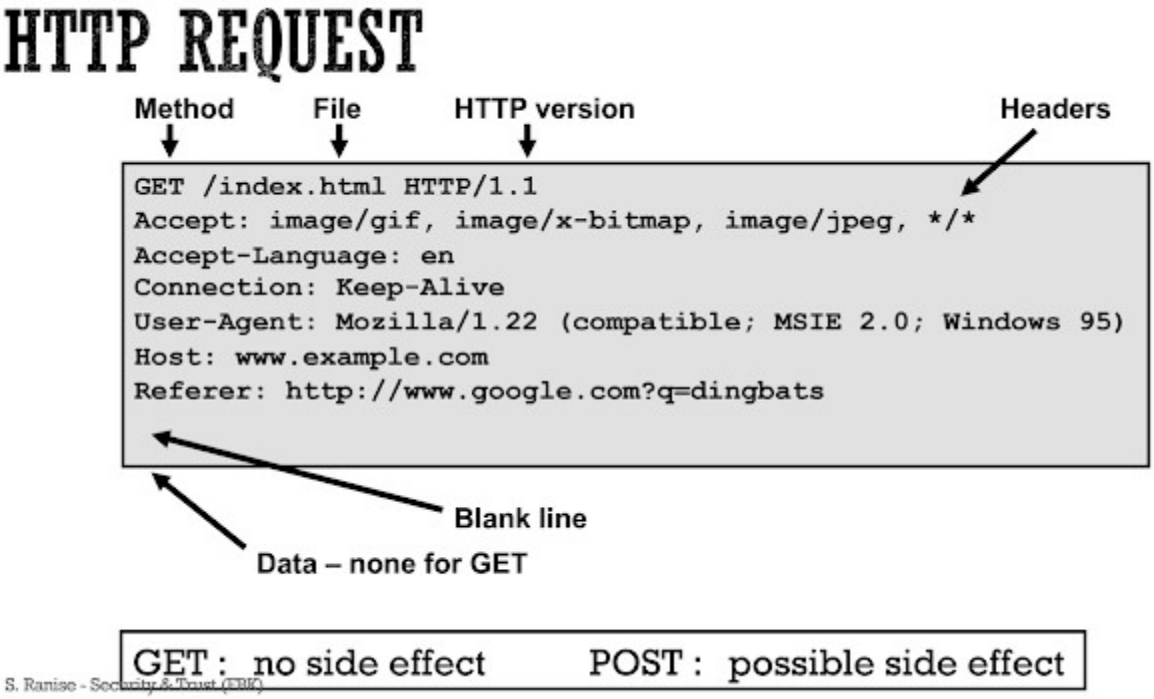
HTTP (Hypertext Transfer Protocol) is a client-server protocol. It is used for distributed, collaborative, hypermedia information systems.
Some HTTP features are:

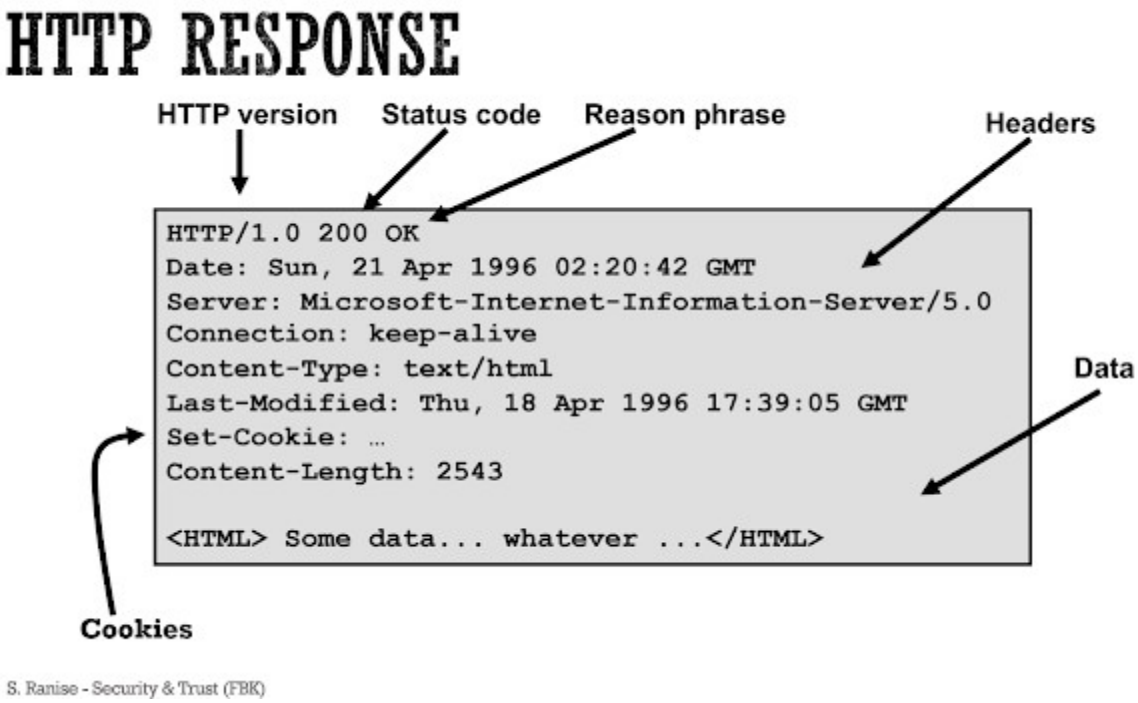
- **Connectionless**: client (e.g., a browser) initiates an HTTP request and after a request is made, the client disconnects from the server and waits for a response. The server processes the request and re-establishes the connection with the client to send a response back

- **Media independent**: any type of data can be sent by HTTP as long as both the client and the server know how to handle the data content
- **Stateless**: the server and client are aware of each other only during a current request. Afterwards, both of them forget about each other. Due to this nature of the protocol, neither the client nor the browser can retain information between different requests across the web pages



Let's see example of an HTTP request and an HTTP response:

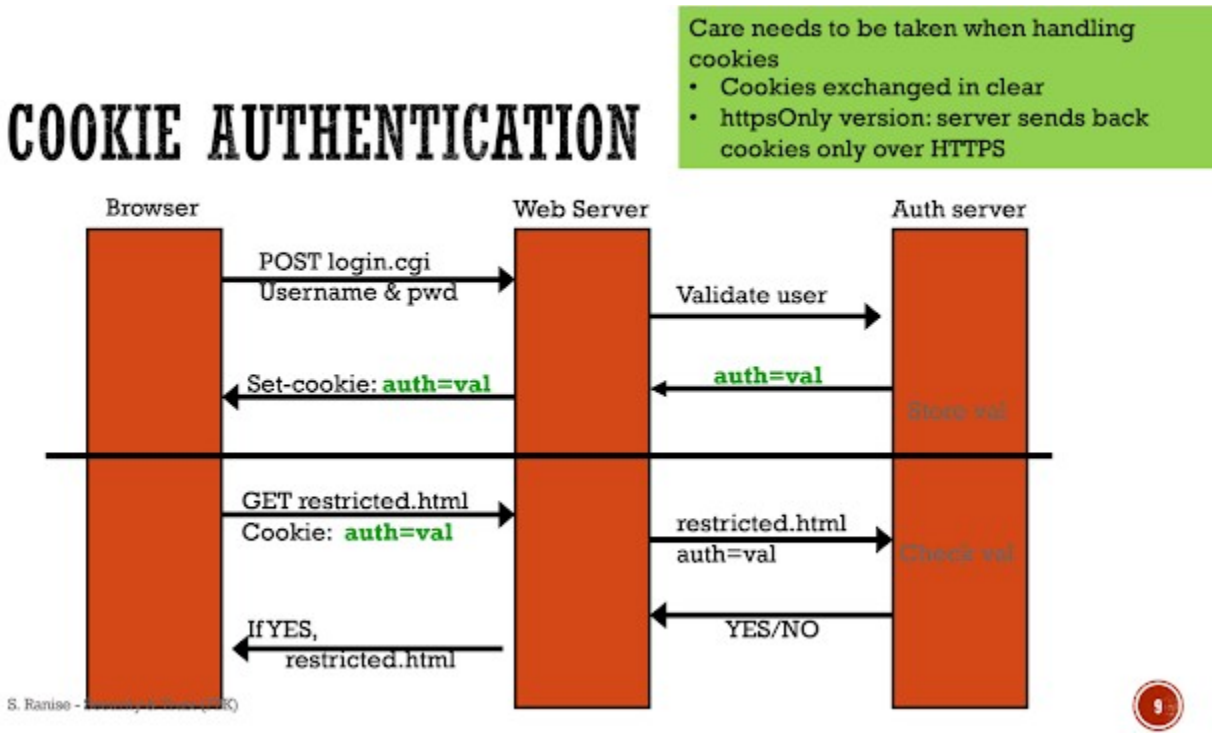




To insert the concept of state in this stateless protocol, **cookies** are used.
An HTTP cookie (also called web cookie, Internet cookie, browser cookie, or simply cookie) is a small piece of data sent from a website and stored on the user's computer by the user's web browser while the user is browsing. It is designed to be a reliable mechanism for websites to **remember stateful information** (such as items added in the shopping cart in an online store) or to record the user's browsing activity (including clicking particular buttons, logging in, or recording which pages were visited in the past).
Authentication cookies are the most common method used by web servers to know whether the user is logged in or not, and which account they are logged in with. Without such a mechanism, the site would not know whether to send a page containing sensitive information, or require the user to authenticate themselves by logging in.
Other cookies can track your action or store some general information.

The security of an authentication cookie generally depends on the security of the issuing website and the user's web browser, and on whether the cookie data is encrypted. Some security vulnerabilities may allow a cookie's data to be read by a hacker, used to gain access to user data, or used to gain access (with the user's credentials) to the website to which the cookie belongs (see cross-site scripting and cross-site request forgery for examples).

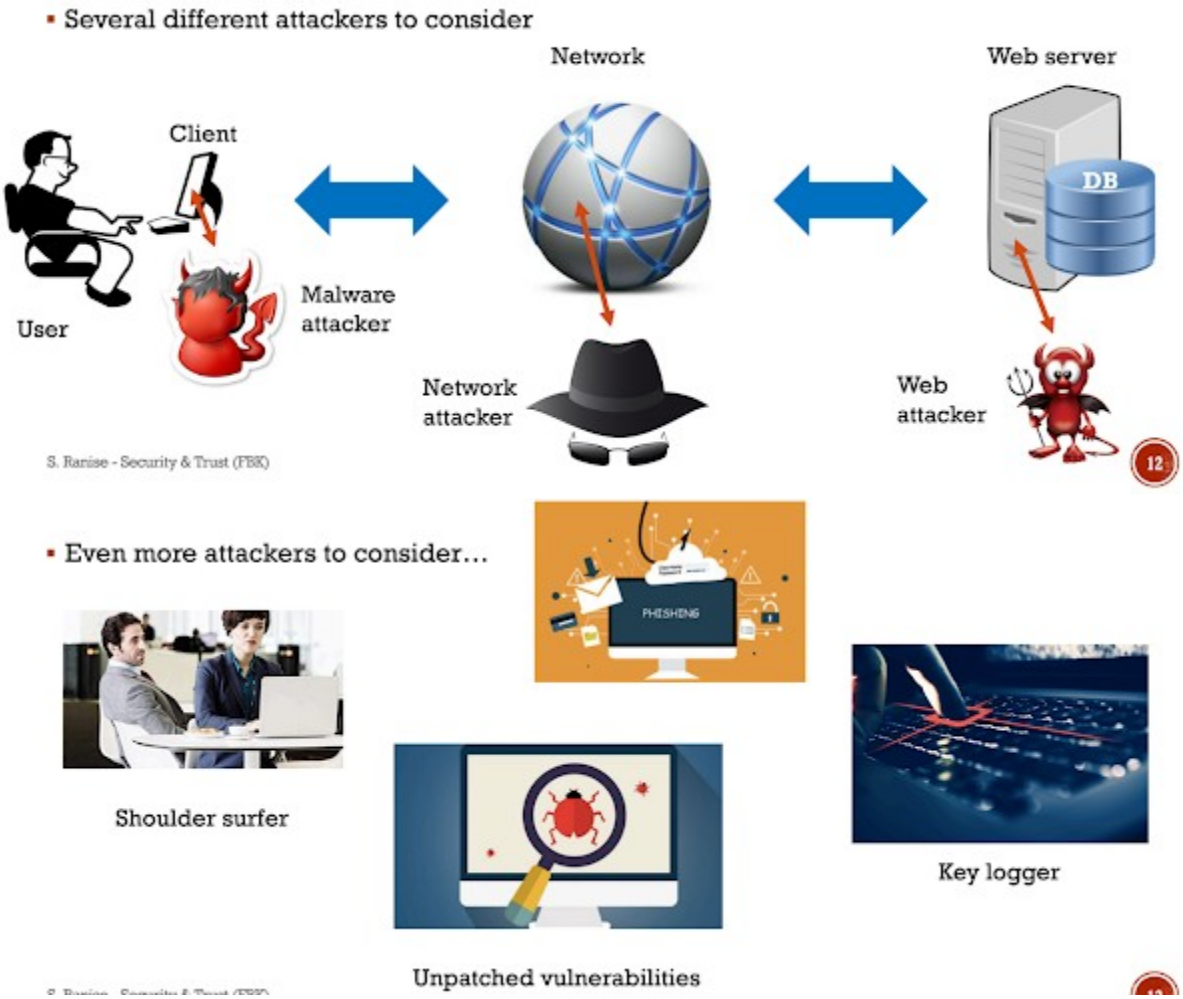
With cookies, some privacy concerns were introduced. Tracking cookies, and especially third-party tracking cookies, are commonly used as ways to compile long-term records of individuals' browsing histories. European law requires that all websites targeting EU member states gain "informed consent" from users before storing non-essential cookies on their device.



Since cookies can contain sensitive information, they must be exchanged in a secure manner (like using HTTPS). For example if a cookie contains an authentication token, then an attacker can steal it and doesn't need anything more to authenticate as the user.

Now that we explain what a web application is, let's make some considerations of the security of them.
Creating a Web application is relatively easy, but creating a secure Web application is hard and tedious. Because of the multi-tiered architecture, security flaws may appear at many levels. In fact we need to secure:

- Database
- Server
- Application
- Network



Let's see some type of attack that can be done:

- **User-Layer**
  For example phishing, keylogging and malware
- **Application-Layer**
  For example SQL injection, Cross-Site-Scripting (XSS), Cross-Site Request Forgery (CSRF), Broken Authentication and Unvalidated Input
- **Network-layer**
  For example Packet-Sniffing, Man-In-The-Middle Attacks (MITM) and DNS attack
- **Server-layer**
  For example Denial-of-Service (DoS) and OS Exploitation

We will concentrate on User-Layer and Application-Layer threats.

So the requirements in terms of security of a web application are:

- **Authentication** (you want to know who you are communicating with)
- **Authorization or Access Control** (user must have access to only those resources that they are entitled to)
- **Confidentiality** (you want to keep information secret e.g., credit card number)

- **Integrity** (you want to know that a message has not been modified in transit)
- **Non-repudiation** (if someone has sent a message, it should be impossible to deny it later, this means legal implications)

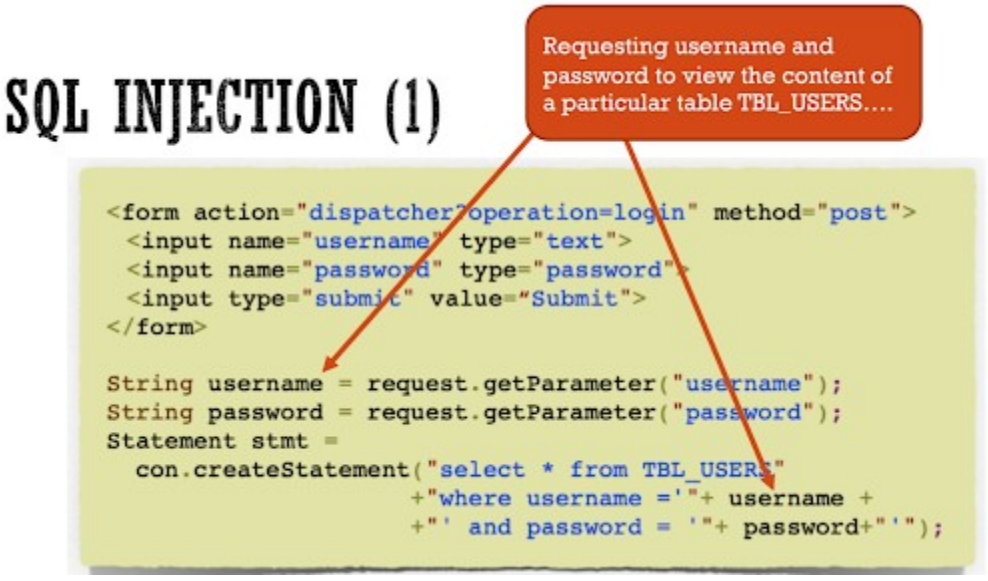
Web application security is a branch of information security that deals specifically with security of websites, web applications and web services. At a high level, web application security draws on the principles of application security but applies them specifically to Internet and web systems.

Application security encompasses measures taken to improve the security of an application often by finding, fixing and preventing security vulnerabilities.

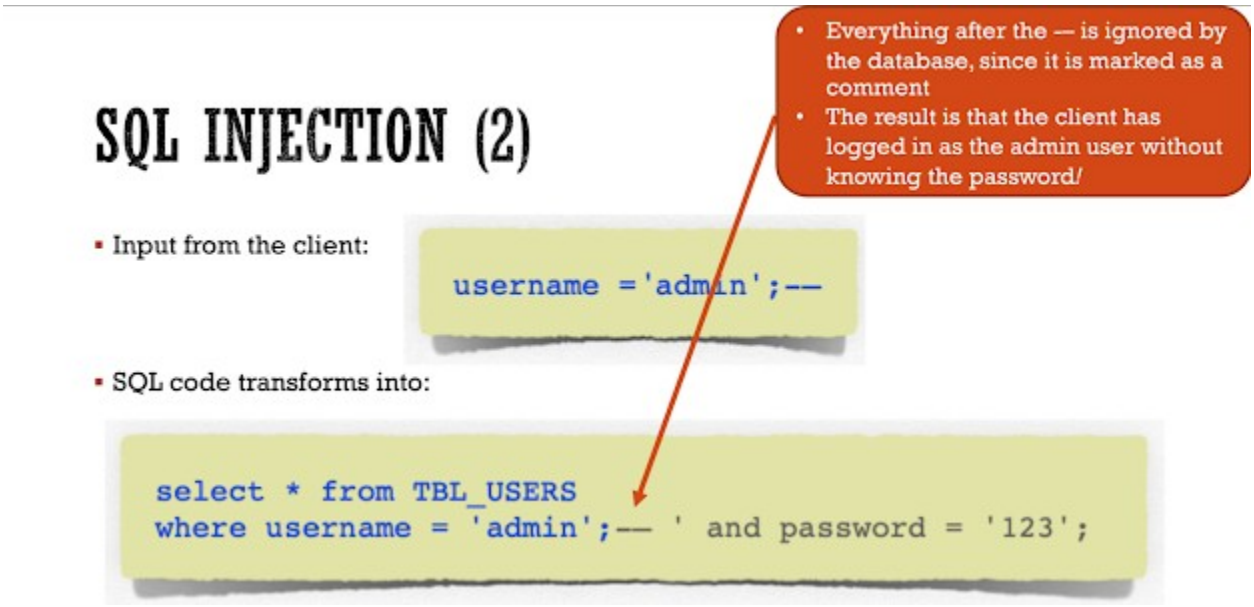
In brief, the goals of web application security are:
- Safely browse the web (visit a variety of web sites without incurring harm)
- Support secure web apps (apps provided over the web can have same security properties as stand-alone applications)
- Support secure mobile apps (web protocols and content standards are used as back end of many mobile apps)

Let's see the first possible attack, that is **injection attacks**.
This attack consists in sending some data to an interpreter which can trick the interpreter into executing unintended commands or accessing data without proper authorization. So the attacker injects some kind of malicious code into a script executer. This attack originated in the browser of the client but it is "executed" in the Server Side Script.
In this the attacker can change in a malicious way the original scope of the script.

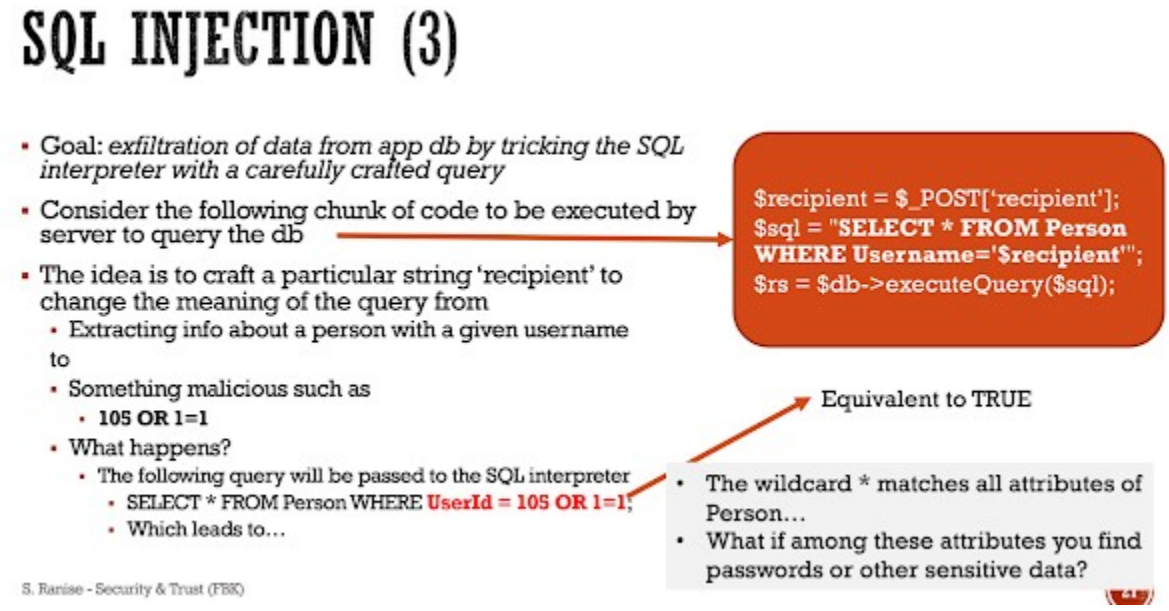
For example if you have this script on your application:

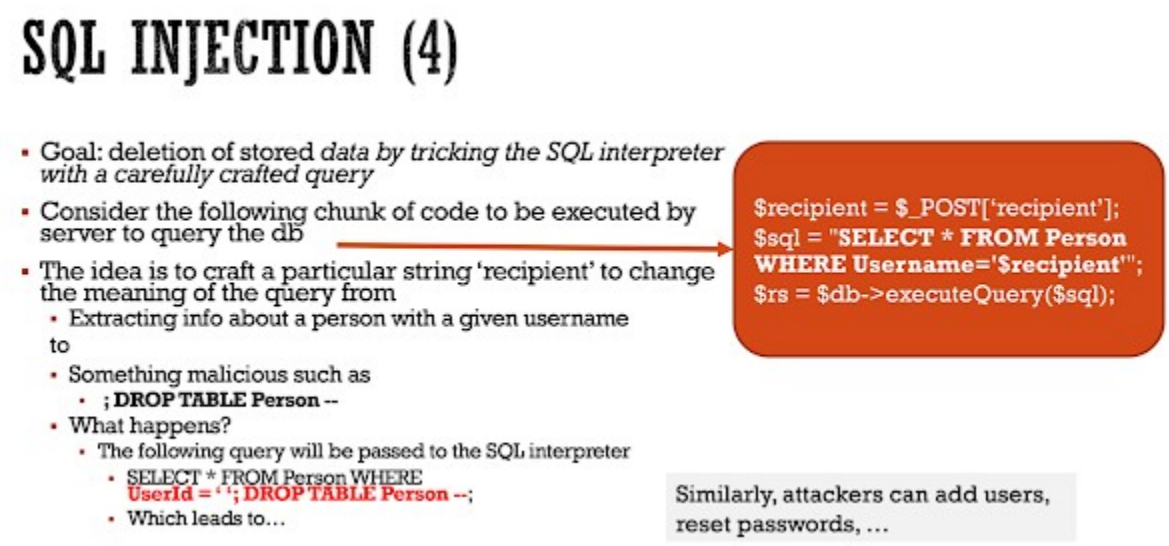


You can simply put into the form *admin';--* and you have made an SQL injection:



In this case we can authenticate as the user which has the username “admin”.
We can also retrieve all the data from a database or delete all the data from a database in these ways:





A famous SQL injection attack was done in June 2005 to CardSystems, which is a credit card payment processing company. This SQL injection attack put the company out of business because 263,000 credit cards were stolen from the database (the credit cards were stolen in an unencrypted way).
This attack was done despite the fact that CardSystems was compliant with a financial systems security standard called PCI-DSS.

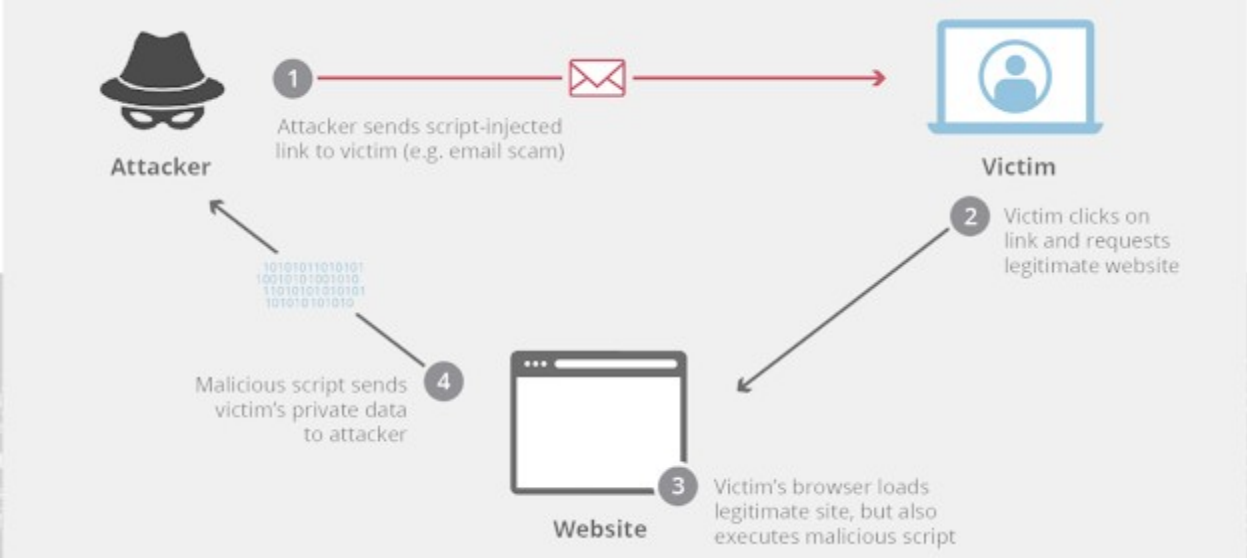
To summarize SQL injection, we have the problem that a client can input SQL code using input parameters (e.g., in a form) and these parameters are then used to dynamically construct SQL queries. The consequences are:

- **Loss of Confidentiality** (attackers can access sensitive data)
- **Authentication & Authorization** (attackers can gain access to privileged accounts or systems without passwords)
- **Integrity** (attackers can modify the information stored in the database)

The mitigations to this attack are:

- **Input sanitization**
  Using parameterized/prepared SQL queries instead, building SQL queries by properly escaping arguments
  In this way attackers can't create hand-made SQL injection.
- **Apply the principle of least privilege**
  That consists in giving least privilege to your application. So don't permit actions that aren't required by the application.

Let's see another type of attack, that is **Cross-Site Scripting** (XSS) attacks.
XSS allows attackers to execute scripts in the victim's browser which can hijack user sessions, deface web sites or redirect the user to malicious sites.



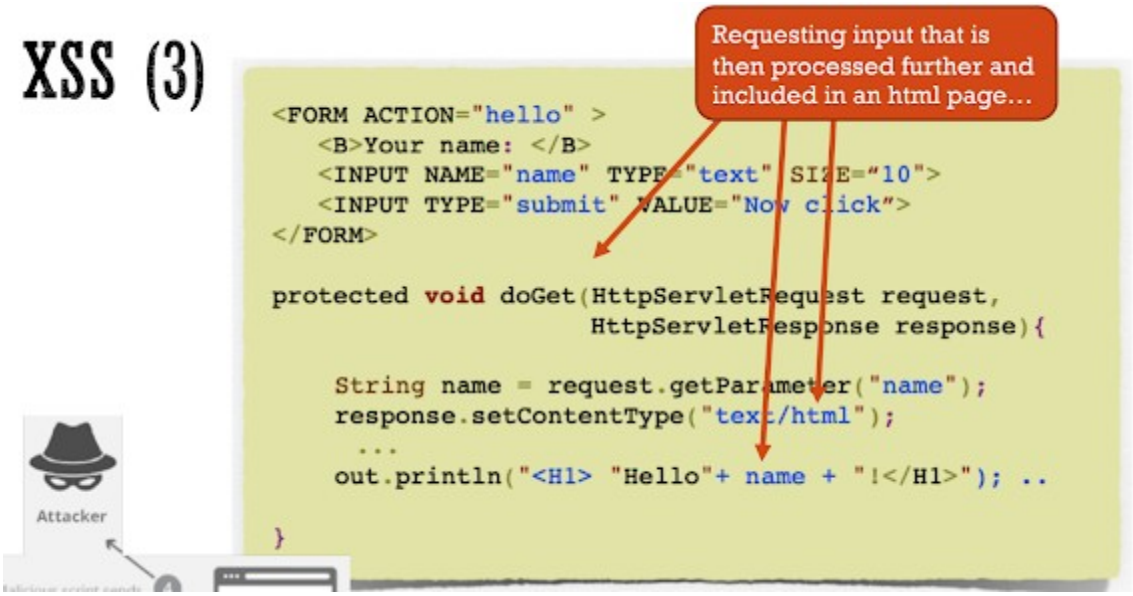
The first part consist in the victim receiving an URL by the attacker controlling a web site (in the example www.badguy.com)



The client is induced to click on the link in some way (because it is a normal website), but the website is vulnerable to this type of attack and in fact it is injected with a malicious script.
So, when the client open this URL, it will be redirected to a web page, which will be injected with the following scripts, that is executed:



At the end the attacker will receive all the data inside the cookie file of the vulnerable web site in a silent and transparent way to the user:

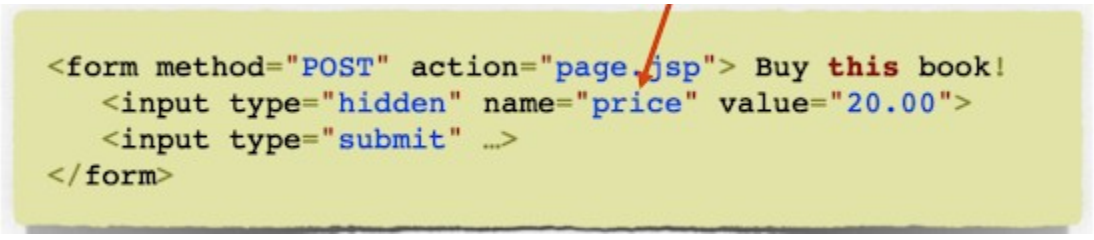


In brief, an attacker attaches a script with a HTTP response. The script executes with privileges available to the responding web application and the attacker is able to access privileged information only available to the user or the web application.
Since cookies often contain authentication information, this could allow the attacker to impersonate the victim.
Two variant of this type of attack exist:

- **Reflected XSS** (using a constructed URL, as in previous slides)
- **Stored XSS** (using POST to store the bad URL inside a comment/forum)

The mitigation to XSS is filter all input parameters from HTTP GET and POST Characters with special meaning in HTML and JavaScript, e.g., , (,), #, & should be removed or substituted (e.g., < becomes <). This may also require filtering all types of active content; e.g. JavaScript. In this way the client doesn't accept the Script inside the URL.
But notice that it is easy to forget something, so it's better to specify what characters are allowed, e.g., [A-Za-z0-9] (better using positive than negative filtering).

Other types of attacks are **unvalidated input**. This consists in editing values in a form of HTML page for example that shouldn't be modified.



The client can download this page, change the form and edit the value of the price input (and modify the action attribute of the form element).
Despite this seems like a bad way to build a web app, there are apps that use this approach.

SO Clients can easily circumvent checks in the HTML code itself, such as hidden parameters (e.g., price) and JavaScript code. Client-side validation is useful for performance reasons, but useless from a security point of view.
The solution is to never trust any input from the user, and never trust client side input validation. All parameters must be validated on the server side before they are used (also in this case, positive filtering is better than negative filtering).

Another is **broken authentication**: username and password combinations are commonly used and commonly broken. Collections of username and passwords are on sale on the dark web.
The causes are:

- Insecure storage of password hash (SQL injection)
- Weak hashing algorithms employed
- Faulty session management (session identifiers exposed)
- Long sessions or too many attempts at password recovery

The mitigation to this type of attack are:

- Disallow weak passwords
- Use a stronger hash algorithm
- Salt the passwords
- Use HTTPS for encrypting session identifiers to prevent MITM
- Do not expose credentials in untrusted locations (hidden fields, cookies, urls)
- Implement account lockouts
- Implement Multi Factor Authentication (MFA)

Another important topic is **session management**. Poor management of session identifiers can lead to different attacks such as:

- Cross-Site Request Forgery (CSRF)
- Session spoofing and hijacking
- Broken Authentication
- Privilege Escalation
- Sensitive Data Leakage

So a session identifier must be considered as an important asset to secure with strategies such as:

- Implementing Strict Timeouts
- Session-ID must be renewed when an authentication state is passed (on logging in/out)
- Ensuring session IDs cannot be easily guessed (use a large space and generate ids with a good random number generator)

For this reason **Proper Cookie Management** must be done. In fact, session identifiers are stored on the client (browser) side in cookies.
Therefore, cookies should be managed properly to ensure security of sessions:

- Use "Domain" and "Path" attributes to restrict the scope of cookies to narrow subdomains
- Use "Secure" attribute to force browsers to send cookies over HTTPS
- Use "HttpOnly" attribute to prevent scripts from accessing the cookies

- Use "X-XSS-Protection" header to allow browsers to detect XSS attacks
- Use "Content-Security-Policy" headers to instruct browsers to only load resources from whitelisted locations

Let's now talk about **access control for web applications**, which are a neglected area that should be much more considered in IT service deployment.
Let's immediately view a real attack, which results in a disclosure of a huge amount of sensitive information.
Equifax was a kind of financial organization containing all this data.



This happened because the permissions on the server (in particular the database and the engine which extracts the information from the database) was not properly configured, and this made possible a **command injection attack**. The attack was done by exploiting a bug in the file upload mechanism in the Jakarta Multipart parser in some versions of Apache Struts (with the possibility to execute arbitrary commands).
It is important to denote that **patches were available but not applied**.
The vulnerability was imported in the system by a particular external library, which is an open-source software.

To make sure to use SW components without **known** vulnerabilities, you can use specific tools (like OWASP Dependency check) which identifies project dependencies (import list) and checks if there are any known, publicly disclosed, vulnerabilities. Obviously these tools will prune false positives, because there could be that you don't use the part of the library which includes the vulnerability.

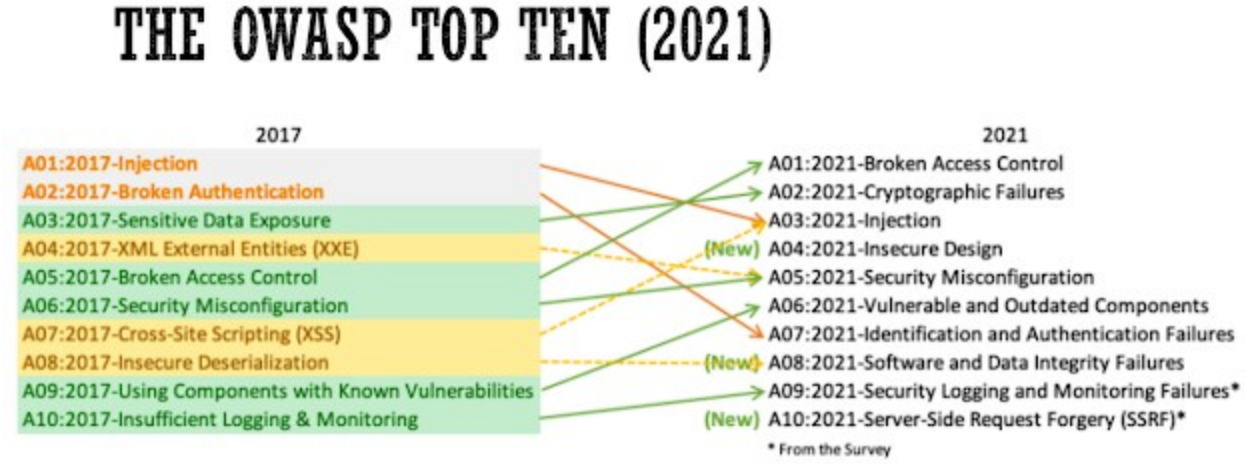
But we still have the problem of making sure to use SW components without **unknown** vulnerabilities. The answer (and the only way) is to use **Access Control**.
An important concept is: "Security best practices dictate that this user have as little privilege as possible on the server itself, since security vulnerabilities in web applications and web servers are so commonly exploited."
In other words, follow the **Principle of Least Privilege**: every subject (such as a process, a user, or a program) must be able to access only the information and resources that are necessary for its legitimate purpose.
In this way even if commands are injected, if these are run with the lowest possible privileges, then less harm can be performed!
This seems to be an effective mitigation measure to reduce the impact of unknown vulnerabilities and to avoid the elimination of false positives libraries.

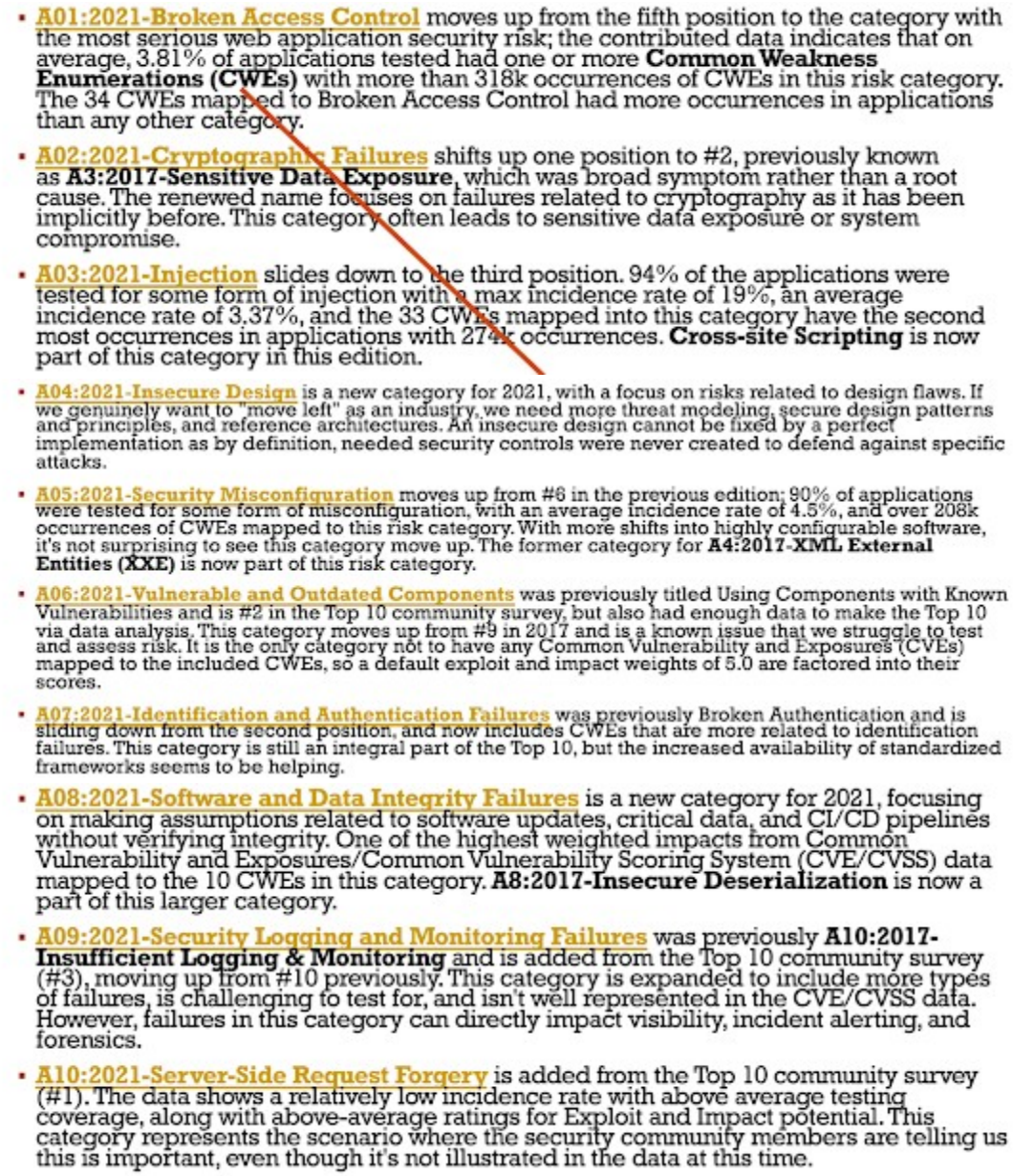
This approach to security is also known under the name of **risk based** and can be summarized as follows: a security breach is not a matter of **if** but **when**.

Let's make a short summary of this chapter on web application security.
We have only scratched the surface, in fact there are many more concepts and issues in web applications. When using cloud, mobile, and edge computing, the situation gets even more complex and the attack surface enlarges significantly…

A good idea to mitigate most of the vulnerabilities is to use automated solutions for analysis and enforcement of security policies. As always it is better to consider the risk assessment rather than try to get absolute security.

This image represents the most popular vulnerabilities and sources of risk in 2017 and in 2021:



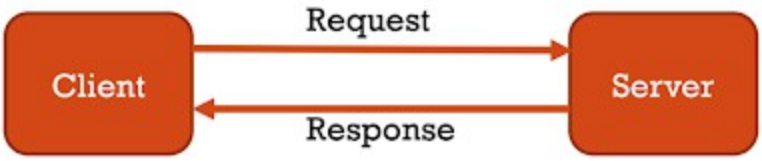


- **A01:2021-Broken Access Control** moves up from the fifth position to the category with the most serious web application security risk; the contributed data indicates that on average, 3.81% of applications tested had one or more **Common Weakness Enumerations (CWEs)** with more than 318k occurrences of CWEs in this risk category. The 34 CWEs mapped to Broken Access Control had more occurrences in applications than any other category.
- **A02:2021-Cryptographic Failures** shifts up one position to #2, previously known as **A3:2017-Sensitive Data Exposure**, which was broad symptom rather than a root cause. The renewed name focuses on failures related to cryptography as it has been implicitly before. This category often leads to sensitive data exposure or system compromise.
- **A03:2021-Injection** slides down to the third position. 94% of the applications were tested for some form of injection with a max incidence rate of 19%, an average incidence rate of 3.37%, and the 33 CWEs mapped into this category have the second most occurrences in applications with 274k occurrences. **Cross-site Scripting** is now part of this category in this edition.
- **A04:2021-Insecure Design** is a new category for 2021, with a focus on risks related to design flaws. If we genuinely want to "move left" as an industry, we need more threat modeling, secure design patterns and principles, and reference architectures. An insecure design cannot be fixed by a perfect implementation as by definition, needed security controls were never created to defend against specific attacks.
- **A05:2021-Security Misconfiguration** moves up from #6 in the previous edition; 90% of applications were tested for some form of misconfiguration, with an average incidence rate of 4.5%, and over 208k occurrences of CWEs mapped to this risk category. With more shifts into highly configurable software, it's not surprising to see this category move up. The former category for **A4:2017-XML External Entities (XXE)** is now part of this risk category.
- **A06:2021-Vulnerable and Outdated Components** was previously titled Using Components with Known Vulnerabilities and is #2 in the Top 10 community survey, but also had enough data to make the Top 10 via data analysis. This category moves up from #9 in 2017 and is a known issue that we struggle to test and assess risk. It is the only category not to have any Common Vulnerability and Exposures (CVEs) mapped to the included CWEs, so a default exploit and impact weights of 5.0 are factored into their scores.
- **A07:2021-Identification and Authentication Failures** was previously Broken Authentication and is sliding down from the second position, and now includes CWEs that are more related to identification failures. This category is still an integral part of the Top 10, but the increased availability of standardized frameworks seems to be helping.
- **A08:2021-Software and Data Integrity Failures** is a new category for 2021, focusing on making assumptions related to software updates, critical data, and CI/CD pipelines without verifying integrity. One of the highest weighted impacts from Common Vulnerability and Exposures/Common Vulnerability Scoring System (CVE/CVSS) data mapped to the 10 CWEs in this category. **A8:2017-Insecure Deserialization** is now a part of this larger category.
- **A09:2021-Security Logging and Monitoring Failures** was previously **A10:2017-Insufficient Logging & Monitoring** and is added from the Top 10 community survey (#3), moving up from #10 previously. This category is expanded to include more types of failures, is challenging to test for, and isn't well represented in the CVE/CVSS data. However, failures in this category can directly impact visibility, incident alerting, and forensics.
- **A10:2021-Server-Side Request Forgery** is added from the Top 10 community survey (#1). The data shows a relatively low incidence rate with above average testing coverage, along with above-average ratings for Exploit and Impact potential. This category represents the scenario where the security community members are telling us this is important, even though it's not illustrated in the data at this time.

IoT Applications

The main difference between Web Applications and IoT Applications is the architecture. In fact Web Applications are based on a Client-Server architecture:

- Communications take place following the request/reply (pull) interaction model
- Interaction is limited to two entities (one-to-one)
- Each entity must know how to address its partner in the communication
- The two entities must be available at the same time in order to communicate
- Communication is inherently synchronous
- Communication is only pull-based



Instead, **IoT Applications have Publish-Subscribe Communication**. Publish/subscribe is a comprehensive solution for client-server problems and the features are:

- **Many-to-many communication model**
  Interactions take place in an environment where various information producers and consumers can communicate, all at the same time. Each piece of information can be delivered at the same time to various consumers. Each consumer receives information from various producers
- **Space decoupling**
  Interacting parties do not need to know each other. Message addressing is based on their content
- **Time decoupling**
  Interacting parties do not need to be actively participating in the interaction at the same time. Information delivery is mediated through a third party
- **Synchronization decoupling**
  Information flow from producers to consumers is also mediated, thus synchronization among interacting parties is not needed
- **Push/Pull interactions**
  Both methods are allowed

In IoT applications this solution is perfect because you have a lot of devices which generate a large amount of data that should be processed by a backend server. But the amount of data is so huge that the server algorithm can't process them in real time.
So we need to decouple the production of the data with the consumption of the data. To do this we insert a sort of queue between the producers and the consumers.
The system which manages this queue is called **broken**.



The **publishers** produce data in the form of events.
The **subscribers** declare interests on published data with subscriptions.
Each **subscription** is a filter on the set of published events and an Event Notification Service (ENS) notifies to each subscriber every published event that matches at least one of its subscriptions.
Events represent information structured following an event schema. The event schema is fixed, defined a-priori, and known to all the participants. It defines a set of fields or attributes, each constituted by a name and a type. The types allowed depend on the specific implementation, but basic types (like integers, floats, Booleans, strings) are usually available.

Given an event schema, an event is a collection of values, one for each attribute defined in the schema.

So, subscribers express their interests in specific events issuing **subscriptions**. A subscription is a constraint expressed on the event schema.
The Event Notification Service will notify an event e to a subscriber x only if the values that define the event satisfy the constraint defined by one of the subscriptions s issued by x (in this case we say that e matches s).
Subscriptions can take various forms, depending on the subscription language and model employed by each specific implementation:

- Topic-based
- Hierarchy-based
- Content-based
- Type-based
- …

In **Topic-based subscriptions** the data that is published in the system is mostly unstructured, but each event is "tagged" with the identifier of a topic it is published in.
Subscribers issue subscriptions containing the topics they are interested in.
A topic can be thus represented as a "virtual channel" connecting producers to consumers (data distribution in topic-based publish/subscribe systems is close to group communication).



In **Hierarchy-based subscriptions**, as in topic-based subscription, the event is "tagged" with the topic it is published in, and subscribers issue subscriptions containing the topics they are interested in.
Contrary to topic-based subscription, topics are organized in a hierarchical structure which express a notion of containment between topics. When a subscriber subscribes to a topic, it will receive all the events published in that topic and in all the topics present in the corresponding sub-tree.

The **event notification system** is usually implemented as a:

- **Centralized service**: the ENS is implemented on a single server
- **Distributed service**: the ENS is constituted by a set of nodes, event brokers, which cooperate to implement the service (it is usually preferred for large settings where scalability is a fundamental issue)

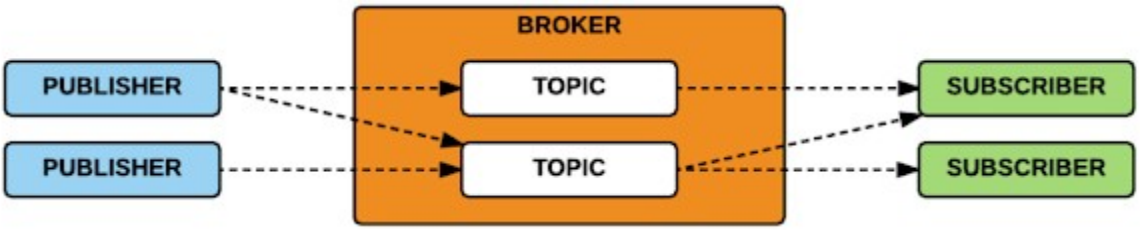
Let's now talk about **MQTT**, which is used just for the purpose of sending messaging containing data collected from typically IoT devices to some kind of backend server in order to analyze this data and make decisions on it.

MQTT (Message Queue Telemetry Transport) is a publish/subscribe, simple and lightweight messaging protocol, designed for constrained devices and low-bandwidth, high-latency or unreliable networks. It runs over TCP, it is hierarchy based and it has a simple packet format (binary payloads).
The design principles of this protocol are:

- Minimize network bandwidth
- Consider small set of device resource requirements
- Ensure reliability and some degree of assurance of delivery

These principles also turn out to make the protocol ideal for the emerging Machine-to-Machine (M2M) or Internet of Things world of connected devices, and for mobile applications where bandwidth and battery power are at a premium.

So the architecture of this protocol is Publish-Subscribe:



The **Publisher** publishes a message to one (or many) topic(s) in the broker.
The **Subscriber** subscribes to one (or many) topic(s) in the broker and receives all the messages sent from the publisher.
The **Centralized broker** routes all the messages from the publishers to the subscribers.
The **Topic** consists of one or more levels that are separated by a forward slash (Example: /smarthouse/livingroom/temperature).

The format of the message of this protocol is the following:

Bit	7	6	5	4	3	2	1	0
Byte 1	MQTT Control Packet type				Flags specific to each MQTT Control Packet type			
Byte 2	Remaining Length							

And the messages can be of these types:

Name	Value	Direction of flow	Description
Reserved	0	Forbidden	Reserved
CONNECT	1	Client to Server	Client request to connect to Server
CONNACK	2	Server to Client	Connect acknowledgment
PUBLISH	3	Client to Server or Server to Client	Publish message
PUBACK	4	Client to Server or Server to Client	Publish acknowledgment
PUBREC	5	Client to Server or Server to Client	Publish received (assured delivery part 1)
PUBREL	6	Client to Server or Server to Client	Publish release (assured delivery part 2)
PUBCOMP	7	Client to Server or Server to Client	Publish complete (assured delivery part 3)
SUBSCRIBE	8	Client to Server	Client subscribe request
SUBACK	9	Server to Client	Subscribe acknowledgment
UNSUBSCRIBE	10	Client to Server	Unsubscribe request
UNSUBACK	11	Server to Client	Unsubscribe acknowledgment
PINGREQ	12	Client to Server	PING request
PINGRESP	13	Server to Client	PING response
DISCONNECT	14	Client to Server	Client is disconnecting
Reserved	15	Forbidden	Reserved

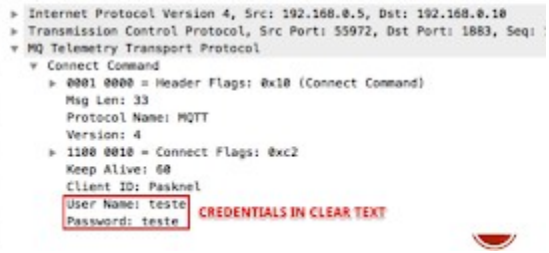
Since it is possible to pass a username and password with an MQTT packet, we want to pass them in a secure way, but the MQTT protocol has nothing about security.
Two simple mechanisms to add security are:

- Encryption across the network can be handled with TLS, independently of the MQTT protocol itself (it is worth noting that TLS is not the lightest of protocols, and does add significant network overhead)
- Additional security can be added by an application encrypting data that it sends and receives, but this is not something built-in to the protocol, in order to keep it simple and lightweight

MQTT AND CREDENTIALS

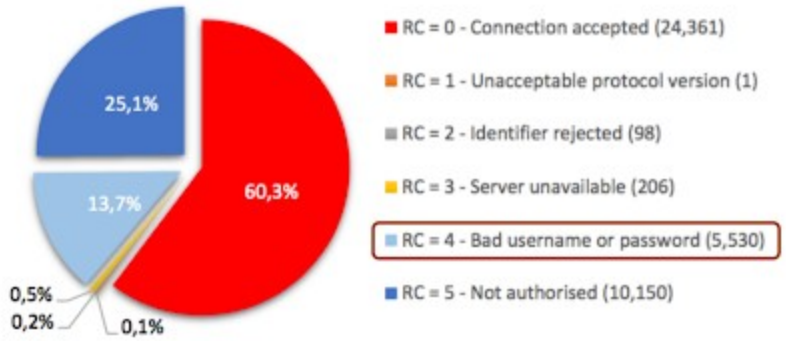
- Clients can authenticate to the MQTT Broker sending a user name and password with the CONNECT packet
- The CONNACK Packet is the packet sent by the MQTT Broker in response to a CONNECT Packet received from the client
- The CONNACK packet header contains a "return code" field that represents the result of the authentication (e.g., Connection Accepted)

Value	Return Code Response	Description
0	0x00 Connection Accepted	Connection accepted
1	0x01 Connection Refused, unacceptable protocol version	The Server does not support the level of the MQTT protocol requested by the Client
2	0x02 Connection Refused, identifier rejected	The Client identifier is correct UTF-8 but not allowed by the Server
3	0x03 Connection Refused, Server unavailable	The Network Connection has been made but the MQTT service is unavailable
4	0x04 Connection Refused, bad user name or password	The data in the user name or password is malformed
5	0x05 Connection Refused, not authorized	The Client is not authorized to connect
6-255		Reserved for future use



In most cases MQTT brokers are configured very badly, in fact in this test we can see that a very large number of servers accepted the connection with any kind of authentication (only 14% implements authentication, but in this 14% very few doesn't transmit the credentials in clear).

MQTT AND CREDENTIALS (CONT'D)





Let's see other security considerations and issues:

- Even when authentication is used, one can try a brute-force attack, since there is no mechanism is available for limiting the number of attempts
- With wildcards the client is able to receive all the messages sent by other clients (Subscribing to "#") and also to receive internal control messages of brokers (Subscribing to "$SYS/#")

- Writing packets to “$SYS/#” in an attempt to crash the broker
- Writing packets to user defined topic

So in conclusion, in the IoT world there is a very lack of security.

RECAP QUESTIONS (1)

- What is web application security?
- For which kind of security threats found in web applications, TLS is not an adequate countermeasures?
- Which kind of attackers threaten web applications?
- What is an injection attacks? Give at least two examples of such an attack.
- What is a CSRF attack? Give a high level description of how to mount it.
- What is a XSS attack? Give a high level description of how to mount it.
- What is a Phishing attack? Give a high level description of how to mount it.
- What are the main mitigation measures for injection attacks? And for CSRF attacks? And For XSS attacks? And for Phishing attacks?

RECAP QUESTIONS (2)

- Explain the publish-subscribe pattern and how it differs from the client-server pattern. What are the advantages of using the publish-subscribe pattern for IoT applications?
- Explain how MQTT implements the publish-subscribe pattern
- What are the main security issues of MQTT?
- Why it may be problematic to use TLS to secure the communication between the MQTT broker and the IoT devices?

Privacy and GDPR

Privacy and data protection

Privacy is a different topic of security but it is correlated to it. It is very difficult to capture exactly what privacy and data protection means.
Privacy can be decomposed over several different dimensions but it is very difficult to find an exact set of dimensions that really characterizes all the aspects of privacy.

So the notion of privacy isn’t well defined. The more general definition is **informational self-determination**, that is that everyone gets to control information about herself/himself.
This means a lot of things, for example:

- Who gets to see what (related to **access** control)
- Who gets to use what (related to **usage** control)
- What they can use it for (related to **purpose** control)

- Who they can give it to (related to **data transfer** control)

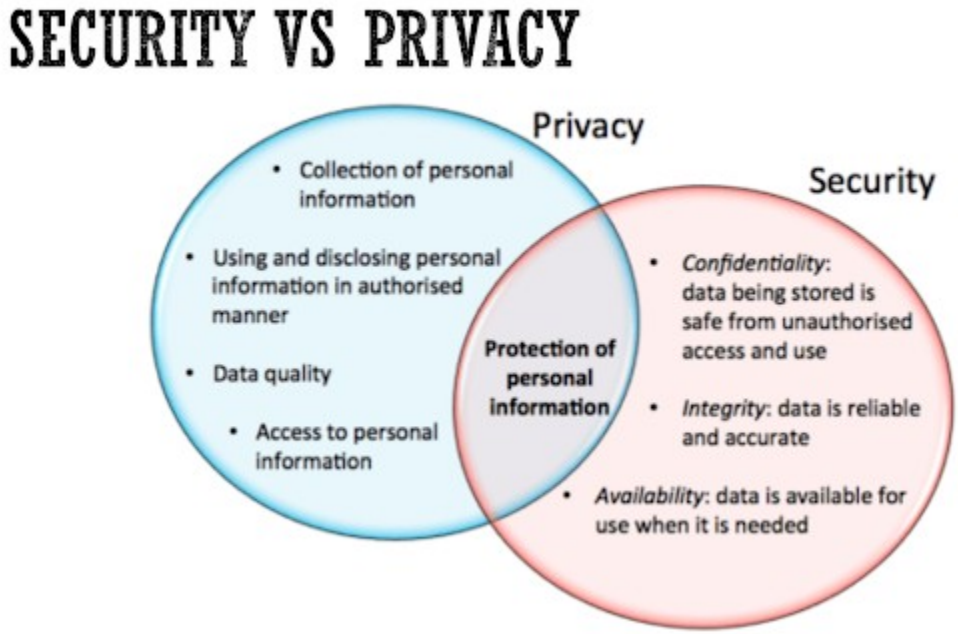
An important example is how to manage cookies.

Another definition of the notion of privacy is the high-level of difficulty in **correlating data/actions**, which means that the actions you perform online aren't linked to each other. This is because otherwise, people can understand your behavior online.
But we have to find a sort of trade off, because if people become completely anonymous, then they can do illegal things without being tracked.

So this two definitions can explain (not completely but in a sufficiently way) what privacy is:

- The control of information
- The capability to perform action in an anonymous way

Let's see the correlation between privacy and security:



There are some security mechanisms that can help with Privacy requirements (in particular Confidentiality). For example Access Control can be useful for Privacy if it is managed by the user on his personal information.
So there exists overlaps between privacy and security but they aren't the same thing.
Privacy is typically evaluated in terms of the impact that the violation of a Privacy property may have on the user (whose data has been violated).

A more complete definition of privacy is called **LINDDUN**, which is an acronym of the threats which privacy take in count:

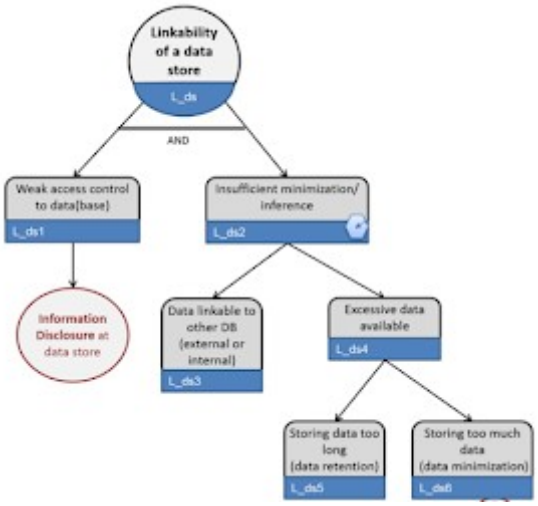
- Linkability
- Identifiability
- Non-repudiation
- Detectability
- Disclosure of information

- Unawareness
- Non-compliance

Type	Property	Description	Threat
Hard privacy	Unlinkability	Hiding the link between two or more actions, identities, and pieces of information.	**L**inkability
	Anonymity	Hiding the link between an identity and an action or a piece of information	**I**dentifiability
	Plausible deniability	Ability to deny having performed an action that other parties can neither confirm nor contradict	**N**on-repudiation
	Undetectability and unobservability	Hiding the user's actvities	**D**etectability
Security	Confidentiality	Hiding the data content or controlled release of data content	**D**isclosure of information
Soft Privacy	Content awareness	User's consciousness regarding his own data	**U**nawareness
	Policy and consent compliance	Data controller to inform the data subject about the system's privacy policy, or allow the data subject to specify consents in compliance with legislation	**N**on compliance

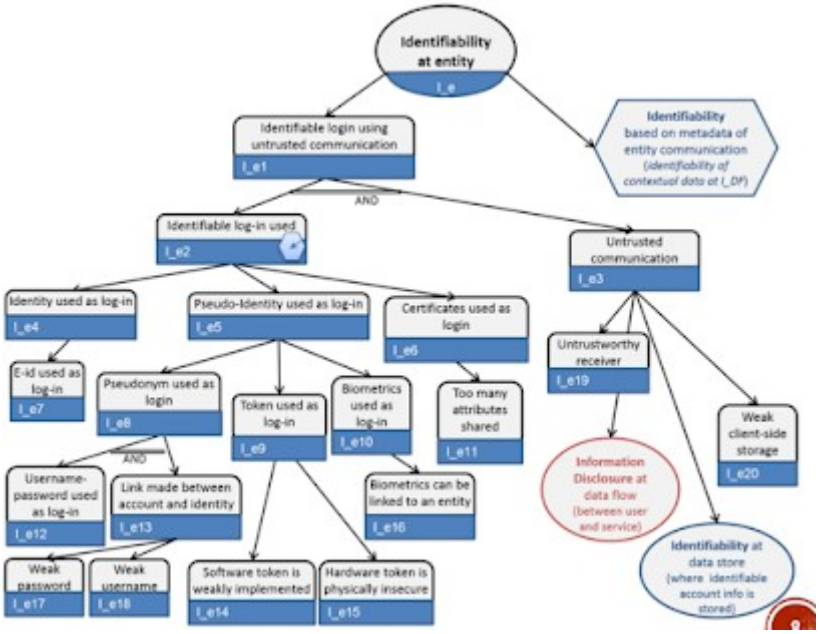
The difference between hard privacy and soft privacy is that soft privacy properties cannot be reached only by technical techniques, but there are some user's aspect that must be considered.

Linkability consists of being able to sufficiently distinguish whether 2 **items of interest** (ioi) are linked or not, even without knowing the actual identity of the subject of the linkable ioi.
For example web page visits by the same user or entries in databases related to the same person.
The consequences of linkability are Identifiability (when too much linkable information is combined) and Inference (when "group data" is linkable, this can lead to societal harm, like discrimination, in prices for example).

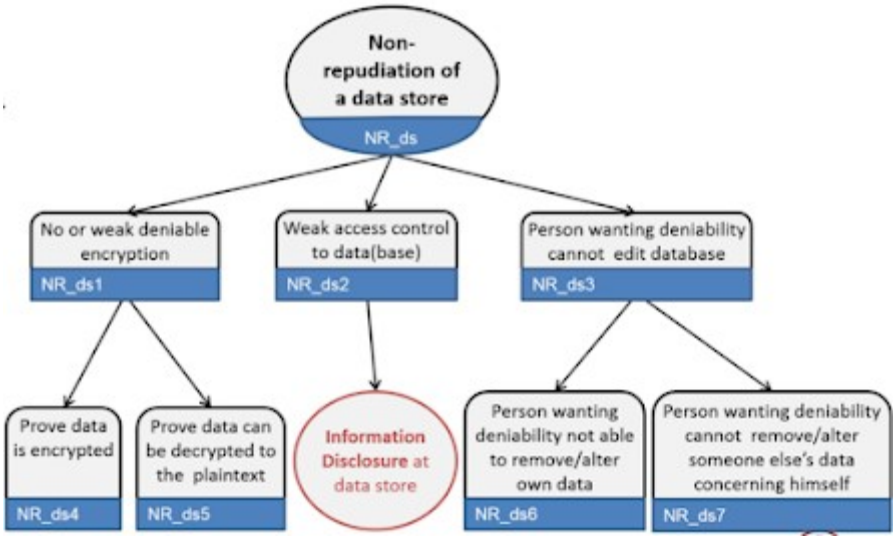


Identifiability consists of being able to sufficiently identify the subject within a set of subjects (i.e. the anonymity set), or in other words, not being able to hide the link between the identity and the ioi (an action or piece of information).

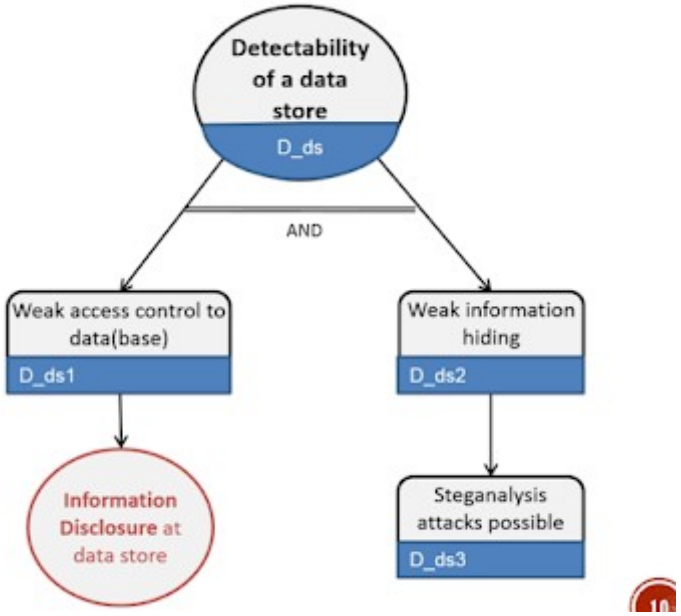
For example identifying the reader of a web page or the person to whom an entry in a database relates.
The consequences are severe privacy violations (when the subject assumes he is anonymous).



Non-repudiation consists of not being able to deny a claim. The attacker can thus prove a user knows, has done or has said something. He can gather evidence to counter the claims of the repudiating party. Notice that it is exactly the opposite of what security wants.
For example this can cause problems in on-line voting/whistleblowing
The consequence is that when a person is not able to repudiate an action or piece of information, he can be held accountable (e.g. a whistleblower can be prosecuted).

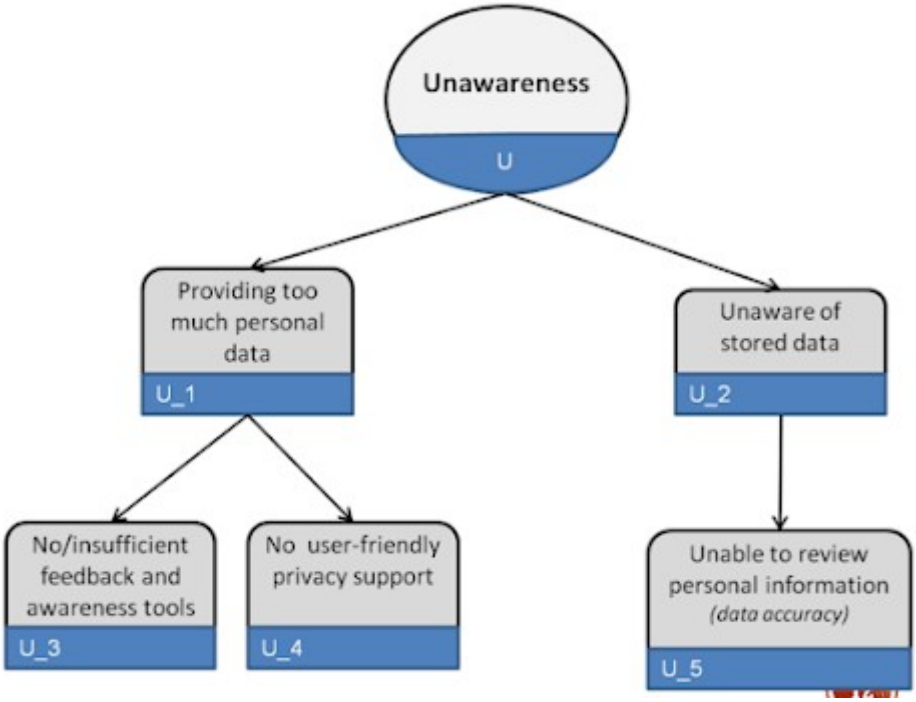


Detectability consists of being able to sufficiently distinguish whether an ioi exists or not. Detectability concerns iois of which the content is not known (to the attacker).
For example, determine the existence of a pregnancy test.
The consequence is that by detecting whether an ioi exists, one can deduce certain information, even without actually having access to that information.

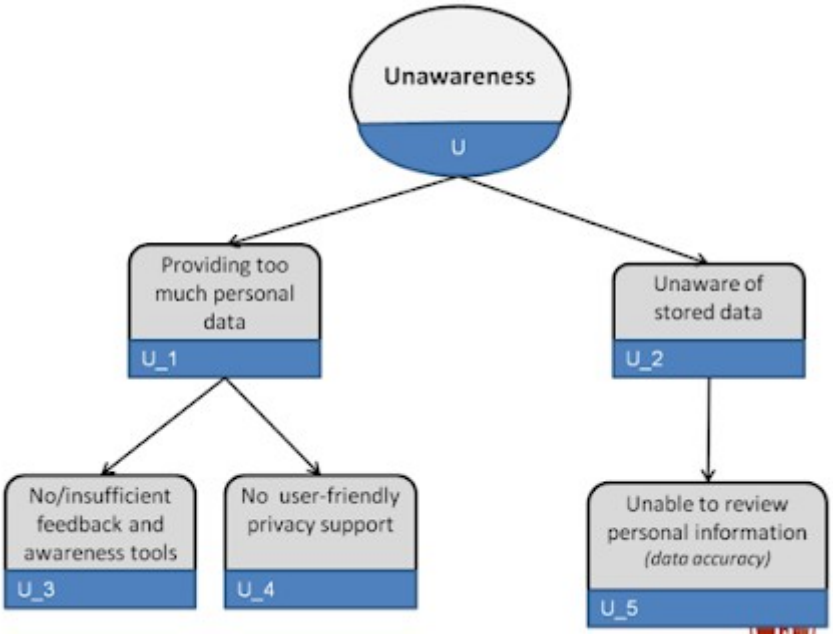


Disclosure of information is related to (violation of) confidentiality and thus more to security. We've already seen this part.

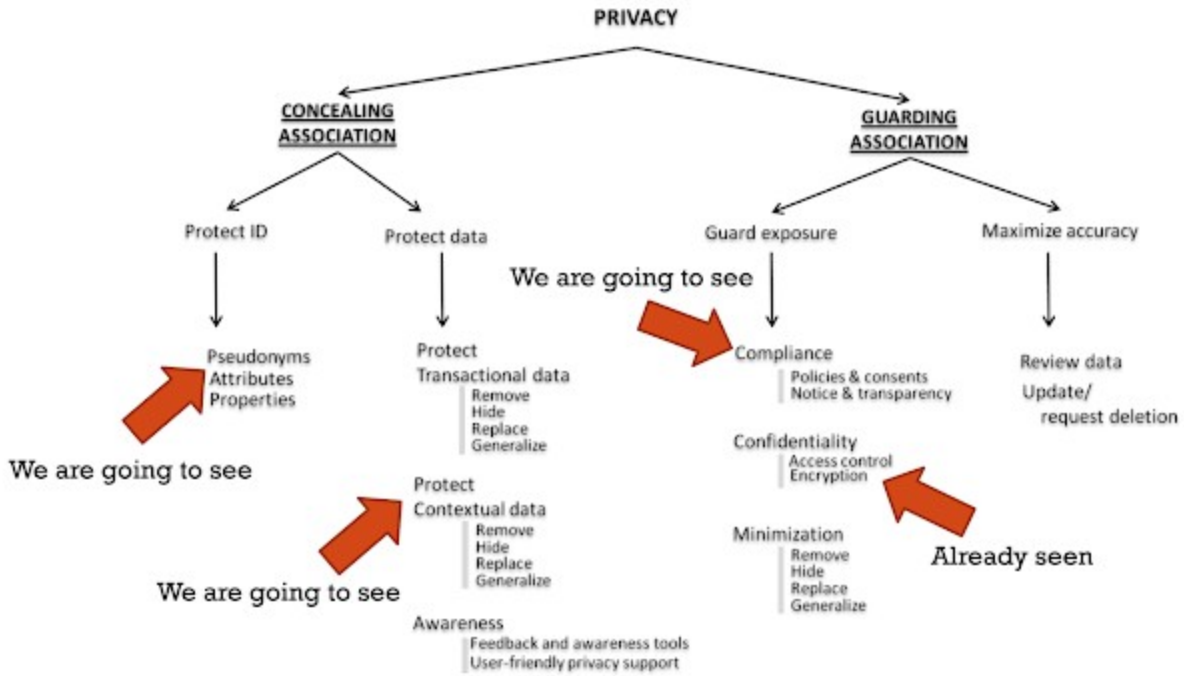
Unawareness consists of being unaware of the consequences of sharing information. Often users are not aware of the impact of sharing data. This can be data shared to friends on facebook, but also personal information shared with other services (i.e. loyalty cards, online services, ...). Ideally, all users (data providers) should be clearly informed and educated of the consequences of sharing data using (online) services. It is a combination of technical means and the interaction of users.
The consequence is that the more information is available, the easier it can be linked (and identified).



Non-compliance consists of not being compliant with legislation, regulations, and corporate policies.
The consequences are fines (when violating legislation, or not adhering to the communicated corporate policies) and loss of image, credibility, etc.



Let’s see an overview of the various kind of mitigation take in count by LINDDUN to guarantee privacy:



We are going to see only some of these.

Anonymization

Data anonymization consists in “protecting” **Personally Identifying Information** (PII), like Name and Social Security number.
The problem is that PII doesn't have a technical meaning, in fact it could be interpreted and it depends a lot on the context.
In privacy breaches, any information can be personally identifiying, even those that don’t seem like it.

Let’s see an example.
Once Netflix launched the Netflix prize, a contest to improve movie recommendations algorithm of at least 10% of the used algorithm and get a $1 million prize (if best solution).
For the first edition in 2006, Netflix released 100 million supposedly anonymized movie ratings, each included a unique subscriber ID, the movie title, year of release and the date on which the subscriber rated the movie.
The problem was that two University of Texas researchers announced that they had identified some of the Netflix users in the data set, because researchers were able to identify targets by matching their Netflix reviews with data from other sites (like IMDb).
This was a problem because it can reveal personal sensitive information, in fact a woman (a lesbian mother who is not open about her sexual orientation) filed a suit against Netflix.

Massachusetts hospital discharge dataset

Medical Data Released as Anonymous

SSN	Name	...city	Date Of Birth	Sex	ZIP	Marital Status	Problem
			09/27/64	female	02139	divorced	hypertension
			09/30/64	female	02139	divorced	obesity
		asian	04/18/64	male	02139	married	chest pain
		asian	04/15/64	male	02139	married	obesity
		black	03/13/63	male	02138	married	hypertension
		black	03/18/63	male	02138	married	shortness of breath
		black	09/13/64	female	02141	married	shortness of breath
		black	09/07/64	female	02141	married	obesity
		white	05/14/61	male	02138	single	chest pain
		white	05/08/61	male	02138	single	obesity
		white	09/15/61	female	02142	widow	shortness of breath

Voter List

Name	Address	City	ZIP	DOB	Sex	Party	.....
.....	.....	.....	.....	.....	.....	.....	
Sue J. Carlson	1459 Main St.	Cambridge	02142	9/15/61	female	democrat	.....
.....	.....	.....	.....	.....	.....	.....	

Figure ...e-identifying anonymous data by linking to external data

Public voter dataset

As we can see in the example, an attacker learns sensitive data by joining two datasets on common attributes, in general one dataset is an anonymized dataset with sensitive attributes, and the other is a non-sensitive dataset with individual identifiers.

Quasi-identifiers are crucial: (birthdate, ZIP code, gender) uniquely identifies 63% of the US population nowadays. **Quasi-identifiers** are pieces of information that are not themselves unique identifiers, but are sufficiently well correlated with an entity that they can be combined with other quasi- identifiers to create a unique identifier.
So, publishing a record with a quasi-identifier is as bad as publishing it with an explicit identity; but eliminating quasi-identifiers is not desirable (for example, users of the dataset want to study distribution of diseases by age & ZIP code)

- Identifiers and sensitive attributes

Key Attribute		Quasi-identifier		Sensitive attribute
Name	DOB	Gender	Zipcode	Disease
Andre	1/21/76	Male	53715	Heart Disease
Beth	4/13/86	Female	53715	Hepatitis
Carol	2/28/76	Male	53703	Brochitis
Dan	1/21/76	Male	53703	Broken Arm
Ellen	4/13/86	Female	53706	Flu
Eric	2/28/76	Female	53706	Hang Nail

To mitigate this type of attack, let's define **k-anonymity**. You have k-anonymity if the information for each person contained in the released table cannot be distinguished from at least $k-1$ individuals whose information also appears in the release.
In other words, any quasi-identifier present in the released table must appear in at least k records.

To achieve k-anonymity you can apply generalization, which consists in replace quasi-identifiers with less specific but semantically consistent values until get k identical. When generalization causes too much information loss, you must apply suppression.

K=2
Quasi-identifiers: Race, Birth, Gender, ZIP

	Race	Birth	Gender	ZIP	Problem
t1	Black	1965	m	0214*	short breath
t2	Black	1965	m	0214*	chest pain
t3	Black	1965	f	0213*	hypertension
t4	Black	1965	f	0213*	hypertension
t5	Black	1964	f	0213*	obesity
t6	Black	1964	f	0213*	chest pain
t7	White	1964	m	0213*	chest pain
t8	White	1964	m	0213*	obesity
t9	White	1964	m	0213*	short breath
t10	White	1967	m	0213*	chest pain
t11	White	1967	m	0213*	chest pain

- By linking these two tables, you still don't learn Andre's problem

Released table

	Race	Birth	Gender	ZIP	Problem
t1	Black	1965	m	0214*	short breath
t2	Black	1965	m	0214*	chest pain
t3	Black	1965	f	0213*	hypertension
t4	Black	1965	f	0213*	hypertension
t5	Black	1964	f	0213*	obesity
t6	Black	1964	f	0213*	chest pain
t7	White	1964	m	0213*	chest pain
t8	White	1964	m	0213*	obesity
t9	White	1964	m	0213*	short breath
t10	White	1967	m	0213*	chest pain
t11	White	1967	m	0213*	chest pain

External data source

Name	Birth	Gender	ZIP	Race
Andre	1964	m	02135	White
Beth	1964	f	55410	Black
Carol	1964	f	90210	White
Dan	1967	m	02174	White
Ellen	1968	f	02237	White

But we need to find a tradeoff, since the more usable data is, the less protected it is (because you collect more personal data). In the other way, the more protected data is, the less usable it is.

The thing we are missing is that every row in a dataset must have a sort of unique identifier. In order for the researchers to be able to extract some information from the dataset, **pseudonyms** are used. Pseudonyms are identifiers that are uniquely associated with attributes in a consistent way. It is very important (for the reasons we've seen before) to make it difficult to correlate the pseudonyms to the real identity. So we need to find a way to create identifiers with this property.

A pseudonym may contain some information on the original identifier, therefore, every such type of pseudonym carries the risk of being subject to a re-identification attack.
In many cases, the additional information on the original identifier contained in the pseudonym is kept for linkage among pseudonyms themselves.
This may be an intentional feature of the pseudonymisation function allowing for some utility of the pseudonyms created, taking into account the potential loss of protection caused by this pseudonymisation approach.

A **pseudonymisation function** maps identifiers to pseudonyms.
The requirement is that, let $id1$ and $id2$ be two distinct identifiers and $pseudo1$ and $pseudo2$ be their corresponding pseudonyms, a pseudonymisation function must verify that $pseudo1$ is different than $pseudo2$ (otherwise, the recovery of the identifier could be ambiguous).
However, a single identifier id can be associated to multiple pseudonyms ($pseudo1$, $pseudo2$,…) as long as it is possible for the pseudonymisation entity to invert this operation.

In all cases, there exists some additional information that allows the association of the pseudonyms with the original identifiers (called **pseudonymisation secret**). The simplest case of pseudonymisation secret is the pseudonymisation mapping table.

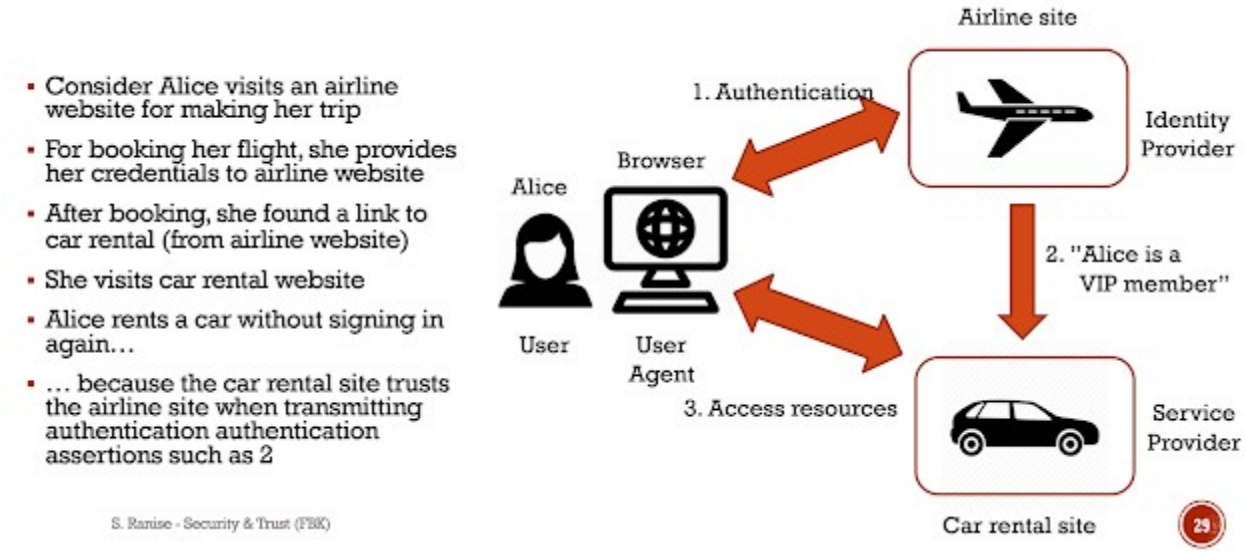
A pseudonymisation function can be implemented as:

- **Counter**
  Identifiers are substituted by a number chosen by a monotonic counter.
  A seed s is set to 9 (for instance) and then it is incremented. Values produced by the counter never repeat to prevent any ambiguity
  The pros are the simplicity and the cons are that sequential character of the counter can provide information on the order of the data and scalability issue for large datasets (full pseudonymisation mapping table needs to be stored)
- **Pseudo Random Number Generator**
  It is similar to counter although more complex solution, difficult to implement (avoid repetitions), but no sequential issue
- **Cryptographic Hash Function**
  The digest of the identifier is the pseudonym. It is considered inadequate for pseudonymisation as it is prone to brute force and dictionary attacks
- **Encryption**
  Usually, block ciphers like the AES are used for pseudonymization.
  The block cipher is used to encrypt an identifier using a secret key, which is both the pseudonymisation and recovery secret.
  Padding is used as the size of identifiers is usually less than the block size

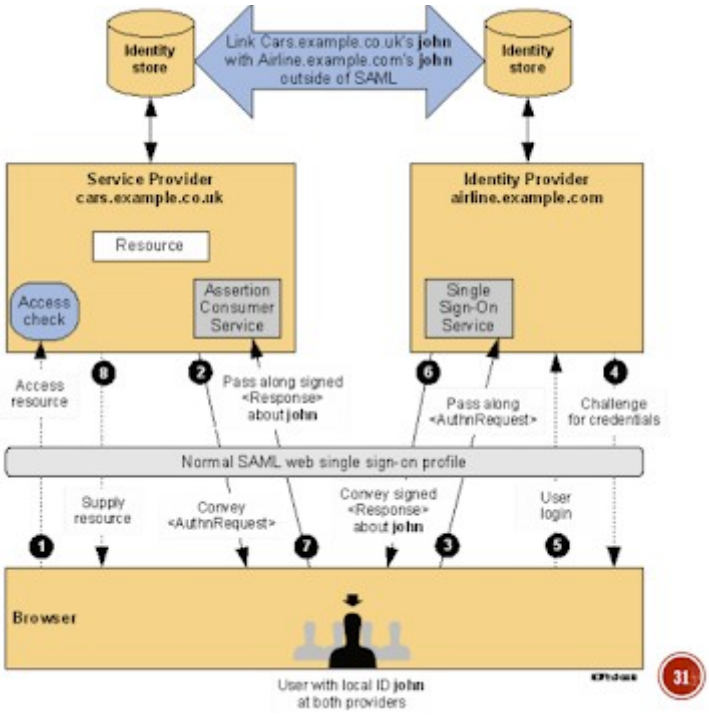
For example, SAML supports privacy because Identity Providers send pseudonyms to Service Providers to authenticate the user.
But they have to pay attention to how to use these pseudonyms. For example if they send the same pseudonyms to different services, then an attacker can understand the behavior of that user. Generating different pseudonyms every time a user logs in is very complex.

Now we are going to see different types of SAML federations implemented with pseudonyms, using the scenario of Alice, the Airline site and the Car Rental site.
We will understand how the message exchanged among users, identity providers, and service providers are tied to individual local and federated user identities shared between participants.
Without using pseudonyms, an attacker can view all the actions that a user makes on different service providers, and he can associate them with the user.
With the use of pseudonyms, the attacker can only view the actions that a user makes on a single service provider, and he can't easily associate them with the user.
When pseudonyms are used, they can be of two kinds:

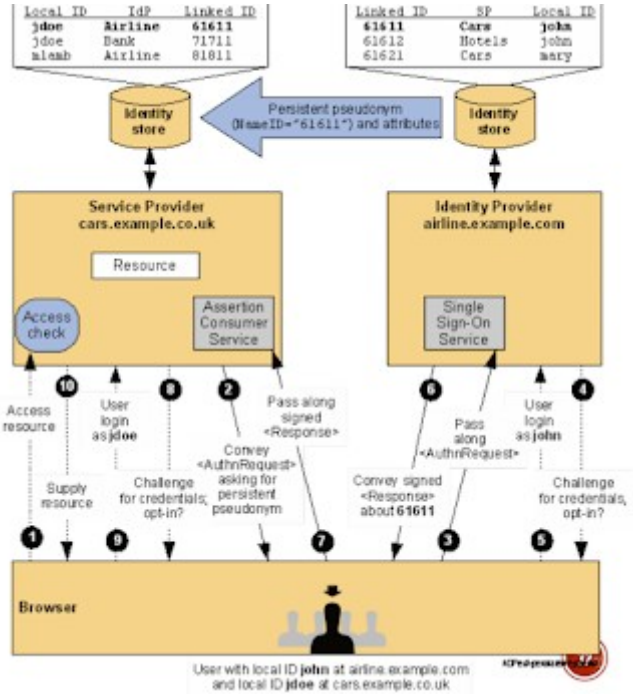
- **One-time or transient identifiers**
  Ensure that every time a certain user accesses a given service provider through a SSO operation from an identity provider, that service provider will be unable to recognize them as the same individual as might have previously visited.
- **Persistent pseudonyms**
  Established between an identity and a service provider.



The first SAML federation is **Federation via Out-of-Band Account Linking**.
In this case pseudonyms aren't used and the establishment of federated identities for users and the association of those identities to local user identities can be performed without the use of SAML protocols and assertions. This is the standard way of performing identity federation, but no privacy is provided.



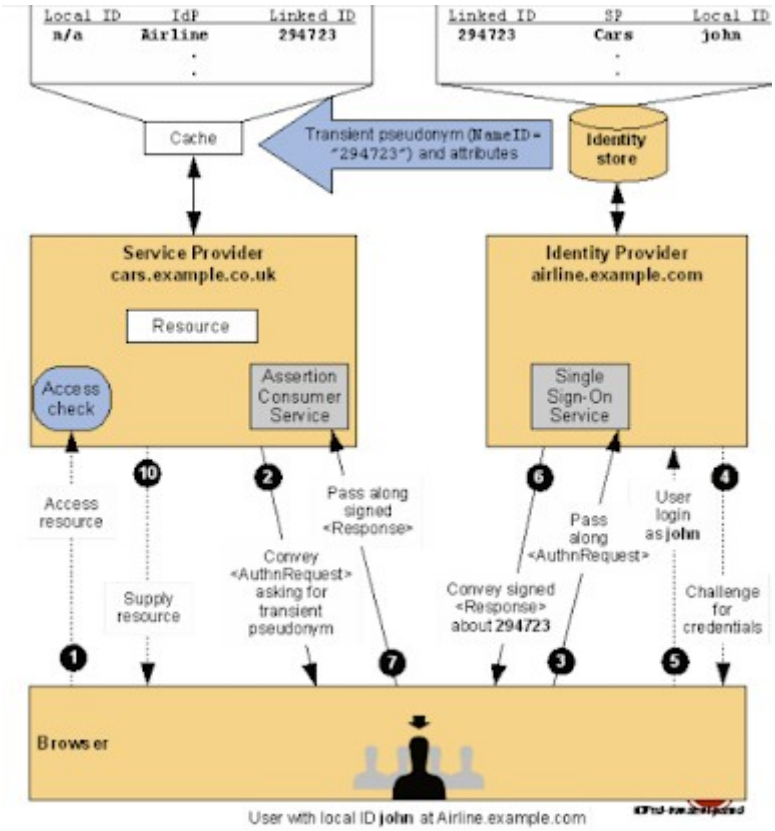
The second SAML federation is **Federation via Persistent Pseudonym Identifiers**.
The car rental site takes advantage of SAML 2.0's ability to dynamically establish a federated identity for a user as part of the web SSO message exchange. SAML 2.0 provides the NameIDPolicy element on the AuthnRequest to allow the SP to constrain such dynamic behavior.
The user jdoe on cars.example.co.uk wishes to federate this account with his john account on the IdP, airline.example.com.



The flow is:

1. Service provider initiated flow
2. User is challenged to enter credentials in the airline.example.com identity provider for username john (local to the identity provider)
3. If authentication is successful, the identity provider creates a persistent identifier (61611) to be used for the session at the service provider: the authentication assertion does not include the local name john
4. Upon reception and validation of the authentication assertion, the service provider checks if there is already a federation
    - If so, the username jdoe (local to the service provider) is retrieved from the association table and the SP checks if the user has the right to access the requested resource (this means checking if she is a VIP member)
    - If not, the service provider challenges the user to enter valid credentials for the username jdoe (local to the service provider), an association between the two usernames is entered in the table and the flow continues in a similar way as in the previous case

The last SAML federation is **Federation Using Transient Pseudonym Identifiers**.



The flow is:

1. Service provider initiated flow
2. User is challenged to enter credentials in the airline.example.com identity provider for username john (local to the identity provider)
3. If authentication is successful, the identity provider creates a temporary identifier (294723) to be used for the session at the service provider: the authentication assertion

does not include the name john but it contains contains a statement about the fact that the authenticated user is a VIP member

4. Upon reception and validation of the authentication assertion
    - The supplied transient name identifier is used to dynamically create a session for the user at the SP
    - The membership level attribute might be used to perform an access check on the requested resource

The techniques that we have seen above allow for a better protection of the anonymity of users with respect to the services providers since tracking user activities is made more complex by the use of pseudonyms (especially transient ones).
Still, identity providers can track user activities as they know how to de-anonymize the pseudonyms (since they implement anonymization).
However, notice that

- If the identity providers use the pseudonyms also internally, instead of the user identifiers, this already offers a better protection to the privacy of users in case of data breaches if the pseudonymization secret is adequately protected by suitable security mechanisms (e.g., cryptography and access control)
- If the identity providers delete from time to time the pseudonymization secret (there may be legal constraints that regulate the time interval between deletion), then even identity providers may have difficulty in reconstructing the entire tracking histories of users

General Data Protection Regulation (GDPR)

GDPR is a set of regulations which has been set by the European Union (in order to mitigate the fact that data privacy laws and regulations vary from country to country and even from state to state). GDPR went into effect on May, 25 2018.
Since GDPR must be guaranteed for all citizens of Europe, it also has an effect on companies which aren't based in Europe (like Google, Microsoft, etc). So the GDPR covers all EU citizens' data regardless of where the organization collecting the data is located.
Compliance with any one set of rules is complicated and challenging.

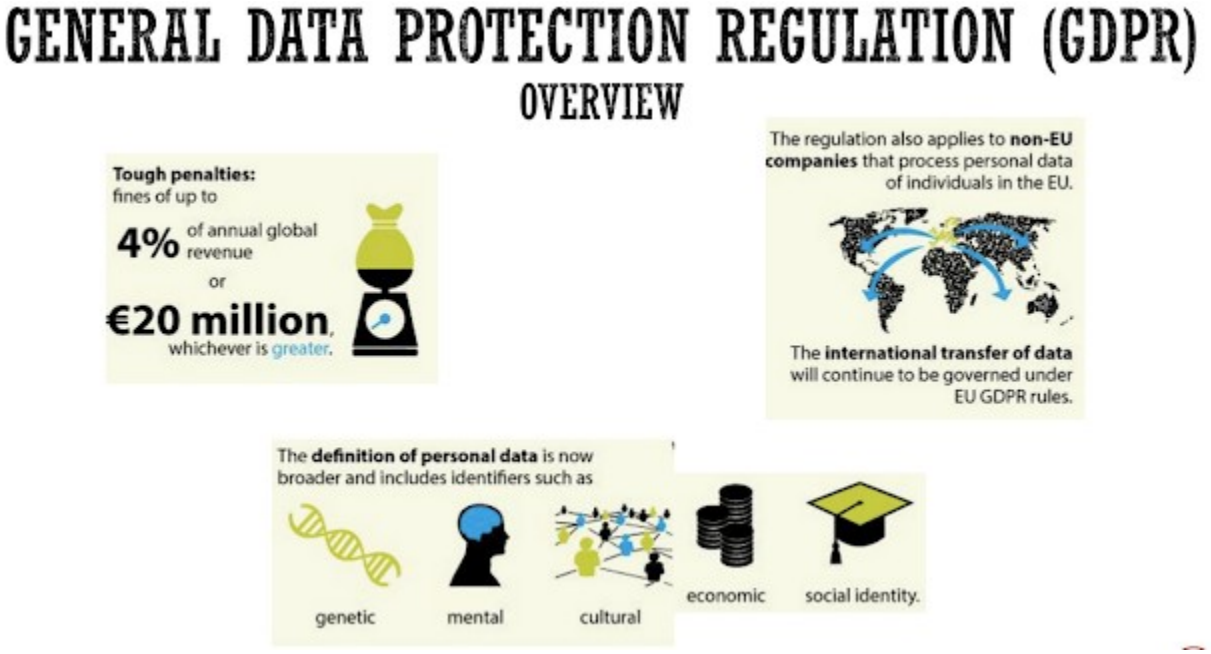
The basics of data protection and privacy apply across the board and include:

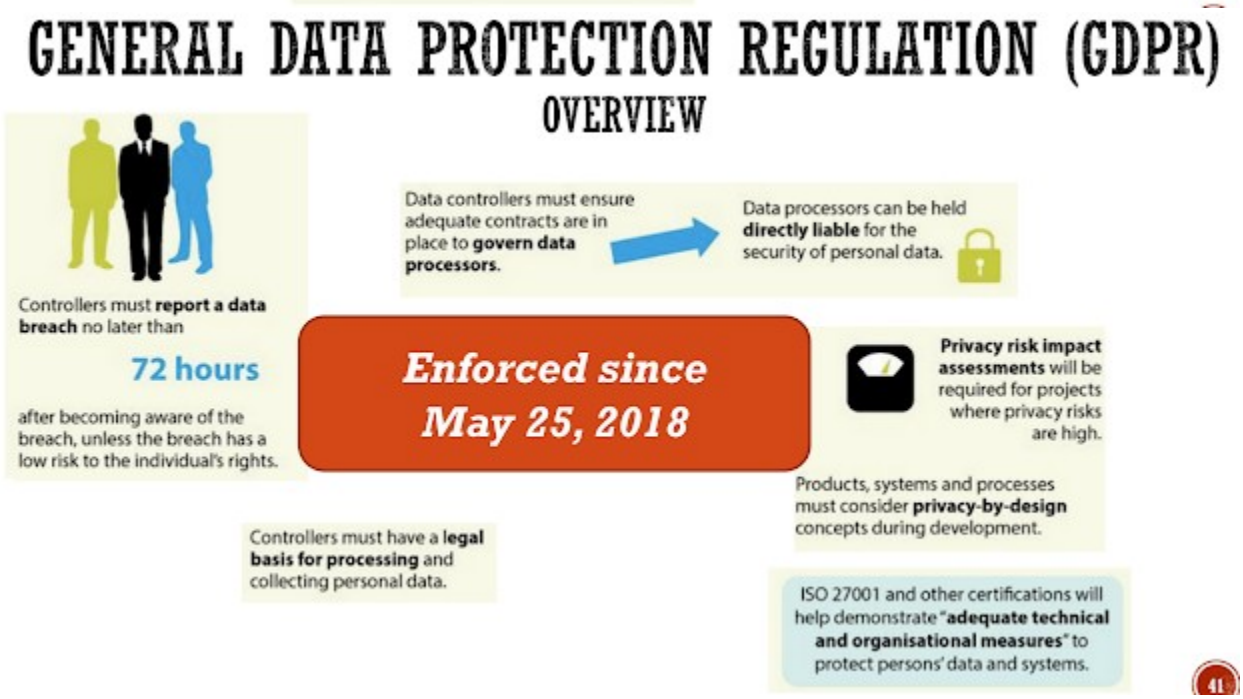
- safeguarding data
- getting consent from the person whose data is being collected
- identifying the regulations that apply to the organization in question and the data it collects
- ensuring employees are fully trained in the nuances of data privacy and security

GDPR requirements include:

- Barring businesses from storing or using an individual's personally identifiable information without that person's express consent
- Requiring companies to notify all affected people and the supervising authority within 72 hours of a data breach

- For businesses that process or monitor data on a large scale, having a data protection officer who's responsible for data governance and ensuring the company complies with GDPR
- Fines for not complying can be as much as €20 million or 4% of the previous fiscal year's worldwide turnover, depending on which is larger
- Performing the Data Protection Impact Assessment (DPIA) for every data processing activity. Data controllers must ensure adequate contracts are in place to govern data processors





The difficulty of having a legal basis for processing and collecting personal data is that there are several legal provisions (national and EU) and they are technology neutral, so we need for harmonization.
Some generic provisions are:

- all data is accurate and, where necessary, kept up to date
- data be kept for no longer than necessary
- data is kept secure
- data is transferred only to countries that offer adequate data protection

But these provisions depends on a lot of things (the scenario, the technologies, etc)

When we talk about privacy risk impact, it's not a matter of if a cybersecurity breach will happen, but **when** (risk-based approach to cybersecurity). So GDPR is focused on the impact rather than the likelihood.

Let's now see some articles from GDPR, with the impact of risks referring to the data subjects.

The **article 4** specifies the **definitions** of some words that are used in GDPR:

- **Personal data**
  Means any information relating to an identified or identifiable natural person ('data subject'); an identifiable natural person is one who can be identified, directly or indirectly, in particular by reference to an identifier such as a name, an identification number, location data, an online identifier or to one or more factors specific to the physical, physiological, genetic, mental, economic, cultural or social identity of that natural person;
- **Processing**
  Means any operation or set of operations which is performed on personal data or on sets of personal data, whether or not by automated means, such as collection, recording, organization, structuring, storage, adaptation or alteration, retrieval, consultation, use, disclosure by transmission, dissemination or otherwise making available, alignment or combination, restriction, erasure or destruction;
- **Pseudonymisation**
  Means the processing of personal data in such a manner that the personal data can no longer be attributed to a specific data subject without the use of additional information, provided that such additional information is kept separately and is subject to technical and organizational measures to ensure that the personal data are not attributed to an identified or identifiable natural person;
- **Controller**
  Means the natural or legal person, public authority, agency or other body which, alone or jointly with others, determines the purposes and means of the processing of personal data; where the purposes and means of such processing are determined by Union or Member State law, the controller or the specific criteria for its nomination may be provided for by Union or Member State law;
- **Processor**
  Means a natural or legal person, public authority, agency or other body which processes personal data on behalf of the controller;

- **Consent** of the data subject
  Means any freely given, specific, informed and unambiguous indication of the data subject's wishes by which he or she, by a statement or by a clear affirmative action, signifies agreement to the processing of personal data relating to him or her;
- **Personal data breach**
  Means a breach of security leading to the accidental or unlawful destruction, loss, alteration, unauthorized disclosure of, or access to, personal data transmitted, stored or otherwise processed;

The **article 7** specifies the **conditions for consent**:

1. Where processing is based on consent, **the controller shall be able to demonstrate that the data subject has consented to processing** of his or her personal data.
2. If the data subject's consent is given in the context of a written declaration which also concerns other matters, **the request for consent shall be presented in a manner which is clearly distinguishable from the other matters, in an intelligible and easily accessible form, using clear and plain language**. Any part of such a declaration which constitutes an infringement of this Regulation shall not be binding.
3. **The data subject shall have the right to withdraw his or her consent at any time**. The withdrawal of consent shall not affect the lawfulness of processing based on consent before its withdrawal. Prior to giving consent, the data subject shall be informed thereof. **It shall be as easy to withdraw as to give consent**.

The **article 32** specifies the **security of processing**:

1. Taking into account **the state of the art**, **the costs of implementation** and the nature, scope, context and purposes of processing as well as the risk of varying likelihood and severity for **the rights and freedoms of natural persons**, the controller and the processor shall implement appropriate **technical and organizational measures to ensure a level of security appropriate to the risk**, including inter alia as appropriate:
   a. the pseudonymisation and encryption of personal data;
   b. the ability to ensure the ongoing **confidentiality, integrity, availability** and resilience of processing systems and services;
   c. …
2. In assessing the appropriate level of security account shall be taken in particular of **the risks that are presented by processing**, in particular from accidental or unlawful destruction, loss, alteration, unauthorized disclosure of, or access to personal data transmitted, stored or otherwise processed.

The **article 33** specifies the **notification of a personal data breach to the supervisory authority**:

1. In the case of a personal data breach, **the controller shall without undue delay and, where feasible, not later than 72 hours after having become aware of it**, notify the personal data breach to the supervisory authority competent [...] **unless the personal data breach is unlikely to result in a risk to the rights and freedoms of natural persons** [...]

2. The processor shall notify the controller without undue delay after becoming aware of a personal data breach.
3. The notification referred to in paragraph 1 shall at least:
    a. describe the nature of the personal data breach including where possible, the categories and approximate number of data subjects concerned and the categories and approximate number of personal data records concerned;
    b. communicate the name and contact details of the data protection officer or other contact point where more information can be obtained;
    c. describe the likely consequences of the personal data breach;
    d. describe the measures taken or proposed to be taken by the controller to address the personal data breach, including, where appropriate, measures to mitigate its possible adverse effects

The **article 34** specifies the **notification of a personal data breach to the data subject**:

1. When the personal data breach is **likely to result in a high risk to the rights and freedoms of natural persons**, the controller shall communicate the personal data breach to the data subject without undue delay.
2. The communication to the data subject referred to in paragraph 1 of this Article shall describe in clear and plain language the nature of the personal data breach and contain at least the information and measures referred to in points (b), (c) and (d) of Article 33(3).
3. The communication to the data subject referred to in paragraph 1 shall not be required if any of the following conditions are met:
    a. the controller has implemented appropriate technical and organizational protection measures, and those measures were applied to the personal data affected by the personal data breach, in particular those that render the personal data unintelligible to any person who is not authorized to access it, such as encryption;
    b. the controller has taken subsequent measures which ensure that the high risk to the rights and freedoms of data subjects referred to in paragraph 1 is no longer likely to materialize;
    c. it would involve disproportionate effort. In such a case, there shall instead be a public communication or similar measure whereby the data subjects are informed in an equally effective manner.

The **article 35** specifies the **data protection impact assessment**:

1. **Where a type of processing in particular using new technologies, and taking into account the nature, scope, context and purposes of the processing, is likely to result in a high risk to the rights and freedoms of natural persons, the controller shall, prior to the processing, carry out an assessment of the impact of the envisaged processing operations on the protection of personal data** [...]
3. A data protection impact assessment referred to in paragraph 1 shall in particular be required in the case of:

a. a systematic and extensive evaluation of personal aspects relating to natural persons which is based on automated processing, including profiling, and on which decisions are based that produce legal effects concerning the natural person or similarly significantly affect the natural person;
b. processing on a large scale of special categories of data referred to in Article 9(1), or of personal data relating to criminal convictions and offenses referred to in Article 10;
c. a systematic monitoring of a publicly accessible area on a large scale.

7. The assessment shall contain at least:
    a. a systematic description of the envisaged processing operations and the purposes of the processing, including, where applicable, the legitimate interest pursued by the controller;
    b. an assessment of the necessity and proportionality of the processing operations in relation to the purposes;
    c. **an assessment of the risks to the rights and freedoms of data subjects** referred to in paragraph 1;
    d. the measures envisaged to address the risks, including safeguards, security measures and mechanisms to ensure the protection of personal data and to demonstrate compliance with this Regulation taking into account the rights and legitimate interests of data subjects and other persons concerned.

An important thing that we have to say is that GDPR is very difficult to enforce.

So the **risk**, and in particular the impact, is the main concept here. Let's say something about risk.
In the context of risk measurement, a **risk metric** (abstract & somewhat subjective attribute) is the concept quantified by a **risk measure** (a concrete & objective attribute). When choosing a risk metric, an agent is picking an aspect of perceived risk to investigate, such as volatility or probability of default.
In a general sense, a measure is a procedure for quantifying something. A metric is that which is being quantified. In other words, the method or formula to calculate a risk metric is called a risk measure.

Organizations should have multiple levels of metrics, each geared toward a particular type of audience:

- Lower-level metrics for tactical decisions (e.g. malicious code detection capabilities)
- Higher-level metrics for strategic decisions (e.g. overall effectiveness of incident prevention & handling capabilities)
- Lower-level metrics used as input to higher-level metrics

The accuracy of the metrics depends on the accuracy of the measures. The problems with the measures could be:

- Imprecise definition
- Ambiguous terminology

- Inconsistent measurement methods

The definition of measure and its measurement method are driven by what the organization is trying to accomplish.
Only measures supporting selected metrics are needed and you have to clearly and accurately specify dependencies among measures in the metrics using them.
Some measures may be more important than others in the scope of a metric (it is difficult to quantify what weight each measure should be given).

The **risk** of an adverse event is the **likelihood of the adverse event** multiplied by the **impact of the adverse event**.
The likelihood is evaluated over a specific time period. The likelihood depends on the probability that exploitable weaknesses are in the system and on the probability that those weaknesses are exploited.
When statistically independent multiple causes are considered, the individual risks need to be added to obtain overall risk.

An adverse event is equal to a vulnerability which is equal to a weakness which allows an attacker to reduce the system's information assurance.
Stochastic models can be used to evaluate the likelihood of adverse events (e.g. stochastic model of SW vulnerability lifecycle in).
Metrics from available vulnerability databases can be used to evaluate the impact of adverse events (e.g. for SW vulnerabilities can be used the Common Vulnerability Scoring Systems CVSS).

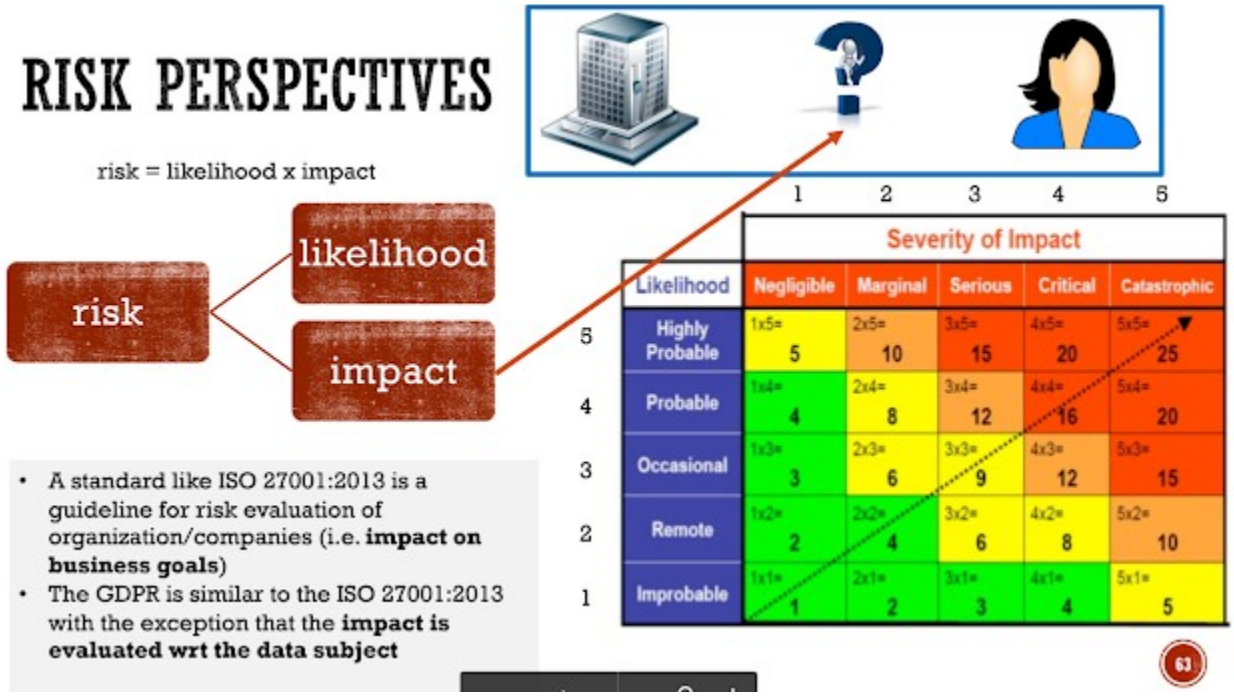
We list the several vulnerabilities of a system with $i = 1, \ldots, n$.
For each vulnerability i we define Li that is the likelihood of exploitation of vulnerability i and Ii that is the impact of exploitation of vulnerability i.
We have a risk matrix which defines the various levels of impact and likelihood.

	CONSEQUENCE				
LIKELIHOOD	Insignificant 1	Minor 2	Moderate 3	Major 4	Catastrophic 5
Almost Certain 5	5	10	15	20	25
Likely 4	4	8	12	16	20
Possible 3	3	6	9	12	15
Unlikely 2	2	4	6	8	10
Rare 1	1	2	3	4	5

The system risk is $\sum_{i=1}^{n} \left(Li \times Ii \right)$.

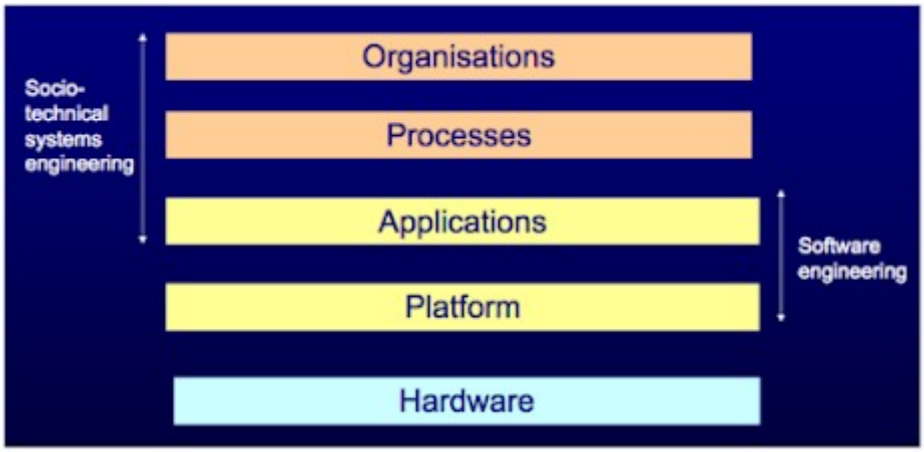
To give value to likelihood and impact CVSS can be used. It is an industrial standard for assessing security vulnerabilities and attempting to evaluate the degree of risks posed by vulnerabilities.
CVSS scores for known vulnerabilities are available on major public vulnerability dbs. For a score, few qualitative levels defined and numerical value associated with them
Base score: intrinsic characteristics of vulnerability, include 2 sub-scores:

- Exploitability: attempt to measure how easy it is to exploit the vulnerability
- Impact: measures how a vulnerability will impact an IT asset in terms of losses in Confidentiality, Integrity & Availability



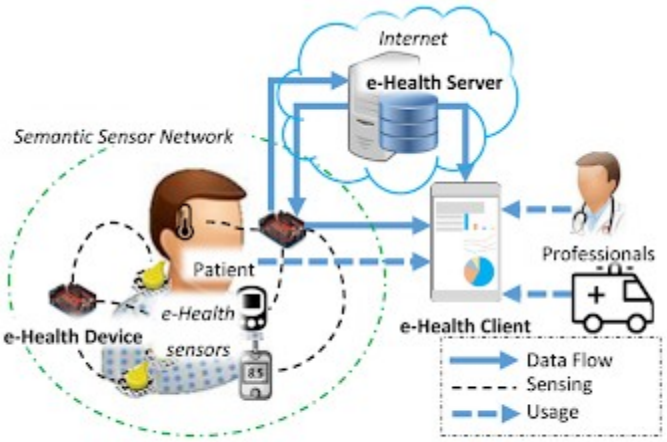
Two examples of data protection problems

Let's view examples about socio-technical systems, which touches both aspects (human and technical).
A **Socio-Technical System** (STS) is any system that involves interaction between humans, machines, and environmental aspects of the work system through technology.



Many areas of study from business to health look for ways to bridge social interaction with technology, a STS can frame these studies into easy to read models that will help businessmen and health experts find a middle ground between technology and social interactions to develop new technologies that will benefit them and the people.

Two examples of socio-technical systems are healthcare systems and social networks.







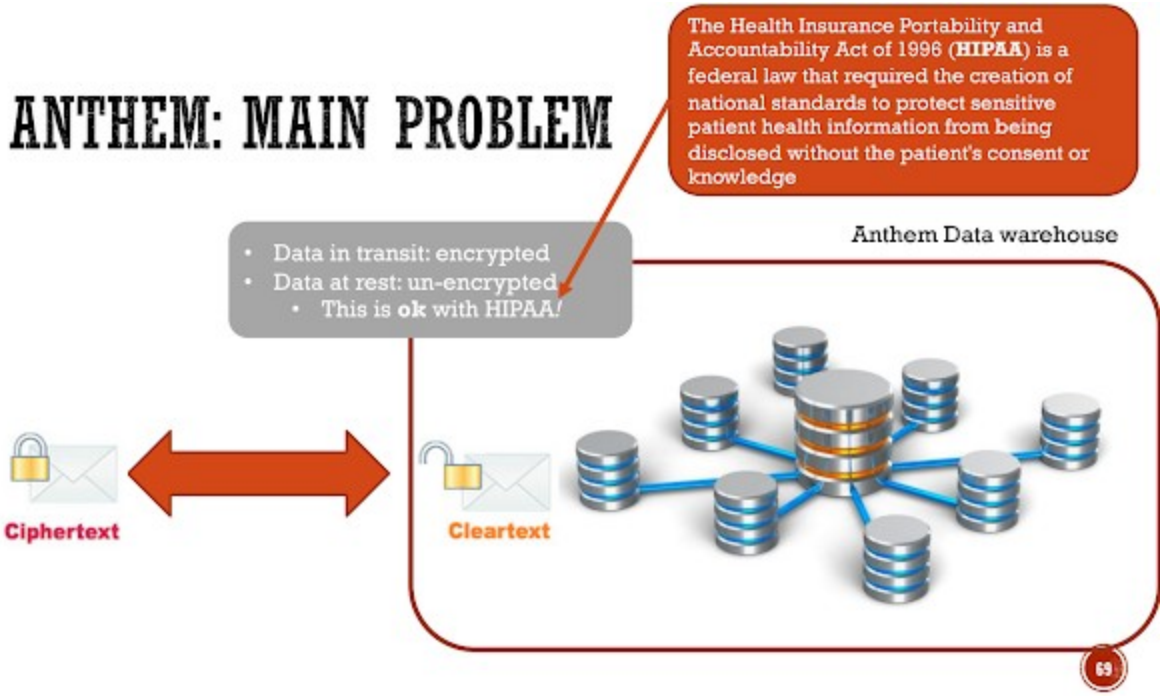
Let's start with the healthcare system, talking about **Anthem**, which is the biggest healthcare breach to date (due to a technical problem).
On January 29, 2015, 78.8 million patient records were stolen. There was a leakage of highly sensitive data, including names, Social Security numbers, home addresses, and dates of birth. The impact was that persons whose data was stolen could have resulting problems about **identity theft for the rest of their lives**!!!

Let's see the sequence of step of the attack:

1. user within one of Anthem's subsidiaries opened a phishing email containing malicious content

2. download of malicious files to the user's computer and allowed hackers to gain remote access to that computer and dozens of other systems within the Anthem enterprise, including Anthem's data warehouse
3. attacker was able to move laterally across Anthem systems and escalate privileges, gaining increasingly greater ability to access information and make changes in Anthem's environment
4. The attacker utilized at least 50 accounts and compromised at least 90 systems within the Anthem enterprise environment including, eventually, the company's enterprise data warehouse
5. Queries to that data warehouse resulted in access to an exfiltration of approximately 78.8 million unique user records

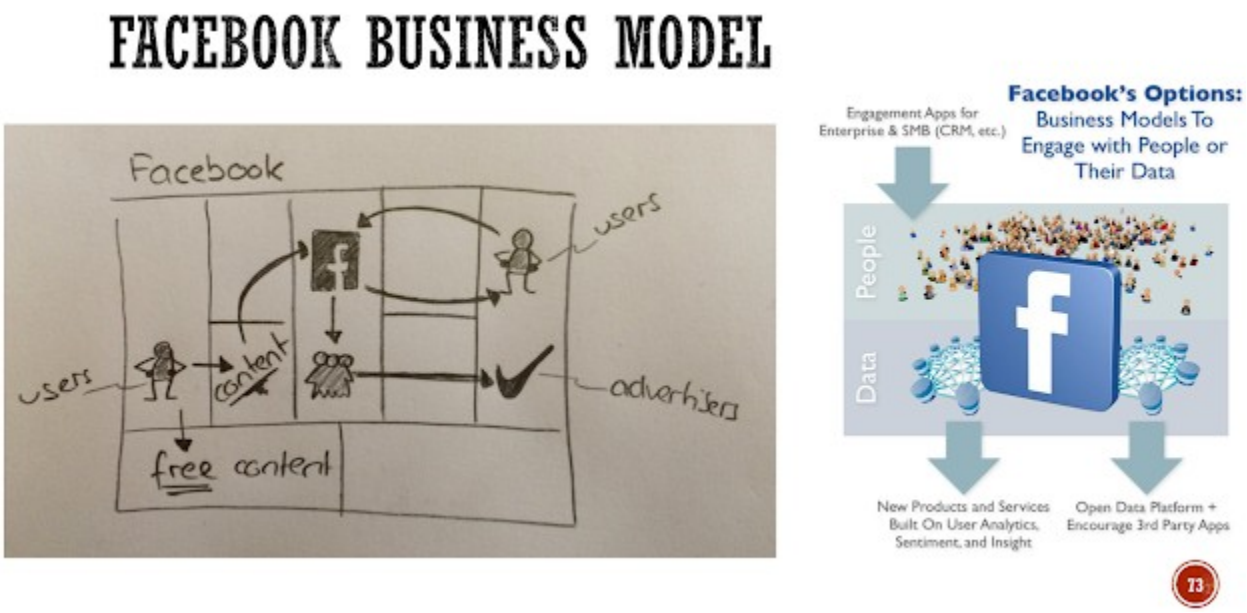
The main problem is that the data at rest in the data warehouse was unencrypted.



But also with data at rest encrypted, since the admin's credentials were compromised, with a little more effort also the decryption key would be (if they are stored in clear somewhere). Other security mechanisms (e.g., access control) have to protect these keys.

So, cryptography (alone) is rarely ever the solution to a security problem. Other security mechanisms are also important, such as Authentication and Access control (More than 80% of data breaches due to authentication and/or access control problems!)

Let's now talk about social networks, with **facebook**.
The scope of facebook is to collect a lot of data and make it available to third parties applications.

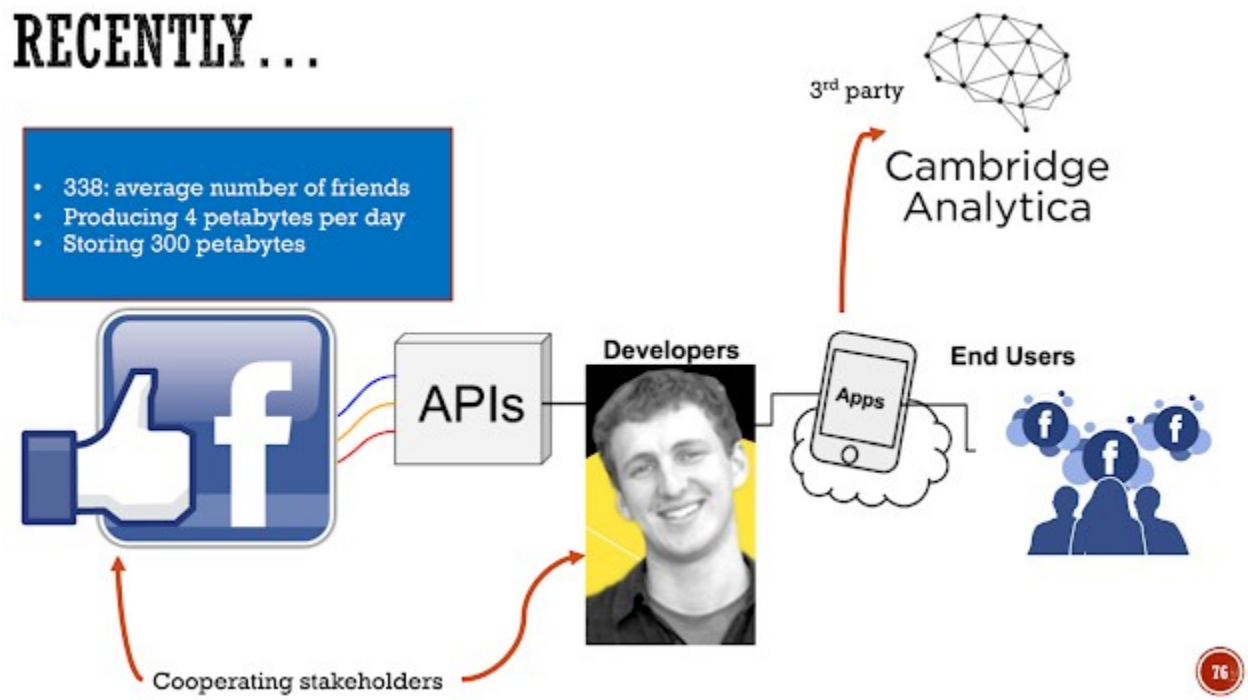


When someone uses Facebook Login to connect with apps and services, they grant those apps access to a range of information from their Facebook profile.

In 2015, Facebook also allowed apps to access some information from the friend networks of people who used Facebook Login, even though those friends may not have agreed to share their data.

Although Kogan (a company) gained access to this vast data set in a legitimate way and through the proper channels that governed developers on Facebook at that time, he shared it with Cambridge Analytica, violating Facebook's policies

Plus, the app was presented as a personality quiz, but the data collected was used for political marketing



It exploited the data to understand the political orientation of people and it ended up selling this data to Cambridge Analytica.
The point is that this guy collected a lot of information (telling users that he's collecting data for research purposes) and then sold it. So there's no technical issue.



So the problem is how can Facebook (or other social networks) prevent bad actors from using platforms to harvest personal data? The idea was to offer better visibility into how user information is handled by third-party apps.

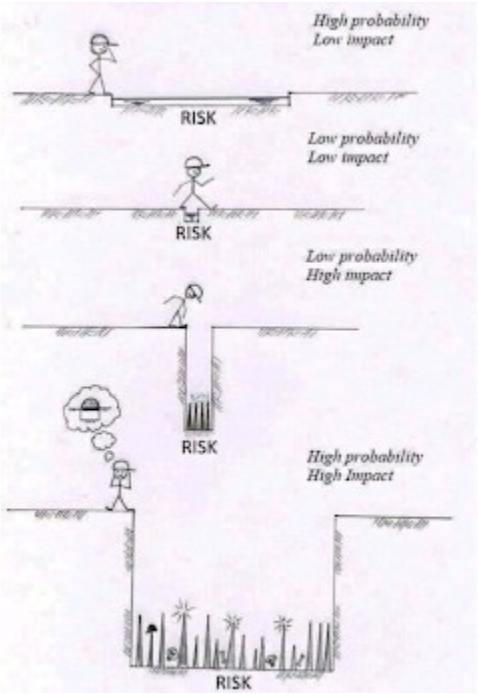


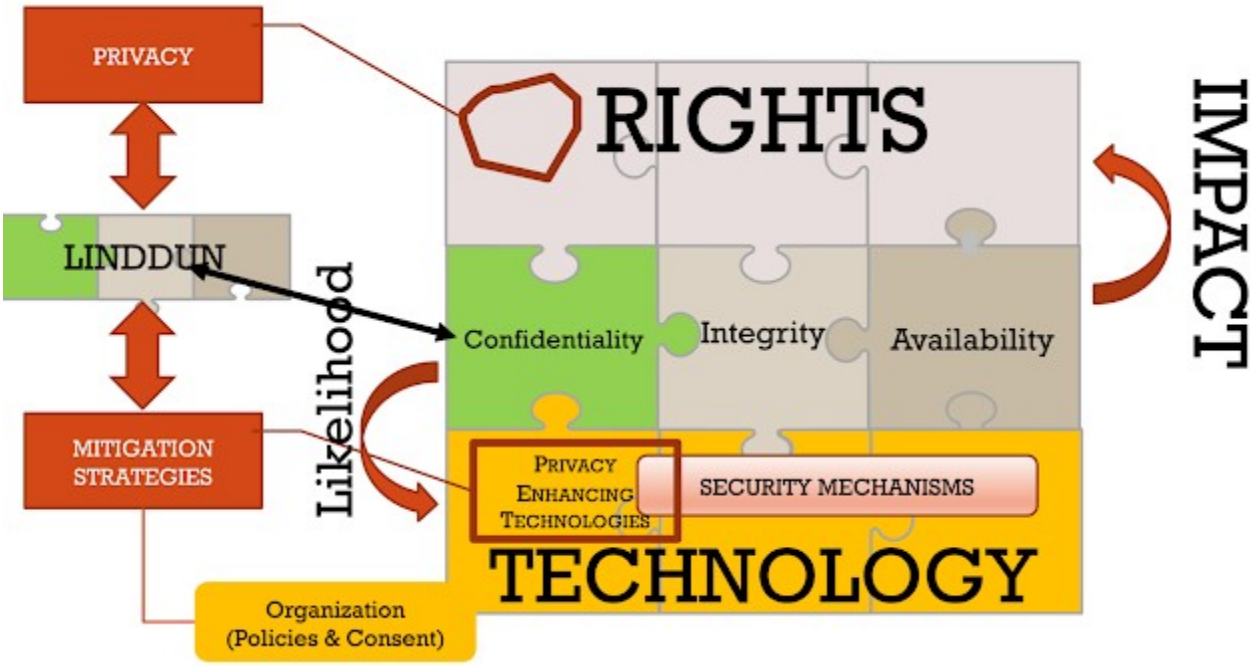
Key to combine
- Technolgical
- Legal

aspects of security, privacy, and fundamental rights

RISK MATRIX

		1	2	3	4	5
	Likelihood	Negligible	Marginal	Serious	Critical	Catastrophic
5	Highly Probable	1x5= 5	2x5= 10	3x5= 15	4x5= 20	5x5= 25
4	Probable	1x4= 4	2x4= 8	3x4= 12	4x4= 16	5x4= 20
3	Occasional	1x3= 3	2x3= 6	3x3= 9	4x3= 12	5x3= 15
2	Remote	1x2= 2	2x2= 4	3x2= 6	4x2= 8	5x2= 10
1	Improbable	1x1= 1	2x1= 2	3x1= 3	4x1= 4	5x1= 5





In other words, data protection (as concieved by the GDPR) is an even broader and more subtle notion than privacy!

TWO EXAMPLES

- Violation of integrity
  - Consider a database containing the data of citizens (e.g., salary, owner of house, …) enabling them to ask for tax reductions
  - If an attacker modify such data, it is possible that a citizen will be denied the tax reduction even though he/she is will entitled to do so

- Violation of availability
  - Consider an on-line system to register kids to school; the system accepts requests only during a limited period of time
  - If an attacker mount a denial of service attack during the time window during which it is possible to submit registration requests, then the right to education can be reduced

RECAP QUESTIONS

- What is the likelihood of an event? What is the impact of an event? What is the risk of an event? What is the risk matrix?
- Define the notion of privacy. What is k-anonimity?
- What is data protection? What is LINDDUN?
- What is the scope of application of the GDPR? Who is the data controller? Who is the data processort? What is the Data Protection Impact Assessment? Why does the GDPR propose a risk-based approach to data protection?
- What is the difference between the risk evaluated for an organization and the risk of a data processing activity in the GDPR?